

Principles of Data Engineering

Matthias Wong

The book is available on principlesofdataengineering.org.

Downloadable PDF and EPUB editions are available for offline reading. The online version is canonical and may be updated over time.

Contents

1	The author	1
1.1	About the author	1
2	The book	3
2.1	Dedication and acknowledgements	3
2.2	Origin and release	3
2.3	Attribution	3
	Foundations	5
3	What is data engineering?	7
3.1	The aim is insight	7
3.2	Why is it engineering	7
3.3	Where data engineering is different	7
3.3.1	Easy substitutes	7
3.3.2	Open-ended questions	8
3.3.3	Ambiguous goals	8
3.4	The fluid nature of data engineering	8
4	What is data?	11
4.1	The central premise	11
4.2	How is data captured?	11
4.3	How is data used?	12
5	Data and organisations	15
5.1	Data and organisational self-understanding	15
5.2	The organisation agency cycle	15
5.3	Implications for data engineering	17

Creating information	19
6 Expressiveness and fragment modelling	21
6.1 From data to reusable information	21
6.2 Expressiveness	21
6.3 Fragment modelling	22
7 Mapping the data world	25
7.1 From records to entities	25
7.2 Primary key	25
7.3 Immutable and mutable entities	27
8 Entity processing	29
8.1 Processing immutable entities	29
8.2 Building the pipeline	30
8.2.1 First pass—Filter	30
8.2.2 Second pass—Map	32
8.2.3 Third pass—Reduce	33
8.2.4 Summary of the three passes	34
8.3 Common problems	35
9 Entity tracking	37
9.1 Tracking mutable entities	37
9.2 Two ways to track	38
9.3 Building the pipeline	38
9.3.1 First pass—Filter and compress	38
9.3.2 Second pass—Build the timeline	39
9.3.3 Third pass—Create snapshots and infer behaviour	41
9.3.4 Summary of the three passes	42
9.4 Common problems	42
10 Reference data	45
10.1 A stable place for meaning	45
10.2 A point of reference	45

10.3	Building and applying shared references	46
10.3.1	Step 1—Model the local system	46
10.3.2	Step 2—Build shared references	47
10.3.3	Step 3—Apply during presentation	48
10.4	Why these separations	49
10.5	When to simplify	49
10.6	The discipline of separation	50
11	Conforming systems	51
11.1	Integration without distortion	51
11.2	Vertical integration	51
11.2.1	Step 1—Model systems individually	52
11.2.2	Step 2—Build conformed references	52
11.2.3	Step 3—Integrate the transactions	53
11.3	Horizontal integration	55
11.3.1	Horizontal integration through mapping fragments	57
11.4	Choosing between vertical and horizontal integration	58
12	Storytelling	61
12.1	Meaning beyond correctness	61
12.2	Good and bad entities	61
12.3	Trading detail for insight	62
12.3.1	Creating categories	63
12.3.2	Pivoting combinations	64
12.3.3	Highlighting special cases	65
12.4	Storytelling dimensions	66
12.4.1	Step 1—Build the reference table	68
12.4.2	Step 2—Map to facts	69
12.4.3	Step 3—Visual check	70
12.4.4	Multiple storytelling dimensions	70
13	Meaningful fragments	73
13.1	Fragment modelling	73

13.2	Why giant tables fail	73
13.3	Meaningful fragments	73
13.3.1	Restaurant analogy	74
13.3.2	The impact on legibility	74
13.4	Symptoms of poor fragmentation	74
13.5	Common fragment patterns	75
13.5.1	Summaries of details and storytelling	75
13.5.2	Milestone datetimes	76
13.5.3	Current or primary version	76
13.5.4	Timeline and end-of-period	77
13.5.5	Mapping	77
13.5.6	Distribution weights	78
13.5.7	Hubs	80
13.6	The discipline of meaningful fragments	81

Presenting insights

83

14 The craft of dimensional modelling

85

14.1	From information to insight	85
14.2	The craft	85

15 Anticipating questions

87

15.1	Designing for questions not yet asked	87
15.2	All the facts	88
15.3	All the dimensions	90
15.4	All the relationships	92
15.5	Conclusion	96

16 A good dimensional model

99

16.1	What makes a good dimensional model?	99
16.2	What should a good model achieve?	99
16.2.1	Resonate with the view from business intent	99
16.2.2	Be intuitive and unambiguous	100

16.2.3	Anticipate questions	100
16.2.4	Support both summary and detail	100
16.2.5	Perform quickly	101
16.3	What are the signs of a good model?	101
16.3.1	Names and metadata	101
16.3.2	Dimensions	102
16.3.3	Facts and measures	103
16.3.4	Relationships	104
16.4	Conclusion	105
17	Dimensional modelling for UX	107
17.1	Facts and dimensions as interaction	107
17.2	Understanding facts and dimensions	107
17.2.1	An analogy of robots	108
17.3	A repertoire of facts and dimensions	108
17.4	Facts	109
17.4.1	Measurable fact	109
17.4.2	End-of-period fact	110
17.4.3	Annotation fact	111
17.5	Dimensions	112
17.5.1	Business dimension	112
17.5.2	Role-playing dimension	112
17.5.3	Histogram dimension	113
17.5.4	Combination dimension	114
17.5.5	Choices dimension	115
17.5.6	Sankey dimension	117
17.5.7	Storytelling dimension	118
17.5.8	Search dimension	119
17.5.9	Degenerate dimension	120
17.6	Pushing detail back to the entity	121
18	Filtering behaviour	123

18.1	Filtering as interaction design	123
18.2	Ways of filtering	124
18.2.1	Relationship filtering	124
18.2.2	Common-fact filtering	125
18.2.3	Non-blank measure filtering	126
18.2.4	Visual-level filtering	127
18.2.5	Measure-defined filtering	127
18.3	Filtering scenarios	128
18.3.1	Displaying unit records for a single business process	129
18.3.2	Displaying unit records for two business processes	132
18.3.3	Displaying sales	133
18.3.4	Displaying refunds	133
18.3.5	Displaying sales and refunds together	134
18.3.6	Displaying unit records for three or more business processes	137
18.3.7	Cascading filters	138
18.3.8	Aggregating dimension values	139
18.3.9	Filter by having	140
18.3.10	Dynamic Type I	142
18.4	Buttons and effects	145
19	Designing measures	149
19.1	Measures as compression and re-expression	149
19.2	Signs of good measures	149
19.2.1	Business centricity	149
19.2.2	Technical simplicity	153
19.3	Interface roles of measures	155
19.3.1	Aggregating measures	155
19.3.2	Dimensional measures	156
19.3.3	Filtering measures	156
19.3.4	Dashboard measures	157
19.4	Technical patterns for measures	158
19.4.1	Base measures	158

19.4.2	Derived measures	160
19.4.3	Context-aware measures	160
19.4.4	Advanced scenario: polymorphic keys	161
19.4.5	Advanced scenario: embedded grain	162
19.4.6	Advanced scenario: unsupported relationships	163
20	Measure of measures	165
20.1	Measures as a modelled structure	165
20.2	The problem	165
20.3	The measure of measures pattern	166
20.3.1	Step 1—Create a measure table	167
20.3.2	Step 2—Create a switch measure	168
20.3.3	Step 3—Define derived measures	168
20.3.4	Step 4—Handle formatting	170
20.4	Benefits	170
20.4.1	Simplified DAX management	170
20.4.2	Explicit business structure	171
20.4.3	More powerful reporting	171
20.4.4	Avoiding awkward fact-table unions	172
20.5	Dangers	172
20.5.1	Performance	172
20.5.2	Dynamism for its own sake	172
20.5.3	Hiding important information behind clicks	172
20.5.4	Ambiguous aggregation across measures	173
20.6	Relationship to switch measures	173
21	Interactive row level security	175
21.1	Security without losing context	175
21.2	Limiting what users can see	175
21.2.1	Static roles	176
21.2.2	Dynamic roles	176
21.3	Preserving population context	177

21.3.1 Anonymous facts	178
21.3.2 Pseudonymous dimensions	179
21.4 Choosing an approach	181
Quality & reliability	183
22 The foundations of trust	185
22.1 Quality and reliability	185
22.2 Trust requires both	186
22.3 The third principle: anticipate errors	186
23 Quality metadata	187
23.1 When metadata is missing	187
23.2 Names answer: what is this thing?	188
23.3 Descriptions answer: what does this mean?	189
23.4 Keys answer: how does this relate?	190
23.5 Metadata should be data	191
24 Three approaches to data quality	193
24.1 Quality means fitness for intent	193
24.2 Human curation: when judgement is needed	195
24.2.1 Data annotation	195
24.2.2 Applying assumptions	196
24.2.3 Data quality report	197
24.3 Precise rules: when intent can be formalised	198
24.3.1 Defining analytical concepts	198
24.3.2 Defining primary keys	202
24.3.3 Defining the primary record	205
24.3.4 Defining relationships	206
24.4 Fuzzy logic: when intent can only be approximated	210
24.4.1 Step 1—Loose-tight iteration	211
24.4.2 Step 2—Random validation	213
24.4.3 Step 3—Monitoring for drift	214

25 Tests and assumptions	217
25.1 Anticipating errors	217
25.2 Tests: checking the same result two ways	218
25.2.1 Row counts	219
25.2.2 Checksums on key results	220
25.2.3 Bypassing mapping tables	221
25.2.4 Checking boundary cases	222
25.2.5 Using subject matter knowledge	223
25.3 Monitored assumptions: surfacing records that require attention	224
25.3.1 Is source data up to date?	225
25.3.2 Is reference data complete?	225
25.3.3 Are there unanticipated values?	226
25.3.4 Are there data quality issues?	226
25.3.5 Is there data drift?	227
25.4 Tests and assumptions together	227
25.5 Conclusion	228
26 Fault tolerance	231
26.1 Containing failure	231
26.2 Uniqueness	232
26.3 Existence	233
26.4 Stability	235
26.5 Conclusion	237
Efficient & stable pipeline	239
27 Efficiency and stability	241
27.1 Correspondence between information and computation	241
27.2 Efficiency	241
27.3 Stability	242
27.4 Efficiency and stability together	243
28 Load mechanics	245

28.1	Controlling change	245
28.2	The three-step load pattern	246
28.2.1	Step 1—Stage the incoming data	246
28.2.2	Step 2—Check the incoming data	247
28.2.3	Step 3—Apply the changes	251
28.3	After the load	252
28.3.1	Clean up temporary artefacts	253
28.3.2	Log, bookmarks and change statistics	253
28.4	Is the extra work worth it?	254
29	Load orchestration	257
29.1	Load stack	257
29.2	Orchestrating a load	257
29.2.1	Dependency metadata	258
29.2.2	The load stack	259
29.2.3	Load candidates	259
29.3	Using the load stack	260
29.3.1	Step 1—Refresh the load stack	260
29.3.2	Step 2—Start workers	260
29.3.3	Step 3—Claim a candidate	261
29.3.4	Step 4—Load, end, and repeat	262
29.3.5	Handling failure	264
29.4	Consequences of the load stack	264
29.4.1	Correct order is computed, not hard-coded	264
29.4.2	Parallelism follows from readiness	265
29.4.3	Execution state is visible and manipulable	265
29.4.4	Higher-frequency loads	266
29.4.5	Cross-technology loading requires shared state, not shared tooling	266
30	Load dependencies	267
30.1	Dependency as coupling	267
30.2	Load dependencies and view dependencies	267

30.3	The dependency trade-off	268
30.4	Valuable dependencies	268
30.5	Targeted dependencies	269
30.6	Stable dependencies	270
30.6.1	Row-wise instability	271
30.6.2	Column-wise instability	271
30.7	The case of surrogate keys	272
30.8	Views as an alternative	273
30.9	Healthy dependency depth	274
31	Incremental load: tracking changes	277
31.1	Change observability	277
31.2	The problem of time	277
31.3	The simple update-datetime approach	278
31.4	Refresh bookmarks	279
31.5	The source time problem	280
31.6	Polling tables	281
31.7	When polling tables can be skipped	285
31.8	The role of the filter step	285
32	Incremental load: responding to change	287
32.1	Change translation	287
32.2	Source changes are not target actions	288
32.3	Analysing the query	288
32.3.1	Common query shapes	289
32.3.2	Robustness	290
32.4	Worked examples	291
32.4.1	Worked example 1—Filter	291
32.4.2	Worked example 2—Aggregation	292
32.4.3	Worked example 3—Anti-join	293
32.5	Applying the change	295
32.5.1	Step 1—Write the full query	295

32.5.2	Step 2—Fetch the refresh bookmark	296
32.5.3	Step 3—Create the upsert driver	296
32.5.4	Step 4—Create the delete driver	297
32.5.5	Step 5—Apply load mechanics	298
32.5.6	Full incremental extract pattern	298
32.6	Why this pattern works	301
32.6.1	Uniformity	301
32.6.2	Idempotency	301
32.6.3	Graceful fallback	301
32.7	Best-practice workflow	301
32.7.1	Step 1—Create a full-load comparison test	301
32.7.2	Step 2—Create a realistic change window	302
32.7.3	Step 3—Build the upsert driver	302
32.7.4	Step 4—Test the minimal staging table	302
32.7.5	Step 5—Build the delete driver	302
32.7.6	Step 6—Simulate deletes before applying them	303
32.7.7	Step 7—Apply changes through load mechanics	303
32.7.8	Step 8—Run a zero-load benchmark	303
32.7.9	Step 9—Run incrementally over time	303
32.8	Using AI as a reviewer	303
32.9	Conclusion	304

33 Optimising Power BI load 305

33.1	Power BI as the final load boundary	305
33.2	Avoiding load with DirectQuery	306
33.3	Efficient underlying source tables	306
33.4	Partitioning the model	307
33.4.1	Analogy for partitions	307
33.4.2	Parallel partition refresh	308
33.4.3	Rolling windows	309
33.4.4	Incremental partition refresh	309
33.4.5	Custom polling tables	310

33.4.6 Example refresh bookmarks and polling values	310
33.5 Conclusion	312
Judgement under ambiguity	315
34 The six principles of data engineering	317
34.1 The problem with shallow curation	317
34.2 Six principles	318
35 Working with stakeholders	319
35.1 Guided attention	320
35.1.1 Focus on trust	320
35.1.2 Lead by listening	321
35.1.3 Own the business intent	322
35.1.4 Anchor the vision	323
35.1.5 Gather around the solution	324
35.1.6 Design for workflows	325
35.1.7 Spot the 20%	326
35.2 Conclusion	327
36 Construction planning	329
36.1 An effective plan	329
36.2 Formulating an effective plan	330
36.2.1 Stage 1 – Discovery	331
36.2.2 Stage 2 – Vision	331
36.2.3 Stage 3 – Scope	332
36.2.4 Stage 4 – Build	333
36.3 Stakeholder requirements	336
36.4 Conclusion	336
37 Sound judgement	339
37.1 A diagnostic framework for sound judgement	339
37.1.1 A simple example	340

37.1.2 Step 1: Symptoms — seeing clearly before acting	340
37.1.3 Step 2: Triggers — locating the point of exposure	341
37.1.4 Step 3: Root causes — acting at the level that matters	341
37.1.5 Step 4: Final effect — knowing when you are done	342
37.2 Sound judgement throughout data engineering	343
37.3 Conclusion	343
38 Getting started as a data engineer	345
38.1 Developing good habits	345
38.1.1 Aim to be helpful, especially on small tasks	345
38.1.2 Talk to the team	345
38.1.3 Talk to the business	346
38.1.4 Understand the architecture	346
38.1.5 Technical hygiene	346
38.1.6 Check, don't assume	346
38.1.7 Estimate and refine	346
38.1.8 Focus on the principles	347
38.2 Conclusion	347
39 Closing essay: Hallmarks of quality	349
39.1 Expressive entities	350
39.2 Well-written explanation	350
39.3 Thoughtful unit-tests	351
39.4 Assumptions are monitored	352
39.5 Anticipate errors	352
39.6 Adherence to patterns	353
39.7 Optimised code	354
39.8 Three aspects of quality	355
39.8.1 Expressiveness	355
39.8.2 Error handling	355
39.8.3 Elegant code	355
39.9 Final words	355

The author

Matthias Wong writes from experience at the intersection of theory, systems, and delivery.

1.1 About the author

Matthias Wong's work examines how organisations see, understand, and act in complex environments. His writing spans organisational theory, data, AI, business, and people leadership.

As part of this work, he has led and trained advanced data engineering teams working on high-stakes systems, with a focus on transforming fragmented operational data into usable, decision-ready information.

Principles of Data Engineering forms the technical segment of this broader work: an account of how organisations make reality visible through data, and how engineers can build the representations organisations rely on to exercise sound judgement.

He is especially passionate about uplifting staff from junior to advanced capability, and providing senior advisory to organisational leaders on the effective use of data to steer complex institutions.

Matthias holds a PhD in Computational Mathematics.

For contact information, see the author's website: matthiaswong.me.

The book

The context, origin, and attribution of this work.

2.1 Dedication and acknowledgements

This book is dedicated to N.H. and G.H.

It was born from years of ongoing dialogue with developers and business stakeholders at the Department of Agriculture, Fisheries and Forestry between 2017 and 2025.

It would not have been possible without the many data engineers and analysts who contributed their thinking, challenged assumptions, and worked through complex problems in practice. Their experiences, questions, and solutions form the foundation of much of what is written here.

Special thanks to E.D. and M.T. for their contribution and support throughout this journey, and to M.C., Z.B., D.S., S.S., S.M., and R.D. for their review and feedback.

2.2 Origin and release

Principles of Data Engineering was written in the Department of Agriculture, Fisheries and Forestry, and released under Freedom of Information.

The site principlesofdataengineering.org is a web edition of that manuscript. It has been transcribed, edited, restructured, and supplemented for online publication by Matthias Wong.

2.3 Attribution

Department of Agriculture, Fisheries and Forestry. 2026. *Principles of Data Engineering manual* [FOI Disclosure Log Reference 35670](#).

The book *Principles of Data Engineering* includes adapted and additional material by Matthias Wong.

© Department of Agriculture, Fisheries and Forestry 2026 Adaptation and additional material
© Matthias Wong 2026

Foundations

What is data engineering?

The aim of data engineering is insight.

3.1 The aim is insight

The aim of data engineering is insight.

But what is insight? Insight is information analysed in the light of intent.

Data engineering, therefore, is an activity in which business intent is applied to data to create first information, then insight.

3.2 Why is it engineering

There are other activities besides data engineering that seek insight. These include staff consultation, customer feedback, and market research. In the case of government agencies, these activities include overt and covert means such as diplomacy and espionage.

Data engineering distinguishes itself from other means of gaining insight through the nature of its methodology.

Like other engineering disciplines, it focuses on systematic rigour, efficiency, and reproducibility—achieving these through the application of scientific and mathematical principles. As with other engineering disciplines, data engineering arose in today's technological society as an effective tool to tackle complexity.

3.3 Where data engineering is different

Despite similarities in method, data engineering differs from other engineering disciplines in crucial ways.

3.3.1 Easy substitutes

There are easy substitutes to data engineering.

The business engaging a data engineer is seeking insight, and in many instances, there are seemingly simple alternatives to serious data engineering. For example, business clients may choose to rely on their own judgement or use one of the many available self-service data tools to rapidly create a visually stunning report with little effort.

This is not the case in other disciplines. The client who engages a bridge engineer needs a bridge. The client who engages a software engineer needs software. There are no easy alternatives to achieving these aims, and therefore no easy alternatives to the engineering effort.

3.3.2 Open-ended questions

The search for business insight is open-ended.

If a client asks for a bridge, the bridge is the end of the engagement. A bridge is not likely to extend into a longer bridge, nor is the client likely to settle for half a bridge. This is not the case with business insight: a question provokes another question, and a question may be half answered.

This interminability of aim means that a data engineering project is likewise open-ended. In no other engineering discipline is the engineer working with such a degree of open-endedness.

3.3.3 Ambiguous goals

The search for business insight is ambiguous.

Quite often, when stakeholders begin a data engineering project, they do not know what business insights they are after. Partly this is because business areas first need to discover the information before they know what questions to ask. Partly it is due to the wide-ranging and sometimes conflicting business intents—are we concerned about profit, cost, market share, or negative externalities?

And partly it is because of the wide range of stakeholders who need to be engaged—senior executives and operational staff have valid but different concerns—yet the data engineer is expected to meet all interests.

This ambiguity of aim is akin to the challenge of planning a new city suburb to satisfy the needs of all residents. Again, this is a unique challenge for a data engineer.

3.4 The fluid nature of data engineering

The search for business insights is shaped by three intrinsic characteristics:

- the availability of easy substitutes
- the open-endedness of the search
- the ambiguity of aim

We summarise these as the **fluid nature** of data engineering.

Certainly, the data engineering team can shut its eyes to these problems and insist on hard-and-fast requirements from stakeholders. This approach is unlikely to lead to quality outcomes. Instead, the data engineer's approach must adapt to the fluid nature of business insights.

At the end of the day, technical approach alone is insufficient. Instead, the data engineering team has an unparalleled need to combine flexibility, endurance of vision, and the ability to negotiate.

Moreover, compared to other engineering contexts, the non-technical capabilities sit in a small team—sometimes within a sole data engineer. A data engineering team must confidently wield these non-technical capabilities if it is to achieve excellence in an organisation.

The search for business insights is both demanding in complexity and fluid in nature. This dual aspect makes data engineering one of the most engaging and rewarding disciplines.

Key ideas.

Data engineering is an activity in which business intent is applied to data to create first information, then insight.

The availability of easy substitutes, the open-endedness of the search, and the ambiguity of aim define the fluid nature of data engineering.

What is data?

Data is a fragment of reality.

4.1 The central premise

Insight is information analysed in the light of intent. However, the data engineer starts one step further back than information. Data engineering starts with data.

Data is what is given before interpretation. It has no fixed meaning by itself, and is not yet information. More precisely:

Data is a fragment of reality captured by process.

Consider a customer whose seat is cancelled because a flight was overbooked. The experience is real, but it is internal to the customer. The decision-maker does not know about it unless the experience is captured as data and made available to someone who was not there.

Without data, a decision-maker attempting to understand overbooked flights must rely on guesswork, intuition, anecdote, or high-profile complaints. These can all be important, but none is systematic.

The same applies to shopping experiences, cargo delays, transactions, legal decisions, medical observations, and countless other events. Nothing becomes systematically available to an organisation unless it is recorded as data.

This definition means data is neither reality itself nor mere record-keeping. It sits between the world and the organisation's understanding of the world.

This becomes especially important at scale. In a small organisation, leaders may retain a direct sense of what is happening. In a large organisation, that direct visibility breaks down. Data becomes the means by which reality remains visible across distance, hierarchy, and complexity.

4.2 How is data captured?

Data is captured by process.

This creates two sources of imperfection. The first is the process. The second is the capture. Between both sits the digital system: the medium through which business activity is encoded into data.

A process may be imperfect because it was designed for operational work rather than analytical understanding. Most workflows exist to get something done. The digital systems that support those workflows are usually built around that operational goal. Therefore, the process may not try to record everything a future analyst may need for business intent. This is especially the case when the source of data is not controlled by the business seeking to analyse it.

The capture itself may also be imperfect. It can be technically difficult, costly, or impractical to record business events accurately. This challenge is intensified by a rapidly evolving technological landscape.

Data is therefore never a simple copy of the world. It is an imperfect projection of the real world onto the data world. The work of the data engineer begins by understanding these imperfections.

4.3 How is data used?

Data is captured by business processes. Once collected, it can be used to return understanding to the business.

In this perspective, data engineering is the task of taking data projected by business processes and reshaping it into a form required by business intent. This can be summarised in Figure 1.

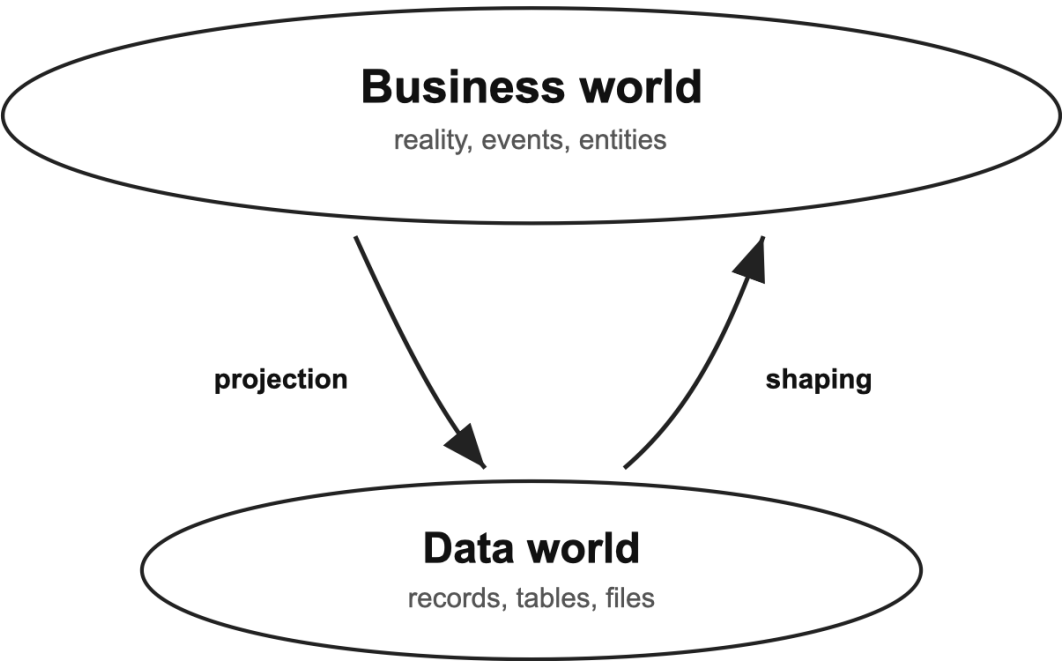


Figure 1. Business reality is projected into the data world, then shaped back into business understanding.

We will return frequently to the concepts illustrated in Figure 1 throughout this book.

Key ideas.

Data is a fragment of reality captured by process. At scale, data is the means by which reality remains visible.

The data world can be seen as an imperfect projection of the business world.

Data engineering reshapes the data world into forms the business world can use.

Data and organisations

Data helps organisations see themselves.

5.1 Data and organisational self-understanding

Data is a fragment of reality. This includes the reality of the environment in which they operate, but also the reality of the organisation itself.

This has implications for how data engineering work is done in practice. The veteran data engineer instinctively works within the entanglement between data and organisation.

To understand this, it is useful to visualise an organisation as a feedback loop.

5.2 The organisation agency cycle

The central premise "Data is a fragment of reality captured by process" can be interpreted as a feedback loop around three components: business processes, digital systems, and captured data. This relationship is illustrated in Figure 1.

Business intent

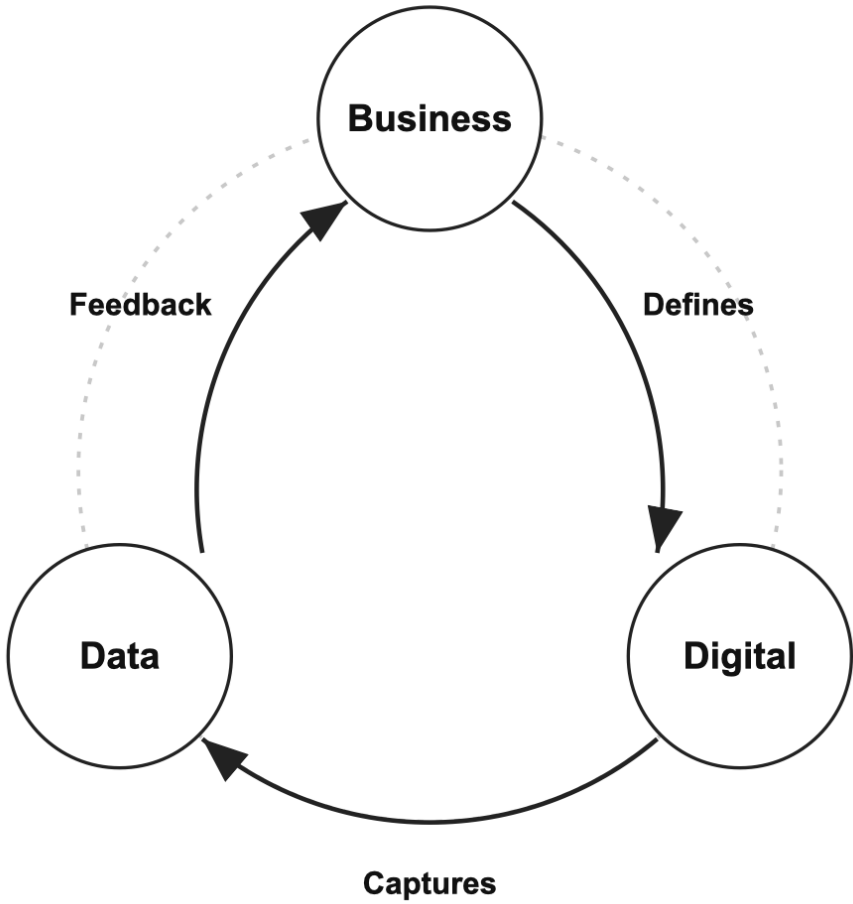


Figure 1. Business intent defines business process, business process defines digital systems, digital systems capture data, and data feeds back into business understanding.

Like many cycles, an organisation's feedback loop can become vicious or virtuous. Business intent is the determinant. An organisation moving toward its intent is in a virtuous cycle; an organisation drifting away from its intent is in a vicious cycle.

A key point of the cycle is that data is not passive “facts and figures” from the world. Data consists of records of the world the organisation is interacting with, and records of that interaction. The feedback loop is therefore a model of how an organisation exercises its goals in the world.

The implication is that what the organisation does is shaped by what it sees, and what it sees is shaped by what it does. Data is the medium of this double movement.

Consequently, data is central to an organisation's self-understanding. Data engineering is not one capability among others. It is the capability through which other capabilities become visible.

5.3 Implications for data engineering

There are two important implications for data engineering work.

First, recorded data is not simply “what is in the system.” It is downstream of business processes and digital capture. The data engineer is always working inside this context, and this context shapes the scope and options for action within a project.

Second, the data engineer’s work itself is not neutral. The output of data engineering is often the first time an organisation encounters the outcome of its own work in a stable form. For this reason, the output may be contested, especially when multiple business areas are involved. Interpretation becomes inseparable from the work: what is good, what is bad, what is reportable, what is not, which record is authoritative, and which definition should be canonical?

Organisations do not simply use data. Data is also where meaning is negotiated.

The data engineer cannot stand outside this negotiation. The engineer can help the organisation navigate meaning with expertise, defer to consensus without judgement, or allow the work to fail because meaning was never clear.

The agency cycle therefore shows why no part of data work is neutral. The business process that defines the digital system is not neutral. The digital capture is not neutral. Most importantly, the data engineer’s work to create feedback is not neutral. All are evaluated by intent. This is why data engineering reshapes data in light of intent.

Key ideas.

What the organisation does is shaped by what it sees; what it sees is shaped by what it does. Data is the medium of this loop.

Organisations do not simply use data; data is also where meaning is negotiated.

Creating information

Expressiveness and fragment modelling

Data engineering begins by shaping data into expressive, reusable fragments.

6.1 From data to reusable information

Data engineering reshapes [fragments of reality](#) into forms the business can use.

This work must be done efficiently, reproducibly, and over sustained periods of time. The goal is not merely to produce isolated reports or one-off datasets, but to create a cost-effective and robust pipeline of valuable, reusable blocks of information that are conducive to business insight.

How does the data engineer do this effectively?

This book introduces six principles of data engineering. The first two are **expressiveness** and **fragment modelling**. The two principles are:

Instead of accepting garbage-in, garbage-out with raw data, add value through expressive entities.

Instead of building giant tables, create meaningful fragments.

6.2 Expressiveness

Insight is [information analysed in the light of intent](#). A data engineer who takes this seriously becomes invested in the business world, asks how data relates to it, and is driven by the need to organise data in a way that makes sense for business decisions.

This is the task of applying business intent to reshape data. When done successfully, the output is expressive of business intent.

The first and most important principle of data engineering is therefore the search for **expressiveness**.

The data engineer's work is expressive when its outputs do not merely reflect data as it was found, but correspond strongly to real business entities and processes. It is expressive because the consumer or reviewer of the model can clearly recognise the world the model is attempting to approximate.

If a reasonably competent layperson cannot easily relate the model to the real world, then the model is not expressive. Correspondence to the world by the organisation trying to influence it is therefore the final arbitrator of whether a data model is successful.

Expressiveness has three qualities.

It is **faithful**: it preserves meaningful correspondence with the business reality it represents.

It is **legible**: a competent business user can recognise what the model is attempting to show.

It is **operable**: the model can be used in workflows or decisions with minimal rework.

Expressiveness encompasses all aspects of a good data engineer's work. It lies in everything: the way the engineer designs tables, chooses names for tables and columns, connects entities through primary and foreign keys, and describes the intent of code to others.

6.3 Fragment modelling

The [fluid nature of business insight](#) means that the data engineer cannot know in advance exactly what needs to be built.

At a micro level, this applies to a single data engineering project. Requirements shift as information becomes visible and stakeholders encounter their own business through data. At a macro level, discovering business insight can be a journey of years over multiple teams. In a large organisation, different teams will also want to see the same information in different ways to reflect specific business interests.

This is why the goal of a data engineer is not only to build complete data products, but to produce reusable blocks of information.

When the data engineer shifts focus from building complete products to creating blocks that can be reused for multiple products, the warehouse is no longer seen merely as a repository of data models. It becomes a fragment store of valuable information.

A data engineering approach focused on creating such a store is **fragment modelling**.

Modularity of code is a cornerstone in software engineering. Fragment modelling applies this concept to the modularity of information creation in a data pipeline. The result, as in software engineering, is clarity, flexibility, and reuse.

The data engineer therefore sees the pipeline primarily as software code, and tables as functions.

A good table in a pipeline performs a defined transformation—it accepts upstream data, applies a specific piece of business meaning, and produces an output that can be reused downstream.

In this sense, much of the wisdom in the Zen of Python carries to fragment modelling.

Fragment modelling is concerned not only with the informational value of a fragment, but also with its intended use. The data engineer does not rest until the fragment becomes plug-and-play for the end user who can make use of the information with minimal additional engineering.

In most cases, this means designing fragments that integrate cleanly into traditional dimensional models for self-service business intelligence. However, it also means ensuring that fragments can be adapted for other analytical scenarios, such as data science feature engineering.

This combined focus on both information and usability makes fragment modelling especially suited to the demands of the data age, where data analytics is employed in a growing range of contexts.

Fragment modelling is also an important step towards expressiveness.

A wide table with a large number of miscellaneous columns buries multiple business concepts into one undifferentiated "thing" that does not recognisably map to any real-world entity. On the other hand, tightly defined fragments bring clarity to business attributes by giving them sharp relief.

It is impossible to achieve a truly expressive data model without deconstructing source data into meaningful fragments and reorganising them to better reflect real-world entities.

This chapter introduces fragment modelling as a principle. The later chapter [Meaningful fragments](#) explores the practice in detail.

Fragment modelling can be disorienting for an engineer used to traditional warehousing approaches. Once familiar, it becomes a powerful approach to creating sustainable and flexible pipelines. The following chapters explain how this is done.

Key ideas.

The first two principles of data engineering are **expressive entities** and **meaningful fragments**.

Expressiveness is the principle that data models should recognisably correspond to the business world they represent.

Expressive data is faithful, legible, and operable.

Meaningful fragment is the principle of creating reusable blocks of information rather than giant tables.

In a well-designed pipeline, tables can be understood as functions that transform fragments of data into reusable information.

Mapping the data world

The data engineer's first task is to relate records back to the business entities behind them.

7.1 From records to entities

Business is not interested in data for data's sake. It is interested in the business reality behind the data.

From this perspective, data is not an end in itself. Its function is to bridge the business processes that collect the data and the business intent that seeks insight.

Consequently, the data engineer sees each data record as an intermediary of the business entity that generated it.

A business entity can be a concrete object, such as a customer or a product. It can also be an abstract event, such as a business transaction.

This shift from data record to business entity is the first practical step in expressiveness. For many data engineers, it is also the first threshold: learning to see not the record itself, but the entity behind it.

For a data engineer who starts with the data and reaches back to the business world, the tool of choice is the primary key.

7.2 Primary key

Primary keys play a special role in establishing the correspondence between business entities in the real world and data records in the data world.

A primary key is the handle by which business processes use to accurately retrieve a business entity's data record from a data store.

For example, an order number identifies a sales transaction, and a customer number identifies a client record. These keys are visible to end users who use them to relate back to business processes, such as calling a company to follow up on an order.

Systems may also employ internal keys, invisible to users, to support subsidiary information retrieval. In every case, the function remains the same: reliable lookup of records.

The quality of a primary key is underpinned by its business process. This process may be:

- **Rigorous:** A strict identity-proofing protocol, such as issuing social security numbers, ensures unique assignment of a key to a personal identity. Mistakes can occur. However, the overall system remains robust across a large population.
- **Fuzzy:** Smaller organisations may register customers by phone number or email. When the database is small, this may return accurate results. However, increased numbers of customers may lead to duplicate records or accidental access if contact details overlap.
- **Log-like:** Modern online shops often implement guest checkout to cater to buyer preference. Guest checkout creates a new record for every transaction. In this case, each row is effectively logged as a separate event.

The final example is a case of having no true primary key.

This absence may be by design, as in the case of guest checkout. The data engineer can treat each row as its own entity and assign a random, unique number for a primary key. If the business needs to know the underlying business entity driving each record, such as the identity of the guest shopper, the data engineer or data scientist may apply entity resolution techniques to deduplicate the data.

The absence of a primary key can also be accidental. For example, staff may record information each day on a spreadsheet without a clear key for each row, leaving the data engineer to infer how those rows relate to business entities.

This way of looking at the primary key departs from treating it only as a technical constraint, or even as a pure link between a record and a real-world entity.

The emphasis on lookup is significant. The ability of a primary key to serve as a reliable link to the real world is directly proportional to the quality of the lookup process that supports it. It helps the data engineer see not only that primary keys can fail, but where the failure is likely to sit within the business process.

Implementing and maintaining a strong primary-key process carries cost. Quite often, this cost becomes a barrier to maintaining accurate correspondence between data records and business entities. The result is duplication or loss of information:

- When record creation is easier than updates, duplicate entries proliferate.
- When creation or updates become burdensome, entities may go unrecorded, overwritten, or repurposed.

Any primary-key issue inevitably traces back to the underlying business process.

If there are issues with the primary key, the data engineer must do more than accept the issue at face value. The engineer must work to find the root cause in the business process.

Given its foundational role, the primary key is the natural starting point for a data engineer examining a new system.

Often the database does not come with keys implemented as constraints, for example when uniqueness is enforced in the application layer. In these cases, a data engineer investigating a new system would do well to infer the primary keys for each table and then ask how those keys are created, retrieved, and deleted in business processes.

A direct corollary of the primary key's significance is that relational algebra becomes indispens-

able.

Experienced engineers think in keys. They rely on algebraic operations to track and communicate data rather than memorise every detail of source processes. Discussion of relational algebra lies beyond this text’s scope.

7.3 Immutable and mutable entities

It is helpful to divide business entities into two broad types: immutable and mutable.

Immutable entities are those that do not change. If they do change, the change is treated as the creation of a different entity, and the latest version is considered the correct one. In other words, neither the entity nor its attributes evolve over time.

Examples include a completed bank transfer or an importer lodging an import declaration into a country. If the import declaration is altered, it is regarded as a new entity. Products can also be treated as immutable. A new model of phone is considered a different product from the previous version.

Mutable entities are those that may change over time while still being considered the same entity.

A customer may change purchasing habits. An employee in an HR system may update personal details or gain new qualifications. These changes are important but, according to business intent, still part of the same entity.

This distinction is helpful because most business processes are built with one of these perspectives in mind.

A digital system may have sub-systems that manage both. For example, a sales system may record purchases as immutable entities, treat products themselves as immutable, but also register information about customers whose attributes and behaviour are mutable.

One way to distinguish mutable from immutable entities is the presence of a datetime component in the primary key to indicate the validity period of the record.

In warehousing terminology, such a record is a Type II record. The datetime signals that it is important to track changes to the entity’s attributes over time. This tracking may be implemented in the source system and maintained by business processes, or it may be absent and later added by a data engineer to meet business intent.

The two major types of business entity, immutable and mutable, correspond to two major types of data engineering methods: **entity processing** and **entity tracking**.

The distinction between immutable and mutable entities shapes the overall approach for the downstream pipeline.

	Immutable entities	Mutable entities
Basic question	What happened?	What changed?
Entity behaviour	Treated as fixed once created; changes are new versions	Changes over time while remaining the same entity

	Immutable entities	Mutable entities
Typical examples	Sales orders, bank transfers, import declarations	Customers, employees, suppliers, accounts
Data engineering approach	Entity processing	Entity tracking
Pipeline focus	Process the entity through business logic	Track attributes, behaviour, and state over time

These are the subjects of the next two chapters: [Entity processing](#) and [Entity tracking](#).

Key ideas.

A data record is an intermediary of the business entity behind it.

Primary keys establish correspondence between data records and business entities through reliable lookup.

The quality of a primary key depends on the quality of the business process that creates, retrieves, and maintains it.

Immutable entities lead to entity processing.

Mutable entities lead to entity tracking.

Entity processing

Entity processing applies to business processes built around immutable transactional entities.

8.1 Processing immutable entities

Entity processing is the main pattern for **immutable business entities**: entities that are treated as fixed once created.

Examples include a sales order, a bank transfer, or an import declaration. If such an entity changes, the change is treated as though it were a different entity. For immutable entities, the business objective is to process the entity according to defined business logic. Hence the term **entity processing**.

Immutable entities share common characteristics:

- **Versions**—each version can be considered a new entity, with the latest version replacing all others as the correct version.
- **Header and details structure**—the header represents the main entity, while the details represent its components. A sales order and its line items are a typical example.
- **Subprocesses**—multiple business processes are needed to complete the business activity on the entity.
- **Good or bad**—the entity is evaluated by outcome, such as profitable or unprofitable, compliant or non-compliant, fraudulent or legitimate.
- **Actors**—actors persist across multiple entities, such as a customer making multiple purchases over time.

These characteristics explain the goals of entity processing:

- Surface insight from the underlying business process.
- Identify macro-level trends, such as sales by region.
- Measure time between process milestones.
- Provide drill-through to specific events for troubleshooting and root-cause analysis.
- Provide a foundation for predictive analytics by engineering attributes that can be used to predict good or bad outcomes.
- Understand actors whose behaviour affects business outcomes.

The first task is to identify the **entity of interest**.

Usually this is clear. Sometimes it is not. A sales transaction may have multiple line items.

Depending on the business question, the transaction may be the entity of interest, or each line item may be the entity of interest.

The correct lens is the one that supports business action.

Once the entity of interest is identified, the data engineer tells the high-level story of that entity while preserving the necessary detail. [Fragment modelling](#) is suited to this task because different aspects of the entity can be maintained in different fragments.

8.2 Building the pipeline

A pipeline for entity processing follows three broad passes:

- **Filter**—remove irrelevant data and establish the minimum meaningful structure.
- **Map**—compute reusable blocks of business meaning.
- **Reduce**—aggregate to the grain required for analysis.

8.2.1 First pass—Filter

The first pass works predominantly with incoming raw data. Its aim is to establish structure and meaning, beginning with keys.

It has three stages: identifying keys, defining reference tables, and extracting transactions.

Identifying keys

The first task is to identify the business keys that serve as primary keys to the raw data. These may be defined as database constraints in the source data, or inferred through exploratory analysis.

Defining primary and foreign keys is a simple but powerful way to [bridge the data world and the business world](#).

In some cases, key columns need to be created. For immutable entities, the two common scenarios are versioning and sequence numbers.

In versioning, a typical application might use `[Sales ID]` and `[Previous sales ID]` to indicate that one record replaces another. A clearer approach is to introduce `[Original sales ID]` and `[Sales version number]`, using partitioning and ordering to establish lineage. The original `[Sales ID]` can still be retained for joins, but should be renamed `[Sales version ID]`.

In header-detail structures, detail rows are often stored as simple lists. For example, a `[Sales ID]` may be associated with multiple `[Sales item ID]` entries. Introducing `[Sales item sequence number]` can clarify the relationship between the order and its line items.

Keys do not need to be enforced as physical constraints. They can be stored in metadata tables. What matters is that they communicate meaning and can be looked up by consumers of the data.

Defining reference tables

Reference tables are small, slow-moving, descriptive tables that describe the content of larger, fast-moving transaction tables.

Examples include **RefProduct**, which describes the products being sold, and **RefLocation**, which describes the store where the sale occurred.

Reference tables are important for two reasons:

- **Expressiveness**—they provide a central place to describe the attributes of a business entity, rather than embedding all information in the transaction table.
- **Efficiency**—updates can be made to a small reference table rather than to large volumes of transaction data.

The data engineer can use source-system reference tables for inspiration, but should not be limited by them. Application reference tables are usually designed to support system functionality, not business insight.

The data engineer can make reference data more expressive by:

- Renaming incoming tables and columns to reflect business intent.
- Combining fragmented application reference tables when this clarifies the business concept.
- Creating new reference tables for analytical use.
- Adding default rows such as **Unknown product** where analytical use requires them.
- Using reference data to identify conformance across the warehouse.
- Adding metadata such as business definitions and descriptions.

Extracting transaction tables

Transaction tables are fast-moving tables that record business events. They are the primary source of informational value for the business.

When extracting transaction tables, use names that pair with the reference tables describing them. For example, **Sales** should be matched with **RefSales**, **Inspection** with **RefInspection**, and **AuditResult** with **RefAuditResult**.

Filter out noisy data, including unnecessary columns and, where justifiable and computationally simple, unnecessary rows.

Removing rows requires judgement. In large tables, reversing such a decision later can be costly for the pipeline. A prudent approach is to remove only rows that are clearly irrelevant, such as stray test records, and handle more ambiguous filtering scenarios in later passes through mapping or classification tables.

Filtering is not only for efficiency. It removes noise and improves clarity for both the data engineer and the business.

Transactional inconsistency is an important case. During batch extraction, inconsistency can arise within the batch. Depending on business use, early records may need to be filtered out until the next batch, provided the next extraction picks up the skipped records.

The transaction table should be as narrow as possible. Pass as much descriptive information as possible to reference tables.

Example extracted tables

SalesItem

Sales ID	Item ID	Quantity	Sale value
1001	A	2	40
1001	B	1	20
1002	A	3	60

ItemSupplierCost

Item ID	Supplier cost
A	12
B	15

When these steps are applied, the data engineer will have:

- Extracted the minimum information needed to remove noise and preserve clarity.
- Built a basic map of information through primary and foreign key relationships.
- Created expressive meaning through clear naming and reference tables.
- Applied change detection to incoming records to support downstream change tracking.
- Set up a foundation for future optimisation, including incremental extraction.

8.2.2 Second pass—Map

The first pass extracts raw data, clarifies basic meaning, and establishes the foundation for deeper analytical work.

The second pass builds on this foundation. It creates useful information—derived, calculated, and shaped for business insight.

This pass should avoid working directly from raw data. It should use the outputs from the first pass.

The mindset here is storytelling. The data engineer should not be limited by source-system concepts, or even by the business terms initially provided by stakeholders. The work requires appropriate creativity: reshaping language and logic so the data makes sense for business decision-making.

The guiding question is:

What valuable pieces of information would be useful for business insight?

Often, this information is latent in the data and must be explicitly calculated. For example,

the concept of a non-compliant transaction may not be stated in the source data. It must be inferred.

The result should be stored in standalone tables rather than added as extra columns to extracted tables. This follows fragment modelling and provides a clean container for complex logic.

As in the first pass, information should be offloaded to reference tables and transaction tables kept narrow. This may require new reference tables because the concept does not exist in the source data.

For example, if the first pass creates `Sales`, `SalesItem`, `ItemSupplierCost`, and `SalesItemRefund`, the second pass may create `SalesItemMargin` to compute profitability on each sales item.

The full set of tables would look like this.

First pass from the source data

- `Sales`—header for the sales event.
- `SalesItem`—items sold in the sale and their prices.
- `ItemSupplierCost`—cost to supply each item, stored as a Type II table because supply cost can change.
- `SalesItemRefund`—refunds recorded against sales items.

Second pass of derived computation

- `SalesItemMargin`—profitability of each sales item after accounting for sale value, refunds, and costs.

Example structure of `SalesItemMargin`

Sales ID	Item ID	Sale value	Supplier cost	Margin
1001	A	40	24	16
1001	B	20	15	5
1002	A	60	36	24

8.2.3 Third pass—Reduce

The third pass focuses on aggregation.

At this stage, the data engineer takes the detailed fragments from the first and second passes and rolls them up to the grain of the entity of interest. Source systems often contain fine-grained detail, while business attention is centred on the entity itself.

This stage is a major opportunity for the data engineer to add value. Aggregations can be technically complex, especially when they involve multiple sources or complex business rules. By building them into the pipeline, the data engineer delivers ready-made, trusted information blocks that can be used without repeated manual work.

Continuing the earlier example, suppose the first pass produced `Sales`, `SalesItem`, `ItemSupplierCost`, and `SalesItemRefund`, and the second pass created `SalesItemMargin`.

In the third pass, this information can be aggregated to the sales level to produce **SalesProfit**. This table combines the total margin from all items in the sale, applies any additional business rules, and arrives at a final profit figure for the sale. A reference table, **RefSalesProfit**, can include a binary flag indicating whether the sale was profitable.

This pass is also where the data engineer computes when an entity reaches specific milestones. A sales order may have milestones for placement, confirmation, shipment, and completion. These dates can be recorded in **SalesProcessMilestone**.

Example structure of SalesProfit

Sales ID	Total margin	Is profitable
1001	21	true
1002	24	true

Example structure of SalesProcessMilestone

Sales ID	Placed date	Confirmed date	Shipped date	Completed date	Is meeting SLA
1001	2023-01-03	2023-01-04	2023-01-06	2023-01-09	true
1002	2023-01-05	2023-01-06	2023-01-08	2023-01-15	false

The outputs of this pass are valuable building blocks.

In business intelligence scenarios, they provide ready-to-use measures for dashboards and reports. In data science scenarios, they are equally important: much of feature engineering involves aggregating detailed attributes back to the correct grain.

8.2.4 Summary of the three passes

When complete, the pipeline produces reusable information blocks: expressive of business intent, supported by metadata, and delivered through an efficient and robust process.

These blocks are ready for downstream use, including self-service dimensional models and feature engineering.

An overview of the three passes and examples is summarised in Figure 1.

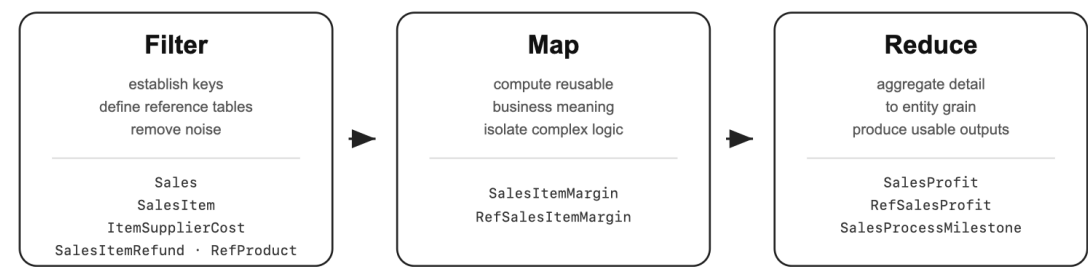


Figure 1. The three passes of entity processing, showing both the purpose of each pass and

example artefacts produced in a sales pipeline.

8.3 Common problems

New data engineers should watch for common mistakes in entity processing.

Starting with the details. Work should follow the shape of the business process. Start with the main entity, such as **Sales**, before moving to associated detail tables.

Neglecting reference tables. A common mistake is to store too much information in the transaction table instead of keeping it narrow and letting reference tables carry the descriptive load.

Accepting “garbage in, garbage out.” Source systems often have poor or cryptic names because presentation is handled in the user interface. Renaming and reshaping data to reflect business meaning is one of the most direct ways to make the model recognisable and useful.

Skipping aggregation. The third pass aggregates information to the entity level. New engineers often stop at the first or second pass without completing this step.

Compounding logic. Placing too much computation into a single table makes the pipeline difficult to understand and maintain. Fragment modelling suggests breaking these computations into standalone tables.

New engineers often mix the first and second passes: extracting raw data while performing complex transformation. This compounds computational complexity and weakens future options for incremental extraction.

The same temptation applies to all three passes. Convenience at the beginning creates fragility later.

Key ideas.

Entity processing applies to immutable business entities.

The first task is to identify the entity of interest.

The three passes are filter, map, and reduce.

The reduce pass returns detailed fragments to the grain required by business intent.

Entity tracking

Entity tracking applies when business entities change over time while remaining the same entity.

9.1 Tracking mutable entities

Entity tracking is the main pattern for **mutable business entities**: those that change over time while still being considered the same entity.

Examples include a customer whose contact details, preferences, or purchasing behaviour evolve; an employee whose qualifications, role, or performance record changes; or a supplier whose compliance status shifts over time.

Entity processing asks what happened to a fixed entity. Entity tracking asks how an entity changed while remaining itself.

Mutable entities share common characteristics:

- **Status**—Instead of "versions", the entity has statuses that change over time.
- **Changing attributes**—the entity has attributes whose changes define its mutation.
- **Actions**—the entity acts within business processes, and its performance or behaviour is of ongoing concern.
- **Audit**—the entity may be audited periodically or in response to events.
- **Registration**—the entity is usually underpinned by a registration process that establishes its initial presence in the system.

These characteristics explain the goals of entity tracking:

- Monitor macro-level trends in the number of entities over time, including changes in the composition of the population.
- Track management activities associated with these entities, such as registration and audit.
- Monitor when entities reach performance milestones defined by business thresholds.
- Detect anomalous entities for targeted action and early intervention.
- Provide drill-down information about the entity and the changes it has undergone.
- Understand the effect of entities on other business outcomes.

9.2 Two ways to track

There are two common ways to track a mutable entity. The first is by its attributes. The second is by its behaviour.

Tracking by attributes records what the entity is over time. An entity is registered into a database, and tables record attributes such as name, address, status, account type, account level, and so on. These tables are usually Type II tables, where each row is labelled with a validity period using columns such as `[Start date]` and `[End date]`.

Tracking by behaviour records what the entity does over time. Behaviour may be observed through audits, transactions, events, or actions taken in business processes.

Many systems have both. A bank, for example, may track accounts in two ways:

- **Attribute**—bank accounts have evolving attributes such as account holder, account type, and account level.
- **Behaviour**—deposits, withdrawals, transfers, and interest accruals are actions associated with the account.

The same three-pass structure applies to both modes, but with different emphasis. Attribute tracking introduces distinctive temporal machinery early. Behaviour tracking enters in the third pass when actions are reduced into entity-level signals over time.

9.3 Building the pipeline

Much of the approach from entity processing still applies. The data engineer must identify the entity of interest, craft [expressive reference tables](#), preserve necessary detail, and tell a coherent story.

The difference is that mutable entities introduce time into the entity itself. In entity processing, time usually appears around the entity: orders, confirmation, shipment, completion. In entity tracking, time defines the state of the entity.

9.3.1 First pass—Filter and compress

The first pass removes noise, establishes keys, and builds the basic semantic map.

For attribute tracking, this pass also introduces **temporal compression**. Temporal compression collapses consecutive rows that carry identical information across adjacent time periods. It is akin to run-length encoding.

Consider a bank account whose attributes are recorded over time.

Example before compression

Account ID	Start date	End date	Account type	Account level	Account status
12345	2022-01-01	2022-06-30	Savings	Silver	Active
12345	2022-07-01	2023-06-30	Savings	Silver	Active

Account ID	Start date	End date	Account type	Account level	Account status

After compression

Account ID	Start date	End date	Account type	Account level	Account status
12345	2022-01-01	2023-06-30	Savings	Silver	Active

In addition to compression, the first pass should assign a surrogate key to each validity period. For example, `[Account SK]` may represent the combination of `[Account ID]` and `[Start date]`.

The first pass should also normalise validity periods.

Open-ended histories should be given explicit boundaries. A missing start date can be replaced with a minimum sentinel date such as `1900-01-01`; a missing end date can be replaced with a maximum sentinel date such as `9999-12-31`. This gives every row a bounded interval and makes later timeline construction simpler.

The data engineer should also decide how to handle gaps. In some systems a record may be deleted and later re-created even though the business regards the entity as continuous. A row may need to be inserted to indicate non-existence, with `lag` functions to fill the dates.

For behaviour tracking, the first pass usually resembles entity processing. The data engineer filters and structures behaviour records, such as transactions, audit results, observations, or actions. The important temporal requirement is to preserve the event time of each behaviour.

9.3.2 Second pass—Build the timeline

An entity usually has attributes stored in multiple tables.

For immutable entities, retrieving attributes from multiple tables is usually a matter of joining on the appropriate keys. This is not the case for mutable entities. In a mutable entity, each attribute table may be Type II, meaning each row is defined by a validity period.

Joining attributes in this scenario means solving the overlapping-window problem.

The timeline table is the central artefact of attribute tracking.

A source system may track each changing attribute separately. Account type, account level, and account status may each have their own history table, with their own validity periods. The difficulty is that the business does not usually ask for these histories separately. It asks what the account was at a point in time.

The timeline table answers this question. It converts multiple overlapping histories into one coherent sequence of entity states.

An account may have attributes stored across multiple tables:

- `Bank.AccountTypeHistory`

- Bank.AccountLevelHistory
- Bank.AccountStatusHistory

The goal is to create `Bank.AccountTimeline`, which expresses the valid combination of attributes for each period.

Example structure of Bank.AccountTimeline

Account ID	Account type SK	Account level SK	Account status SK	Start date	End date
12345	1	1	1	2022-01-01	2023-06-30
12345	1	2	1	2023-07-01	2024-12-31

The core SQL pattern is an interval overlap join.

```
select
    th.[Account ID]
  , th.[Account type SK]
  , lh.[Account level SK]
  , sh.[Account status SK]
  , greatest(th.[Start date], lh.[Start date], sh.[Start date]) as [Start date]
  , least( th.[End date], lh.[End date], sh.[End date] ) as [End date]
from Bank.AccountTypeHistory th
inner join Bank.AccountLevelHistory lh on lh.[Account ID] = th.[Account ID]
                                     and greatest(th.[Start date], lh.[Start
    ↪ date]) <
    ↪ least(th.[End date], lh.[End date])
inner join Bank.AccountStatusHistory sh on sh.[Account ID] = th.[Account ID]
                                     and greatest( th.[Start date], lh.[Start
    ↪ date], sh.[Start date]) < least(th.[End
    ↪ date], lh.[End date], sh.[End date] );
```

The timeline table is produced through a temporal join. Notice that, without the time component, this is an ordinary join on the entity key. This is worth remembering. The timeline table is simply what the join would have looked like "at that point in time", but pre-computed for all time intervals in the most computationally compact form.

In this example, we use inner join for simplicity. Real systems also require careful handling of gaps in history.

Advanced note: two kinds of time.

Some tracking problems involve more than one temporal dimension.

For example, a user may report an account status for a business-effective period, then later change that reported period. The status has business dates describing when it was effective, but the system also has architectural dates describing when that report was recorded or changed.

This creates questions such as:

How many valid account holders were there at time X, according to what was known at time Z?

These problems can become extremely messy because the data engineer is no longer tracking only what was true in business time, but also what was known in system time.

We do not develop the full pattern here. The important point is that the timeline approach generalises. A timeline can be extended across an additional time dimension, giving the user a disciplined way to answer these questions while maintaining the clarity on the multiple time dimensions.

9.3.3 Third pass—Create snapshots and infer behaviour

The third pass produces entity-level outputs over time.

For attribute tracking, the main output is an end-of-period table: a snapshot of the entity at regular reporting intervals. This is created by joining the timeline to a calendar table and selecting the row that represents the entity’s state at the end of each period.

For example, joining `Bank.AccountTimeline` on the validity period to a distinct list of end-of-month dates returns `Bank.AccountEndOfMonth`.

Example structure of `Bank.AccountEndOfMonth`

Account ID	Period end date	Account type SK	Account level SK	Account status SK
12345	2023-06-30	1	1	1
12345	2023-07-31	1	2	1
12345	2023-08-31	1	2	1

This table is a building block. It gives downstream consumers regular access to the entity state without requiring them to solve overlapping validity windows.

For behaviour tracking, the third pass aggregates behaviour back to the entity over time.

Behaviour records are usually at the grain of the action. A bank transaction table, for example, may be at the grain of `[Transaction ID]`. The tracking question is usually at the grain of `[Account ID]` over time.

The data engineer therefore aggregates behaviour into signals about the entity: balances, counts, failures, first occurrences, repeated events, thresholds, or changes in pattern.

Following the previous example, `Bank.Transaction` holds records of an entity’s actions, the withdrawals and deposits of an account. It is at the grain of `[Transaction ID]`, which is much finer than `[Account ID]`. To track behaviour of the bank account, the data engineer can aggregate by `[Account ID]` to produce `Bank.AccountValue`, described by `Bank.RefAccountValue`. Example columns in `Bank.RefAccountValue` would be `[Is a millionaire]` or `[Is a new millionaire in the last 3 months]`.

Example structure of `Bank.AccountValue` with `Bank.RefAccountvalue`

Account ID	Period end date	Closing balance	Is millionaire	Is new millionaire in last 3 months
12345	2023-06-30	8700.00	false	false
12345	2023-07-31	9200.00	false	false
12345	2023-08-31	13700.00	false	false

With this table, a consumer should not need to recalculate balances or scan transaction history to find millionaire accounts. The pipeline should produce a fragment where that business state is already visible.

At an advanced level, behavioural fragments can themselves become Type II attributes. For example, once `Bank.AccountValue` identifies whether an account is a millionaire at each period, the data engineer can convert that repeated period-level result into a history of millionaire status over time.

9.3.4 Summary of the three passes

When complete, the pipeline produces reusable tracking blocks: expressive business entities, clear management of time, in computationally compact form.

These blocks are ready for downstream use where users can access information without struggling with error-prone process of writing time-based queries.

An overview of the three passes and examples is summarised in Figure 1.

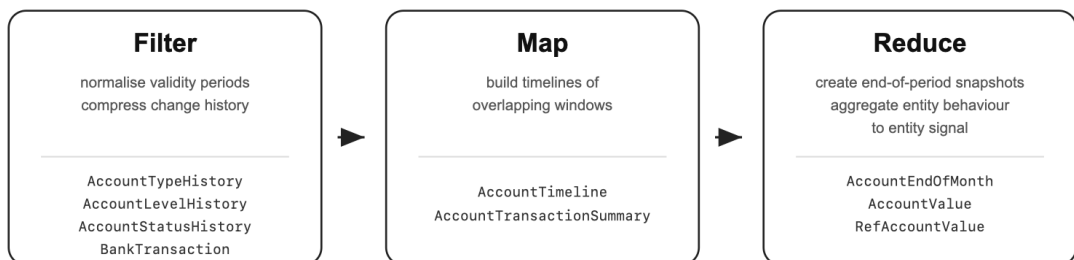


Figure 1. The three passes of entity tracking, showing both the purpose of each pass and example artefacts produced in a bank account pipeline.

9.4 Common problems

Compared with entity processing, entity tracking has additional difficulties.

Hidden tracking problems. It is not always obvious that the business is dealing with an entity tracking problem. Sometimes tracking is hidden within what appears to be a processing task.

Unregistered entity. A lack of entity registration undermines the clarity of identity. If the

entity is not clearly registered, changes cannot be reliably attributed to the same thing over time.

Undefined memory. Behaviour informs performance. Memory of behaviour informs the view of performance. But it may be unclear how much behaviour history is relevant for analysis.

Gaps in change history. Tracking mutable entities requires reliable records of historical change. This is difficult when the source system does not track changes natively.

Multiple timelines. Confusion arises when there is a lag between when a change occurs, when it is entered, and when it is recorded by the system.

There are three common types of datetime columns:

- **Architectural**—system-generated timestamps, such as those from database triggers.
- **Application**—timestamps defined by application logic, such as `[Update datetime]`.
- **Business**—timestamps tied to the actual business event, such as `[Sales order date]`.

The data engineer must engage early with stakeholders to understand the strengths and weaknesses of each type. Clear naming conventions and consistent usage are essential to avoid confusion and promote expressiveness.

Temporal representation introduces another important decision. Since attributes can change, the data engineer must decide how to represent the entity at a given point in time:

- **Type I**—show the latest available attribute value.
- **Type II**—show the attribute value as it was at a specific point in time.
- **Dynamic Type I**—show the latest value as defined by the user’s context, such as a time filter.

The choice depends on the analytical scenario. The same attribute may require different representations depending on the business question. Fragment modelling and timeline tables are well suited to supporting this flexibility.

Computational performance can also become a bottleneck, especially in end-of-period reporting. Many of these challenges require detailed handling rather than a ready-made approach, and they often require direct engagement with the business.

Key ideas.

Entity tracking applies to mutable business entities.

Entity processing asks what happened to a fixed entity. Entity tracking asks how an entity changed while remaining itself.

Mutable entities can be tracked through attributes, behaviour, or both.

Attribute tracking requires timelines and end-of-period snapshots.

Behaviour tracking aggregates actions into information about the entity over time.

The central difficulty of entity tracking is representing change without losing identity.

Reference data

Reference data gives business meaning a stable place to live.

10.1 A stable place for meaning

In the chapters [Entity processing](#) and [Entity tracking](#), reference tables appeared as a way to make business meaning expressive. They gave names, descriptions, and analytical attributes a stable place to live.

This chapter takes the next step. The goal is to introduce reference data into the warehouse and connect individual systems to it. Local systems may continue using their own terminology, codes, and classifications, while the warehouse provides shared points of reference for business meaning.

Two constructs are important for this purpose: **reference tables** and **mapping tables**. Reference tables define the meanings the business wants to use. Mapping tables connect local system values to those shared meanings.

This chapter focuses on applying these ideas within a single system. The next chapter extends the same pattern across [multiple systems](#).

10.2 A point of reference

A reference table that applies to multiple business processes becomes a shared point of reference.

The calendar is a simple example. Most business processes can map directly to a calendar table through a date column. A country table is another common example. Product categories, organisational structures, geographic hierarchies, and customer classifications often play the same role.

Such tables are commonly called **conformed reference tables** because multiple business processes conform to the same definition. This makes reference data one of the main ways a warehouse gives stable form to [negotiated organisational meaning](#).

The term **golden record** is also used, particularly when a single reference table represents the authoritative version of a concept.

In simple cases, systems can map directly to the shared reference. In more complex cases, each system requires its own mapping table.

The goal is not to force every source system to use identical codes. The goal is to provide a common reference that allows information from different sources to be understood consistently.

10.3 Building and applying shared references

It can be tempting to select a source-system table and declare it to be the golden reference, asking all other systems to conform to it.

This should be avoided.

The golden reference should be treated as physically and conceptually separate from any individual source system. This preserves flexibility and reduces coupling. If a source system changes, the warehouse reference remains stable.

A practical approach consists of three steps:

- Model the local system.
- Build shared references.
- Apply the references during presentation.

The following example uses a sales process from a fictional system called **Cake**.

10.3.1 Step 1—Model the local system

Begin by modelling the source system in isolation. The objective at this stage is not integration. The objective is to understand and represent the system faithfully.

The resulting tables may look like:

- `Cake.Sales`
- `Cake.RefSales`
- `Cake.RefCountry`

Example structure of `Cake.RefCountry`

Cake country code	Cake country name
AUS	Aus
NZ	Aotearoa
UK	Britain

The local model should stand on its own before any reference-data work begins.

This separation allows the engineering work to progress while mapping decisions are still being refined.

10.3.2 Step 2—Build shared references

Next, construct the warehouse reference table and the mapping table.

The warehouse reference captures the business concept independently of any individual source system.

Example structure of `Location.RefCountry`

Country ID	Country name	Region
1	Australia	Oceania
2	New Zealand	Oceania
3	United Kingdom	Europe

Example structure of `Cake.CountryMap`

Cake country code	Country ID
AUS	1
NZ	2
UK	3

The reference table can now hold information that may never have existed in the source system. Additional classifications, descriptions, metadata, and analytical attributes can be added without modifying operational systems.

If multiple systems require mapping to the same country list, the mappings may be stored separately:

- `Cake.CountryMap`
- `Finance.CountryMap`
- `Logistics.CountryMap`

An alternative is to combine them into a single table.

Example structure of a consolidated mapping table

System	Source country code	Country ID
Cake	AUS	1
Cake	NZ	2
Finance	AU	1
Finance	NZL	2

While convenient, this introduces a polymorphic key. The meaning of the source code depends on the value of another column. Such designs are practical but no longer correspond cleanly to traditional entity-relationship semantics.

10.3.3 Step 3—Apply during presentation

The final step is to apply the mapping during presentation.

This usually occurs through a dimensional model, reporting view, semantic model, or presentation layer.

The process is straightforward:

1. Retrieve the local system value.
2. Look up the mapping table.
3. Retrieve the shared reference.

The result is that reports and analytical models operate on shared business concepts rather than system-specific codes.

For example, a report consuming `Cake.Sales` no longer sees country code `AUS`. It sees the warehouse country reference and any associated classifications such as region.

Example application

Suppose `Cake.Sales` contains the local country code:

Sales ID	Country code	Sales value
1001	AUS	125.00
1002	UK	320.00

The presentation layer can translate the local code into the warehouse reference by joining through the mapping table:

```
select
    s.[Sales ID]
  , s.[Sales value]
  , rc.[Country name]
  , rc.[Region]
from      Cake.Sales s
left join Cake.CountryMap      cm on cm.[Country code] = s.[Country code]
left join Location.RefCountry rc on rc.[Country ID]    = cm.[Country ID];
```

The result is:

Sales ID	Sales value	Country name	Region
1001	125.00	Australia	Oceania
1002	320.00	United Kingdom	Europe

The report consumes the shared reference rather than the local system code. This allows multiple systems to present countries consistently even when their underlying codes differ.

In summary, the complete pattern introduces four tables, each table having a distinct responsi-

bility:

Table	Responsibility
<code>Cake.Sales</code>	Local business transaction
<code>Cake.RefCountry</code>	Local system country reference
<code>Cake.CountryMap</code>	Translation from local country code to shared country reference
<code>Location.RefCountry</code>	Shared warehouse country reference

This structure provides flexibility, stability, and reuse while keeping responsibilities clear.

10.4 Why these separations

The pattern depends on several separations:

- the local system is separated from the warehouse reference;
- the reference table is separated from the mapping table;
- the building of reference data is separated from its application.

These separations guard against a common mistake: introducing shared reference data too early and too deeply into the local system model.

The local system should first be modelled on its own terms. `Cake.Sales` and `Cake.RefCountry` describe how the Cake system records sales and countries. `Location.RefCountry` describes the warehouse's shared country reference. `Cake.CountryMap` connects the two.

Keeping these responsibilities separate has practical advantages.

First, the local system can be developed while reference mapping is still being reviewed. Mapping countries, product categories, and organisational structures often requires detailed business attention and cannot always be completed immediately. This matters in projects that require tight iteration loops.

Second, the design remains modular. Reference tables and mapping tables can be changed independently, which aligns with fragment modelling.

Third, reference data has an amplification effect. A change to a single reference row may affect large portions of the warehouse. If shared references are introduced too early into local transaction tables, changes to the warehouse reference can ripple through the local model. Delaying the dependency until presentation reduces unnecessary coupling.

10.5 When to simplify

Smaller systems may not require the full pattern.

One option is to skip `Cake.RefCountry` if the mapping to the golden reference can be created directly.

If a mapping can be expressed directly and maintained safely, `Cake.CountryMap` may be replaced by code. Where possible, this should be implemented through a temporary table or CTE rather than a large inline conditional, so that the pattern is still visible in code.

The full pattern exists because it scales well. Simpler systems can often adopt lighter variations without losing the underlying principle.

10.6 The discipline of separation

The pattern is simple, but easy to damage in practice.

Most mistakes come from collapsing distinctions that should remain separate: treating a source-system table as the warehouse reference, hard-coding mappings into reports, applying shared references too early, or merging mapping logic into the reference table itself.

The pattern in this chapter has been tested in complex settings. It is the path of least regret.

Above all, it is a discipline of separation, not a mechanical implementation recipe.

Key ideas.

Reference data gives business meaning a stable place to live.

Reference tables define shared business concepts.

Mapping tables connect local system values to those shared concepts.

Build the local system first, then construct shared references, then apply them during presentation.

Shared references allow a warehouse to remain coherent even when source systems use different codes and classifications.

Changes to shared references can amplify across the warehouse; delay the dependency until presentation when possible.

Conforming systems

Systems should be integrated only after the nature of their sameness is understood.

11.1 Integration without distortion

Integration is common, tempting, and dangerous because it invites the data engineer to connect tables before clarifying what kind of sameness is involved.

Business often needs to see multiple systems together. A legacy system may be replaced by a new platform, but the business still needs to analyse across both systems. Different business processes may also need to be compared through common references. A company may want to compare production and sales by region and month. A government agency may want to compare applications, inspections, and enforcement actions by location or program.

New engineers often fall into two traps: forcing a union of tables that do not naturally fit, or performing large joins that create ambiguous grain and duplicated data. Both approaches can produce outputs that run but no longer mean what they appear to mean.

Conforming systems is the discipline of integrating information without distorting the [business entities](#) being represented. Fragment modelling handles this by refusing to collapse meaning into convenience.

There are two approaches:

- **Vertical integration**—the same kind of entity is recorded across multiple systems.
- **Horizontal integration**—different entities can be compared through shared references.

11.2 Vertical integration

Vertical integration applies when the same kind of entity is recorded across multiple systems.

This commonly occurs when a legacy system is replaced by a new platform. The systems differ, but the business entity continues. The data engineer's task is to preserve that continuity without pretending the two systems are identical.

Take a cake company using two systems, **CakeV1** and **CakeV2**, to record sales. The newer system adds fields and improves logic, but both systems record sales.

Integration proceeds in three steps:

- Model the systems individually.
- Build conformed reference tables.
- Integrate the transaction tables.

11.2.1 Step 1—Model systems individually

A common mistake is to integrate too early.

Each system should first be modelled on its own terms. This preserves local meaning and makes the later integration safer.

Suppose both systems have sales and status, but **CakeV2** also has a marketing campaign.

The local tables might be:

- **CakeV1.Sales**
- **CakeV1.RefStatus**
- **CakeV2.Sales**
- **CakeV2.RefStatus**
- **CakeV2.RefCampaign**

At this stage, no union is required. The goal is to represent each incoming system clearly.

11.2.2 Step 2—Build conformed references

The second step is to build **reference data** that expresses shared business meaning across both systems.

For example, suppose the two systems record sales status differently.

CakeV1.RefStatus

V1 status code	V1 status name
O	Open
C	Completed
A	Abandoned

CakeV2.RefStatus

V2 status code	V2 status name
OP	Open
CO	Completed
WD	Withdrawn
RF	Refunded

A proposed conformed reference might be:

`Cake.RefStatus`

Status ID	Status name	Is finished sale
1	Open	false
2	Completed	true
3	Withdrawn	true
4	Refunded	true

The mapping table then connects each system-specific status to the conformed reference.

`Cake.StatusMap`

System	Source status code	Status ID
CakeV1	O	1
CakeV1	C	2
CakeV1	A	3
CakeV2	OP	1
CakeV2	CO	2
CakeV2	WD	3
CakeV2	RF	4

But this is only a hypothesis. It must be validated with the business. Even subtle differences in meaning can undermine the integration.

In addition to aligning codes, the golden reference tables should:

- Include default rows for unknown values
- Add analytical columns such as `[Is finished sale]` to support downstream use
- Be documented with metadata to support clarity and reuse

11.2.3 Step 3—Integrate the transactions

Only after the local systems have been modelled and the references conformed should the transaction tables be integrated.

The integrated table might be:

- `Cake.Sales`

This table is created as a union of `CakeV1.Sales` and `CakeV2.Sales`.

If a column exists only in one system, the other system should be populated with an appropriate default value. This includes default foreign keys to reference tables where required.

During the union, the mapping tables translate system-specific codes to conformed references. This allows `Cake.Sales` to use shared meanings rather than system-specific values.

Example SQL would be:

```
select
    v1.[Sales ID]
  , 'CakeV1'      as [Source system]
  , v1.[Sales date]
  , sm.[Status ID] as [Status ID]
  , -1            as [Campaign ID]
  , v1.[Sales value]
from      CakeV1.Sales v1
left join Cake.StatusMap sm on sm.[System] = 'CakeV1'
                        and sm.[Source status code] = v1.[Status code]

union all

select
    v2.[Sales ID]
  , 'CakeV2'      as [Source system]
  , v2.[Sales date]
  , sm.[Status ID] as [Status ID]
  , cm.[Campaign ID] as [Campaign ID]
  , v2.[Sales value]
from      CakeV2.Sales v2
left join Cake.StatusMap sm on sm.[System] = 'CakeV2'
                        and sm.[Source status code] = v2.[Status code]
left join Cake.CampaignMap cm on cm.[System] = 'CakeV2'
                        and cm.[Source campaign code] = v2.[Campaign code];
```

A sample result is:

Sales ID	Source system	Sales date	Status ID	Campaign ID	Sales value
10001	CakeV1	2025-06-01	1	-1	120.00
10002	CakeV1	2025-06-03	2	-1	340.00
10001	CakeV2	2025-06-02	1	5	180.00
10002	CakeV2	2025-06-05	4	7	95.00

CakeV1 and CakeV2 may use overlapping [Sales ID] values, the primary key for the union is [Sales ID], [Source system]. A new [Sales SK] surrogate key may be necessary if a single-column primary key is needed.

The union does not preserve the system-specific status codes. It replaces them with the conformed [Status ID]. CakeV1 has no campaign, so it receives the default campaign key. The resulting Cake.Sales table is therefore not merely a stack of two source tables. It is a conformed sales table.

The final structure includes:

- **Cake.Sales**—the unified transaction table
- **Cake.RefStatus**—the conformed status reference

- `Cake.RefCampaign`—the conformed campaign reference

The separation into three steps—modelling, reference building, and integration—allows each system to be developed independently, supports incremental delivery, and makes it easier to refactor or extend the model later. In practice, simplifications may be appropriate. The decision to simplify should be left to the data engineer, guided by expressiveness, fragment modelling, and the need to build a stable pipeline.

The overall workflow would look like:

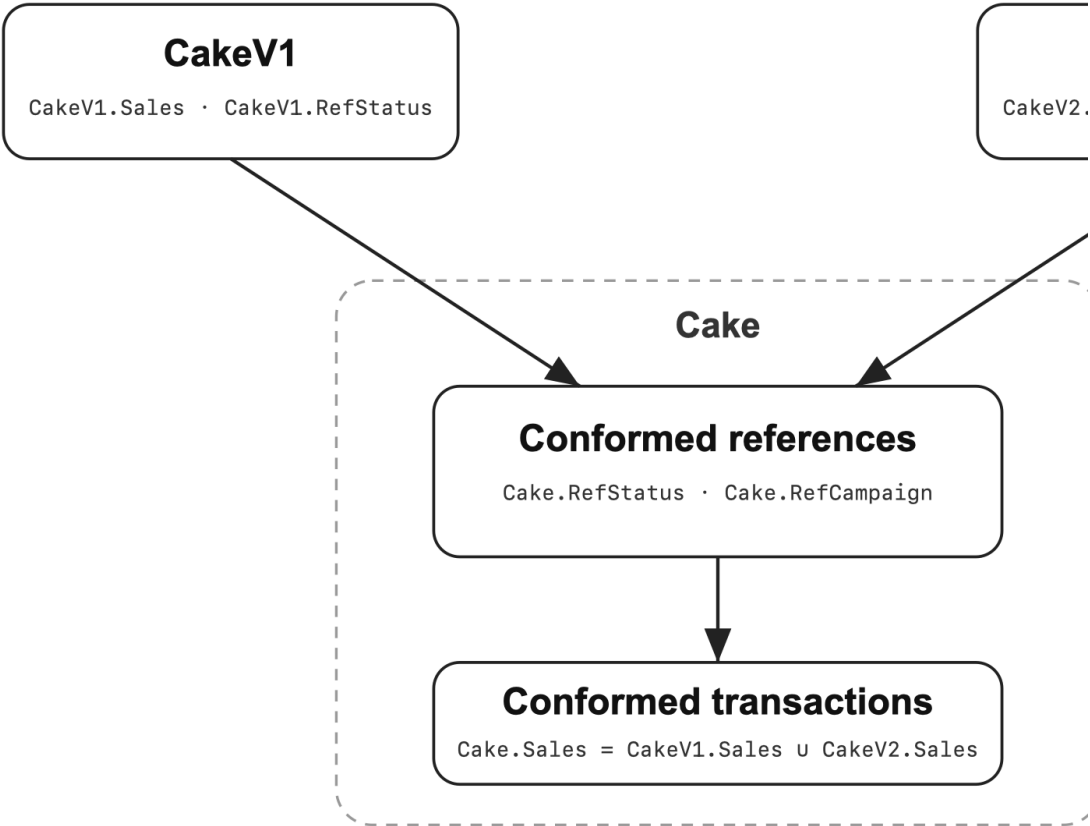


Figure 1. Vertical integration. Local systems are modelled separately, mapped to conformed references, and then integrated into a unified transaction table.

11.3 Horizontal integration

Horizontal integration applies when the entities differ conceptually but share enough commonality to make comparison valuable. This pattern is closely related to Kimball's notion of conformed dimensions and the bus matrix. Different business processes remain separate while becoming comparable through shared references.

A cake company may both produce and sell cakes. The core tables are:

- `Cake.Production`
- `Cake.Sales`

Production records manufacturing. Sales records transactions. However, they share common attributes such as region, time, cost, and staff, making comparison meaningful.

These should not be collapsed into a single abstract table such as `Cake.Event`. This creates a model that is difficult to understand and prone to error.

They can still be compared through shared references.

For example, both production and sales may use:

- `Cake.RefCalendar`
- `Cake.RefRegion`

This allows comparisons such as:

- production volume and sales volume by region and month
- production cost and sales value by region and month
- staff count and sales performance by site

An example SQL would be

```
with production as (
    select
        [Production date]
        , [Region ID]
        , sum([Production volume])      as [Production volume]
        , sum([Production cost])       as [Production cost]
        , count(distinct [Staff ID])   as [Production staff count]
    from Cake.Production
    group by
        [Production date]
        , [Region ID]
),
sales as (
    select
        [Sales date]
        , [Region ID]
        , sum([Sales volume])          as [Sales volume]
        , sum([Sales value])          as [Sales value]
        , count(distinct [Staff ID])  as [Sales staff count]
    from Cake.Sales
    group by
        [Sales date]
        , [Region ID]
)
select
    c.[Month start date]
    , r.[Region name]
```

```

, sum(p.[Production volume])      as [Production volume]
, sum(s.[Sales volume])           as [Sales volume]
, sum(p.[Production cost])        as [Production cost]
, sum(s.[Sales value])            as [Sales value]
, sum(p.[Production staff count]) as [Production staff count]
, sum(s.[Sales staff count])      as [Sales staff count]
from      Cake.RefCalendar c
cross join Cake.RefRegion  r
left join production      p on p.[Production date] = c.[Calendar date]
                        and p.[Region ID]         = r.[Region ID]
left join sales           s on s.[Sales date]      = c.[Calendar date]
                        and s.[Region ID]         = r.[Region ID]

group by
    c.[Month start date]
    , r.[Region name];

```

The result is:

Month start date	Region name	Production volume	Sales volume	Production cost	Sales value	Production staff count	Sales staff count
2025-01-01	North	12,500	11,900	82,000	145,000	18	7
2025-01-01	South	9,800	10,200	63,000	126,000	15	6
2025-02-01	North	13,200	12,600	86,500	152,000	19	8
2025-02-01	South	10,400	10,100	66,000	128,500	15	6

The comparison occurs through the shared references of calendar and region. Neither `Cake.Production` nor `Cake.Sales` has been altered or merged into a common transaction table. This is integration without collapse of meaning.

11.3.1 Horizontal integration through mapping fragments

Sometimes shared references are not enough.

The business may want to understand how specific production batches relate to specific sales. However, the source data may not record that relationship directly.

Suppose the business proposes a rule: Cakes produced in the same region and month are sold in that region and month, in production order.

This relationship is inferred. It is not native to either table.

New engineers may try to enforce the relationship by adding `[Production ID]` to `Cake.Sales`, or `[Sales ID]` to `Cake.Production`.

Both approaches damage the grain of the table.

A better approach is to create a mapping fragment.

`Cake.SalesProductionMap`

Sales ID	Production ID	Sales month	Production batch
S1001	P5501	June	June, Batch 1
S1002	P5501	June	June, Batch 1
S1003	P5502	June	June, Batch 2
S1004	P5502	June	June, Batch 2
S1005	P5601	July	July, Batch 1
S1006	P5601	July	July, Batch 1

Sales in the same region and month are allocated to production batches in production order. The relationship is stored as a fragment rather than forced into either `Cake.Sales` or `Cake.Production`.

`Cake.Sales` remains a sales table. `Cake.Production` remains a production table. The inferred relationship lives in its own fragment, where its logic can be tested, reviewed, and refined.

This is especially important when the relationship is fuzzy, inferred, many-to-many, or likely to change.

11.4 Choosing between vertical and horizontal integration

The distinction between vertical and horizontal integration is not always obvious. A useful starting point is to ask two questions.

First, would a union produce relatively few null columns?

If the two systems record largely the same attributes, a union is often a good sign. If most columns would be null for large portions of the data, the systems may not represent the same business entity.

Second, can the resulting union be given a meaningful name?

For example, it is natural to combine `CakeV1.Sales` and `CakeV2.Sales` into `Cake.Sales`. Likewise, `LegacyCustomer` and `ModernCustomer` may become `Customer`.

In contrast, combining `Production` and `Sales` into a table called `Event` is technically possible but conceptually weak. The name exists to support the union rather than describe a genuine business entity.

When both questions can be answered positively, vertical integration is often the stronger option. It preserves continuity across systems and usually produces a simpler model for downstream use.

When the answer to either question is no, then vertical integration is likely to distort meaning. Horizontal integration would be more appropriate.

Key ideas.

Integration should begin by asking what kind of sameness is involved.

Vertical integration applies when the same kind of entity is recorded across multiple systems.

Horizontal integration applies when different entities can be compared through shared references.

Mapping fragments preserve grain when relationships between entities are inferred, fuzzy, or many-to-many.

A quick test for vertical integration: would the union produce few null columns, and can the resulting table be given a meaningful business name?

The central danger of integration is collapsing meaning for technical convenience.

Storytelling

Data engineering is not finished when the data is correct. It is finished when the business can see.

12.1 Meaning beyond correctness

Source systems record operational events. They do not necessarily expose the outcomes, patterns, and journeys that matter for decision making.

A production system records inspections. A help desk system records escalations. A sales system records transactions. None automatically tell the business whether things are going well or how entities move through business processes.

Data engineering is not finished when the data is correct. It is finished when the business can see.

Storytelling is the process of transforming operational detail into forms that allow the business to recognise outcomes, patterns, and journeys.

This chapter explores three approaches:

- **Good and bad entities**—making business judgement explicit.
- **Trading detail for insight**—reducing noise into categories, combinations, and special cases.
- **Storytelling dimensions**—combining these interpretations into reusable reference data.

12.2 Good and bad entities

Business often sees the world through the lens of “good” and “bad.” A trade makes a profit or a loss. A customer is retained or lost. A production run succeeds or fails.

However, operational systems are often designed to support workflow rather than business judgement. The outcome that matters to the business may not exist explicitly in the source system.

One of the most valuable contributions of a data engineer is therefore to define outcomes that help the business recognise success and failure consistently. Sometimes the definition is obvious to the business user. Other times, it is latent in the way stakeholders talk about the data. The data engineer can proactively surface this latent definition by listening carefully during discussions and asking the right questions.

As a simplified example, a `Cake.Production` process may contain inspection results and operational notes, but not a clear concept of production failure. The data engineer can introduce that concept through a reference table and fragment.

Example structure of `Cake.RefProductionFailure`

Production failure ID	Production failure outcome	Failure reason
1	Production failure	Spoiled cake
2	Production failure	Undercooked cake
3	Production failure	Damaged packaging
4	Production failure	Missing allergen label
5	No production failure	No failure detected

Example structure of `Cake.ProductionFailure`

Production ID	Production failure ID
P1001	5
P1002	1
P1003	5
P1004	3
P1005	4

The source system may record production events and inspection notes, but the business needs to see whether a production run failed and why. The data engineer creates that interpretation as a reusable fragment.

The definition of good and bad can be a significant point of debate in a complex organisation with different business viewpoints. In these situations, it can be extremely challenging to arrive at a consensus. Many organisations fail to do so, crippling their ability to see issues consistently. In this case, it is often [the creativity and technical expertise of the data engineer](#) that can broker between parties by showing the way forward.

12.3 Trading detail for insight

Not every detail contributes equally to understanding. Often the fastest path to insight is to reduce detail rather than increase it.

A sales process may appear noisy when analysed by individual city but reveal a clear trend when grouped by region. A daily pattern may become obvious only after aggregating into weeks or sales seasons. In both cases, detail is traded for visibility.

During development, business stakeholders will often ask for the nth detail. This is understandable, but it can obscure an essential perspective. The problem becomes acute in fast-paced projects where teams tick off requirements and move on. The data engineer must proactively compensate for this tendency. Aggregation should be built into the work plan from the outset.

The data engineer has three common ways of trading details for insights:

- Creating categories
- Pivoting combinations
- Highlighting special cases

12.3.1 Creating categories

Creating categories is the simplest way to surface insight that is hidden by noise. For example, using age bands rather than individual ages, or grouping days into seasons, can reveal patterns that are otherwise obscured.

A special case of this is the use of binary columns. Suppose a help desk workflow has cases in `Helpdesk.Case` and another `Helpdesk.Escalation` table to record escalation events.

Example structure of `Helpdesk.RefEscalationTier`

Escalation tier	Is escalated
Tier 1	false
Tier 2	true
Tier 3	true
Tier 4	true

Creating a column such as [`Is escalated`], defined as false for `Tier 1` and true for `Tier 2` and above, can be immediately useful for business users seeking to understand escalation behaviour.

A further special case of binary columns is to link categories to good and bad. For example, an inspection is a common business process. The system records inspection results selected from a finite list.

Binary columns also play an important role in filtering and creating measures, as will be discussed in following chapters.

Example structure of `Cake.RefInspectionResult`

Inspection result	Is bad cake
Excellent cake	false
Good cake	false
Spoiled cake	true
Not tasty	true

The grouping looks straightforward, but it has a large influence downstream in designing reports, exposing filters, and designing measures.

12.3.2 Pivoting combinations

Another approach is to pivot options in the entity’s detail rows back to the entity grain.

Continuing with the help desk workflow example, escalation tiers can be pivoted into columns such as [Tier 1], [Tier 2], [Tier 3], and [Tier 4] to represent whether each tier was reached.

Source detail rows in Helpdesk.Escalation

Case ID	Escalation tier
H1001	Tier 1
H1001	Tier 2
H1001	Tier 3
H1002	Tier 1
H1003	Tier 4
H1004	Tier 1
H1004	Tier 2

Would be converted to a table of the entity grain.

Pivoted structure at the case grain

Case ID	Tier 1	Tier 2	Tier 3	Tier 4
H1001	true	true	true	false
H1002	true	false	false	false
H1003	false	false	false	true
H1004	true	true	false	false

The detail rows, including information such as the timing of the escalation, have been traded for a compact representation of the path. The model can now describe the case at the case grain without double-counting escalation events.

Once pivoted, the data engineer can summarise the escalation path in a variety of ways.

Example escalation path

Case ID	Tier 1	Tier 2	Tier 3	Tier 4	Escalation path
H1001	true	true	true	false	Progressively escalated
H1002	true	false	false	false	Not escalated
H1003	false	false	false	true	Direct escalation
H1004	true	true	false	false	Progressively escalated

The column [Escalation path] communicates an operational pattern that is difficult to see from detail rows alone. A business user does not need to inspect a deluge of tier values to understand whether the case stayed at the first tier, progressed through escalation, or jumped

directly to a higher tier.

12.3.3 Highlighting special cases

Sometimes a single result tells most of the story. For example, if there are multiple results, the most important one may be the worst result, first result, final result, or best result.

In the escalation example, the highest tier reached is often more important than every individual escalation event.

Extended structure

Case ID	Tier 1	Tier 2	Tier 3	Tier 4	Highest escalation
H1001	true	true	true	false	Tier 3
H1002	true	false	false	false	Tier 1
H1003	false	false	false	true	Tier 4
H1004	true	true	false	false	Tier 2

The [Highest escalation] column highlights the most significant outcome of the escalation journey. Users do not need to inspect every event individually because the most important result has already been surfaced.

A common scenario is the header-detail structure where the header is the entity and the detail is the result.

For example, in `Cake.InspectionResult`, a single cake may have multiple inspection criteria and therefore multiple inspection results. A simple yet powerful way of storytelling is to compute the worst of the inspection result.

Source detail rows in `Cake.InspectionResult`

Cake ID	Inspection criterion	Inspection result
C1001	Appearance	Excellent cake
C1001	Taste	Good cake
C1001	Packaging	Good cake
C1002	Appearance	Excellent cake
C1002	Taste	Not tasty
C1002	Packaging	Good cake
C1003	Appearance	Good cake
C1003	Taste	Good cake
C1003	Packaging	Spoiled cake

Suppose the reference table includes a binary classification:

`Cake.RefInspectionResult`

Inspection result	Is bad cake
Excellent cake	false
Good cake	false
Not tasty	true
Spoiled cake	true

The worst inspection result for each cake can then be calculated as the maximum of [Is bad cake].

Result at the cake grain

Cake ID	Is bad cake
C1001	false
C1002	true
C1003	true

The business no longer needs to inspect every individual inspection result. The most significant outcome has been surfaced at the grain of the cake itself.

12.4 Storytelling dimensions

A storytelling dimension brings multiple acts of interpretation together into a single reusable business view.

Categories, combinations, special cases, and journeys can each be useful individually. A storytelling dimension combines them into a coherent way of seeing the entity. The aim is to tell the overall story of the entity. This means stepping back from what is written in the database and asking what kind of journey the entity undertook through business processes. The emphasis is on the business view of the situation. The story is then expressed through [reference data](#) rather than in the transaction table itself.

Continuing the help desk example:

Example structure of Helpdesk.RefCaseEscalation

Case escalation						Highest escalation	Escalation path	Escalation summary	Display order
ID	Tier 1	Tier 2	Tier 3	Tier 4	Is escalated				
1	true	false	false	false	false	Tier 1	Not escalated	Closed without escalation	1
2	true	true	false	false	true	Tier 2	Progression escalation	Closed after escalation	2

Case escalation						Is escalated	Highest escalation	Escalation path	Escalation summary	Display order
3	3	true	true	true	false	true	Tier 3	Progressive escalation	Closed after escalation	2
	4	true	true	true	true	true	Tier 4	Progressive escalation	Closed after escalation	2
5	5	false	true	false	false	true	Tier 2	Direct escalation	Closed, started with escalation	3
	6	false	false	true	false	true	Tier 3	Direct escalation	Closed, started with escalation	3
7	7	false	false	false	true	true	Tier 4	Direct escalation	Closed, started with escalation	3
	8	true	false	true	false	true	Tier 3	Escalated with skipped tier	Closed after escalation	2
...	...	...	...	...	...	...	...	...	...	...
	16	false	false	false	false	false	Not applicable	Not applicable	Case still open	99

This single reference table contains several layers of interpretation:

- Tier combinations
- Special cases
- Journey descriptions
- Business summaries

Together they tell the story of the case.

Creating and applying a storytelling dimension follows three basic steps:

- Build the reference table
- Map to the facts

- Visual check

12.4.1 Step 1—Build the reference table

The storytelling dimension begins with a reference table that describes the journeys that matter to the business.

Returning to the help desk example, the starting point is the escalation pattern. The data engineer can represent whether each tier was reached using columns such as `[Tier 1]`, `[Tier 2]`, `[Tier 3]`, and `[Tier 4]`.

Those tier columns can then support `[Highest escalation]`, which identifies the most severe escalation reached by the case.

The same combination can also support `[Escalation path]`, with values such as:

- Not escalated
- Progressive escalation
- Direct escalation
- Escalated with skipped tiers

These values describe how the case moved through the escalation process, not merely where it ended.

The story can then be enriched with `[Escalation summary]`, which combines escalation path, highest escalation, and case status into a small set of business-facing categories.

Example summaries include:

- Closed without escalation
- Closed after escalation
- Closed, started with escalation
- Case still open

When creating a summary, resist the temptation to include too much detail. A summary is useful because it compresses many possibilities into a handful of categories the business can hold mentally. Adding detail creates more combinations and defeats the point of the summary.

These summaries often imply a ranking. A `[Display order]` column should be included to support this ranking in visualisations and filters.

The reference table should be heavily annotated with useful business descriptions. Each column should describe the business rule it represents in a way that is close to pseudo-code for business understanding.

Such tables are often composed from underlying binary flags. The exact SQL depends on the platform, but the pattern is usually:

- generate all combinations of the binary flags;
- assign each combination a stable key;
- compute mechanical columns such as `[Highest escalation]`;

- add business interpretation columns manually or through controlled rules.

```
select
    1
    + cast([Tier 1].value as int) * 1
    + cast([Tier 2].value as int) * 2
    + cast([Tier 3].value as int) * 4
    + cast([Tier 4].value as int) * 8           as [Case escalation ID]

    ,      cast([Tier 1].value as bit)           as [Tier 1]
    ,      cast([Tier 2].value as bit)           as [Tier 2]
    ,      cast([Tier 3].value as bit)           as [Tier 3]
    ,      cast([Tier 4].value as bit)           as [Tier 4]

    ,      ...                                   as [Is escalated]
    ,      ...                                   as [Highest escalation]
    ,      ...                                   as [Escalation path]
    ,      ...                                   as [Escalation summary]
    ,      ...                                   as [Display order]

from      string_split('0,1', ',') [Tier 1]
cross apply string_split('0,1', ',') [Tier 2]
cross apply string_split('0,1', ',') [Tier 3]
cross apply string_split('0,1', ',') [Tier 4];
```

In dimensional modelling, this type of reference table is sometimes called a junk dimension or transaction profile dimension.

12.4.2 Step 2—Map to facts

Once the storytelling dimension exists, the entity of interest can be mapped to it.

Example structure of Helpdesk.CaseEscalation

Case ID	Case escalation ID
H1001	3
H1002	1
H1003	7
H1004	2

The calculation should be done at the grain of the entity of interest during the reduce stage of the pipeline, as described in the chapter on [Entity Processing](#).

The calculation can be added to an existing aggregation table if appropriate or implemented as a new fragment. Continuing the help desk example, this would result in a `Helpdesk.CaseEscalation` table paired with `Helpdesk.RefCaseEscalation`. **Example result after joining to `Helpdesk.RefCase`**

Case ID	Is escalated	Highest escalation	Escalation path	Escalation summary
H1001	true	Tier 3	Progressive escalation	Closed after escalation
H1002	false	Tier 1	Not escalated	Closed without escalation
H1003	true	Tier 4	Direct escalation	Closed, started with escalation
H1004	true	Tier 2	Progressive escalation	Closed after escalation

A table like this tells the business far more about the case than a simple list of escalation events is able to.

12.4.3 Step 3—Visual check

Storytelling dimensions should always be reviewed visually.

A quick chart or summary table often reveals whether the intended story is actually visible.

For example:

Escalation summary	Cases
Closed without escalation	8,450
Closed after escalation	2,130
Closed, started with escalation	18
Case still open	742

The distribution may reveal unexpected patterns, data-quality issues, or opportunities to refine the storytelling dimension itself. A quick bar chart may reveal that there are almost no directly escalated records, and these cases occur only due to data entry errors. Alternatively, the chart may show a high number of directly escalated cases, suggesting a deeper operational issue or a more nuanced story worth exploring.

Visual checks are therefore not merely validation. They are part of the iterative design process.

12.4.4 Multiple storytelling dimensions

Complex business processes may require multiple storytelling dimensions.

A help desk workflow may need one dimension describing escalation behaviour and another describing service-level agreement outcomes. A production process may need one dimension describing inspection outcomes and another describing manufacturing quality. ## Storytelling and the organisation

Storytelling is important because correctness is not enough. A table can be accurate and still

fail to show the business what happened.

The data engineer's task is to create forms of information that make the business visible: its outcomes, patterns, and journeys.

This involves decisions and compromise: details to discard, or journeys to emphasise. Consequently, storytelling is a central place where the [organisation negotiates meaning](#), and where the data engineer plays an influential role in how organisation understands its own business.

Key ideas.

Data engineering is not finished when the data is correct. It is finished when the business can see.

Storytelling creates business meaning that is not explicitly present in the source system.

Defining outcomes is one of the most powerful ways to make business performance visible.

Trading detail for insight reveals patterns that are often hidden at the lowest level of granularity.

Storytelling dimensions combine categories, combinations, special cases, and journeys into reusable business interpretations.

Storytelling dimensions should be implemented as reference data and mapped back to the entity of interest.

Visual checks are an essential part of developing storytelling dimensions.

Meaningful fragments

Fragment modelling makes complex business meaning easier to build, test, reuse, and change.

13.1 Fragment modelling

The earlier chapters introduced [entity processing](#), [entity tracking](#), [reference data](#), [conforming systems](#), and [storytelling](#). Each depended on the same discipline: break complexity into individual fragments and connect them deliberately.

This approach is **fragment modelling**.

This chapter closes the section on creating information by making fragment modelling explicit. It steps back from those specific patterns and names the broader practice. It then introduces additional fragment patterns that commonly appear in real business pipelines: milestones, current-version selectors, timelines, mappings, distribution weights, and hubs.

The key idea is summarised by the [second principle of data engineering](#):

Instead of building giant tables, create meaningful fragments.

13.2 Why giant tables fail

New engineers often build giant tables to answer queries. This feels convenient because everything is in one place, and there are fewer artefacts to create and deploy.

However, the convenience is temporary.

Wide tables accumulate unrelated meanings. Business logic becomes buried and entangled. Change detection becomes harder because every column is treated as part of the same object. Being wide, they also become a magnet for dependencies for downstream use, creating additional risks.

The result is a pipeline that looks simple from far away and becomes fragile up close.

13.3 Meaningful fragments

Fragment modelling takes the opposite approach.

A **meaningful fragment** is a narrow table that packages one reusable piece of business meaning. Each fragment should answer a clear question:

What information does this table create?

The fragment then isolates the calculation of this information to its own table, and gives the table an expressive name.

By packaging up complex calculation into a meaningful fragment, it becomes operable.

Consistent use of meaningful fragments makes the pipeline easier to understand, test, change, and reuse.

13.3.1 Restaurant analogy

Fragment modelling is like *mise en place* in a restaurant kitchen.

The raw data are the whole potatoes and unwashed greens. Fragments are the prepared ingredients: chopped vegetables, cured sauces, and marinated proteins. They are not the final dish, but they make service possible.

A good pipeline works the same way. Once the fragments are well prepared, insights can be assembled quickly, consistently, and without repeating the same preparation work.

13.3.2 The impact on legibility

A further benefit of meaningful fragments is legibility.

Creating a fragment and giving it a name elevates a concept into a first-class object within the warehouse. Users become aware that the concept exists and can discover it through the schema itself.

For example, milestone datetimes could be implemented as additional columns within `Cake.Sales`. While technically correct, users browsing the warehouse may not realise that milestone information has been prepared and curated. By creating a dedicated `Cake.SalesMilestone` fragment, the concept becomes visible. The table name itself communicates that milestones are important and available for use.

In this sense, fragments do more than package information for easy operability. They help communicate the structure of the business domain. A well-designed warehouse is not merely a collection of tables, but a map of the concepts that the organisation has chosen to recognise.

13.4 Symptoms of poor fragmentation

When meaningful fragments are missing, the symptoms are usually visible:

- the same logic appears repeatedly in **where** clauses, joins, reports, or measures;

- tables become wider as constructed columns are bolted onto fact tables;
- reference data is duplicated into transaction tables;
- users need excessive joins, filters, and deduplication to answer basic questions;
- complex transformations are hidden inside one large table;
- small changes create large downstream effects.

These symptoms indicate that the pipeline has not been decomposed into its minimal informational components, and made easy for access.

13.5 Common fragment patterns

The following examples illustrate common fragment patterns. They are not exhaustive. Any self-contained piece of reusable business meaning can become a fragment.

13.5.1 Summaries of details and storytelling

Complex business processes generate detail rows that must be summarised to become useful. The computation for these summaries can be complex and should be placed in a separate fragment rather than compounded onto existing tables.

For example, a cake may undergo multiple inspections.

`Cake.InspectionResult`

Inspection ID	Cake ID	Inspection result
I1001	C1001	Excellent cake
I1002	C1001	Good cake
I1003	C1001	Good cake
I1004	C1002	Good cake
I1005	C1002	Spoiled cake

The business may only need to know whether the cake had a bad result.

`Cake.CakeStory`

Cake ID	Is bad cake
C1001	false
C1002	true

The fragment turns scattered detail into entity-level meaning.

This is a special case of the the storytelling dimension studied in the last chapter.

Once computed, the storytelling dimension and the fragment often becomes the primary window through which the business sees the data.

It may become the default filter in reports and queries because it lets users stay at the grain of the entity of interest. They do not need to go lower into subprocess tables, joining across grain, and risk double-counting or inconsistent logic.

Thus, the storytelling fragment absorbs that complexity once, so ordinary queries can remain simple. Across a warehouse with many users, the option to skip over the detailed grain can vastly reduce errors and improve maintenance.

13.5.2 Milestone datetimes

Business is always interested in measuring the time between milestones. This can be difficult if steps repeat or spread across multiple tables. The data engineer can add value by precomputing these milestones into a dedicated fragment.

A milestone datetimes fragment is simply a table at the grain of the entity of interest, with one datetime column per milestone.

A help desk may want to know when a case was received, escalated, and closed.

`Helpdesk.CaseEvent`

Case ID	Event	Datetime
H1001	Received	2025-01-10 09:05
H1001	Escalated	2025-01-11 11:20
H1001	Closed	2025-01-12 15:40

This can be simplified into a milestone fragment.

`Helpdesk.CaseMilestone`

Case ID	Received datetime	Escalated datetime	Closed datetime
H1001	2025-01-10 09:05	2025-01-11 11:20	2025-01-12 15:40

The fragment makes duration calculations straightforward and avoids repeated event-processing logic throughout the model.

Milestone tables are also known as accumulating snapshots in dimensional modelling.

13.5.3 Current or primary version

Many entities have multiple versions. The full version history may be useful for audit or lineage, but the business often needs the current or primary version.

In many systems, this logic is handled in the application tier and is not obvious in the database. A fragment makes this explicit.

`Sales.CustomerVersion`

Customer ID	Version	Customer name
C1001	1	John Smith
C1001	2	John A. Smith

A fragment can isolate the selection logic.

Sales.CurrentCustomerVersion

Customer ID	Current version
C1001	2

Users can then retrieve the current state without reconstructing the version-selection rule.

This is one case where fragment modelling can reduce immediate expressiveness if naming is poor. Most users do not expect to retrieve a row from one table and then consult another table to find whether it is current. The data engineer should compensate by naming the main table clearly, for example by using **version** in the table name or key columns.

13.5.4 Timeline and end-of-period

When entities change over time, users often need consistent point-in-time views. This can be error-prone for users when the attributes are spread across multiple tables, because joins require careful handling of time conditions. The data engineer can make these results easily accessible through corresponding fragments.

The relevant fragments—timelines and end-of-period tables— are discussed in [Entity tracking](#).

13.5.5 Mapping

Business processes often do not record relationships at the grain required for analysis.

Sometimes this granularity is impractical because processes do not operate in the neat way expected by the business.

Sometimes the recording is imperfect, such as through nearest temporal join. These relationships can be approximated using fuzzy logic.

These are often many-to-many relationships. Adding foreign key columns to the core tables would disturb their grain.

Rather than including complex code in core tables, or disturbing their grain, the mapping can be stored in separate fragments. These are two-column tables with foreign keys pointing to each primary table. This is the mapping table.

For example, the business may want to relate sales to production batches, even though the source system does not record the link directly.

Cake.SalesProductionMap

Sales ID	Production ID
S1001	P5501
S1002	P5501
S1003	P5502

Rather than disturbing the grain of either table, the relationship is represented in its own fragment.

13.5.6 Distribution weights

Sometimes a value is recorded only in aggregate form. However, the business wants to understand that value at the detail level and proposes a rule for distributing it. This logic can be complex and should be calculated in its own fragment. This is the distribution weights table.

Suppose a system records total effort for a work package.

`Cake.WorkPackageEffort`

Work package ID	Total work hours
WP1001	100
WP1002	80

The same work package contains multiple tasks, but the system does not record actual effort per task.

`Cake.Task`

Task ID	Work package ID	Task type	Worker	Task start datetime
T1001	WP1001	Design	Alex	2025-01-10 09:00
T1002	WP1001	Build	Priya	2025-01-10 13:00
T1003	WP1001	Test	Mei	2025-01-11 10:00
T2001	WP1002	Design	Alex	2025-01-12 09:00
T2002	WP1002	Build	Priya	2025-01-12 14:00

The business may define a rule for distributing the total effort across tasks. The result should be stored as a fragment.

`Cake.TaskWeight`

Task ID	Estimated work hours
T1001	50
T1002	30
T1003	20
T2001	35
T2002	45

Task ID	Estimated work hours

The fragment does not claim that the source system directly recorded these task-level hours. By placing the allocation in its own table, and naming the column appropriately, the data engineer keeps the assumption visible, testable, and replaceable.

In many cases, mapping tables and distribution weights may be combined if their ambiguities stem from the same business phenomenon.

A common variation occurs when the detailed records contain repeated item types, while the measure is recorded only at the header level.

Suppose a restaurant records the bill amount by table.

Restaurant.TableBill

Table ID	Bill month	Bill amount
T1001	2025-01	120.00
T1002	2025-01	80.00

The table also records drink orders.

Restaurant.DrinkOrder

Table ID	Drink item
T1001	Beer
T1001	Wine
T1001	Wine
T1001	Spirits
T1002	Wine
T1002	Wine

If the business assumes each drink order has equal value, the data engineer can create a distribution-weight fragment.

Restaurant.DrinkOrderWeight

Table ID	Drink item	Item order count	Total order count
T1001	Beer	1	4
T1001	Wine	2	4
T1001	Spirits	1	4
T1002	Wine	2	2

The final two columns can be used as a fractional weight. For example, T1001 has a bill amount of 120.00, and wine accounts for 2 / 4 drink orders, so the estimated wine value is 60.00. This

also simplifies whole-of-month measures at the item level. A monthly estimate of wine sales can be calculated without repeatedly scanning and counting the raw order rows.

13.5.7 Hubs

A hub creates a consistent surrogate key for the same entity appearing across multiple systems. This is a concept from the Data Vault method of data warehousing, but it is also useful as a fragment pattern. The hub centralises identity logic that should not belong to any one subsystem.

For example, multiple systems may record staff by first name and last name.

Payroll.Staff

Payroll staff ID	First name	Last name
P1001	Jane	Smith
P1002	John	Wong
P1003	Alex	Spelic

Training.Staff

Training staff ID	First name	Last name
T884	Jane	Smith
T921	John	Wong
T312	Mary	Zimm

A hub table assigns a consistent surrogate key to the shared identity.

Workforce.HubId

Staff SK	First name	Last name
1001	Jane	Smith
1002	John	Wong
1003	Alex	Spelic
1004	Mary	Zimm

This key can then be used for light anonymisation, relationship modelling, or downstream tools such as Power BI.

The hub belongs to no single subsystem. It is a central fragment where pairing and key assignment occur.

13.6 The discipline of meaningful fragments

Fragment modelling is a discipline of decomposition.

The data engineer looks at a data product and asks:

- What are the entities?
- What meanings are being created?
- Which computations are complex enough to isolate?
- Which relationships should not be forced into core tables?
- Which pieces of information need to be reused?
- Which dependencies should be delayed or reduced?

A meaningful fragment is justified when it creates business meaning, isolates complexity, reduces dependency, supports reuse, or makes change easier to manage.

In practice, anything can become a meaningful fragment. It does not need to fall under a predefined category. If the information is self-contained and its creation is justified by business meaning, complex computation, or dependency management, it belongs in its own place.

This is one of the hallmarks of mature data engineering: the ability to see a data product not as a pile of tables, but as a set of minimal informational components.

Key ideas.

The second principle of data engineering—Instead of building giant tables, create meaningful fragments.

A meaningful fragment is a narrow table with a specific informational purpose.

Fragments isolate complex computation, reduce dependencies, support reuse, and make change easier to manage.

Common fragment patterns include storytelling summaries, milestones, current-version selectors, timelines, mappings, distribution weights, and hubs.

Mature data engineering depends on seeing the data product as a set of minimal informational components.

Presenting insights

The craft of dimensional modelling

Information does not become insight until users can see it, touch it, and ask it questions.

14.1 From information to insight

Insight is information analysed in light of intent. The previous section, [Creating information](#), focused on building reusable blocks of information from data.

This section, **Presenting insights**, asks the next question.

How should that information be assembled into a product that business users can explore, understand, and trust?

Modern data engineers may present insights through a variety of ways. Yet one method has remained central: **dimensional modelling**.

In the Microsoft ecosystem, Power BI semantic models make dimensional modelling especially important because they provide the layer through which many business users interact with data.

A dimensional model in Power BI is therefore best understood as a product interface.

14.2 The craft

Building a data pipeline is largely an engineering exercise. It requires correctness, reproducibility, performance, and maintainability.

Building a dimensional model also requires those things. But the work becomes much easier when the upstream tables are ready for assembly: meaningful fragments become facts; expressive reference tables become dimensions; binary flags simplify measures. The technical work of dimensional modelling is greatly simplified when it is built on a strong foundation.

The central difficulty of dimensional modelling lies elsewhere.

When building a pipeline, the data engineer is mainly concerned with informational value: how data can be transformed into faithful, reusable, business-relevant information. When building a dimensional model, the data engineer must also anticipate how users will think, ask, filter, drill, and otherwise use the model.

The model must not only be correct. It must feel natural to use and intuitively aligned to business intent.

This is why dimensional modelling is a craft.

In practice, this craft appears in modelling decisions, many of which are not questions of correctness alone: which tables should be visible, which columns should be hidden, which measures should be defined, which relationships should be active, and how the model should guide the user's attention.

The hard question is therefore not only: "How do I build the dimensional model?"

It is also:

How should the dimensional model behave to anticipate user needs?

Three guiding questions organise the chapters in this section:

- How do we know what the user will do with the model?
- What does a good dimensional model look and feel like?
- What modelling tools are available to anticipate a range of user behaviours?

Anticipating questions

A strong dimensional model preserves the structure needed to answer questions before they are formally asked.

15.1 Designing for questions not yet asked

A good dimensional model does not merely answer the questions already written in requirements. It preserves enough business structure to answer all the questions users are likely to ask during the model's lifetime.

This expectation can surprise data engineers who come from an application-development background. In application development, the work often begins by defining requirements with stakeholders and building accordingly. Even when development is iterative, it is still usually organised around delivering stated requirements.

That approach has its place, but analytics behaves differently.

In data work, answering one question often creates the next question. Users discover what they want to ask by interacting with the data. They may begin with a simple requirement, such as “show quality-control failures by month”, but once the data is visible, they will ask follow-up questions: Which products failed? Which batches? Which suppliers? Which customers were affected? Did shipping outcomes change afterward?

A data project focused only on stated requirements is therefore likely to frustrate everyone. If stakeholders have little interest, the stated requirements may be shallow. If stakeholders are engaged, the requirements will keep changing as the data reveals new possibilities.

The data engineer cannot predict every future question. But good design can preserve the structure from which future questions can be answered.

This is the purpose of anticipating questions.

Power BI dimensional models are well suited to this task because facts represent business processes, dimensions represent business information, and relationships determine which questions can be answered through filtering.

Instead of ticking off stated requirements, the data engineer should identify the business processes, the information known to those processes, and the information shared across processes. These become the facts, dimensions, and relationships of the dimensional model.

To develop a model that anticipates questions and captures the breadth of information, the data

engineer can remember a simple three-step process:

1. All the facts—business processes
2. All the dimensions—business information
3. All the relationships—real-world relationships mirrored in the model

Each step has an artefact to help the data engineer and business stakeholders check the comprehensiveness of the model.

15.2 All the facts

The first step to building a dimensional model that can anticipate all reasonable questions is to ensure it captures all the relevant business processes. For ease of memory, this is the step to capture “all the facts.”

This step can be non-trivial for two reasons.

First, in a complex business, many teams work together to complete a single goal. These teams often operate across different business processes. The data engineer and project team are usually working with a particular stakeholder group that focuses on specific processes rather than the full set. For example, some may focus on manufacturing, some on ordering, and others on quality control.

Second, different stakeholders often take different views of the same process. Operational staff may focus on detailed steps, while others take a broader, strategic view. Speaking with any one group always results in a skewed perspective.

In other words, the data engineer faces two challenges—how much to include, and how zoomed out the view should be.

Consider the following example of business processes:

1. Research and development
2. Sourcing materials
3. Manufacture
4. Quality control
5. Orders
6. Shipping
7. Customer feedback

For the question “what are the different business processes to include?”, the data engineer should be as comprehensive as possible during horizon scanning. This means being persistent in asking “where does this come from?” and “where is this going to?” until the end-to-end is covered.

A useful rule of thumb is to trace the process beyond the immediate team: where did the work come from, and where does it go next? For end-to-end process discovery, it can help to ask where the work first enters the organisation and where its effect eventually leaves.

Once all the processes are identified, it becomes easier for stakeholders to collectively decide which ones to include. Usually, there is a shared sense of what is core and what is peripheral.

This is a different question from “what are your requirements?”

Requirements tend to narrow the perspective.

For example, a manager of quality control may focus narrowly on their part of the process. If the project is driven by this stakeholder—perhaps because quality control initiated the engagement—the requirements may be framed as “how much is spent on quality control?” or “how many products are failing?” Such a view is unhelpfully narrow.

Instead, the data engineer can help by identifying the full business process. The manager may then recognise that, even though reporting on the cost of manufacture is not a “must-have requirement” for quality control, it is still a core business process.

Research and development, by contrast, may be considered peripheral.

For the question “how zoomed out should the view be?”, the answer is to work at a level that makes sense for decision-making. It does not help to model the lowest-level details such as sending emails or looking up documents. On the other hand, it is also not helpful to see manufacturing and quality control as one large step.

Zooming out often involves grouping sub-processes into a single process, or denormalising detail rows into header-level structures that expose a broader grain.

For example, the quality control process may involve multiple testing steps for different criteria. Rather than creating one fact table for quality samples and another for quality sample tests, it may be more expressive to model the process as a single fact called 'Quality control', with the lower-level testing detail embedded where appropriate.

Power BI’s flexibility with its DAX engine supports this approach. In simple cases, a `distinctcount` can recover the embedded grain, such as counting headers by their ID. In more complex cases, DAX can still retrieve the embedded grain, as explained in [Designing measures](#).

This question often challenges engineers trained in traditional dimensional modelling who are learning Power BI for the first time. The classical approach tends to split facts by grain rather than by business process. This can obscure the business view.

The artefact for this step is the linear process diagram. A linear process diagram is a business process diagram using only linear steps with no branching, cycles, or decision trees. Readers find linear diagrams much easier to understand than diagrams with arrows pointing in all directions. The constraint of linearity forces the business analyst to abstract the process to a level that is useful for themselves and others. This is especially true for executive audiences.

It can also be helpful to categorise the business processes into higher-level categories.

Continuing with the example above:

- Design—Research and development
- Production—Sourcing, Manufacturing, Quality control
- Sales—Orders, Shipping, Customer feedback

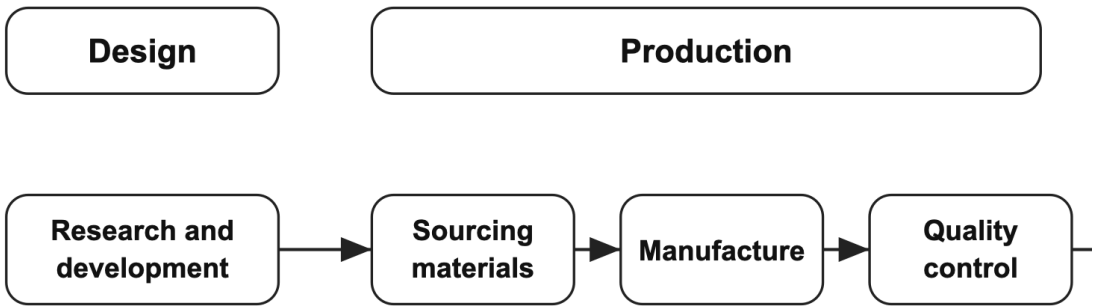


Figure 1. Linear process diagram. Business processes are arranged in chronological order and grouped into Design, Production, and Sales.

Making linear process diagrams does not mean that business analysts cannot create more complex diagrams for other purposes. However, these are not linear process diagrams for model planning.

The identified business processes form the facts of the dimensional model. Hence this step can be remembered as “all the facts.” The processes selected for the model define its scope. For example, if the decision is to focus on manufacturing onwards, then questions about research and development or sourcing will fall outside the model’s scope.

Identifying the business processes is the hardest part of anticipating questions. It involves making business decisions and requires a high degree of judgement and practical experience. Once the processes are identified, the next step of identifying the business information follows more mechanically.

15.3 All the dimensions

The second step, after identifying the business processes in scope, is to identify all the business information known to those processes. This ensures that all reasonable questions about the processes included in the model can be answered. For ease of memory, this is the step to capture “all the dimensions.”

The key distinction is between information a process captures and information a process knows.

This step is more straightforward than identifying the processes. Business stakeholders usually know what information is captured. While there may be debate about the relative importance of one attribute over another, the answer is often that they are all important. If they were not, they would not be captured in the first place.

The nuance for the data engineer is to distinguish between what is captured by a business process and what is known to it. The general rule is that downstream processes inherit information from upstream ones.

Consider the following example. Suppose each process captures the following information:

- Manufacturing—Product type, Manufacture details (date, batch number)
- Quality control—Testing details (manufacture sample, date, criteria, results)

- Orders—Product type, Customer, Order details (date, order units, sales amount, order number), Order status
- Shipping—Shipping order (order number, line items), Shipping logistics (shipping method, shipping company), Shipping status (date, status)
- Customer feedback—Customer, Product type, Feedback details (method, star rating, comments)

In this example, the order process captures product type, customer and order units.

The shipping process does not capture these directly but knows them through the order number. This is inheritance. However, not all downstream processes inherit everything.

For example, customer feedback may not record the order number and may only capture the product type. In that case, it does not know the order date, even though it is downstream.

Once the information is identified, it can be presented using a cumulative information diagram. This is a table with business information as rows and business processes as columns, both listed in chronological order. The diagram is cumulative because information typically accumulates downstream through inheritance.

Rows are business information. Columns are business processes. **C** means the process captures the information directly. **I** means the process inherits the information from an upstream process.

It is also helpful to annotate the diagram with where the information is physically stored—which database or table holds each attribute. This helps assess what can be reasonably answered and informs project planning.

The example below shows what information is known to each business process.

The linear process diagram describes the real-world business processes. The cumulative information diagram builds on the linear process diagram by showing how information flows through those processes and where it is stored in the database world.

Business information	Manufacture	Quality control	Orders	Shipping	Customer feedback
Product type	C	I	C	I	C
Manufacturing details	C	I			
Testing details		C			
Order status			C	I	
Order details			C	I	
Customer			C	I	C
Shipping order				C	
Shipping logistics				C	
Shipping status				C	
Feedback details					C
Database storage	XYZ SQL database	ABC diagnostics database	XYZ SAP ERP	XYZ SAP ERP	Salesforce analytics

Business information	Manufacture	Quality control	Orders	Shipping	Customer feedback
-------------------------	-------------	-----------------	--------	----------	----------------------

In other words, the cumulative information diagram maps real-world processes to database storage.

Since business information becomes dimensions in a dimensional model, this step can be remembered as “all the dimensions.”

15.4 All the relationships

The final step is to ensure that the business processes and their known information are reflected in the dimensional model through filtering relationships. If all relevant processes are captured, all relevant information is identified, and valid relationships are correctly implemented, then the model can anticipate all reasonable questions.

A relationship is not only a technical join. In a Power BI model, a relationship is a path by which a user’s question can reach a fact.

The rule of thumb is simple. If a business process knows a piece of information, whether by direct capture or by inheritance, then the corresponding dimension should filter that fact table.

Continuing with the earlier example, suppose `'Order ID'`, `'Order calendar'`, and `'Customer'` are dimensions, and `'Orders'` is the fact table. These dimensions should directly filter `'Orders'`. The `'Shipping'` fact table should also be filtered by `'Order ID'`, since shipping needs to know the order number. In this case, `'Order ID'` is a conformed ID dimension for both processes. The `'Order calendar'` and `'Customer'` dimensions should also filter `'Shipping'`, because this process inherits them through the order number.

New data engineers often forget about inheritance. This creates a frustrating experience for users who expect to retrieve all shipments for a customer but find the model does not support the question due to a missing relationship.

There are exceptions to inheritance. Sometimes an upstream process has a zero-to-many relationship with a downstream one. In these cases, including the upstream information as a filter may be problematic.

For example, suppose the quality control process usually tests manufactured batches, but sometimes tests other aspects not tied to a specific batch, such as testing factory equipment itself. In this case, `'Manufacture ID'` should filter `'Quality control'` so users can look up testing for a batch. However, it may be better not to implement a filtering relationship between `'Manufacture calendar'` and `'Quality control'`. In Power BI, if a user selects a manufacture date, this relationship will also filter `'Quality control'` and affect any measures, even when the test was unrelated to a batch.

It may be sufficient to use a role-playing `'Reporting calendar'` that filters `'Manufacture'` on manufacture date and `'Quality control'` on testing date. This avoids misleading results. If the relationship is implemented, the report design must be careful not to mislead users with incorrect numbers caused by unintended filters.

Usually, upstream processes tend not to know about downstream processes. This means that 'Customer' should not filter 'Manufacture' because this information does not exist at the time of manufacture. However, the data engineer has the benefit of hindsight, so some information can be brought back to the earlier process. For example, while the quality-control results of a manufactured batch are not known at the time of manufacture, the result can be associated to the batch number afterwards.

To express this at the batch-number grain, multiple quality control criteria may need to be rolled back to the batch for an overall pass or fail. A dimension for 'Batch quality outcome' can be used to filter 'Manufacture'. In general, storytelling dimensions summarise the journey of an entity through the whole business process and can be associated to all steps of the process in hindsight.

The artefact for this step is the chronological bus. A bus matrix is a table that lists dimensions as rows and facts as columns, showing which dimensions filter which facts. In Power BI, this should reflect the actual filtering relationships, including their cardinality. A chronological bus matrix is one where the facts and dimensions are sorted in chronological order of their occurrence in the business.

The chronological bus is the main planning artefact of the dimensional model. It shows, at a glance, which information can filter which business processes.

The diagram for our example is below. The symbol 1 -> * is used to indicate one-to-many, while * -> *, which is used for product type and customer feedback, indicates a many-to-many relationship. The latter means one customer-feedback record may relate to several products at once.

Dimension	Manufacture	Quality control	Orders	Shipping	Customer feedback
Reporting calendar	1 -> *	1 -> *	1 -> *	1 -> *	1 -> *
Manufacture ID	1 -> *	1 -> *	1 -> *	1 -> *	
Manufacture calendar	1 -> *		1 -> *	1 -> *	
Batch number	1 -> *	1 -> *	1 -> *	1 -> *	
Batch quality outcome	1 -> *	1 -> *	1 -> *	1 -> *	
Product type	1 -> *	1 -> *	1 -> *	1 -> *	* -> *
Testing ID		1 -> *			
Testing date		1 -> *			
Quality criteria		1 -> *			
Testing result		1 -> *			
Order ID			1 -> *	1 -> *	
Order calendar			1 -> *	1 -> *	
Customer			1 -> *	1 -> *	1 -> *
Order status			1 -> *	1 -> *	
Shipping calendar				1 -> *	

Dimension	Manufacture	Quality control	Orders	Shipping	Customer feedback
Shipping company				1 -> *	
Shipping status				1 -> *	
Arrival date				1 -> *	
Feedback calendar					1 -> *
Feedback sentiment (star ratings)					1 -> *

The chronological bus can be used to evaluate what questions are answered by the model.

In the above example, because '**Customer**' is a conformed dimension for '**Orders**', '**Shipping**', and '**Customer feedback**', it is possible to identify customers by the volume of orders, shipping delays, and feedback results.

Because '**Product type**' is a conformed dimension on all processes, it is the attribute that the model can answer most questions for, such as comparing quality-control results with customer feedback. Note, however, the many-to-many relationship between '**Product type**' and '**Customer feedback**' indicates that any assignment of feedback to product is approximate and may double-count.

Because '**Manufacture ID**' is a conformed ID dimension from '**Manufacture**' to '**Shipping**', the model can support traceback of faulty products that have been shipped.

On the other hand, it is not possible to directly analyse the effect of shipping on customer feedback because shipping information does not filter the customer-feedback fact.

Such analysis may be possible indirectly through '**Reporting calendar**', '**Product type**' and '**Customer**', which together may narrow the transactions down to a segment that correlates shipping status and shipping company with feedback sentiment. If this can be done systematically, it can be introduced as precomputed information in the data pipeline.

Ensuring that the model's relationships match the real-world flow of information means the model behaves in ways users intuitively expect. Downstream processes tend to inherit from upstream ones. By default, upstream processes do not know downstream outcomes. When downstream outcomes are brought back upstream, they should be deliberately modelled.

As a result, the chronological bus is typically dense in the upper right. This structure makes gaps easy to spot. These gaps may be accidental, reflect a limitation in the source system, or be a deliberate design choice.

In the example above, '**Manufacture calendar**' does not filter '**Quality control**'. This is a design decision. It does not filter '**Customer feedback**' either, which is a limitation because the information is not recorded. This example does not have any accidental gaps. If there were, they would stand out.

The chronological bus only works if the data engineer has modelled information as dimensions,

processes as facts, and listed them in chronological order.

The same information becomes much less useful when sorted alphabetically:

Dimension	Customer feedback	Manufacture	Orders	Quality control	Shipping
Arrival date					1 -> *
Batch number		1 -> *	1 -> *	1 -> *	1 -> *
Batch quality outcome		1 -> *	1 -> *	1 -> *	1 -> *
Customer	1 -> *		1 -> *		1 -> *
Feedback calendar	1 -> *				
Feedback sentiment (star ratings)	1 -> *				
Manufacture calendar		1 -> *	1 -> *		1 -> *
Manufacture ID		1 -> *	1 -> *	1 -> *	1 -> *
Order calendar			1 -> *		
Order ID			1 -> *		1 -> *
Order status			1 -> *		1 -> *
Product type	* -> *	1 -> *	1 -> *	1 -> *	1 -> *
Quality criteria				1 -> *	
Reporting calendar	1 -> *	1 -> *	1 -> *	1 -> *	1 -> *
Shipping calendar					1 -> *
Shipping company					1 -> *
Shipping status					1 -> *
Testing date				1 -> *	
Testing ID				1 -> *	
Testing result				1 -> *	

In comparison, this matrix is difficult to read. This remains true even though the dimension names have been deliberately prefixed to group similar attributes together, and “Manufacture”, “Order” and “Shipping” happen to sort roughly chronologically. In any other instance, the bus matrix would be incomprehensible.

Consequently, the three-step process of linear process diagram, cumulative information diagram, and chronological bus is not an accident. It is designed to make it simple for the data engineer to keep track of information, visualise what questions can be answered, and thus accomplish the goal of anticipating questions.

The bus matrix is also a chance to check naming. Dimensions should be nouns because they represent business attributes or information. Fact names should usually be process-oriented

because facts represent business processes.

The relationship between business information and business processes is mirrored in the relationship between dimensions and facts. Hence this step can be remembered as “all the relationships.”

15.5 Conclusion

The three-step approach guides the data engineer to work on the information captured in business processes, rather than the stated requirements of “I would like to see X by Y”. The focus is on preserving information, not on answering specific questions.

The three steps and their artefacts can be summarised in a table like this:

Step	Content	Artefact
All the facts	Identify all the business processes.	Linear process diagram. Shows all the major steps in a business, set at a level that makes sense to decision-making.
All the dimensions	Identify all the information captured or inherited by the processes.	Cumulative information diagram. Shows all the information relevant to the business, how it flows, and where it is physically stored. Used to inform what can be reasonably answered.
All the relationships	Relate information to its processes in the dimensional model.	Chronological bus. Shows the dimensional model that is implemented. Used to identify which possible questions can or cannot be answered, particularly any gaps.

Following these three steps ensures the full range of information is preserved, and questions relating to this information can be answered because the relationships exist in the model.

This breadth of information requires a variety of modelling techniques. The later chapters in this section explain how filtering, measures, measure families, and security patterns make the preserved structure usable.

In future, when a user asks a different question, the model can answer it if the relevant information has already been articulated in the dimensional model.

This is how the data engineer anticipates questions.

Key ideas.

A strong dimensional model preserves enough business structure to answer all reasonable future questions.

Requirements tend to narrow attention; analytical modelling must preserve the structure from which follow-up questions can be answered.

“All the facts” means identifying the business processes that should become fact tables.

“All the dimensions” means identifying the information captured or inherited by those processes.

“All the relationships” means implementing the filtering paths that mirror what each process knows.

The three planning artefacts are the linear process diagram, the cumulative information diagram, and the chronological bus.

The chronological bus shows which questions the model can and cannot answer.

A good dimensional model

A good dimensional model lets users ask real business questions without guessing.

16.1 What makes a good dimensional model?

A dimensional model is successful when users can ask real business questions and obtain the expected answers without guessing how the model works.

This chapter describes a good dimensional model through two questions:

- What should a good model achieve?
- What are the signs of a good model?

16.2 What should a good model achieve?

A good dimensional model in Power BI should:

- Resonate with the business view
- Be intuitive and unambiguous—it should “just work”
- Anticipate questions
- Support both summary and detail
- Perform quickly

16.2.1 Resonate with the view from business intent

A dimensional model is where expressiveness is tested by use.

In earlier chapters, expressiveness meant creating information that corresponds to the business world in a recognisable way. In a dimensional model, this becomes even more important. Users must be able to recognise the business through the model.

In some cases, technically correct representations may need to give way to what better reflects how the business sees the world. For example, strict grain consistency can sometimes produce a proliferation of objects that are technically correct but difficult to understand. A simpler model may better match business understanding.

Resonance begins with naming. Models, tables, columns, and measures should use language that is immediately meaningful to the business.

This may also require creating perspectives the business has not yet articulated. Familiarity can cause business areas to miss insights in their own processes. The data engineer, working across domains, is often well placed to offer a fresh view.

Metadata also plays a key role. Descriptions should speak the business language, state the intent clearly, and, in complex cases, explain the logic in near-pseudocode using business terms.

16.2.2 Be intuitive and unambiguous

As a self-service data model, a Power BI model should let users drag and drop fields to get answers. For this purpose, the model should just work.

If users must memorise complex filtering rules, understand the map of relationships, or second-guess which column to use, the model is not intuitive. An intuitive model behaves in a way that matches expectation without requiring a heavy manual.

A related issue is ambiguity. An ambiguous model gives users multiple apparent ways to get the same information, or presents the same information in ways open to interpretation.

For example, ambiguity can occur when a column has been denormalised into multiple fact tables, leaving the user unsure which version to use. It can also occur when a column name such as `[Calendar date]` is too generic, making it unclear which business date the column refers to.

An unambiguous model gives the user one obvious way to answer a question. The model should either answer the question accurately or make it apparent that it cannot.

16.2.3 Anticipate questions

A good model does not merely satisfy the initial requirements. It anticipates the full range of reasonable business questions.

This requires the data engineer to think beyond the immediate request and consider what the model may need to answer over time. There is no escaping this reality. If a reasonable question is not answered now, a business area will eventually return and ask for it later.

In practice, this means a business process should either be outside the model, or, if included, represented in a form that supports its expected usage. All information captured by that process should be accessible in a way that allows users to ask sensible questions of it.

This is subject of the chapter [anticipating questions](#).

16.2.4 Support both summary and detail

One of the most powerful features of Power BI is the ability for users to move interactively through data by cross-filtering and drill-through.

Users often begin with a high-level view. If something catches their attention, they drill into

the details to see exactly what happened. This is an important part of how users build trust in the model.

The opposite path is also common. Users who operate at the detailed level often begin with transactions for troubleshooting, but eventually want to zoom out in search of broader patterns. This movement from detail to pattern is a sign of trust and maturity.

A data engineer cannot ignore the need to support both high-level and detailed views. While individual reports may emphasise one or the other, the underlying model often needs to support both.

16.2.5 Perform quickly

Most queries should return in under a second. Complex or infrequent queries should complete in under two seconds. Longer runtimes are acceptable only in rare cases.

Performance is not merely a user-experience consideration. It can also signal deeper issues in the articulation of information.

A slow model often indicates that key information has not been pre-computed through [meaningful fragments](#). The model is then forced to compensate with work at query time. In this sense, a slow model may be a concern of business expressiveness, not only technical performance.

16.3 What are the signs of a good model?

The following signs do not guarantee that the above objectives are met, and some of the signs may occasionally be absent for good reasons. But a strong Power BI dimensional model will usually show many of them. Their absence or frequent violation would hint at a model that is difficult to use.

The signs can be grouped into four areas:

- Names and metadata
- Dimensions
- Facts and measures
- Relationships

16.3.1 Names and metadata

Names should be business-centric rather than system-centric. It is a common mistake for data engineers to use acronyms or terms from the source system rather than names that reflect the business content itself. This should be avoided unless the model is intentionally about the system.

Names should also avoid technical implementation terms. Words such as **dim**, **fact**, or **bridge** should not appear in visible names. These are implementation details, not business concepts.

Names should be explicit. For example, **[Sales date]** is better than **[Date]**. It may even

be useful to repeat the table meaning in the column name, such as changing **Sales[Date]** to **Sales[Sales date]**. This is because the column name, and not the table name, is what appears in report visuals.

Visible names should not repeat across the model. When a user searches for a name in the field list, it should return one result. If a column appears more than once, it presents an immediate ambiguity.

If the repeated columns are genuinely the same, but all are visible, the model may need to be reworked to remove duplication. If the columns are different but share the same name by accident, they should be renamed rather than relying on table context to resolve the ambiguity.

Dimensions represent information or attributes, while facts represent business processes. As such, dimensions should be nouns, while business process facts should be verbal nouns or carry a sense of action.

For example, in a business system with three processes—manufacture, order, and shipping—the facts may be called **Manufacture**, **Order**, and **Shipping**. The dimensions may be called **Product**, **Region**, **Manufacture date**, **Order date**, and **Shipping date**. The facts name the business processes; the dimensions name the attributes those processes capture.

Naming facts after business processes is harder than it sounds. It requires a clear focus on business intent and enough familiarity with Power BI to design the model with simplicity.

A common case is the transaction and reference table pair, such as **Sales** and **RefSales**. One approach is to use **Sales** as the fact name and **Sales description** as the dimension name.

Finally, a strong indicator of a good dimensional model is the presence of rich, business-centric descriptions in the hover text of tables, columns, and measures.

16.3.2 Dimensions

In Power BI, dimensions are the window through which users interact with the model. A good dimensional model consciously treats dimensions as the user's point of access. They are the biggest influence on the model's look and feel.

On the surface, dimension table names should be nouns that convey the type of information available to the user.

Dimensions should also have a reasonable number of columns, often around five to ten. There can be more if some columns are simple derivations of others, such as text versions of binary flags.

Too few columns in a dimension may mean that it carries too narrow a business meaning. For example, a dimension called **Gender** with a single column **[Gender]** is likely too thin. A large number of dimensions with very few columns indicates a fragmented rather than consolidated view of the business. It may be more useful to combine **Gender** with related attributes into a broader **Demographic** dimension.

Too few columns may also indicate missing helpful attributes. For example, a **Gender** table should include **[Gender display order]** so the values can be sorted usefully.

On the other hand, too many columns in a dimension may suggest that the grouping of attributes is too complicated and may overwhelm users. It also reflects a view of the business that is insufficiently nuanced. In that case, the dimension may need to be broken into several more manageable dimensions.

A dimensional model often includes two special types of dimensions that indicate thoughtful design.

The first is the ID dimension. This is a table of the primary keys of a business process. It allows users to retrieve detailed transactions for a business entity by looking up a familiar business key.

The second is the storytelling dimension. It sits at the opposite end of the spectrum. As explained in the Storytelling chapter, a storytelling dimension categorises business entities into a digestible number of journeys. It is often the front door of the model: the first high-level lens through which users enter the data.

The presence of both ID dimensions and storytelling dimensions suggests that the data engineer has considered a broad range of user needs. It is not a guarantee of quality, but it is a useful sign.

The full set of useful dimensions is covered in the next chapter on the [components of a dimensional model](#).

16.3.3 Facts and measures

Power BI is designed so that users primarily interact with dimensions and measures rather than facts. This is explained in greater depth in the chapter on [filtering behaviour](#).

For now, it is enough to note that a good Power BI dimensional model usually reduces the prominence of fact tables. In the ideal case, fact tables are hidden from the user.

A related sign is the absence, or at least limited use, of degenerate dimensions (attributes stored directly in the fact table rather than in a separate dimension) in fact tables. A proliferation of degenerate dimensions suggests that the model is working against the natural filtering behaviour of Power BI. It also indicates that business information has been left as miscellaneous attributes of business processes rather than articulated as standalone properties that deserve dimension tables.

Measures become more prominent as fact tables become less visible. A good dimensional model invests heavily in measures.

As a rough rule of thumb, each fact table representing a business process should usually have at least ten measures. This reflects the reality that there are often many metrics needed to understand a business process. For example, a sales process may require measures for sales volume, turnaround time, profit, cost, and related percentages.

If the model has implemented the concept of "good" and "bad", this often leads to sub-measures for each category. Percentages and other derivatives quickly add more.

If users can grab ready-made measures and arrive at useful answers with minimal effort, this is a strong sign of a good model. It shows that the data engineer has thought carefully about the business process and placed important information at the user's fingertips.

Conversely, a lack of ready-to-use measures means that the data engineer has not sufficiently engaged with the business intent of the user.

When a model contains many measures, measure management becomes important. A good pattern is to place all measures into a single table at the top of the field list. A suitable name for this table is simply **Measure**.

Power BI supports this behaviour. If all columns in a table are hidden and the table contains at least one measure, Power BI places the table at the top of the field list. This indicates a model that elevates measures to first-class status.

Measures should be organised by display folders. By default, these folders should be grouped by business process. For example, a model with the processes **Manufacture**, **Order**, and **Shipping** could have folders called "**Manufacture**", "**Order**", and "**Shipping**".

The prominence of measures, their business-centricity, and their organisation through business processes are strong contributors to the look and feel of a good data model.

Measures are also easier to define when the pipeline has already created meaningful fragments at the right grain. A good model should not rely on complex measures to compensate for missing upstream structure.

16.3.4 Relationships

Relationships in a Power BI model define user interactivity.

Following the theme of using dimensions as the main point of interaction, dimensions should filter facts and not the other way around.

A good sign is that, if a piece of information is known to a business process, the dimension representing that information should filter the corresponding fact.

For example, consider a model with two processes: **Manufacture** and **Shipping**.

The **Product** dimension should filter both facts if both processes know the product. **Manufacture** captures the product directly. **Shipping** inherits the product because shipped items trace back to what was manufactured.

By contrast, the **Shipping date** dimension should filter only the **Shipping** fact. It should not filter **Manufacture**, because shipping occurs at the shipping grain. One manufactured batch may be shipped across multiple dates, so there is no single shipping date that naturally belongs to the manufacture process.

This does not mean that each business process is filtered only by its own attributes. For example, all fact tables may be filtered by **Manufacture date** if the shipment tracks back to a manufacture batch.

The criterion is whether the dimension belongs naturally to the grain of the fact being filtered.

Dimension	Manufacture	Shipping
Product	1 → *	1 → *
Manufacture date	1 → *	1 → *
Shipping date		1 → *

Dimension	Manufacture	Shipping

Relationships are explored in greater depth in the chapter on [anticipating questions](#).

Since dimensions are the user's access point to the model, the column on the filtered side of the relationship, usually in the fact table, should be hidden to avoid ambiguity. The user should use the column on the filtering side, usually in the dimension.

16.4 Conclusion

This chapter sketched what a good dimensional model looks like.

The purpose is not to provide a definitive checklist that guarantees quality. It is to attune the data engineer to the look and feel of a good model in Power BI by taking the perspective of the user.

A list of signs is helpful, but the best test is for the data engineer to use the model personally and ask a range of real business questions. This is part of conducting visual checks.

This task requires deep familiarity with the business and strong curiosity about the subject matter. It cannot be reduced to ticking off formal requirements.

The common theme is business centricity and intuitive use. A good model implements answers explicitly so that they appear naturally. It does not require the user to interpret technical structure or apply guesswork.

The Zen of Python is a useful guide to designing a dimensional model. Its emphasis on explicit over implicit, and on having one—and preferably only one—obvious way of doing something, applies directly to Power BI model design.

Key ideas.

A good dimensional model lets users ask real business questions without guessing.

A dimensional model is judged by the experience it creates for users, not only by whether has the technically correct information.

It should resonate with the business view, feel intuitive and unambiguous, anticipate reasonable questions, support both summary and detail, and perform quickly.

The signs of a good model are business-friendly names and metadata, useful dimensions, ready-to-use measures, and relationships that support natural filtering.

A strong model makes answers appear naturally because the data engineer has already made the hard design decisions.

Dimensional modelling for UX

In an interactive model, dimensions are what users touch; facts are what respond.

17.1 Facts and dimensions as interaction

Dimensional modelling is usually taught as a distinction between table types—facts record activity; dimensions provide context.

That distinction is correct, but not enough to achieve the expectations of a [good dimensional model](#). The more important distinction is between what the user can touch and what responds—dimensions are what the user touches; facts are what respond.

A model is good when the user can answer questions without guessing. This chapter explains facts and dimensions from that point of view. That is, dimensional modelling for user experience (UX).

17.2 Understanding facts and dimensions

Classical dimensional modelling revolves around two core concepts: facts and dimensions.

In a technical view:

- Facts are transactions, events, or activities that occur in the business.
- Dimensions are reference tables that provide context for those transactions.

Fact tables are fast-moving tables. They grow quickly, have high cardinality, and record frequent business activity. Dimension tables are slower-moving tables. They have lower cardinality and provide meaningful categories for filtering, grouping, and comparison.

From the business view:

- Facts are business processes. They convey action, such as manufacture, order, shipping, inspection, payment, or case handling.
- Dimensions are business objects, attributes, or categories. They convey context, such as product, region, customer, officer, status, or date.

A common explanation says that dimensions filter facts. This is true, but it hides the practical point: filtering is a form of control.

17.2.1 An analogy of robots

To make the distinction concrete, think of dimensions as levers and facts as robots.

If filtering is a form of control, then:

1. Levers control robots.
2. Levers may control other levers.
3. Robots may occasionally control other robots.
4. Robots should not control levers.

This analogy helps reinforce a key design principle: interactivity should be driven by dimensions.

Levers are what users control. Robots are what respond. In the same way, dimensions are the user's interface to the model, while facts contain the machinery that responds to that interaction.

This is also why the controls should not sit on the robots themselves. They should sit on the controller. In practical terms, users should interact with dimensions to drive filtering, grouping, searching, comparison, and other model effects—not with columns embedded in fact tables.

This is the main problem with degenerate dimensions. They may work technically, but they place controls inside the fact table. They make the robot carry its own levers.

For this chapter, we stick with the simpler view of dimensions as tables.

But the wider principle is more general. A dimension is not merely a lookup table. It is anything in the model the user interacts with to produce an effect.

In more advanced scenarios, a dimension may function more like a button than a filter. The user clicks or selects it, and the model responds. The fact tables contain the ingredients that make the response possible.

In this view, the physical relationship pattern is secondary. A dimension may have one relationship, many relationships, or no direct relationship to a fact table. It may work alone, or as part of a set of tables. What matters is that the user interacts with it to produce a controlled effect. If it does that, it is functioning as a dimension.

This abstract view is useful when tackling demanding usage scenarios.

17.3 A repertoire of facts and dimensions

The categories below are not a complete taxonomy of dimensional modelling. They are a practical repertoire for handling usage scenarios. They name the roles tables can play when a model is designed for interaction.

There are 3 types of facts and 9 types of dimensions.

Facts:

1. Measurable fact
2. End-of-period fact

3. Annotation fact

Dimensions:

- 1. Business dimension
- 2. Role-playing dimension
- 3. Histogram dimension
- 4. Combination dimension
- 5. Choices dimension
- 6. Sankey dimension
- 7. Storytelling dimension
- 8. Search dimension, a sub-type being the ID dimension
- 9. Degenerate dimension

17.4 Facts

Facts are the responsive machinery of the model. They contain business activity, business state, or display detail that becomes visible when the user interacts with dimensions.

17.4.1 Measurable fact

A **measurable fact** records business activity that can be counted, summed, averaged, or otherwise aggregated.

Examples include inspections, sales, orders, payments, helpdesk cases, audit events, or shipments. These are the facts people usually mean when they talk about fact tables.

A measurable fact is designed to work through measures. Users should normally not consume the raw fact table directly. They should interact with dimensions, and the fact should respond through measures such as inspection count, failure rate, total value, or average processing time.

Example structure of 'Inspection'

Inspection		Inspection		Inspection	
Inspection SK	date	Location SK	Officer SK	status SK	duration minutes
1001	2024-01-15	12	31	1	45
1002	2024-01-16	18	44	2	70

This fact can support measures that use functions such as COUNTROWS, SUMX, and COUNTX. The user interacts with dimensions such as 'Location', 'Officer', and 'Inspection status'. The measures respond from 'Inspection'.

This example is at the grain of a single inspection, but a measurable fact does not have to be at event grain. It may already be aggregated before it reaches the model. What matters is not the physical grain of the table, but the role it plays: it supplies measurable material for measures to calculate against and for dimensions to control.

Example structure of 'Inspection daily aggregates'

Inspection date	Location SK	Inspection status SK	Inspection count	Total inspection duration minutes	Failed inspection count
2024-01-15	12	1	18	720	0
2024-01-15	18	2	7	490	7
2024-01-16	12	1	22	860	0

This is also a measurable fact. It is not at the grain of a single inspection. It has already been aggregated by inspection date, location, and inspection status before it reaches the model.

The measures still respond from the fact. A measure may sum [Inspection count], divide [Failed inspection count] by [Inspection count], or calculate average duration from [Total inspection duration minutes] and [Inspection count].

The physical grain is different, but the modelling role is the same: 'Inspection daily aggregates' supplies measurable material for measures to calculate against and for dimensions to control.

Measurable facts are the default facts for [entity processing](#) scenarios.

17.4.2 End-of-period fact

An **end-of-period fact** records the state of an entity at a regular reporting point.

This pattern applies when the business needs to ask what existed, or what state something was in, at the end of a period. Examples include an account at the end of each month, an employee at the end of each pay cycle, or a supplier at the end of each reporting quarter.

End-of-period facts are usually produced from [entity tracking](#) pipelines. They let the model answer point-in-time questions without requiring users to solve validity-window logic inside the report.

Example structure of 'Account end of month'

Account SK	Period end		Account status SK	Account level SK	Closing balance
	date				
501	2024-01-31		1	2	8700.00
501	2024-02-29		1	3	13700.00

This fact supports questions such as:

- How many accounts were active at the end of February?
- How many accounts were gold-level accounts at the end of the quarter?
- What was the total closing balance for the selected period?

The important point is that the fact records state at a reporting boundary. It turns a changing entity into something the user can query at a stable point in time.

17.4.3 Annotation fact

An **annotation fact** records display detail attached to another fact, usually where the detail is too fine-grained or too high-cardinality to belong in the main fact table.

Examples include inspection comments, certificate details, notes, attachments, or free-text descriptions. These details are often not designed for aggregation. They are designed for display after the user has already arrived at a transaction.

Example structure of 'Inspection comments'

Inspection SK	Comment sequence	Comment datetime	Comment text
1001	1	2024-01-15 10:32	Missing supporting document.
1001	2	2024-01-15 10:47	Officer requested follow-up evidence.

The reason to move these details into a separate fact is grain.

One inspection may have many comments. If the comments are placed directly in '**Inspection**', the data engineer must either change the grain of '**Inspection**' or concatenate the comments into a single field. Both options are undesirable.

Changing the grain bloats '**Inspection**' by repeating inspection-level information once for every comment. This increases model size and load cost, but it also creates a deeper pipeline problem: the inspection and its comments may change on different rhythms. Inspection status may be updated while comments remain unchanged, or comments may be appended while the inspection itself is stable. Combining them forces change detection and partitioned loading to treat these different behaviours as one table.

Concatenation preserves the grain of '**Inspection**', but destroys the structure of the comments. The comments become a block of text rather than records with their own sequence, datetime, author, category, and display behaviour. The user may be able to read the comments, but the model can no longer sort them, filter them, display them individually, or apply separate security and formatting logic to them.

An annotation fact avoids both problems. It lets '**Inspection**' remain at the inspection grain, while '**Inspection comments**' carries the finer-grain detail.

Annotation facts are one of the rare cases where fact-to-fact filtering can be appropriate. Here, the [**Inspection SK**] column would be the related column to the same in '**Inspection**' fact. The typical usage scenario is one where the business user has arrived at a set of transaction records through a dimension and would like to see additional details about that transaction. This is also why they are filtered through another fact, not a dimension.

Often, a measure is still needed to display annotation detail in a controlled way. For example, **SELECTEDVALUE** can ensure that a comment or detail field is only displayed when the relevant transaction has been selected.

An annotation fact is therefore not a second business process. It is an adjunct fact. It carries detail that describes another fact.

17.5 Dimensions

Dimensions are the controlled surfaces of interaction. They give users handles for filtering, grouping, searching, comparing, and storytelling.

A dimension is not merely a lookup table. In an interactive model, it is a control.

17.5.1 Business dimension

A **business dimension** is an ordinary control surface that lets users filter and group facts using familiar business categories.

Examples include statuses, locations, officers, inspection types, customers, products, regions, suppliers, and programs. These dimensions often come from source-system reference tables, but the data engineer may supplement them with additional columns that make business meaning clearer.

Example structure of 'Inspection status'

Inspection status SK	Inspection status	Is completed	Is failed
1	Passed	true	false
2	Failed	true	true
3	In progress	false	false

This dimension filters 'Inspection'. A user can place [Inspection status] on a slicer, group inspection counts by [Is failed], or compare completed and incomplete work.

The important point is not only that 'Inspection status' is a low-cardinality lookup table. The important point is that it is a natural thing for the user to touch.

17.5.2 Role-playing dimension

A **role-playing dimension** is a dimension that takes different meanings depending on which fact column it filters.

The most common example is a calendar. The same 'Reporting calendar' table may be used against multiple date columns. It may filter orders by order date, shipments by shipment date, and completions by completion date.

Example structure of 'Reporting calendar'

Reporting date	Financial year	Month name	Month end date
2024-01-15	2023-24	January	2024-01-31
2024-02-15	2023-24	February	2024-02-29

The same 'Reporting calendar' table may filter a fact through different columns, such as

[Order date], [Shipment date], and [Completion date].

A user selecting January may be asking for orders placed in January, shipments sent in January, or cases completed in January. Role-playing dimensions play an important role in comparing different business processes using a common denominator, as explained in the chapter [Conforming systems](#).

The model should make that role clear. If a role-playing dimension is exposed to users, the field names, measure names, or hover text should explain which role is being used.

A dimension is not role-playing merely because it filters many facts. The related columns must have different meanings.

For example, suppose [Manufacture date] appears on 'Manufacture', 'Order', and 'Shipping', and 'Reporting calendar' relates to each fact through [Manufacture date]. Even though the calendar filters three facts, it is not role-playing. It means the same thing in each case: manufacture date. It is better understood as a business dimension, and may be better named 'Manufacture calendar'.

17.5.3 Histogram dimension

A **histogram dimension** turns numeric values into a filterable and visualisable dimension.

This pattern is useful when the user needs to filter or chart records by a numeric value, such as days to resolve, hours to process, number of attempts, or value bands.

Measures do not work as ordinary slicer fields. If users need to filter by a numeric result, that result often needs to be expressed as a dimension.

Example structure of 'Days to resolve'

Days to resolve SK	Minimum days	Maximum days	Days to resolve band
1	0	1	0 to 1 days
2	2	5	2 to 5 days
3	6	10	6 to 10 days
4	11	9999	More than 10 days

The fact table can carry [Days to resolve SK], or it can carry a numeric value that maps to 'Days to resolve'.

Care is needed around boundaries. The data engineer must decide whether boundaries are inclusive or exclusive and ensure that maximum and minimum values are handled consistently.

As well as allowing the user to filter the model on a numeric value, the dimension is useful for creating histogram visuals—hence the name. The visual counts entities against a dimension of days, hours, attempts, amounts, or other numeric bands.

This is especially useful for time-to-process metrics. Instead of showing only an average processing time, the model can show the distribution: how many cases took 0 to 1 days, 2 to 5 days, 6 to 10 days, or more than 10 days.

17.5.4 Combination dimension

A **combination dimension** represents combinations of related binary properties at the entity grain.

This pattern is useful when an entity can have several related properties at once, and the business wants to filter by combinations of those properties.

For example, a business entity may pass through multiple systems. The detail may be recorded at a lower grain, with one row per system touchpoint. But the user often wants to filter the business entity, not the system touchpoints.

Example structure of 'System combination'

System combination SK	Has System A	Has System B	Has System C	System combination
1	false	false	false	No system
2	true	false	false	System A
3	false	true	false	System B
4	false	false	true	System C
...	...	...	...	...
8	true	true	true	System A, System B, System C

The fact table carries [System combination SK]. The user interacts with the flags in 'System combination'.

A combination dimension pushes lower-grain detail back to the entity level, then exposes the result as a controlled surface of interaction.

A common case is the header-detail pattern. A consignment may be the header entity, while the cargo line items are the detail rows. Some line items may be fresh produce. Others may be inanimate goods. The user often wants to filter consignments by the types of cargo they contain, not analyse each line item separately.

Example structure of 'Cargo combination'

Cargo combination SK	Has fresh produce	Has inanimate cargo	Cargo combination	Has mixed cargo
1	false	false	No cargo	false
2	true	false	Fresh produce only	false
3	false	true	Inanimate cargo only	false
4	true	true	Fresh produce and inanimate cargo	true

The consignment fact carries [**Cargo combination SK**]. The user interacts with 'Cargo combination' to find consignments containing fresh produce, inanimate cargo, or both.

Combination dimensions are especially useful for **and** logic. They allow the user to ask for entities that have this property and that property while keeping the model at the correct grain.

17.5.5 Choices dimension

A **choices dimension** is a low-cardinality, multi-valued dimension that lets users filter for entities that have any selected attribute.

Choices dimensions are related to combination dimensions, but the interaction is different. A combination dimension is useful when the user cares about combinations of properties. A choices dimension is useful when the user wants any matching choice.

The simplest example is a drink preference question where a person may choose tea, coffee, both, or neither. The fact table stays at the grain of the person. It does not expand to one row per drink.

Example structure of 'Table order'

Table order ID	Order date	Drink choice group SK
1	2024-01-15	2
2	2024-01-15	3
3	2024-01-16	3
4	2024-01-16	1
5	2024-01-17	4
6	2024-01-17	4

Example structure of 'Drink choice'

Drink choice group SK	Drink choice
1	No drink
2	Tea
3	Coffee
4	Tea
4	Coffee

The fact table carries [**Drink choice group SK**]. The choices dimension contains one row per selected choice within each group.

If the user selects **Tea**, the model returns table orders 1, 5, and 6. If the user selects **Coffee**, the model returns table orders 2, 3, 5, and 6. Table orders 5 and 6 appear in both results because their group contains both choices. The grain of 'Table order' has not changed.

Without a choices dimension, there are two common alternatives.

The first is to store each choice as a separate Boolean column in the fact table, such as [**Has**

tea] and [Has coffee]. But it is awkward when the user wants to find tables with tea *or* coffee—the filter will pick rows that have tea *and* coffee instead.

The second is to change the grain of the fact table so that each order appears once per drink choice. This makes the choice easy to filter, but 'Table order' is no longer at the grain of an order. It is now at the grain of an order-choice. That may be correct for some models, but often not.

A choices dimension avoids both problems. The fact remains at the order grain, while the user can still select individual choices using *or* logic. Thus, a choices dimension is a set-membership bridge expressed as a dimension. It preserves the entity grain while still letting the user interact with individual choices.

A common business example is data quality. A transaction row may have multiple issues: missing date, missing officer, invalid status, or missing location. A data quality report needs to show records that have any selected issue, while still preserving one row per transaction.

Example structure of 'Transaction'

Transaction SK	Transaction date	Issue group SK
1001	2024-01-15	2
1002	2024-01-16	3
1003	2024-01-17	4
1004	2024-01-18	1

Example structure of 'Data quality issue'

Issue group SK	Data quality issue
1	No issue
2	Missing date
2	Missing officer
3	Missing location
3	Invalid status
4	Missing date
4	Missing officer
4	Missing location

The fact table carries [Issue group SK]. The choices dimension contains one row per issue within each group.

This allows a user to select **Missing date** and see all transactions whose issue group includes **Missing date**. It also allows the user to select several issues without forcing 'Transaction' to expand to one row per issue.

Physically, a choices dimension is created by taking the possible sets of choices from a finite list and assigning each set a group key. The relationship between the fact and the choices dimension is on that group key.

Choice and combination dimensions are closely related. A combination dimension can often be unpivoted to create a choices dimension.

For example, 'System combination' may contain [Has System A], [Has System B], and [Has System C]. This can be unpivoted into a 'System touch point' choices dimension with values such as System A, System B, and System C.

Example structure of 'System touch point'

System combination SK	System touch point
1	No system
2	System A
3	System B
4	System C
5	System A
5	System B
6	System A
6	System C
7	System B
7	System C
8	System A
8	System B
8	System C

The decision depends on the question. If the business wants combinations, use a combination dimension. If the business wants any selected value, use a choices dimension. Since they are lightweight, it is always possible to have both if beneficial.

17.5.6 Sankey dimension

A **Sankey dimension** represents possible paths through a sequence of checkpoints.

This pattern is useful when the business wants to visualise movement through a process. The source fact should not be reshaped just to create a flow visual. Instead, each possible path can be represented as a dimension.

For example, an entity may move through follow-on systems: System A, System B, and System C. Some entities begin in System A and continue through all systems. Some begin in System B. Some skip System B and move directly from System A to System C.

A 'System path' dimension can represent each possible path as a set of edges.

Example structure of 'System path'

System combination SK	From system	To system
1	Start	No system
2	Start	System A
3	Start	System B

System combination SK	From system	To system
4	Start	System C
5	Start	System A
5	System A	System B
6	Start	System A
6	System A	System C
7	Start	System B
7	System B	System C
8	Start	System A
8	System A	System B
8	System B	System C

The fact table carries [System combination SK]. The Sankey dimension expands that path into the rows required by the Sankey visual.

This lets the user see process flow without changing the grain of the fact. The fact remains at the entity grain. The dimension supplies the visual structure.

Notice that [System combination SK] is the same key used by 'System combination', 'System touch point', and 'System path'.

This is the same underlying structure expressed for different usage scenarios.

'System combination' presents the systems as a combination of binary choices. It answers: which exact combination of systems was involved?

'System touch point' presents the same structure as a choices dimension. It answers: did the entity involve any selected system?

'System path' presents the same structure as a Sankey dimension. It answers: how did the entity move through the systems?

Computationally these are low costs and low complexity translations.

The key is to recognise the underlying axes of binary choice. Once those axes are identified, the data engineer can express them in different dimensional forms depending on the interaction the user needs. Thus, the data engineer is not merely arranging tables, but recognising the latent structure of interaction in the business problem.

17.5.7 Storytelling dimension

A **storytelling dimension** groups entities into meaningful business journeys.

This pattern is used when the raw process is too detailed for the user's first point of entry. Instead of beginning with many statuses, events, checkpoints, or paths, the model gives users a small set of meaningful journeys.

Where a Sankey dimension shows movement, a storytelling dimension translates these movements into manageable categories that are business-meaningful.

For example, the [Storytelling](#) chapter describes a help desk escalation dimension that combines tier combinations, highest escalation, escalation path, escalation summary, and display order. The point is not merely to record the path a case took, but to express what that path means to the business.

A storytelling dimension is created when the data engineer steps back from raw data and asks how the business understands the movement of the entity.

In a self-service model, storytelling dimensions often act as the first portal into the data. They divide the population into meaningful strata and help users begin analysis before drilling into detail.

17.5.8 Search dimension

A **search dimension** is a high-cardinality dimension that gives users a controlled way to find, retrieve, drill through, or cross-filter records.

Most dimensions are useful because they support aggregation. Search dimensions are useful because they provide a way into the model.

In traditional dimensional modelling, a transaction is usually modelled as a fact table, not as a dimension. Transaction identifiers are often kept inside fact tables as degenerate dimensions. In an interactive Power BI model, this is not always enough. Sometimes the user needs to search for a specific transaction, use it for drillthrough, or cross-filter related facts across multiple business processes.

In that case, a transaction-level value can become a search dimension.

An important special case is the **ID dimension**.

An ID dimension is a minimal search dimension that contains only the key fields needed for search, usually the surrogate key and one or more business identifiers.

Example structure of 'Inspection ID'

Inspection SK	Inspection number
1001	INS-2024-0001
1002	INS-2024-0002

This dimension lets a user retrieve records using an identifier they already know. In traditional dimensional modelling, this pattern may be treated as an exception. In interactive reporting, it is often practical and necessary because users frequently arrive with a known business key. They want to paste or search an ID and see the relevant records.

Search dimensions can also support high-cardinality text values that users treat as search handles.

Example structure of 'Package search'

Package SK	Package reference	Package description
7001	PKG-2024-0001	Refrigerated seafood consignment

Package SK	Package reference	Package description
7002	PKG-2024-0002	Machinery parts and spare components

The modelling decision depends on the user’s intent.

If the user expects to search using the field, model it as a search dimension. If the user only expects to display the field after arriving at a record, model it as an annotation fact.

Search dimensions can significantly increase model size. They should be used deliberately, and they are not ideal for aggregation.

Due to Power BI filtering behaviour, search dimensions often need to be used with a unit-record display measure. This is explained in the [Filtering behaviour](#) and [Designing measures](#) chapters.

17.5.9 Degenerate dimension

A **degenerate dimension** is a dimension-like field retained inside a fact table rather than lifted into a separate dimension table.

This usually occurs for one of two reasons:

- The field does not have enough business weight to justify a standalone dimension.
- The field has high cardinality and would increase model size if lifted into a dimension.

Example of a degenerate dimension [Inspected goods] inside 'Inspection'

Inspection SK	Inspection number	Inspection date	Inspection status SK	Inspected goods
1001	INS-2024-0001	2024-01-15	1	Refrigerated seafood consignment
1002	INS-2024-0002	2024-01-16	2	Machinery parts and spare components
1003	INS-2024-0003	2024-01-16	1	Fresh mangoes and tropical fruit
1004	INS-2024-0004	2024-01-17	3	Timber pallets and packaging materials
1005	INS-2024-0005	2024-01-18	2	Laboratory testing equipment

Here, [Inspected goods] is the degenerate dimension, and can be used for searching. This works technically, but it can weaken the interaction logic of the model. It makes the fact table behave like a control surface.

Degenerate dimensions should therefore be treated with caution. They are sometimes necessary, but they are often a sign that a field has not been properly expressed as a business dimension, search dimension, or annotation fact.

For example, if **Inspected goods** is only needed for display, the column can remain hidden in the model and be exposed through a measure using **SELECTEDVALUE**. If it is needed for search or cross-filtering, it may be better expressed as an '**Inspected goods**' search dimension related by [**Inspection SK**].

The question is not whether the field can remain in the fact table. The question is whether the user needs to touch it. The three problems of degenerate dimensions are discussed in the next chapter.

If the user needs to touch it, it probably belongs on the controller, not on the robot.

17.6 Pushing detail back to the entity

One common theme in this chapter is the movement from information at a finer grain than the entity back to the entity level, and then into a dimension.

Combination dimensions, choice dimensions, Sankey dimensions, and storytelling dimensions all follow this pattern.

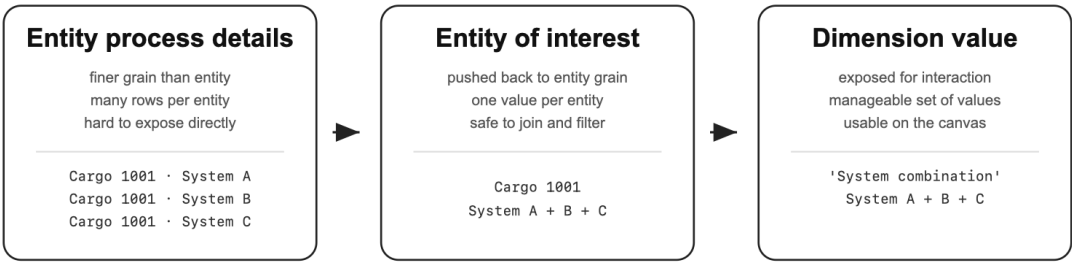


Figure 1. Lower-grain detail is pushed back to the entity grain, then exposed as a dimension value that users can interact with safely.

This movement is powerful because it keeps the focus on the entity of interest, both technically and business-wise.

Technically, the entity of interest remains the common denominator for joins. The user gets a meaningful control surface without forcing the model into lower-grain chaos.

Business-wise, user interactions remain centred on the entity the business actually cares about.

The purpose of this chapter is not to define an exhaustive list of all possible artefacts in a dimensional model. Ultimately, each table is either a dimension or a fact, and the sub-categorisation does not significantly change this.

Rather, the purpose of listing these elements is to help data engineers shift to a mindset of seeing dimensions and facts through usage scenarios, and to recognise that they can play functionally different roles depending on the interactivity requirements.

In the end, the final test is this: can the business user answer a question without needing to guess?

That is the heart of dimensional modelling in an interactive model.

Key ideas.

In an interactive model, dimensions are what users touch and facts are what respond.

Facts contain business activity, business state, or display detail.

Dimensions provide controlled surfaces for filtering, grouping, searching, comparing, and storytelling.

A fact table should not become the user's control panel. The controls belong on the controller, not on the robot.

Some dimensions exist to push lower-grain complexity back to the entity grain and expose it as something users can interact with safely.

A good dimensional model lets users interact with business activity reliably without guessing how the model works.

Filtering behaviour

Filtering design is what makes a Power BI model feel intuitive—or frustrating.

18.1 Filtering as interaction design

Filtering design is what makes a Power BI model feel intuitive—or frustrating.

Many engineers stop too early, treating the job as complete once the correct answer is reachable through some combination of filters.

That is not enough.

A self-service model is not a puzzle for the user to solve. The user should not have to discover the one valid route through the model. They should be able to approach the model from different directions—starting with a date, a product, an ID, a measure, or a business process—and still receive sensible results at each step.

The data engineer’s task is therefore not only to make the correct answer possible. It is to anticipate how users will try to reach that answer. More generally, it is to anticipate what buttons the user will click, and what effects the model should produce.

A good dimensional model is designed around the interaction between [buttons and effects](#). Filtering is the mechanism through which this interaction takes place.

This chapter explains how filtering works so the data engineer can anticipate user interaction and design the model’s response deliberately.

The chapter proceeds in two parts.

First, it explains the main ways filtering occurs in Power BI. These are the basic mechanisms the data engineer needs to understand: relationship filtering, common-fact filtering, non-blank measure filtering, visual-level filtering, and measure-defined filtering.

Second, it applies those mechanisms to tricky modelling scenarios. These include displaying unit records, coordinating multiple business processes, cascading filters, aggregating dimension values, filter-by-having, and dynamic Type I behaviour.

The purpose is not to memorise tricks. The purpose is to understand how Power BI responds when users interact with the model, so the data engineer can design those responses deliberately.

18.2 Ways of filtering

There are five common ways filtering appears in Power BI:

- Relationship filtering
- Common-fact filtering
- Non-blank measure filtering
- Visual-level filtering
- Measure-defined filtering

18.2.1 Relationship filtering

Relationship filtering is the standard case of filtering.

When table X filters table Y, a user who selects a value from X narrows the rows in Y based on the relationship between the two tables. In a dimensional model, the most important case is when dimensions filter facts.

The user touches the dimension. The fact responds.

Example structure of 'Product'

Product SK	Product name
1	Cake
2	Bread
3	Pastry

Example structure of 'Sale'

Sale ID	Product SK
S1001	1
S1002	1
S1003	2

If the user selects **Cake** from 'Product', the model filters 'Sale' to S1001 and S1002. If the user selects **Bread**, it filters to S1003. If the user selects **Pastry**, there are no matching sales rows.

New engineers often ask, “Why do we need dimensions at all? Why not leave all the information in the fact table? It is simpler.” That is, why not rely solely on degenerate dimensions?

There are three common problems.

First, non-existence becomes impossible to express. If product were only stored as a degenerate column inside 'Sale', then **Pastry** would not exist anywhere in the model. A user would not be able to know that no pastries were sold.

Second, the business meaning becomes buried. Attributes such as region, product, demographic group, or status are no longer presented as coherent business objects. They become miscellaneous columns inside a fact table. A user would have to read every single row to reverse engineer the entity and its grain.

Third, cross-filtering and drillthrough across multiple business processes become harder. A conformed dimension such as 'Calendar', 'Product', or 'Region' can filter several facts. A degenerate column inside one fact cannot naturally serve as a shared point of interaction.

18.2.2 Common-fact filtering

Filtering also appears indirectly through common filtered tables.

This behaviour is often missed by new engineers, but it is essential to the intuitive experience of Power BI. When two dimensions both filter the same fact, Power BI can use the fact to determine which combinations of dimension values are valid.

Consider a simple sales model.

Example structure of 'Product'

Product SK	Product name
1	Cake
2	Bread
3	Pastry

Example structure of 'Region'

Region SK	Region name
1	North
2	South
3	West

Example structure of 'Sale'

Sale ID	Product SK	Region SK	Sales amount
S1001	1	1	100
S1002	1	2	150
S1003	2	1	200

If a visual contains 'Product'[Product name] and 'Region'[Region name], Power BI returns only product-region combinations that exist in 'Sale'.

The fact table acts as the common filtered table that makes the dimension combination valid.

The result would be:

Product name	Region name
Cake	North
Cake	South
Bread	North

Pastry does not appear because there is no matching row in **'Sale'**. **West** does not appear because there is no matching row in **'Sale'**. The missing combinations are not shown because they do not exist in the common fact.

This common-fact behaviour is part of what makes a dimensional model feel natural. Users can place dimension fields together, and the model silently removes combinations that do not exist.

If two dimensions do not share any filtered fact table, Power BI cannot determine the valid combinations between them. In that case, the visual may return the Power BI error message: **Can't determine relationships between the fields.**

If multiple fact tables are filtered by both dimensions, Power BI shows combinations where any fact table has a matching row. This behaviour is powerful, but it also means the data engineer must think carefully about which facts share which dimensions.

This “upward” filtering from facts to dimensions enables the intuitive experience of a dimensional model. It is why dimensions alone can serve as the interface, even when facts are hidden. Mastering this behaviour is central to designing good Power BI models.

18.2.3 Non-blank measure filtering

The previous method relies on implicit filtering through relationships and common facts. In simple models, this is often enough. In complex models with several fact tables, it may not be.

Measures can also control what appears in a visual.

Suppose **'Product'** filters **'Sale'**, and **[Total sales amount]** sums the sales amount in **'Sale'**.

Visual before adding [Total sales amount]

Product name
Cake
Bread
Pastry

Visual after adding [Total sales amount]

Product name	Total sales amount
Cake	250
Bread	200

`Pastry` disappears because `[Total sales amount]` is blank for that product in the current filter context.

This is non-blank measure filtering. When a measure is added to a table or matrix visual, Power BI tends to retain rows where the measure returns a non-blank value.

If several measures are added, the visual may retain rows where any measure returns a value.

This mirrors fact table filtering, but gives the data engineer more control. Instead of relying only on row existence in a fact table, the data engineer can define the measure logic that determines which values should remain visible.

18.2.4 Visual-level filtering

Sometimes a measure cannot be placed directly in the visual, or doing so would interfere with the visual's layout.

For example, a slicer cannot simply display a helper measure as one of its fields. A table visual may also need to be filtered by a measure without showing that measure to the user.

In these cases, the report developer can apply a visual-level filter using the measure value.

For example, `[Has sales]` may return 1 when there is at least one sale in the current context. A slicer on `'Product'` `[Product name]` can then use `[Has sales]` as a visual-level filter so that only products with sales appear.

This method is useful, but should be used with care. The evaluation cost can be significant when the visual has a large number of possible values and the measure must be calculated for each one.

18.2.5 Measure-defined filtering

The previous methods control which dimension values appear in a visual. Sometimes the data engineer needs to apply filters directly inside the measure calculation itself.

This is measure-defined filtering.

Power BI supports this through DAX expressions such as `userrelationship` and `crossfilter`. These expressions allow the data engineer to activate a relationship, modify a relationship direction, apply a virtual relationship, or otherwise change the filter context for the duration of the calculation.

Consider a sales model where `'Sale'` has both `[Order date]` and `[Delivery date]`.

Example structure of `'Calendar'`

Date	Month name
2024-01-15	January
2024-01-20	January
2024-02-05	February

Example structure of 'Sale'

Sale ID	Product SK	Order date	Delivery date	Sales amount
S1001	1	2024-01-15	2024-01-20	100
S1002	1	2024-01-16	2024-02-05	150
S1003	2	2024-02-01	2024-02-06	200

Suppose 'Calendar'[Date] has an active relationship to 'Sale'[Order date] and an inactive relationship to 'Sale'[Delivery date].

A normal measure such as [Total sales amount] responds to the active relationship. If the user selects January, the measure returns sales ordered in January.

```
Total sales amount =  
sum ( 'Sale'[Sales amount] )
```

But the user may also need sales delivered in January. The data engineer can define a second measure that activates the delivery-date relationship only for that calculation.

```
Total delivered sales amount =  
calculate (  
    [Total sales amount],  
    userrelationship ( 'Calendar'[Date], 'Sale'[Delivery date] )  
)
```

With the example above, selecting January gives [Total sales amount] = 250, because S1001 and S1002 were ordered in January. The delivered-sales measure gives 100, because only S1001 was delivered in January.

The model relationship does not permanently change. The filtering path changes only while [Total delivered sales amount] is being evaluated.

Measure-defined filtering is powerful because it allows precise behaviour without permanently changing the model. It is also dangerous because the filtering rule becomes hidden inside the measure. Used well, it solves specific interaction problems. Used carelessly, it creates a model where results are technically correct but difficult to explain.

18.3 Filtering scenarios

The five filtering mechanisms above can be combined to support a wide range of interactivity goals while preserving the standard dimensional model pattern: dimensions filter facts using single-direction relationships.

The scenarios below are not exhaustive. Their purpose is to show how filtering design supports user interaction in real models.

18.3.1 Displaying unit records for a single business process

A common requirement is to show one row per business transaction.

The obvious approach is to expose the fact table. For example, if the transaction is a sale, the user might expect to drag fields directly from 'Sale'.

This is usually the wrong interface.

A better approach is to hide 'Sale' and let dimensions and measures reconstruct the transaction view.

The point of hiding the fact table is not tidiness. It is to protect the user experience. If users interact directly with the fact table, they can produce technically valid but strange results. If they interact through dimensions and measures, the model is much more likely to behave sensibly no matter which direction they approach it from.

The model setup

Suppose there is a business process represented by the 'Sale' fact table.

The setup is as follows:

- A fact table 'Sale'
- An ID dimension 'Sales ID' with [Sales order number]
- A calendar dimension 'Sales calendar'
- A business dimension 'Sales product'
- A measure [Total sales amount] that sums the sales amount in 'Sale'

Example structure of 'Sale'

Sales order number	Sales date	Product SK	Sales amount
SO1001	2024-01-15	1	100
SO1002	2024-01-16	2	150

Example structure of 'Sales ID'

Sales order number
SO1001
SO1002

Example structure of 'Sales calendar'

Sales date	Month name
2024-01-15	January
2024-01-16	January

Example structure of 'Sales product'

Product SK	Product name
1	Cake
2	Bread

Reconstructing the transaction row

The user should reach the sale through:

- 'Sales ID'[Sales order number]
- 'Sales calendar'[Sales date]
- 'Sales product'[Product name]
- [Total sales amount]

Together, these fields give the user the full sale record without exposing the fact table itself.

This works because Power BI returns only combinations of dimension values that correspond to existing rows in 'Sale' (common-fact filtering). Since 'Sales ID'[Sales order number] uniquely identifies the sales transaction, adding 'Sales ID'[Sales order number], with either 'Sales calendar'[Sales date] or 'Sales product'[Product name] downfilters 'Sales' to the transactions that exist.

Why use a measure for the amount?

'Sale' has an amount recorded in [Sales amount]. A simplistic approach is to expose 'Sale'[Sales amount] to the user as though it were an ordinary field. Instead, a better approach is to expose [Total sales amount].

Because 'Sales ID'[Sales order number] uniquely identifies one row, [Total sales amount] returns the sum of that one row in the current context—that is, the same numeric value as 'Sale'[Sales amount] at the transaction level (non-blank measure filtering).

At the sales order level, the measure returns the value of one transaction. At the product level, the same measure aggregates all matching transactions. The user does not need to choose between a transaction amount and an aggregate amount. The measure behaves correctly in both situations. For example:

Current context	Result
S01001	[Total sales amount] = 100
January	[Total sales amount] = 250

This sounds complex in the backend, but it is seamless for the user. In the ideal setup, 'Sale' and 'Sale'[Sales amount] are hidden. The user sees [Total sales amount], not both the measure and the raw fact column.

This is far superior to exposing both. At the transaction level, both may show the same number,

which encourages users to treat them as interchangeable. But in charts, aggregations, and time intelligence, only the measure behaves correctly. That inconsistency is confusing. Something appears to work in one visual, then fails in another.

With this setup, `[Total sales amount]` behaves consistently across situations. The user gets sensible results without needing to remember special cases.

Why this remains intuitive

The benefit is that the model behaves sensibly from multiple directions. The user can start with the sales order number, the date, the product, or the measure. In each case, Power BI's filtering behaviour narrows the model toward valid sales records.

First, suppose the user selects `'Sales ID' [Sales order number]`, then `'Sales calendar' [Sales date]`.

Power BI immediately gives the valid combination of sales order numbers and sales dates because the dimension values are filtered by their common fact. This returns the desired transaction record because the business key selects the correct grain.

Second, suppose the user selects `'Sales ID' [Sales order number]`, then `[Total sales amount]`.

This also gives the transaction record. This time, it is not only because of the common fact. It is also because `[Total sales amount]` returns a non-blank value for the sales order number. If there are sales order numbers without a sales amount, such as unconfirmed sales, those order numbers will not appear. For most use cases, that is the desired outcome.

Third, suppose the user selects `[Total sales amount]`, then `'Sales product' [Product name]`.

The model first returns the total sales amount for the whole model as a single number, then breaks it down by product. When the user adds `'Sales ID' [Sales order number]`, the table expands to unit records.

The experience is correct at every step.

This is the power of a properly designed dimensional model focused on dimensions and measures. Regardless of the sequence of field selection, the model produces sensible business results.

This seamless experience would not occur if the user selected the degenerate `'Sale' [Sales amount]` instead of `[Total sales amount]`. The user would first see a distinct list of sales amount values, then a confusing combination of sales amount and product, and only after adding `'Sales ID' [Sales order number]` would the table make sense.

The experience would be jarring rather than seamless.

Even this simple use case requires careful modelling work. The model must cater for a large number of possible user interaction sequences. These possibilities are handled elegantly when the data engineer focuses on dimensions and measures, and hides facts.

In this example, `'Sales ID' [Sales order number]` is a single-column business key. If the business key has multiple columns, all columns need to be used.

18.3.2 Displaying unit records for two business processes

The single-process case is difficult enough. The two-process case introduces a new problem: different business processes may share some controls but not others.

Continuing the previous example, suppose sales can be refunded.

The model contains:

- A fact table 'Sale'
- A fact table 'Refund'
- A shared ID dimension 'Sales ID'
- A shared product dimension 'Sales product'
- A sales calendar dimension 'Sales calendar'
- A refund calendar dimension 'Refund calendar'
- A measure [Total sales amount]
- A measure [Total refund amount]

Example structure of 'Sale'

Sales order number	Sales date	Product SK	Sales amount
SO1001	2024-01-15	1	100
SO1002	2024-01-16	2	150
SO1003	2024-01-17	1	200

Example structure of 'Refund'

Sales order number	Refund date	Refund amount
SO1001	2024-01-20	50
SO1001	2024-01-20	50
SO1003	2024-01-22	75

The dimensions relate to the two facts differently.

Sales and refunds share a sales order and product. They do not share the same process date. Sales occur on sales dates; refunds occur on refund dates. Sales are usually one row per order; refunds may have multiple rows per order.

Dimension	'Sale'	'Refund'	Meaning
'Sales ID'	1 → *	1 → *	Shared sales order identifier
'Sales product'	1 → *	1 → *	Product sold and later refunded
'Sales calendar'	1 → *		Date of sale
'Refund calendar'		1 → *	Date of refund

The data engineer therefore has to anticipate which process the user is trying to display. Showing sales, showing refunds, and showing sales with refunds are three different interaction problems.

This is the source of the complexity.

Some controls are shared. `'Sales ID'` and `'Sales product'` can filter both sales and refunds.

Some controls are process-specific. `'Sales calendar'` filters sales. `'Refund calendar'` filters refunds.

The model therefore behaves differently depending on what the user is trying to display:

- Sales transactions
- Refund transactions
- Sales and refund information together

18.3.3 Displaying sales

When displaying sales records, everything works as if there were only the `'Sale'` fact table.

This is because every refund comes from a sale. `'Sale'` is therefore a superset of `'Refund'`. Adding `'Refund'` to the model does not interfere with the display of sales transactions.

The superset relationship matters. If there were sales order numbers in `'Refund'` that did not exist in `'Sale'`, then selecting `'Sales calendar'` [`Sales date`] and `'Sales ID'` [`Sales order number`] would not correctly identify the list of sales transactions.

Single-direction filters also matter. If the report canvas has a filter on `'Refund calendar'`, that filter does not interfere with the presentation of sales transactions because `'Refund calendar'` filters `'Refund'`, not `'Sale'`.

This would not be true if the model used bidirectional filters carelessly. The refund date could unexpectedly affect sales visuals.

18.3.4 Displaying refunds

When displaying refund records, the user is interested in [`Total refund amount`], but they may also need the sales context, such as `'Sales ID'` [`Sales order number`] and `'Sales product'` [`Product name`].

While the outcome—displaying refund—is the same, the user may come from two angles: starting with refund information, or starting from sales information. Both need to work. In a properly designed dimensional model, that is the case.

If the user selects [`Total refund amount`] first, the model returns the total refund amount for the whole dataset (non-blank measure filtering). Adding sales attributes breaks that number down. Adding the relevant ID dimension returns unit-level records.

If the user selects sales attributes first, Power BI initially displays the list of sales orders and products, regardless of whether they were refunded. When the user adds either `'Refund calendar'` [`Refund date`] or [`Total refund amount`], the table filters down to refund transactions. The former because of common-fact filtering, the latter because of non-blank measure filtering. And both happens because those fields are linked to `'Refund'`.

Whether the user starts with the sales attributes or the refund-specific fields, the dimensional model gives a sensible path.

However, the situation is more complicated than it first appears.

Suppose a sales order of \$100 (S01001 in the example) is refunded in two partial refunds of \$50 on the same day. Because there is no primary key for refunds, the refund for that sales order may appear as a single transaction with `[Total refund amount] = 100`, rather than two rows of \$50.

For many scenarios, showing the total refund amount, regardless of the partial breakdowns, may be sufficient. In some cases, it is not.

When the user must see each refund action as a separate transaction, the data engineer needs a key that distinguishes each refund transaction.

One option is to create `[Refund SK]`, a unique integer for each refund, and expose it as 'Refund ID' `[Refund SK]`. This is usually not preferable:

- `[Refund SK]` is high cardinality, increasing model size through the 'Refund ID' dimension and relationship columns.
- `[Refund SK]` has no natural business meaning and creates an additional learning burden for users.

A better approach is often to create `[Refund number]`, a sequence number for each refund action within a sales order. The composite key becomes `[Sales order number]` and `[Refund number]`.

`[Refund number]` can be exposed through 'Refund ID' `[Refund number]`. This is a partial ID dimension because it supplies one part of the full key needed to identify a refund.

This dimension is low cardinality, has minimal model-size impact, and is natural for users.

From the user's perspective:

- `[Total refund amount]` gives the refund value for the transaction.
- Adding 'Refund ID' `[Refund number]` breaks the sale into separate refund actions.

This is natural at every step.

The use of `[Total refund amount]`, rather than the degenerate 'Refund' `[Refund amount]`, is essential. Suppose the user selects the raw value 'Refund' `[Refund amount]` instead. In the absence of a refund primary key, Power BI may display one row of \$50 for the sales order rather than the correct total of \$100.

This is another example of why degenerate numeric columns are problematic and why fact tables are better hidden.

18.3.5 Displaying sales and refunds together

The user may want to see sales and refund business processes simultaneously. This is not one problem. There are three common intentions:

- Show the sales and refund processes side by side.
- Show sales transactions with supplementary refund detail.
- Show refund transactions with the original sales amount.

Each intention has a different grain and therefore a different filtering problem.

Side-by-side visuals

Displaying sales and refunds side by side on the same report page is the simplest case.

Suppose there is a table visual for sales with:

- 'Sales ID' [Sales order number]
- 'Sales product' [Product name]
- 'Sales calendar' [Sales date]
- [Total sales amount]

And a table visual for refunds with:

- 'Sales ID' [Sales order number]
- 'Sales product' [Product name]
- 'Refund calendar' [Refund date]
- [Total refund amount]

These two tables work well because each visual keeps its own business-process grain.

Shared dimensions behave well.

If there is a slicer on 'Sales product' [Product name], both visuals filter down to the selected product.

If there is a slicer on 'Sales ID' [Sales order number], the user can look up a specific sales order and retrieve details of both the sale and any refund.

A user can also click on a row in the sales table and cross-filter to any refund. This works because 'Sales ID' is a conformed dimension for both 'Sale' and 'Refund'.

For the business user, this is extremely convenient. It is one reason the ID dimension is one of the most powerful dimensions and an indicator of a good model.

The same experience cannot be achieved if [Sales order number] is used only as a degenerate dimension inside 'Sale'. If degenerate dimensions are used, the filtering experience becomes frustrating because the shared point of interaction is missing.

Process-specific dimensions are harder.

The user experience is seamless when operating on conformed dimensions such as 'Sales ID' and 'Sales product'.

It becomes more complicated with the process-specific calendar dimensions.

Users expect to slice by time. But neither calendar works perfectly in every context:

- A slicer on `'Refund calendar'` has no effect on the sales table because it does not filter `'Sale'`.
- A slicer on `'Sales calendar'` correctly filters the sales table, but in the refund table, it returns refunds for products sold on that date—not refunds that occurred on that date.

In simple examples, users may interpret this correctly. In real-world scenarios with multiple dates, this nuance is often difficult to understand. In the worst case, it silently misleads users.

One solution is to create a role-playing `'Reporting calendar'` that links `[Reporting date]` to both `'Sale'` `[Sales date]` and `'Refund'` `[Refund date]`.

When used as a slicer, both tables return rows for the selected date. This works because the selection filters the sales and refund facts directly, and the visuals then filter out combinations without matching fact rows.

The chain of filtering behind the scenes creates a seamless experience for the user.

Sales with refund details

If the user wants a single table of sales transactions with refund amount, the visual should remain at the sales grain.

Use the sales table columns and add `[Total refund amount]`.

The table visual then has two measures:

- `[Total sales amount]`
- `[Total refund amount]`

The result is:

- All sales transactions display with `[Total sales amount]`.
- Rows with refunds show a value in `[Total refund amount]`.
- Rows without refunds show a blank refund amount.

This works because Power BI displays rows where any measure is non-blank.

In addition, if a refund exists without a sales amount, the row stands out with a blank `[Total sales amount]` and a non-blank `[Total refund amount]`.

However, the user cannot simply add `'Refund calendar'` `[Refund date]` to this sales-grain visual.

The technical reason is that `'Refund calendar'` relates only to `'Refund'`, not `'Sale'`. The business reason is that one sale can have multiple refunds on different days. It is therefore not logically correct to add refund date to a visual whose intent is one row per sale.

If displaying refund dates is necessary, the data engineer can create a measure such as `[Sales refund dates]` that returns the unique refund date if there is one, or concatenates distinct dates if there are several.

Refunds with sales details

The reverse scenario does not mirror cleanly.

If the user creates a refund table and adds `[Total sales amount]`, Power BI may return incorrect results by cross-joining sales and refunds. This happens because 'Refund calendar' `[Refund date]` relates only to 'Refund' and does not filter 'Sale'. There is no valid relationship path for `[Total sales amount]` to work naturally.

Users may expect to drag `[Total sales amount]` into the refund table and see the original sales amount. Instead, they may get a cross-join of rows.

This is one of the situations where the default dimensional model is not intuitive out of the box and requires additional tuning.

There are several solutions while staying within dimensional modelling:

- Define `[Total sales amount]` to behave differently depending on context using DAX functions such as `treatas`.
- Denormalise the sales amount into 'Refund' as a value such as `[Sales amount before refund]`, then expose a specific measure for that use case.

The first option gives the most seamless experience for the user, but adds complexity and performance cost to a core business measure. It is often not advisable to complicate a central measure for a narrow scenario.

The second option adds a more narrowly defined measure, such as `[Sales amount before refund]`. This is more explicit, easier to explain, and usually keeps DAX complexity lower overall.

The measure is unlikely to be useful everywhere, but it is useful in the refund context.

18.3.6 Displaying unit records for three or more business processes

The techniques above rely on conformed dimensions and the existence of fact rows to filter. The ID dimension plays a special role.

This does not scale neatly.

With three or more facts, it becomes difficult to know which combinations of dimension values still have at least one row in any fact. Managing multiple ID dimensions is also complicated because some are conformed across some facts but not others. Many processes also lack a ready business key.

A practical solution is to control display with measures.

The data engineer can create one display measure per business process:

- `[Display sales transaction]`
- `[Display refund transaction]`
- `[Display ... transaction]`

Each measure returns 1 when there is at least one row in its fact under the current filter context.

These measures should be placed in a display folder named `Display unit records`.

A refinement is to return a value only when the relevant ID is in scope. For example, `[Display sales transaction]` may return a value only when `'Sales ID'` `[Sales order number]` is being used. This can be done with functions such as `ISINSCOPE`.

Adding one of these measures to a table visual filters dimension values to those with a matching fact row. This is non-blank measure filtering.

The user can hide the helper column by renaming it and shrinking its width to zero.

An alternative is to use the measure as a visual-level filter. This is convenient but may have a noticeable performance penalty on large facts.

This has a learning curve, but it can be helpful. A model with more than four or five business processes is already likely to be complicated. For a user dealing with this level of complexity, the additional complexity of display measures is minimal. In fact, an explicit menu of display measures can clarify which business process the user is trying to display.

18.3.7 Cascading filters

Cascading filters refer to the behaviour where selecting a value in one filter narrows the choices in another so that only valid options remain.

For example, if a report page has a slicer for `'Sales product'` `[Product name]` and another for `'Sales calendar'` `[Sales date]`, then choosing a product should limit the dates to dates where that product was sold. Choosing a date should limit products to products sold on that date.

This feels obvious to users, but it is not always automatic.

Relationship filtering requires a chain of relationships from one table to the other. In this case, there is no direct relationship path from `'Sales product'` to `'Sales calendar'`. Both filter `'Sale'`, but neither directly filters the other.

Common-fact filtering and non-blank measure filtering usually apply inside a single visual where dimension values and measures are evaluated together. They do not automatically make two slicers filter each other.

The solution is to use visual-level filtering. For example, a measure such as `[Has sales]` can return 1 if there is at least one row in `'Sale'` under the current filter context. The slicer can then use `[Has sales]` as a visual-level filter.

A simple measure might be conceptually:

```
Has sales =  
if (  
    countrows ( 'Sale' ) > 0,  
    1  
)
```

The slicer for `'Sales product'[Product name]` can then be filtered to values where `[Has sales]` is not blank. The same can be done for `'Sales calendar'[Sales date]`.

The solution relies on a common fact table and a measure that passes information back to the dimension against the default filter direction.

This is a case where the dimension is on the receiving end of the button-and-effect relationship. The data engineer can support this without using bidirectional relationships across the whole model.

18.3.8 Aggregating dimension values

Cascading filters aim to filter one dimension by another. Sometimes the goal is to aggregate dimension values by another dimension.

Following the sales and refunds example, suppose the user wants to display a concatenated list of distinct products refunded on each refund date.

If `[Product name]` were already included in `'Refund'` as a raw value, the measure could take the distinct values in `'Refund'[Product name]` and use `CONCATENATEX`.

However, this approach is often not available or desirable.

For example:

- `'Sales product'[Product name]` may be linked to `'Refund'` through `[Product SK]`.
- The user may want to aggregate another column from `'Sales product'`, such as `[Product type]`.
- It is not practical to include all such columns in the fact table.

An inexperienced data engineer might denormalise the required column into the fact table, or even precompute string concatenation in the data layer so the column can be used like a degenerate dimension.

Sometimes this is pragmatic. In general, it is not desirable. It is heavy work to modify a large fact table for every dimension column that users may want to aggregate. Pre-aggregation is also limited because it cannot respond flexibly to user filter context.

The better solution is to notice that the information is already in the model. It only needs to be retrieved under the right filter context.

Measure-defined filtering provides the answer.

The DAX pattern is to concatenate values from `'Sales product'[Product name]` while using `calculate` and `crossfilter` to temporarily allow `'Refund calendar'` to filter `'Sales product'`.

Conceptually:

```
Refunded products =  
calculate (
```

```
concatenatex (
    values ( 'Sales product'[Product name] ),
    'Sales product'[Product name],
    ", "
),
crossfilter ( 'Sales product'[Product SK], 'Refund'[Product SK], both )
)
```

The exact DAX depends on the model, but the principle is the same: temporarily change the filter path for this calculation only.

This avoids compromising the whole model with bidirectional relationships or denormalised columns. The model remains clean, and the special behaviour is contained inside the measure that needs it.

18.3.9 Filter by having

Sometimes the user does not want to filter a dimension by the value it currently has. They want to filter by whether the entity has ever had an attribute.

This is filter by having.

Filter by having means: filter the entity by whether it has, or once had, a matching attribute, rather than filtering only the rows where the attribute currently appears.

Consider an organisation’s HR system. The model contains:

- A dimension 'Employee' that stores details such as name, email, date of birth, and role.
- 'Employee' is Type II, so it stores historical changes to employee attributes.
- The key columns are [Employee ID], [Start datetime], and [End datetime].
- [Employee SK] is used as the surrogate key for Power BI relationships.
- An end-of-month fact table 'Employee end of month' stores the employee’s attributes at the end of each month.

Example structure of 'Employee'

				Employee	
Employee SK	Employee ID	Start date	End date	name	Role
1	E1001	2023-01-01	2023-06-30	Alice Chen	Data analyst
2	E1001	2023-07-01	9999-12-31	Alice Chen	Senior analyst
3	E1002	2023-01-01	9999-12-31	Ben Smith	Finance officer

Example structure of 'Employee end of month'

Employee SK	Period end date	FTE
1	2023-06-30	1.0
2	2023-07-31	1.0
2	2023-08-31	1.0

Employee SK	Period end date	FTE
3	2023-08-31	1.0

'Employee' is a dimension filtering the fact 'Employee end of month'.

Dimension	'Employee end of month'	Meaning
'Employee'	1 → * via [Employee SK]	Point-in-time employee attributes

In this model, filtering 'Employee' [Role] filters the fact through [Employee SK]. This is correct for point-in-time reporting. It returns the employee-month rows where the selected Type II employee row is active.

That is not always the desired search behaviour.

The user wants to search an employee's history by personal attributes such as name or role.

Since 'Employee' is Type II, a direct search returns only the end-of-period states that match the selected point-in-time attribute values. If the user filters 'Employee' [Role] to **Data analyst**, the model returns only rows where the employee was a data analyst. In the example above, it returns Alice's June row, but not Alice's later history as a senior analyst.

That is correct Type II behaviour, but it is not the desired search behaviour.

The desired behaviour is different. The user wants to find employees who have ever had **Data analyst** as a role, then return their full history.

New data engineers often try to solve this by restructuring the fact table. This leads to complex code and confusing artefacts.

A simpler solution is to note that [Employee ID] is the identifier of the "true" employee across time, and add a search dimension keyed on this column.

Create 'Employee search' as a duplicate 'Employee'. It relates to 'Employee' on [Employee ID] in a snowflake configuration. The relationship is many-to-many and single direction.

Example structure of 'Employee search'

Employee ID	Employee name	Role
E1001	Alice Chen	Data analyst
E1001	Alice Chen	Senior analyst
E1002	Ben Smith	Finance officer

The revised model looks like this.

Dimension	'Employee'	'Employee end of month'	Meaning
'Employee search'	* → * via [Employee ID]		Search by any historical employee attribute

Dimension	'Employee'	'Employee end of month'	Meaning
'Employee'		1 → * via [Employee SK]	Point-in-time employee attributes

Using 'Employee search', the user can look up any historical attribute of the employee, retrieve [Employee ID], and return all rows in 'Employee' that match that ID.

When the user selects Data analyst in 'Employee search' [Role], the model first finds [Employee ID] = E1001. It then filters 'Employee' to all rows for E1001, not only the row where the role was Data analyst.

The user searched by a historical attribute, but the model returned the full entity history.

Under this view, 'Employee search' and 'Employee' work together as one logical dimension that acts as a search interface for the model.

This pattern generalises. The data engineer can add 'Compliance search', 'Qualification search', or other derived attribute search dimensions as additional branches of the snowflake.

From the user’s perspective, these are search tables that allow searching employees by their history.

Unlike tampering with fact tables, this solution is non-destructive. The fact table has not changed. The ordinary point-in-time path remains intact. The search dimension adds a new interface path for a different user intention.

18.3.10 Dynamic Type I

A common workforce reporting request sounds like this: *“When I choose August, show me the last 12 months using the organisation structure as at August.”*

This sounds simple, but it is not the default behaviour of a Type II dimension.

Type II dimensions show an entity at its point-in-time attribute. Type I dimensions show the latest attribute value. Neither meets the user intention. What the user would like is Dynamic Type I—latest attribute value as at a user-selected time.

Consider employees and organisation units. The fact table 'Employee end of month' returns employee metrics at the end of each month. The dimension 'Organisation unit' is Type II because organisation structures change over time.

Example structure of 'Organisation unit'

Team unit SK	Team unit ID	Start date	End date	Team name	Group name
1	T1001	2023-01-01	2023-06-30	Data engineering	Group A
2	T1001	2023-07-01	9999-12-31	Data engineering	Group B
3	T1002	2023-01-01	9999-12-31	Finance operations	Group A

Team unit SK	Team unit ID	Start date	End date	Team name	Group name
--------------	--------------	------------	----------	-----------	------------

Example structure of 'Employee end of month'

Team unit SK	Team unit ID	Period end date	Commencements
1	T1001	2023-06-30	2
2	T1001	2023-07-31	1
2	T1001	2023-08-31	1
3	T1002	2023-08-31	1

In the ordinary Type II model, 'Employee end of month' relates to 'Organisation unit' through [Team unit SK].

Dimension	'Employee end of month'	Meaning
'Organisation unit'	1 → * via [Team unit SK]	Point-in-time organisation hierarchy

In the example, Team T1001 belonged to Group A in June, then moved to Group B in July.

If the user selects August and asks for commencements in the last twelve months, they may expect all activity for Team T1001 to appear under Group B, because that is the organisation structure as at August. The correct answer is a total of 4 commencements.

The ordinary Type II relationship does not do that. It reports each fact row using the organisation structure that was valid at the time of the row. June remains under Group A; July and August appear under Group B. This returns 2 commencements, the incorrect result.

Thus, Dynamic Type I changes the lens. It says: use the selected reporting date to decide which organisation-unit row is current, then view the relevant historical facts through that selected-date structure.

Solving Dynamic Type

The problem is notoriously complex to solve by amending the fact table. It requires associating the organisation unit's full history with each fact row. That approach is complex, fragile, and expensive.

The key is to recognise that the model already has the history in 'Organisation unit'. The information is already in the model, and what's missing is to surface it. The better solution is to surface it at query time.

There are two steps.

First, broaden the relationship for evaluation.

The problem with the existing relationship is that it is too "narrow"—locking the dimension into a smaller set of data than intuitive to business sense. The first step is to "unlock" this

relationship between the organisation unit to the activity rows that are relevant to it.

This means relating each fact row not only to a single point-in-time row in 'Organisation unit', but to all rows for that [Team unit ID]. This requires a many-to-many, single-direction relationship on [Team unit ID]. Each fact row now has the full history of its team available during calculation.

The relationship becomes:

Dimension	'Employee end of month'	Meaning
'Organisation unit'	* → * via [Team unit ID]	Relates all rows based on the persistent team unit identity

Second, select the valid row at query time.

The consequence of the relationship is that, though the data is available in the filter context, too many rows are selected for display. We now need to narrow this again.

In the measure, we take the latest date in the current context and filter 'Organisation unit' to the row whose validity covers that date:

```
[Start date] <= selected date < [End date]
```

For a given [Team unit ID], this should return one row because the primary key is [Team unit ID] and [Start date].

Under this filter, the table behaves like a one-to-many join for that evaluation. The measure returns values only for the valid organisation-unit row. Other rows go blank and drop out through non-blank measure filtering.

For August 2023, T1001 resolves to the row where [Group name] = Group B. The measure can then report prior-period activity using the August 2023 organisation view.

This creates a problem. The original 'Organisation unit' no longer behaves as before. This could be problematic. If the original behaviour needs to be preserved, there are two broad implementation options.

Option A—Keep one dimension and switch in measures

The first option is to do the switching in the measure.

In this scenario, we keep the normal one-to-many, single-direction relationship on [Team unit SK], and then add a many-to-many, single-direction relationship on [Team unit ID]. It remains dormant by default.

In measures that need dynamic Type I, deactivate the [Team unit SK] path and allow the [Team unit ID] path to apply for the calculation. Then filter to the valid row as above.

In this implementation, 'Organisation unit' works as Type II by default, and the user chooses the dynamic Type I perspective through measures.

Option B—Add a new dimension

The second option is to switch through dimensions. This is done by duplicating 'Organisation unit' as 'End of period organisation unit', and adding [Team unit ID] to 'Employee end of month'.

The fact now carries:

- [Team unit SK] for the normal relationship to 'Organisation unit'
- [Team unit ID] for the many-to-many relationship to 'End of period organisation unit'

The revised evaluation model is therefore:

Dimension	'Employee end of month'	Meaning
'Organisation unit'	1 → * via [Team unit SK]	Ordinary Type II path
'End of period organisation unit'	* → * via [Team unit ID]	Dynamic Type I evaluation path

Measures that need dynamic Type I behaviour can be wrapped.

For example, rename [Employee commencements] to [_Employee commencements]. Then define [Employee commencements] so that it evaluates [_Employee commencements] inside calculate, filtering 'End of period organisation unit' to the valid row as at the latest date in context.

In this option, the user chooses perspective by choosing the dimension:

- 'Organisation unit' gives Type II.
- 'End of period organisation unit' gives dynamic Type I.

Under both options, the original dimension keeps its normal behaviour. Existing measures continue to work. Both options are non-destructive.

The distinction is whether the user chooses Type II or dynamic Type I through different dimensions, or through different measures.

The important point is that Dynamic Type I does not rewrite history in the fact table. It changes the reporting lens at query time.

18.4 Buttons and effects

The take-away from above is not exhaustive list of filtering techniques. That would be a book in itself.

Their purpose is to illustrate a philosophy of seeing Power BI as an interface that users touch to produce effects.

In this philosophy, every table tends to have one of two roles:

- Tables that define the interface as buttons to click, usually dimensions
- Tables that define the content that drives the effect, usually facts

A good model keeps these roles separate.

Every table should have one clear purpose. Tables should not mix interface and content unless there is a deliberate reason. This is why an indicator of quality is that fact tables are hidden.

The first scenarios showed how dimensions present information from facts through the difficult case of displaying transaction records. They illustrate that fact tables, which are the natural focus for new data engineers, are not the artefacts to expose to users.

Trying to expose fact tables and degenerate dimensions causes issues. Dimensions should act as the entry point to the data.

The same philosophy explains why degenerate dimensions and bidirectional relationships often frustrate users. They blur the line between tables that act as buttons and tables that create effects.

The examples of cascading filters and aggregating dimension values show that the data engineer can handle cases where a dimension is on the receiving end of a button-and-effect relationship without compromising the model with degenerate dimensions or broad bidirectional relationships.

The final examples generalise the idea of a dimension to a set of tables. A dimension is not simply a lookup table. If the user sees it as intuitive and gets the desired outcome unambiguously and quickly, that is what matters.

In this view, a dimension's chief purpose is to be the user's interface to the model.

This is the philosophy of separation of concerns from software design. It maintains a clear distinction between the interface layer and the content layer.

Power BI supports this through the five filtering mechanisms described at the start of the chapter. This perspective pushes the data engineer to see a data model as software and to design a clean interface that is business-centric and intuitive for the user.

Key ideas.

Filtering design is what makes a Power BI model feel intuitive—or frustrating.

A self-service model is not a puzzle for the user to solve. The correct answer should not merely be reachable; it should be reachable through likely paths of interaction.

Filtering is the mechanism that turns model structure into user experience.

Dimensions usually provide the buttons: the things users select, slice, search, group, and compare.

Facts and measures provide the effects: the activity, state, and values that respond.

Power BI filtering commonly occurs through relationships, common facts, non-blank measures, visual-level filters, and measure-defined filtering.

Degenerate dimensions and broad bidirectional relationships often blur the distinction between buttons and effects.

Good filtering design anticipates how users will interact with the model and makes sensible behaviour occur naturally.

Designing measures

Measures compress facts into business meaning, then unpack that meaning in user context.

19.1 Measures as compression and re-expression

A fact table may contain thousands, millions, or billions of rows. A measure compresses those rows into a business expression, then unpacks that expression again in user context.

A good measure therefore does not merely calculate correctly. It states clearly what business meaning is being compressed, and it behaves predictably as that meaning is re-expressed through the user's selected dimensions.

Power BI's implicit calculations can undermine this design. They encourage users to treat raw columns as ready-made measures, even when those columns do not express a clear business answer. For this reason, **Discourage implicit measures** should usually be turned on.

This chapter covers three areas:

- signs of good measures;
- interface roles of measures;
- technical patterns for building measures.

The next chapter, [Measure of measures](#), introduces a pattern for managing structured families of measures.

19.2 Signs of good measures

There are two signs of good measures:

- business centricity;
- technical simplicity.

19.2.1 Business centricity

A measure is not merely a calculation. It is a business expression made available to the user for unpacking.

Every aspect of a measure should reflect business meaning. This includes its definition, name, description, display folder, and placement alongside related measures.

Alignment to business reality

A measure should align with real-world events.

A common mistake is to count a database key and assume that the result is a meaningful business metric.

For example, a data engineer might define `[Inspection count]` as a distinct count of `[Inspection ID]` in the `'Inspection'` fact table. That may be fine if `[Inspection ID]` corresponds to a real inspection. But in many systems, `[Inspection ID]` is only a system-generated record key. It may exist for retrieval, workflow, or database convenience.

If the system later changes so that one real inspection is split into ten records, the measure changes even though inspection effort has not changed. Conversely, if ten records are merged into one, the measure falls even though no real-world activity has disappeared.

That is a measure misaligned with reality.

A better measure quantifies something with real-world business meaning. Depending on the scenario, the data engineer may measure:

- inspection hours;
- entities inspected.

The right choice depends on the business question.

Even `inspection` itself can be unreliable as the basis for a measure. If new inspection types are introduced that are much lighter or heavier than the old ones, then `[Number of inspections]` may drift in meaning over time.

By contrast, `[Total inspection hours]` remains close to concrete reality. An hour is still an hour, even when new inspections get added.

This does not mean `[Number of inspections]` is always wrong. The point is that the data engineer should understand what the measure is really compressing, and where possible, stay closer to concrete reality than abstraction.

Business specificity

Measures should be specific to the business question they answer.

A common misstep is to create a single generic measure and expect users to combine it with filters to answer every question.

For example, the model may contain a generic measure:

```
Employee count =
```



```
distinctcount ( 'Employee end of month'[Employee ID] )
```

The data engineer may then expect users to filter manually for active employees, separated employees, employees on leave, current employees, and other cases.

This turns the model into a puzzle. Users must remember which filters to apply, which dates matter, and which status values are valid for each business question.

A better model presents specific measures that answer common business questions directly.

For example:

- **[Active employees end of period]**—employees with an active employment status at the end of the selected period.
- **[Separated employees end of period]**—employees with a separated employment status at the end of the selected period.
- **[Current employees]**—the latest value of **[Active employees end of period]**.
- **[Current employees on long service leave]**—current employees who are on long service leave.

These measures are not merely variants of **[Employee count]**. They are specific business expressions that make common questions directly available.

A technical measure such as **[_Employee count]** and **[_Employee count end of period]** may still be useful internally. If it is not useful to users directly, it should be hidden to support business-facing measures without cluttering the interface.

Names

Measure names should be explicit, grammatical, and able to stand alone.

Measures can be dragged anywhere on the canvas. Their names should carry enough context to remain meaningful outside their original folder or visual.

Prefer:

```
[Active employees end of period]
```

over:

```
[Count]
```

Avoid non-standard abbreviations. Saving a few characters does not help the user or the next engineer.

Names should also support search. Users often find measures by typing keywords in the field list. If the business calls something **commencements**, the measure name may include **commencements**. Anticipating the user search should be part of the measure design.

Descriptions

Descriptions should explain business meaning.

The description should not simply restate the DAX. Users are not asking for implementation trivia, nor should they be expected to accept complex logic without explanation. They want to know what the measure means and whether they can trust it.

For simple measures, a description may be short:

```
Counts employees with an active employment status at the end of the selected  
↪ reporting period.
```

For more complex measures, near-pseudocode in business language may be appropriate:

```
Counts employees who were active at the end of the selected period and whose leave  
↪ status indicates long service leave. Uses the latest reporting period in the  
↪ current user context.
```

The description is part of the interface. It is often the first explanation the user sees, and an opportunity to build trust.

Display folders

Display folders help measures appear in business-facing groups.

Measures can be grouped by business process, such as:

- Workforce
- Inspections
- Sales
- Refunds

Technical or report-specific measures can be placed in folders such as:

- Dashboard
- Display unit records

Proximity to related measures

The meaning of a measure is influenced by nearby measures.

Related measures should be named so that they appear near each other.

For example:

- [Active employees end of period]
- [Separated employees end of period]

- [Current employees]
- [Current employees on long service leave]

The presence of [Separated employees end of period] reinforces the meaning of [Active employees end of period]. The presence of [Current employees] clarifies that [Active employees end of period] is a point-in-time measure.

These compare-and-contrast relationships help users form a clearer mental model of the data.

There are two practical forms of proximity:

- alphabetical placement within a display folder;
- keyword-based lookup in the field list.

A data engineer can take advantage of the proximity effect during measure naming and placement.

19.2.2 Technical simplicity

Measures should be technically simple.

This does not mean every measure must be trivial. It means complex business logic should usually be pushed upstream into the data pipeline.

A measure should aggregate, filter, compare, or display information that is already well prepared.

There are two important foundations for technical simplicity:

- precomputing complex information;
- preparing binary flags.

Precomputing complex information

Complex business logic should usually be computed in the data layer.

This frees measures from carrying too much responsibility. A measure should not have to reconstruct a complicated business state every time a user clicks a slicer.

For example, suppose [Is on long service leave] requires analysing leave transactions, effective dates, overlapping periods, and employee status. That logic should usually be calculated in the pipeline and stored in a fragment such as `HR.EmployeeLeave` or a related reference table.

Then the measure can remain simple:

```
Current employees on long service leave =
calculate (
    [Current employees],
    keepfilters ( 'Employee leave'[Is on long service leave] )
)
```

The complexity has been moved to the data layer, where it can be tested and reused.

Storytelling dimensions are an important example of this principle. When complex detail has been reduced into business-facing categories, paths, summaries, or special cases, measures can refer to those prepared structures instead of repeating the interpretation in DAX.

Precomputation can be overdone. If the data engineer precomputes too much, the model may lose interactivity. A pre-aggregated string may no longer respond correctly to filters. An average stored at the wrong grain may produce wrong results when aggregated again.

The rule is not “precompute everything.” The rule is:

Precompute business interpretation when doing so preserves interactivity and simplifies the model.

The data engineer should understand the filtering behaviour of Power BI well enough to know which logic belongs in the pipeline and which logic belongs in a measure.

Preparing binary flags

Binary flags are true-or-false columns that express business logic clearly.

Their most common place is in reference tables and dimensions. For example, an 'Employee leave' dimension may contain [Is on long service leave] to indicate whether the employee is on long service leave at the end of the reporting period.

Binary flags simplify measures.

Instead of writing a measure that filters text values directly:

```
Current employees on long service leave =  
calculate (  
    [Current employees],  
    keepfilters ( 'Employee leave'[Leave type] in { "Long service leave", "Extended  
        ↳ long service leave" } )  
)
```

The measure can use the prepared binary flag:

```
Current employees on long service leave =  
calculate (  
    [Current employees],  
    keepfilters ( 'Employee leave'[Is on long service leave] )  
)
```

The reverse can be expressed with not:

```
Current employees not on long service leave =  
calculate (  
    [Current employees],  
    keepfilters ( not 'Employee leave'[Is on long service leave] )  
)
```

```
)
```

Binary flags have two advantages.

First, they are syntactically checked. If a developer mistypes `Long service leave` as text, the formula may still be valid but return the wrong result. If the developer mistypes the column name `[Is on long service leave]`, the formula fails visibly.

Second, binary flags make the business logic readable. The measure says what it means.

Binary flags also participate cleanly in `and` and `or` logic. For example:

```
Current employees on leave or acting =  
calculate (  
    [Current employees],  
    keepfilters (  
        'Employee status'[Is on leave]  
        || 'Employee status'[Is acting]  
    )  
)
```

Using binary flags makes measures faster to write, easier to maintain, and easier to understand. The business logic has been named and surfaced in the model.

19.3 Interface roles of measures

Measures do not all play the same role in the user experience. Some measures answer business questions directly. Some display contextual information. Some control whether rows appear. Some support the presentation of a dashboard.

These are interface roles. There are four common roles:

- aggregating measures;
- dimensional measures;
- filtering measures;
- dashboard measures.

19.3.1 Aggregating measures

Aggregating measures are the measures most people think of first.

Examples include:

- [Total sales amount]
- [Total inspection hours]
- [Average inspection duration]
- [Active employees end of period]
- [Median time to resolve]

For measurable facts, aggregating measures may use functions such as `sum`, `countrows`, `distinctcount`, `averagex`, or `medianx`.

For end-of-period facts, they often need to identify the relevant reporting period before aggregating. For example, `[Current employees]` may evaluate `[Active employees end of period]` at the latest period in the current filter context.

Aggregating measures should usually be business-facing. They are the main way facts become visible to users.

19.3.2 Dimensional measures

A dimensional measure turns dimension values into a measure result.

This sounds unusual because dimensions are normally used for grouping or filtering. But sometimes a user wants a dimension value displayed as a measure.

For example, in [Filtering behaviour](#), the user may want to show all products refunded on a particular day. The product names come from `'Sales product'[Product name]`, but the result is displayed as a measure.

A simplified example is:

```
Refunded products =  
calculate (  
    concatenatex (  
        values ( 'Sales product'[Product name] ),  
        'Sales product'[Product name],  
        ", "  
    ),  
    crossfilter ( 'Sales product'[Product SK], 'Refund'[Product SK], both )  
)
```

This is not an aggregating measure in the normal sense. It is not summing or counting fact rows. It is retrieving dimension values under a particular filter context and turning them into display text.

Dimensional measures can be confused with dimension columns.

A dimension column such as `'Sales product'[Product name]` can be used in slicers, rows, columns, and cross-filtering.

A dimensional measure such as `[Refunded products]` cannot do the same thing. It is a displayed result. It does not provide the same interactive control.

Its chief purpose is display.

19.3.3 Filtering measures

Filtering measures exist to control what appears.

As explained in [Filtering behaviour](#), Power BI often retains visual rows where at least one measure returns a non-blank value. A measure can therefore be used as a filtering device.

Filtering measures typically return 1 or blank.

Examples include:

- [Has sales]
- [Has refund]
- [Display sales transaction]
- [Display refund transaction]

A simple filtering measure might be:

```
Has sales =  
if (  
    countrows ( 'Sale' ) > 0,  
    1  
)
```

This can be added to a visual or used as a visual-level filter to show only products, dates, or IDs that have sales under the current filter context.

Display unit-record measures are a special case. They are used to control whether transaction records should appear. They may include functions such as `isinscope` to ensure that rows appear only when the relevant ID dimension is present.

Filtering measures are technical, but they still need business meaning. Their descriptions should explain how they are intended to be used.

19.3.4 Dashboard measures

Dashboard measures support report presentation.

They may provide:

- dynamic titles;
- selected-value labels;
- data currency messages;
- conditional formatting values;
- colour rules;
- icons;
- arrows;
- warning messages.

For example:

```
Selected product label =  
if (
```

```
hasonevalue ( 'Sales product'[Product name] ),
selectedvalue ( 'Sales product'[Product name] ),
"Multiple products"
)
```

Dashboard measures often inspect user context. They commonly use functions such as `selectedvalue`, `hasonevalue`, `isfiltered`, and `isinscope`.

19.4 Technical patterns for measures

Most measures fall into a small number of technical patterns.

The first three fall under the standard case:

- base measures;
- derived measures;
- context-aware measures.

These patterns cover most ordinary scenarios.

Some measures become difficult because the model contains a more demanding semantic problem. The final part of this section discusses three advanced scenarios:

- polymorphic keys;
- embedded grain;
- unsupported relationships.

These are not measure types. They are situations where the data engineer must be especially clear about grain, meaning, and filter context.

The techniques in this section assume that the model has been prepared properly. They work best when the data engineer has preserved business keys, built useful dimensions, hidden facts, prepared binary flags, and created meaningful fragments in the pipeline.

If those foundations are missing, the same DAX patterns may become difficult or impossible to apply. A measure cannot recover a grain if the key for that grain has been discarded. It cannot express a business category if the category has never been modelled. It cannot follow a relationship path if the relevant identifier has not been preserved.

19.4.1 Base measures

Base measures interact directly with fact tables.

They are the first layer of measure design. They should usually be simple, explicit, and reusable.

For example:


```
Total sales amount =  
sum ( 'Sale'[Sales amount] )
```

```
Sales transaction count =  
countrows ( 'Sale' )
```

```
Total inspection hours =  
sum ( 'Inspection'[Inspection duration hours] )
```

For end-of-period facts, base measures often include logic to select a reporting period.

For example, a current-state measure may select the latest period in context:

```
Employees end of period =  
var latest_period =  
    max ( 'Reporting calendar'[Period end date] )  
return  
    calculate (  
        distinctcount ( 'Employee end of month'[Employee ID] ),  
        keepfilters ( 'Reporting calendar'[Period end date] = latest_period )  
    )
```

The exact formula depends on the calendar design, but the pattern is common: identify the relevant period, then evaluate the base measure there.

For annotation facts, base measures may use `concatenatex` to display detail.

For example:

```
Inspection comments =  
concatenatex (  
    'Inspection comments',  
    'Inspection comments'[Comment text],  
    " , ",  
    'Inspection comments'[Comment sequence number]  
)
```

Base measures should carry as little business complexity as possible. They should aggregate prepared facts and use prepared dimensions. For example, failed inspections should be:

```
Failed inspection count =  
calculate (  
    [Inspection count],  
    keepfilters ( 'Inspection outcome'[Is failed inspection] )  
)
```

19.4.2 Derived measures

Derived measures build on base measures.

They include ratios, rates, comparisons, rolling periods, and time-intelligence measures.

For example:

```
Inspection failure rate =  
divide ( [Failed inspection count], [Inspection count] )
```

Derived measures should usually be built from other measures rather than repeating base logic.

Population comparison measures are another important derived pattern. These measures escape part of the filter context to compare the current value with a larger population.

For example:

```
National inspection count =  
calculate (  
    [Inspection count],  
    removefilters ( 'Region' )  
)
```

The meaning should be explicit. The user should understand which filters are removed and which remain.

19.4.3 Context-aware measures

Context-aware measures inspect the user's current selection and respond accordingly.

Dashboard measures are often context-aware, but aggregating and filtering measures can be context-aware too.

Common functions include:

- `selectedvalue`
- `hasonevalue`
- `isfiltered`
- `isinscope`
- `values`
- `isblank`

A report title may use:

```
Sales title =  
"Sales for "  
    & if (  
        hasonevalue ( 'Sales product'[Product name] ),
```

```

        selectedvalue ( 'Sales product'[Product name] ),
        "all products"
    )

```

A display measure may require an ID dimension before returning a value:

```

Display sales transaction =
if (
    isinscope ( 'Sales ID'[Sales order number] )
    && countrows ( 'Sale' ) > 0,
    1
)

```

Context-aware measures are useful because Power BI users interact with the model in many sequences. A measure can inspect the current context and choose an appropriate response.

But context-aware logic should remain explainable. If a measure behaves differently in different visuals, its description should make that behaviour clear.

19.4.4 Advanced scenario: polymorphic keys

Polymorphic keys occur when the meaning of one key column depends on another column.

For example, a business process may inspect travellers and bags. Both are recorded in the same 'Inspection' fact table. A column such as [Inspection item type] has values such as Traveller and Bag. Another column, [Inspected item SK], stores the inspected item key.

The meaning of [Inspected item SK] depends on [Inspection item type].

Example structure of 'Inspection'

Inspection SK	Inspection item type	Inspected item SK
1	Traveller	1001
2	Traveller	1002
3	Bag	1001
4	Bag	1003

A simple distinct count of [Inspected item SK] gives the wrong answer. It treats traveller 1001 and bag 1001 as the same item.

The correct logic must count distinct items within each item type, then add the results.

In a well-designed model, the determining column should usually be expressed in a dimension such as 'Inspection type'. The measure can then group by the determining column.

Conceptually:

```

Inspected item count =
var inspection_per_type =
  summarize (
    'Inspection type', -- group by dimension
    'Inspection type'[Inspection item type], -- on the polymorphic resolving
    ↪ column
    "Inspection count",
      distinctcountnoblank ( 'Inspection'[Inspected item SK] ) -- aggregating
    ↪ fact per segment
  )
return
  sumx ( inspection_per_type, [Inspection count] ) -- adds up the result from each
    ↪ segment

```

If additional filters are needed, `summarize` can be wrapped inside `calculatetable` with `keepfilters`.

New data engineers often solve this by concatenating item type and item key into a single artificial column. That can work, but it changes the model to solve a calculation problem. The approach above keeps the model stable and handles the polymorphism inside the measure.

The idea is similar to SQL: group by the resolving column, aggregate each segment, then aggregate the result.

19.4.5 Advanced scenario: embedded grain

Embedded grain occurs when a table is physically stored at one grain, but contains enough repeated keys or attributes to recover another grain.

Power BI is more flexible than traditional dimensional modelling. A single fact table can sometimes contain detail rows with header-level values repeated on each row. This can be practical, but it creates a measure problem.

For example, suppose 'Inspection' records inspections at the traveller level, but also contains bag results.

Example structure of 'Inspection'

Inspection SK	Traveller SK	Bag SK	Inspection duration minutes
1	1001	501	45
1	1001	502	45
3	1002	503	30

The inspection duration for SK = 1 is repeated on bag rows. If the data engineer averages or sums directly over 'Inspection', the row may be aggregated twice.

The solution is to reconstruct the intended grain before aggregating.

For example, to calculate median inspection duration per traveller:

```

Median time to inspect traveller =
var inspection_time_per_sk =
    summarize (
        'Inspection',
        'Inspection'[Inspection SK],
        'Inspection'[Inspection duration minutes]
    )
return
    medianx (
        inspection_time_per_sk,
        [Inspection duration minutes]
    )

```

In SQL terms, this is similar to using `select distinct` on the columns that define the intended grain, then aggregating over that result.

ID dimensions can help because they preserve the keys needed to reconstruct the grain. This is another reason ID dimensions are powerful in Power BI models.

The broader lesson is that measures must know the grain of the thing being measured. If the grain is embedded inside a denormalised fact, the measure may need to recover it explicitly.

19.4.6 Advanced scenario: unsupported relationships

Unsupported relationships occur when the standard single-direction dimension-to-fact relationship structure does not naturally support the required calculation.

The answer is not always to change the model. Sometimes the data engineer can use DAX to create the required filter path for one measure.

The most useful tools are:

- `crossfilter`
- `treatas`
- `userelationship`

`crossfilter` temporarily changes relationship direction for a calculation. As shown in [Filtering behaviour](#), it can be used to retrieve dimension values through a relationship path that normally flows the wrong way.

`treatas` applies values from one table as though they filtered another table. It is powerful when the information is already in the model but no physical relationship exists.

For example, a refund table may need to retrieve the original sale amount through sales order numbers. If the default relationship path does not support the calculation, `treatas` may be used to apply the refund's sales order numbers to 'Sale' for that measure.

`userelationship` activates an inactive relationship for the duration of a calculation.

For example, suppose 'Calendar'[Date] has an active relationship to 'Sale'[Order date] and an inactive relationship to 'Sale'[Delivery date]. A delivered-sales measure can activate the

delivery-date relationship:

```
Total delivered sales amount =  
calculate (  
  [Total sales amount],  
  userrelationship ( 'Calendar'[Date], 'Sale'[Delivery date] )  
)
```

These functions are powerful because they allow specific measures to behave differently without changing the whole model.

They should be used deliberately. If many measures require complex relationship overrides, the model may need redesign. But when the exception is narrow and well understood, measure-defined filtering is often cleaner than adding confusing bidirectional relationships or distorting facts.

Key ideas.

Measures compress fact-table content into business meaning, then unpack that meaning in user context.

Good measures should be business-centric and technically simple.

Business-facing measures should answer common questions directly rather than forcing users to memorise filter combinations.

Complex business logic should usually be prepared in the data pipeline, reference data, binary flags, or storytelling dimensions.

Measures can play different interface roles: aggregating, dimensional, filtering, or dashboard-oriented.

Business keys help with difficult measure challenges by preserving recoverable meaning across time, grain, and modelling complexity.

If measure logic becomes difficult to explain, the data engineer should consider whether the model or pipeline should be improved.

Measure of measures

Sometimes the measures themselves have dimensional structure, and that structure should be modelled.

20.1 Measures as a modelled structure

The previous chapter described measures as the face of facts. A measure compresses fact-table content into business meaning, then unpacks that meaning in user context.

This chapter extends that idea.

When the measures themselves have dimensional structure, that structure should be modelled.

The main idea is that a family of measures may have business dimensions of its own. Measures may vary by process stage, metric type, or reporting perspective. If these dimensions exist only inside measure names, the structure is hidden from the model.

A measure of measures makes this structure explicit.

This should be distinguished from other Power BI approaches that look similar on the surface: create a disconnected table of measure names, let the user choose a measure, and use a **switch** measure to return the selected result. That can be useful, but it is not the main idea here.

Instead, **measure of measures** refers to the modelling pattern of making the underlying structure of measures explicit for user interaction. The **switch** measure is only one component of that pattern.

20.2 The problem

Consider a company that manages three operational stages:

- Manufacture
- Orders
- Shipping

Each stage has three operational metrics:

- Process volume
- Median time to process

- Volume processed on schedule

This creates nine base measures.

These measures have a clear two-axis structure:

- Process stage
- Metric

Process stage	Metric	Base measure
Manufacture	Process volume	[Manufacture process volume]
Manufacture	Median time to process	[Manufacture median time to process (days)]
Manufacture	Volume processed on schedule	[Manufacture volume processed on schedule]
Orders	Process volume	[Orders process volume]
Orders	Median time to process	[Orders median time to process (days)]
Orders	Volume processed on schedule	[Orders volume processed on schedule]
Shipping	Process volume	[Shipping process volume]
Shipping	Median time to process	[Shipping median time to process (days)]
Shipping	Volume processed on schedule	[Shipping volume processed on schedule]

That structure matters to business intent. A manager may want to compare process volume across stages. Another may want to see whether time to process is deteriorating across years. Another may want to view operational performance by stage and metric in a single matrix.

If the structure exists only in measure names, the user cannot interact with this structure directly. The report builder has to manually place many separate measures into visuals and repeat derived logic.

Moreover, the complexity quickly explodes. For example, if two additional concepts need to be measured:

- Percentage processed on schedule
- National comparison

Calculating percentage processed on schedule for each process stage leads to 3 additional measures and 12 total. If national versions are created for all 12, the total becomes 24.

20.3 The measure of measures pattern

The measure of measures pattern introduces four elements:

1. A measure table
2. A switch measure
3. Derived measures
4. A formatting calculation group

20.3.1 Step 1—Create a measure table

The measure table is a small table that lists the measures and annotates them with business attributes.

In this example, create a table called 'Operational metric'.

Example structure of 'Operational metric'

Operational metric SK	Measure name	Process stage	Process stage display order	Metric	Metric display order
1	Manufacture process volume	Manufacture	1	Process volume	1
2	Manufacture median time to process (days)	Manufacture	1	Median time to process	2
3	Manufacture volume processed on schedule	Manufacture	1	Volume processed on schedule	3
4	Orders process volume	Orders	2	Process volume	1
5	Orders median time to process (days)	Orders	2	Median time to process	2
6	Orders volume processed on schedule	Orders	2	Volume processed on schedule	3
7	Shipping process volume	Shipping	3	Process volume	1
8	Shipping median time to process (days)	Shipping	3	Median time to process	2
9	Shipping volume processed on schedule	Shipping	3	Volume processed on schedule	3

This table is not a fact table. It is also not a normal business dimension over transactional data. It is a dimension over the measures themselves.

With the table, the model now knows that [Manufacture process volume], [Orders process volume], and [Shipping process volume] are the same kind of metric applied to different process stages.

The display order columns are important. Alphabetical order is rarely the same as business order. For example, [Process stage] is in order of manufacture, orders, and shipping.

20.3.2 Step 2—Create a switch measure

The switch measure returns the correct base measure depending on the selected row in 'Operational metric'.

A simplified version of the measure [Operational metric] looks like this:

```
Operational metric =  
var selected_metric =  
    selectedvalue ( 'Operational metric'[Measure name] )  
return  
    switch (  
        selected_metric,  
        "Manufacture process volume", [Manufacture process volume],  
        "Manufacture median time to process (days)", [Manufacture median time to  
        ↪ process (days)],  
        "Manufacture volume processed on schedule", [Manufacture volume processed on  
        ↪ schedule],  
        "Orders process volume", [Orders process volume],  
        "Orders median time to process (days)", [Orders median time to process  
        ↪ (days)],  
        "Orders volume processed on schedule", [Orders volume processed on  
        ↪ schedule],  
        "Shipping process volume", [Shipping process volume],  
        "Shipping median time to process (days)", [Shipping median time to process  
        ↪ (days)],  
        "Shipping volume processed on schedule", [Shipping volume processed on  
        ↪ schedule]  
    )
```

The measure table provides the selection. The switch measure provides the value.

The ordinary underlying measures can be called base measures. They calculate directly from facts or from ordinary dimensional context. The switch measure is the first measure that operates over the structure of those base measures.

20.3.3 Step 3—Define derived measures

Once the switch measure exists, the data engineer can define derived measures over the structure of the measure table.

Suppose the business wants percentage processed on schedule. Without a measure of measures, the data engineer may create one percentage measure per process stage:

- [Manufacture percentage processed on schedule]
- [Orders percentage processed on schedule]
- [Shipping percentage processed on schedule]

With the measure table, the data engineer can define a more general derived measure.

For example, the model may contain these derived measures:

```
Process volume =  
calculate (  
    [Operational metric],  
    keepfilters ( 'Operational metric'[Metric] = "Process volume" )  
)
```

```
Volume processed on schedule =  
calculate (  
    [Operational metric],  
    keepfilters ( 'Operational metric'[Metric] = "Volume processed on schedule" )  
)
```

The percentage measure can then be written once:

```
Percentage processed on schedule =  
divide (  
    [Volume processed on schedule],  
    [Process volume]  
)
```

A national comparison can also be written once:

```
National operational metric =  
calculate (  
    [Operational metric],  
    removefilters ( 'Region' )  
)
```

The same measure works across process stages because [Process stage] remains in the filter context.

For example, when the process stage is filtered to **Manufacture**, [Process volume] resolves to [Manufacture process volume], and [Volume processed on schedule] resolves to [Manufacture volume processed on schedule].

When the process stage is filtered to **Orders**, the same measure resolves to the order measures.

20.3.4 Step 4—Handle formatting

A switch measure does not automatically preserve the original format of each underlying measure.

In practice, this can be handled with a calculation group or another dynamic format-string pattern.

A simplified format string expression may look like this:

```
switch (
    selectedvalue ( 'Operational metric'[Measure name] ),
    "Manufacture process volume", "#,0",
    "Orders process volume", "#,0",
    "Shipping process volume", "#,0",
    "Manufacture median time to process (days)", "#,0.0",
    "Orders median time to process (days)", "#,0.0",
    "Shipping median time to process (days)", "#,0.0",
    "Manufacture volume processed on schedule", "#,0",
    "Orders volume processed on schedule", "#,0",
    "Shipping volume processed on schedule", "#,0",
    selectedmeasureformatstring ()
)
```

The exact implementation depends on the model and the calculation group design. The important point is that dynamic measure selection also requires deliberate formatting design.

Without this step, the measure of measures may return correct numbers but display them poorly.

Note.

In practice, long **switch** measures and matching format-string expressions should usually be generated by code when this pattern is used regularly.

20.4 Benefits

The measure of measures pattern has several benefits.

20.4.1 Simplified DAX management

Instead of repeating similar logic across many measures, the data engineer can define reusable derived measures.

This reduces duplication and lowers maintenance cost. In some cases, it is the only viable solution without creating a tangle of DAX definitions or nightmare reports.

20.4.2 Explicit business structure

The measure table expresses the structure of the metrics directly.

Process stage, metric, display order, and other annotations become part of the model rather than being hidden inside measure names.

For example, a matrix can place `[Metric]` on rows, `[Process stage]` on columns, and `[Operational metric]` as the value.

Metric	Manufacture	Orders	Shipping
Process volume	12,400	11,850	11,200
Median time to process	2.1	1.4	3.8
Volume processed on schedule	11,780	10,960	9,520

The numbers are returned by different underlying measures, but the row and column structure comes from `'Operational metric'`.

This visual is difficult to build with separate measures.

20.4.3 More powerful reporting

The third benefit is powerful reporting. A visual can place `[Process stage]` and `[Metric]` on rows, `[Reporting year]` on columns, and `[Operational metric]` as the value.

Process stage	Metric	2022	2023	2024
Manufacture	Process volume	10,800	11,600	12,400
Manufacture	Median time to process	2.4	2.2	2.1
Manufacture	Volume processed on schedule	9,900	10,850	11,780
Orders	Process volume	10,200	11,100	11,850
Orders	Median time to process	1.7	1.5	1.4
Orders	Volume processed on schedule	9,300	10,100	10,960
Shipping	Process volume	9,800	10,600	11,200
Shipping	Median time to process	3.2	3.5	3.8
Shipping	Volume processed on schedule	8,950	9,350	9,520

This gives the business a compact view of many measures across time. For example, shipping volume is increasing, but median time to process is also deteriorating. Conditional formatting can then highlight deterioration or improvement.

There are many ways to use the structure expressed by `'Operational metric'`. The `[Process stage]` column can be used for small multiples in line charts, creating a series of visuals in a controlled and automated manner. It can also serve as a legend to compare metrics directly, such as identifying lags or discrepancies in `[Process volume]` between manufacture, orders, and shipping.

If each process is managed by a different team, `[Process stage]` can be used as a report page filter to tailor the view to each manager's concern.

Used appropriately, the measure of measures enables powerful reports that would otherwise be impractical or impossible.

20.4.4 Avoiding awkward fact-table unions

Inexperienced data engineers sometimes try to solve the same problem by forcing fact tables into an awkward union so that different process stages can appear as rows in a single table.

That may be appropriate in some models, but not always. If the processes are genuinely different and already have good measures, it may be better to structure the measures rather than reshape the facts.

The measure of measures allows users to operate on the measures directly. It is a form of [horizontal integration](#) when vertical integration of the underlying facts is not appropriate.

20.5 Dangers

The measure of measures pattern is powerful, but it should be used judiciously.

20.5.1 Performance

Large `switch` measures can perform poorly, especially when the selected field is not the same column used by the switch.

For example, performance may be worse when the visual filters on `[Metric]` or `[Process stage]`, but the switch branches on `[Measure name]`.

With many branches, performance should be tested in practice.

20.5.2 Dynamism for its own sake

Dynamism can be overused.

In theory, any list of measures can be placed into a switch. That does not mean it should be done.

The criterion is business expressiveness. The pattern is appropriate when the measures form a coherent structure: process stages, business lines, quality criteria, product families, or similar.

20.5.3 Hiding important information behind clicks

A dynamic report is not automatically a good report.

If users must keep clicking before they can see what matters, the report may be poorly designed. A report with ten selectable metrics is often less intuitive than a report that shows the important comparisons directly.

The measure of measures should be used to create explicitly expressive visuals, not to hide information behind unnecessary interaction.

20.5.4 Ambiguous aggregation across measures

The measure table often has a composite key. In the example above, `[Process stage]` and `[Metric]` together identify a specific measure.

If the user selects `[Metric] = Process volume` without selecting `[Process stage]`, the switch measure returns blank. This is often correct. The default behaviour prevents summing over items that should not be summed.

If aggregation across measures is required, it should be explicitly defined and tailored to the metric.

20.6 Relationship to switch measures

Many discussions of **switch** measures in Power BI focus on technical convenience: allowing users to choose a measure from a slicer, reducing visual clutter, or reusing a visual for different metrics.

Those uses can be valid, but they are not the main point here.

A measure of measures is not a slicer trick. It is a modelling technique for expressing business intent. The measure table allows the data engineer to:

- make business metrics explicit and navigable;
- build structured visuals by metric and process;
- define derived measures without duplication.

Without anchor in business meaning, dynamic measure selection can easily become frustrating. The model may become clever, but not clearer.

Key ideas.

Sometimes the measures themselves have dimensional structure, and that structure should be modelled.

Measures can themselves be modelled as a business object.

A measure of measures is useful when measures have a coherent structure, such as process stage by metric.

The important idea is not merely that a **switch** measure can change what is displayed. The important idea is that the measure family may have dimensional structure worth modelling.

The pattern uses a measure table, a switch measure, derived measures, and usually a formatting calculation group.

The measure table makes the structure of the measures explicit rather than hiding it in measure names.

Derived measures can often be written once because the process or metric structure is carried by the measure table.

The pattern should be used for business expressiveness, not dynamism for its own sake.

Aggregating across measures must be defined deliberately. Blank results are often a useful safeguard against accidental nonsense.

Interactive row level security

RLS should limit access without destroying unit-record interactivity.

21.1 Security without losing context

Row level security (RLS) in Power BI means exposing to users only the rows they are allowed to see. This is implemented through a DAX filter attached to the user's security role.

The important subtlety is that an RLS filter is not visible to DAX as an ordinary, inspectable filter. Functions such as `isfiltered()` or `isinscope()` cannot detect that RLS is active. A formula simply evaluates over the rows available to the user after security has been applied.

This makes restriction straightforward but comparison difficult. If a regional user is restricted to regional rows, then ordinary measures automatically evaluate over that regional population. A national comparison cannot be recovered by asking DAX to notice and remove the RLS filter. The wider population must be deliberately modelled.

A common solution is to summarise the wider population into aggregate tables. This can reduce disclosure risk, and it may be the right approach for highly sensitive data. However, summary tables do not preserve unit-record interactivity. They also introduce their own modelling complexity, because every supported comparison must be anticipated in the summary design.

This chapter focuses on approaches that preserve unit-record interactivity while still allowing comparison with a wider population.

This leads to two distinct tasks in implementing RLS:

- limiting what users can see;
- preserving population context.

21.2 Limiting what users can see

RLS first needs an access-control pattern. There are two common approaches:

- static roles;
- dynamic roles.

This choice determines how the permitted population is assigned to each user.

21.2.1 Static roles

Static roles use one RLS role per access category.

For example, the model may have separate roles for **North**, **South**, **West**, and **Unlimited**.

The **North** role may apply this filter to 'Region':

```
'Region'[Region name] = "North"
```

The **South** role may apply:

```
'Region'[Region name] = "South"
```

The **West** role may apply:

```
'Region'[Region name] = "West"
```

The **Unlimited** role has no filter. Users assigned to this role can see all rows allowed by the model.

This is simple to understand and easy to test. Each role has its own DAX filter, and users are assigned to the role that matches their access.

The weakness is maintenance. Static roles work when the access categories are few and stable. They become awkward when users have many different access combinations, when access changes often, or when permissions need to follow organisational hierarchy.

A clear naming convention is important. **Limited** and **Unlimited** are often clearer than **Restricted** and **Unrestricted**, because **restricted** can mean either access to restricted data or restriction from data.

21.2.2 Dynamic roles

Dynamic roles use a security mapping table and the user’s identity, usually through `userprincipalname()`.

Instead of creating one role per access category, the model has a role that looks up which rows the current user is allowed to see. The mapping table may connect users to regions, teams, programs, customers, or other secured entities.

For example, a simple user-region mapping table may look like this.

Example structure of 'User access'

User principal name	Region
alice@example.com	North
ben@example.com	South
clara@example.com	West
dana@example.com	North

User principal name	Region
dana@example.com	South
dana@example.com	West

The RLS rule can then filter the model by the regions assigned to the current user.

A DAX expression may look like this:

```
'User access'[User principal name] = userprincipalname ()
```

In practice, the filter often applies through relationships from the access table to a secured dimension or directly to the fact table.

Dynamic roles are more flexible and auditable. They allow access rules to be maintained in data rather than hard-coded into many RLS roles. They are especially useful when permissions are numerous, changing, hierarchical, or managed outside Power BI.

The trade-off is that the access table becomes part of the security infrastructure. It must be governed, tested, and documented. If the mapping logic is complex, the table should include metadata explaining why each assignment exists.

From a performance perspective, the filter should be applied as close as possible to the sensitive data. If the sensitive data resides in fact tables, the filter should ideally reach those fact tables directly. If the sensitive data resides in dimensions, the filter may apply through a snowflake relationship.

Multiple copies of the access table may be needed if filtering applies across multiple dimensions. In such cases, care must be taken to avoid conflicting filters that accidentally combine as an unintended **and** condition.

Whether access is static or dynamic, the result is the same: each user sees only their permitted rows. The harder problem is what happens when the user needs to compare those rows with a wider population.

21.3 Preserving population context

RLS removes rows from the user's visible population. If the user needs comparison against a wider population, that wider population must remain available through another modelling path.

For example, a regional team may need to compare its sales performance against national sales. The comparison may need to remain filterable by product type, sales period, customer segment, or other dimensions.

A measure such as this cannot recover rows removed by RLS:

```
National sales volume =  
calculate (
```

```
[Sales volume],
removefilters ( 'Sales team' )
)
```

`removefilters()` can remove ordinary filter context. It cannot bypass security. If RLS has already restricted the user to regional rows, then the national rows are not available to the measure.

There are two approaches to preserving population context while retaining unit-record interactivity:

- anonymous facts;
- pseudonymous dimensions.

21.3.1 Anonymous facts

The anonymous approach duplicates the fact table and removes sensitive columns and sensitive relationship paths.

For example, suppose the model contains:

- a fact table 'Sales';
- a sensitive dimension 'Sales team';
- a measure [Sales volume].

The restricted user should see only their permitted rows in 'Sales', but should also compare against the wider population.

The data engineer can create a second fact table:

- 'Sales anonymous'

This table contains the population sales rows, but removes columns and relationships that identify the sensitive team or user. The relationship to 'Sales team' is removed. Any sensitive columns are removed. The table is therefore not filtered by the same RLS path.

A population measure can then be defined over the anonymous fact:

```
Population sales volume =
sum ( 'Sales anonymous'[Sales volume] )
```

The user-facing comparison measure might be:

```
Regional sales proportion =
divide (
    [Sales volume],
    [Population sales volume]
)
```

The ordinary [Sales volume] measure reflects the user’s restricted rows. [Population sales volume] reflects the wider anonymous population. Both can still respond to safe shared dimensions such as calendar, product, or region grouping where appropriate.

For example, if both 'Sales' and 'Sales anonymous' relate to 'Calendar' and 'Product', the user can still compare their regional sales with population sales by month and product type.

The anonymous approach preserves interactivity by keeping the population fact at the grain needed for analysis, while removing the sensitive columns and relationship paths that would identify restricted entities.

When there are many pairs of user and population measures, maintaining the measures can become tedious. The measure of measures pattern can help by creating one switch measure for restricted measures and another for population measures. Derived comparison measures can then be defined over the two switch measures rather than repeated for every metric.

21.3.2 Pseudonymous dimensions

The pseudonymous approach assumes sensitive information is contained in dimensions.

For example, the sensitive information may be in 'Sales team', which contains team names, staff names, locations, or other identifying attributes.

Instead of duplicating the fact table, the data engineer duplicates the dimension rows through a union. One set of rows contains the real values. The other set contains masked values.

Example structure of 'Sales team'

Sales team SK	Sales team ID	Team name	Region	Is masked
1	T001	North Metro Team	North	false
2	T002	South Coast Team	South	false
101	T001	(masked)	North	true
102	T002	(masked)	South	true

The access table determines which doubled dimension rows each user principal may see. Since each [Sales team ID] has both a masked and unmasked row, the access table grants access to the appropriate [Sales team SK] for each user.

For example, one user may see the unmasked row for T001 but the masked row for T002. Another user may have the reverse access. A national manager may see unmasked rows for all sales teams. The access table may look like this:

Example structure of 'User sales team access'

User principal name	Sales team SK
alice@example.com	1
alice@example.com	102
ben@example.com	101
ben@example.com	2
clara@example.com	1

User principal name	Sales team SK
clara@example.com	2

This user access table filters the 'Sales team' dimension on [Sales team SK]. The RLS rule is applied to 'User sales team access' using [User principal name]. The filtered access table then filters 'Sales team' through [Sales team SK], leaving only the masked or unmasked rows the user is allowed to see.

The relationship from the fact table to 'Sales team', based on [Sales team ID], becomes many-to-many. Since [Sales team SK] has multiple values in the access table, it is also many-to-many.

The relationships can be summarised as a bus matrix.

Dimension / access table	'Sales team'	'Sales'
'User sales team access'	* → * via [Sales team SK]	
'Sales team'		* → * via [Sales team ID]

Following the previous example, these would be the view for each user.

Alice’s view

Team name	Region	Goals reached
North Metro Team	North	84
(masked)	South	57

Ben’s view

Team name	Region	Goals reached
(masked)	North	84
South Coast Team	South	57

Clara’s view

Team name	Region	Goals reached
North Metro Team	North	84
South Coast Team	South	57

In this case, each user can only see the full details of the teams they are granted access to. The fact table is unaffected because RLS is selecting visible dimension rows rather than removing fact rows directly. This means population measures can still use the same fact table through `removefilters()` where the security design permits it.

Care must be taken when masking. In some cases, the primary key itself is sensitive. For

example, if the business key is a social security number or another identifying value, it should not be exposed as the relationship key. A surrogate key or non-sensitive mapping key is required.

The pseudonymous approach is lighter than duplicating facts, but it is also riskier. Because the same underlying facts remain available, an informed user may infer identities from distinctive combinations of values.

21.4 Choosing an approach

The anonymous and pseudonymous approaches solve the same problem in different ways.

The anonymous approach duplicates the population facts and removes sensitive columns and relationship paths. It is safer and easier to reason about, but it increases model size and requires parallel population measures.

The pseudonymous approach doubles dimension rows and masks sensitive values. It keeps the model smaller and preserves the same fact table. However, because the same underlying fact rows remain available, it exposes more detail than the anonymous approach.

It is most useful when the identity of the row is not itself the main concern, but particular attributes need to be hidden. For example, a user may be allowed to see all traveller records and analyse their activity, but should not see sensitive values such as passport numbers, phone numbers, or personal identifiers.

Both approaches can preserve unit-level interactivity. This is intentional. The goal is to keep dimensional filtering, drillthrough, and detailed comparison available while limiting exposure of sensitive attributes.

This also means both approaches carry inference risk. A user may deduce identities from distinctive combinations of values, even when direct identifiers are removed or masked.

If that risk is unacceptable, the required design is no longer merely anonymous or pseudonymous RLS. It becomes a summary-table problem, with specific applications of mitigation strategies. This is outside the scope of this chapter.

The choice between anonymous population facts and pseudonymous dimensions is not purely technical. It reflects the organisation's appetite for risk, governance requirements, security obligations, and expected user behaviour.

Key ideas.

RLS should limit access without destroying analytical context.

DAX cannot detect RLS as an ordinary filter. Measures evaluate over the rows available after security has been applied.

`removefilters()` cannot recover rows removed directly by RLS. Population context must be preserved through modelling design.

RLS has two distinct tasks: limiting what users can see, and preserving population context.

Anonymous population facts preserve wider population context by duplicating facts and removing sensitive columns and relationship paths.

Pseudonymous dimensions preserve population context by masking sensitive dimension values while keeping the model interactive.

Both anonymous and pseudonymous approaches preserve unit-record interactivity, but both carry inference risk.

Quality & reliability

The foundations of trust

Quality and reliability are the foundations of trust in a data product.

22.1 Quality and reliability

Data is a fragment of reality captured by process.

Because of this, data is an [imperfect projection](#) of the business world into the data world. Data quality issues arise from the gap between the business world and this projection.

For example, key information required for decisions may not be captured. It may be captured in free-text format. It may be captured at the wrong grain.

When business users encounter these gaps, they experience a mismatch between the data product and the business world they already know. In many cases, this means they cannot trust the product for decision-making.

No amount of blaming the source system changes this reality. From the business's point of view, the product simply cannot be used.

In this perspective, one of the data engineer's jobs is to close the gap left by the projection, to the satisfaction of the decision-maker. This is how the data engineer adds to data **quality**.

Beyond the imperfections of the projection itself, the mechanical processes of a data pipeline may introduce further errors.

An example is schema change. Users may experience this when entire tables become empty or when inconsistencies arise between business processes.

On some occasions, this can be disastrous. For example, if the cost forecast has been updated to the latest month but the actual expenditure has not, the final reconciliation may contain an unacceptable discrepancy. However, there may be no obvious signs of such an error.

The issue is not with the source data, but with mechanical **reliability**: the pipeline has produced a data product whose parts are no longer aligned and has nothing to signal this divergence.

Reliability is the discipline of making the data product stable against predictable forms of breakage.

22.2 Trust requires both

Quality keeps users from guessing what the data means. Reliability keeps users from guessing whether the output worked.

When both are available, users can use the product without second-guessing. Thus, quality and reliability are the foundations of trust.

Foundation	Question	User failure mode
Quality	Does this data product represent the business reality well enough for the decision?	“This does not match the world I know.”
Reliability	Does the pipeline consistently produce the data product without hidden breakage?	“This number might be broken.”

22.3 The third principle: anticipate errors

This section introduces the third principle of data engineering: anticipate errors.

This principle follows from the nature of quality and reliability. Data is an imperfect projection of complex reality, and pipelines are mechanical systems that operate in a changing world.

Because of these changes, a trustworthy data product cannot depend on everything continuing as expected. The data engineer does not stop with what works now, but anticipate errors that may occur.

Key ideas.

The third principle of data engineering is to **anticipate errors**.

Trust is produced when a data product is fit for decision.

Quality means the data product represents business reality well enough for its intended use.

Reliability means the pipeline consistently produces the data product without hidden failure.

Quality keeps users from guessing what the data means.

Reliability keeps users from guessing whether the data worked.

Quality metadata

Without metadata, everyone is guessing.

23.1 When metadata is missing

Metadata is the place where meaning is either carried by the data product or abandoned to the user.

Data quality issues reflect the gap between the business world and the data world. Metadata helps bridge that gap by adding the interpretative context that connects the user to the business reality behind the data.

A source column called `[Valid]` may look harmless until someone has to use it.

Valid for what? Valid now, or valid at creation? Valid according to the source system, the business rule, or the reporting model? Does `false` mean expired, rejected, superseded, or never approved?

Renaming `[Valid]` to `[Is current name]` is not cosmetic. It changes the user's ability to understand the data.

This is why metadata is not an add-on task. It is part of modelling. No transformation can help a user who cannot tell what the result means.

In complex cases, writing metadata first is a way to lay out the logic in plain language before implementation.

There are three basic kinds of metadata discussed in this chapter:

Metadata	Question it answers	Danger when compromised
Names	What is this thing?	Unrecognisable
Business descriptions	What does this mean, and when is it unsafe to use?	Incomprehensible
Database keys	How does this relate to reality and to other artefacts?	Lost

Without good names, artefacts become unrecognisable. Without descriptions, artefacts become incomprehensible. Without keys, they become lost: disconnected from reality and from the other artefacts that give them meaning.

23.2 Names answer: what is this thing?

Names make artefacts recognisable.

A table, column, measure, calculation group, or display folder may be technically visible but still unrecognisable in business terms. If a user cannot recognise what business thing an artefact represents, the artefact cannot be used.

This is why names should be business-centric and written in plain language. They should not use idiosyncratic acronyms or abbreviations for the developer's convenience. There is no reason to use `[Employee cnt]` when the meaning is `[Employee count]`. If the infrastructure supports spaces in column names, they should be used.

A recognisable name can also contain useful signals. `Is` and `Has` can indicate binary columns, such as `[Is non-compliant]` or `[Has active case]`. Columns ending with `date` and `datetime` can signal precision. Terms such as `current`, `historical`, `earliest`, `latest`, `start`, and `end` should be used consistently.

These signals are helpful only when they clarify business meaning. They should not advertise technical structure at the expense of user understanding.

This is why a Power BI dimensional model should not prefix tables with `dim` or `fact`. Such prefixes describe the modelling technique, not the user-facing meaning.

Recognisability also depends on consistency. If the same business concept appears under different names across the warehouse, the user cannot tell whether the concepts are genuinely different or merely named differently.

The data engineer should therefore pay particular attention to conformed concepts. Before naming something new, check whether the same concept already appears elsewhere under a different name.

For example, when adding `Cake.RefQualityControl`, it is important to check whether `RefTesting`, `RefAudit`, or `RefInspection` are already being used to mean the same thing. If so, it may be better to reuse an existing name.

Names should also be explicit and unambiguous because ambiguity creates false recognition. The user thinks they know what the artefact means, but may be wrong.

An explicit column name means using `[Inspection first of month]` rather than `[First of month]`.

An unambiguous name means ensuring the word has only one meaning for the user. For example, does `[Last inspection date]` mean the previous inspection date or the final inspection date? Does `restricted` mean less access, or access to restricted data?

Names should also be able to stand alone. Data tables and columns can often be taken out of context. For key concepts, it may be worthwhile for names to carry their own context.

For example, `[Is non-compliant]` may be better as `[Is financially non-compliant]`. This may involve repeating some of the table meaning in the column name.

When in doubt, the data engineer should err on the side of explicitness, because it is easier to shorten a name in a Power BI visual than for a user to creatively extend it.

Another compromise to recognisability is filler language.

Words such as **type**, **group**, **summary**, and **details** are often used by developers without adding meaning. They can function like “um” and “ah” in a sentence: they fill space but obscure meaning.

If a table is called **summary**, it should usually be renamed to reflect what is being summarised. **Details** should be avoided unless it is truly a finer-grain expansion of a header row.

These principles apply just as much to code. Temporary tables and CTEs should be named with business meaning, not cryptic abbreviations like **tmp**. In complex code, column renaming should happen early, on first contact, and persist throughout transformation.

23.3 Descriptions answer: what does this mean?

Descriptions make artefacts comprehensible.

A name may tell the user what an artefact is, but the name cannot carry the whole interpretation. The user may still need to know what business reality is represented, how the value was derived, what assumptions were made, when the artefact is unsafe, and how it differs from related concepts.

This is why business descriptions are part of the meaning rather than an add-on. They enhance data quality by aligning artefacts to business reality through interpretative context.

Too often, descriptions merely restate the artefact name. For example, **Cake.Sales** may be described as “A list of cakes sold”, or **[Is non-compliant]** as “True if found non-compliant.” These descriptions do nothing more than take up storage space. At worst, they harm trust by conveying shallowness.

Like names, descriptions should be business-centric and written in plain language. But unlike names, they should be substantial.

A good description should explain:

- what business reality is represented;
- how the value was derived;
- what assumptions were made;
- when the artefact is unsafe or limited;
- how it relates to similar concepts.

A useful description carries the interpretation that would otherwise have to be supplied by tribal knowledge or guesswork.

A column such as **[Is non-compliant]** should describe what non-compliance means in business terms, not merely say “if the result code is **F**.”

Data engineers and stakeholders often slip into using system names as shorthand for business concepts. Descriptions should refer to business entities and business processes, not merely the systems that implement them.

There will be cases where system names or technical implementation details are necessary for transparency. These should be included as supplementary information, not as standalone descriptions.

For example, the description for `[Is non-compliant]` may be:

True if the transaction failed to meet the minimum regulatory requirements for importation, based on the department’s published compliance criteria. This includes both automatic and manual assessments. Transactions flagged by automated rules are included regardless of whether they were later manually overridden. This column does not include cases where compliance was not assessed. The result code is derived from the `ResultCode` field in the source system, where F indicates failure.

The same principle applies to transformation logic. The transformation should be stated in business terms. Technical statements can be included as elaboration, but not as a substitute for meaning.

For example:

This column is derived from the inspection result and officer notes. It is true if the inspection outcome was `Spoiled cake` or `Not tasty`, and the inspection was conducted by a certified officer. It excludes inspections conducted during training. The logic is implemented in the `InspectionOutcomeFlag` measure using a combination of result codes and officer certification status.

Limitations should also be framed in terms of business interpretation.

“The column can have a null value” is not a useful limitation by itself.

A better statement is:

The inspector may not record a result if there are no issues. This means it is not possible to distinguish between a successful inspection and an inspection not performed.

Clear statements of limitation are essential for transparency and reliable application of business insight. They tell the user not only what the artefact means, but when it becomes dangerous to rely on it.

This is where descriptions protect users from misuse. They do not merely explain what an artefact means in normal cases; they mark the boundary beyond which that meaning breaks down.

23.4 Keys answer: how does this relate?

Database keys include primary keys, foreign keys, and unique keys. They are also metadata.

Keys stop records from becoming lost.

As explained in [Mapping the data world](#), the primary key is what links a database record to its real-world business entity. For the user, they are the anchor between a table and the entity it claims to represent.

Moreover, a business domain is rarely three simple tables. It is usually a swamp of entities, events, statuses, and inferred relationships. Without keys, the user may see the records but have no reliable way to navigate them.

In this sense, keys are a map for the warehouse. They show which row represents which business entity, which records belong together, and how one fragment of reality relates to another.

Keys also recognise that relationships are part of meaning. The relationship between one table and another tells the user something about both tables that neither table can say in isolation.

For example, a foreign key between `Cake.Sales` and `Cake.RefProfitability` immediately signals that some sales may be profitable and some may not.

Each organisation implements keys differently. Some store them only in conceptual diagrams or third-party tools. The disadvantage is that they cannot be easily read or queried.

The best approach is to house them as standalone tables in the warehouse. This makes them visible and open to querying like every other table in the warehouse.

Ultimately, keys are not just technical constraints. They are statements of identity and relationship.

This makes them a non-negotiable part of metadata.

23.5 Metadata should be data

Data without metadata is data without context.

The primary purpose of metadata is not governance, tagging, discoverability, or data exchange. These are all important, but they are secondary effects.

Metadata should be understood in the context of the data engineer's task: shaping data in light of business intent. From this perspective, the primary purpose of metadata is to add interpretative context that aligns the data world to its underlying business reality.

Without this context, the data engineer is abandoning the user to guess.

Finally, whether metadata describes names, descriptions, or keys, it should be stored and made available as data.

Instead of recording metadata only in diagrams, specialist tools, or database constructs, it should also be available as tables. Treating metadata as data enhances the ability to distribute it, automate it, and surface it to appropriate tools through APIs.

Key ideas.

Without metadata, everyone is guessing.

Metadata gives data its interpretative context.

Without good names, artefacts become unrecognisable.

Without descriptions, artefacts become incomprehensible.

Without keys, artefacts become lost.

Metadata should be stored and treated as data.

Three approaches to data quality

When recorded data does not meet business intent, choose the right kind of intervention.

24.1 Quality means fitness for intent

Most people see data quality issues as failures of data capture to reflect reality.

That view is useful, but incomplete.

In practice, errors in the database are rarely neutral. They reflect the incentives and pressures of the process that produced the data. In general, errors tend to be biased toward the advantage of the person providing the information, or toward the path of least resistance in the business process.

Consider personal income.

Both government social services and the tax office may be concerned with underestimation of income, but the direction and consequence of error differ. For tax, a taxpayer is unlikely to complain if they accidentally report less income, because they may receive a higher tax return or pay less tax. For social services, the same person is more likely to notice an underestimate, because they may receive less support than expected.

A bank credit rater faces a different concern again. It may care more about overestimates of income, because overestimated income may lead to excessive lending risk.

The same data element—personal income—therefore has different quality implications depending on the business intent. Social services may actively manage underestimates to meet legislative requirements. Credit raters may actively manage overestimates to reduce risk. The tax office may design its controls around yet another risk profile.

All three may prefer perfect accuracy. But none takes the same approach when under constraint.

There is no accuracy without margin of error, no margin of error without a risk threshold, and no risk threshold without an articulation of business intent.

Consequently, data quality is not merely data versus reality. It is data versus business intent.

A dogmatic insistence on reflecting the real world perfectly is costly and often impractical. It may also conflict with privacy, timeliness, proportionality, or operational burden. The data engineer's job is not to pursue perfect data in the abstract, but to help the business decide what quality means for the decision at hand.

With that in mind, common data quality issues arise because digital systems and business processes were not built to honour the full intent of the business. This may be due to poor design, such as missing data type constraints. It may be due to bias in the business process, such as different incentives for reporting income. It may also be due to a mismatch of purpose, such as operational forms captured for one regulatory purpose but later analysed for another.

The data engineer therefore needs a practical set of interventions. This chapter introduces three broad approaches, with further methods for different scenarios.

Situation	Approach
Business judgement is needed	Human curation
Business intent can be formalised	Precise rules
Business intent can only be approximated	Fuzzy logic

Each approach contains recurring methods.

Human curation

Method	Use when...
Data annotation	Records need to be classified, mapped, enriched, or corrected by a business expert.
Applying assumptions	A practical assumption can handle most cases, but violations need expert review.
Data quality reports	Issues are frequent, numerous, or easier to remediate in bulk.

Precise rules

Method	Use when...
Defining analytical concepts	Operational details need to be turned into usable business concepts, such as good/bad, milestones, or conformed categories.
Defining primary keys	Rows need a clear identity that connects them to real-world entities.
Defining the primary record	Multiple records appear to refer to one underlying entity.
Defining relationships	Analytically important relationships were not recorded by the source system.

Fuzzy logic

Fuzzy logic is handled through iterative approximation.

Step	Question
Loose-tight iteration	Can the pattern be tuned to reduce both false matches and false rejects?

Step	Question
Random validation	Does the pattern hold up when checked against unbiased samples?
Monitoring for drift	Does the pattern continue to behave over time?

One theme runs through all three approaches: assumptions must be monitored. If a data engineer closes a quality gap by applying judgement, rules, or approximation, the data product should also monitor whether that intervention continues to behave as intended.

This is covered in more detail in [Tests and assumptions](#).

24.2 Human curation: when judgement is needed

If data quality issues arise from a gap between recorded data and business intent, one simple remedy is to allow business experts to intervene directly.

Human curation closes the quality gap by allowing business judgement to enter the data product. Its main monitoring need is to detect new records that require human attention.

It is appropriate when business judgement is needed and the data engineer should not pretend the answer can be fully inferred by the system.

This section covers three scenarios where human curation is appropriate:

1. Data annotation
2. Applying assumptions
3. Data quality report

24.2.1 Data annotation

Data annotation is useful when records need to be classified, mapped, enriched, or corrected by a business expert.

For example, a digital system may record store locations as addresses. These addresses may be sufficient for operational use, but not enough for regional reporting. A business expert may later annotate each store with its sales region.

This annotation occurs in the data pipeline, not the source system, and is commonly known as reference data management.

Example structure of 'Store'

Store ID	Store address	Sales region (annotated)
S001	12 Smith St, Northvale	North
S002	88 Market Rd, Southport	South
S003	45 Central Ave, Westhaven	West
S004	9 Harbour Rd, Eastbank	New store not annotated

Store ID	Store address	Sales region (annotated)

The value `New store not annotated` flags the need for human attention.

Another example is harmonising employee identities across systems. In large organisations, the same employee may appear under different accounts for different business processes. A business expert who understands the enterprise view may annotate these accounts to map them to a canonical representation. This is known as master data management.

Human annotation extends beyond reference and master data. It applies wherever a human can interpret records outside the operational context.

There are two legitimate views of data: one fast-paced and operational, suited to digital systems; the other reflective and analytical, suited to business interpretation. Both may require human input. It is not reasonable to expect operational systems to carry the full load of analytical meaning.

Traditionally, data warehousing has treated human curation as an exception. But if human editing can be implemented transparently, with control and auditability, it should be treated as a standard business tool.

Whether the curation concerns reference data, master data, or other forms of annotation, the data engineer must monitor for incoming records that require curation. Examples include uncategorised store locations, unmapped employee accounts, or new values that do not yet belong to a known category.

24.2.2 Applying assumptions

Applying assumptions is useful when a practical assumption can handle most cases, but violations need expert review.

Consider the recording of dates. If users manually enter dates, they may accidentally type 2300 instead of 2030. If the digital system lacks validation, such errors can end up in the database. Even a single mistake can distort a line chart or skew time-based analysis.

One way to address this is to *assume* that future dates must be within 50 years. Dates outside this range are treated as invalid and converted to blank. Offending records should also be flagged so that a business expert can review and correct the source data.

Before applying the assumption, the data may look like this.

Example structure before flagging

Record ID	Event date
1	2030-06-15
2	2300-06-15
3	2025-04-02

After applying the assumption, the invalid date is blanked out and the row is flagged.

Example structure after flagging

Record ID	Event date	Is flagged for invalid date
1	2030-06-15	false
2		true
3	2025-04-02	false

This preserves analytical usefulness while making the intervention visible. The invalid date no longer distorts time-based analysis, but the issue remains available for remediation.

This approach works best when violations are rare and the business expert can correct the issue before the next batch load. It is suitable for irregular data quality issues that do not require bulk remediation.

If data quality issues are frequent or systemic, a data quality report may be more appropriate.

24.2.3 Data quality report

Data quality reports are suitable when issues are frequent, numerous, or easier to treat in bulk on a periodic basis.

An effective implementation is to create a [combination or choice dimension](#). A combination dimension may include one column per issue and describe each transaction. For example, a dimension called 'Sales data quality' might include [Is missing sales amount], [Is invalid sales date], [Is unknown customer], and so on.

Example structure of 'Sales data quality'

Sales data quality SK	Is missing sales amount	Is invalid sales date	Is unknown customer	Data quality category
1	false	false	false	No known issue
2	true	false	false	Missing amount
3	false	true	false	Invalid date
4	false	false	true	Unknown customer
...	...	...	...	...
8	true	true	true	Multiple issues

The fact table can then refer to the data quality dimension.

Example structure of 'Sales'

Sales ID	Sales date	Customer ID	Sales amount	Sales data quality SK
S1001	2025-04-01	C001	120.00	1
S1002	2300-04-01	C002	95.00	3
S1003	2025-04-03	C999	80.00	4

Sales ID	Sales date	Customer ID	Sales amount	Sales data quality SK
S1004	2025-04-04	C004		2

Coupled with the ID dimension, this makes it easy to build a Power BI report that surfaces transactions requiring remediation by a business expert.

A data quality dimension also supports systematic analysis by enabling statistics on the number and type of issues.

Reports are only effective if they embed in the user’s workflow. A decontextualised report becomes forgotten and unused. The same applies to data quality reports. The business should have a workflow trigger that prompts engagement with the report.

24.3 Precise rules: when intent can be formalised

Translating business knowledge into precise rules allows it to be automated and reduces reliance on human intervention.

One way to look at these rules is to see them as “fixing issues” in the data. A more useful way is to see them as rules that bridge recorded data to business intent.

Precise rules close data quality gaps by formalising business intent into repeatable logic. Their main monitoring need is to detect rule violations, edge cases, and changes in the business process that make the rule unsafe.

Precise rules are appropriate when the business can say what should happen clearly enough for the data engineer to implement it.

This section covers four recurring types of precise rule:

1. Defining analytical concepts
2. Defining primary keys
3. Defining the primary record
4. Defining relationships

24.3.1 Defining analytical concepts

Defining analytical concepts closes the quality gap between operational detail and business interpretation.

Since business insight is information analysed in light of business intent, a direct way of improving data quality is to define the analytical concept that the business needs.

Defining analytical concepts takes [leadership and negotiation skill](#). The data engineer is often well placed to broker between stakeholder groups by experimenting with the data and visually communicating possible outcomes.

Good and bad entities

Good and bad entity definitions close the quality gap between detailed operational outcomes and the business's need for a usable judgement.

A business process may collect detailed information such as audit results, inspection results, incidents, sentiment, or sales volume. However, these details may not yet be formalised into a systematic judgement.

Sometimes numeric values such as star ratings need to be translated into good and bad using thresholds. More commonly, a business process records information at a lower grain that needs to be rolled up to the entity of interest.

For example, a manufacturing batch may have multiple quality-control criteria. The business may decide that failing one critical criterion, or multiple non-critical criteria, defines failure at the batch level.

Example structure of 'Batch quality criterion'

Batch ID	Quality criterion	Quality result	Is critical criterion
B1001	Temperature control	Pass	true
B1001	Packaging integrity	Pass	false
B1001	Labelling accuracy	Fail	false
B1002	Temperature control	Fail	true
B1002	Packaging integrity	Pass	false

The analytical concept can then be expressed at the batch level.

Example structure of 'Batch quality outcome'

Batch ID	Critical criteria failed	Non-critical criteria failed	Batch quality outcome
B1001	0	1	Pass
B1002	1	0	Fail

This gives the user an immediate view of the process at the level of business judgement.

It is not reasonable to expect operational systems to always define these concepts. Their primary job is to execute workflows while preserving records. The data engineer plays a role by adding the analytical lens.

This implementation is studied in greater depth in [Entity processing](#).

Milestones

Milestones close the quality gap between messy workflow events and meaningful process control.

A business process may have many detailed steps. Some may loop back. Some may start and stop multiple times. Some may occur repeatedly for the same business entity. This mass of

events can bury insight in operational detail.

A useful analytical view is to define a small number of major checkpoints that can be used to measure performance. These checkpoints are milestones.

The difficulty is choosing the right level of abstraction. A milestone can be too narrow, in which case it merely reproduces operational noise. It can also be too broad, in which case it merges several different responsibilities into one measure.

A practical rule is that each milestone should have one main control point or responsible owner. If a milestone mixes responsibilities, then a delay becomes difficult to interpret. The business can see that something is late, but not who is able to act on it.

For example, an HR recruitment process may contain dozens of operational steps: position approved, advertisement drafted, advertisement cleared, advertisement published, applications received, applications closed, eligibility checked, longlist completed, shortlist completed, interviews scheduled, interviews completed, referee checks completed, delegate approval received, offer issued, offer accepted, and commencement recorded.

In practice, such an operational process may contain more than forty recorded steps. That level of detail is necessary for workflow execution, but too noisy for business decision-making.

The solution is to define key milestones.

Example structure of 'Recruitment event'

Recruitment ID	Event datetime	Event type
R1001	2025-04-01 09:00	Advertisement drafted
R1001	2025-04-03 11:30	Advertisement cleared
R1001	2025-04-04 09:00	Advertisement published
R1001	2025-04-18 17:00	Applications closed
R1001	2025-04-21 10:00	Eligibility checked
R1001	2025-04-23 16:00	Shortlist completed
R1001	2025-04-29 09:00	Interviews scheduled
R1001	2025-05-02 16:30	Interviews completed
R1001	2025-05-07 10:00	Delegate approved
R1001	2025-05-09 14:00	Offer accepted
R1002	2025-04-03 09:00	Advertisement drafted
R1002	2025-04-05 10:00	Advertisement returned for changes
R1002	2025-04-08 13:00	Advertisement cleared
R1002	2025-04-09 09:00	Advertisement published
R1002	2025-04-23 17:00	Applications closed
R1002	2025-04-29 15:00	Eligibility checked
R1002	2025-05-05 15:00	Shortlist completed
R1002	2025-05-14 16:00	Interviews completed

The data engineer can turn the event history into a milestone fragment.

Example structure of 'Recruitment milestone'

Recruitment ID	Advertisement published datetime	Interview completed datetime	Offer accepted datetime	Days from advertisement to interview	Days from interview to offer
R1001	2025-04-04 09:00	2025-05-02 16:30	2025-05-09 14:00	28.3	6.9
R1002	2025-04-09 09:00	2025-05-14 16:00		35.3	

The milestone fragment does not replace the event history. It gives the business a stable analytical view of the process.

In this example, the three milestones correspond to different control points. Advertisement publication depends on the area preparing and clearing the advertisement. Interview completion depends on the assessment process. Offer acceptance depends on post-interview approval and candidate response. If these were collapsed into one broad measure such as `[Days to recruit]`, the business would know the process was slow but not where the delay occurred.

To deal with loops and repeated events, the data engineer can define the earliest or latest relevant occurrence. For example, `[Advertisement published datetime]` may use the first publication event, while `[Offer accepted datetime]` may use the final accepted offer event.

This pattern is studied in [Meaningful fragments](#).

Conformed dimensions

Conformed dimensions close the quality gap between local system categories and enterprise intent.

A large organisation may have many processes, and many of them share similar concepts under different names or codes. A conformed dimension reconciles these local concepts into a shared view.

For example, one system may store country using two-letter country codes, while another system may store country using its own operational codes.

Example structure of Sales.Customer

Customer ID	Customer country code
C001	AU
C002	NZ
C003	US

Example structure of Shipping.Delivery

Delivery ID	Delivery country code
D001	AUS
D002	NZL
D003	USA

Delivery ID	Delivery country code

Both systems refer to the same business concept: country. However, the codes are not the same. If the data engineer leaves them as separate local values, enterprise reporting becomes harder than it needs to be.

The data engineer can create a standard country reference table.

Example structure of 'Country'

Country SK	Country name	ISO alpha-2 code	ISO alpha-3 code
1	Australia	AU	AUS
2	New Zealand	NZ	NZL
3	United States	US	USA

The local system codes can then be mapped to the conformed dimension.

Example structure of 'Country code map'

Source system	Source country code	Country SK
Sales	AU	1
Sales	NZ	2
Sales	US	3
Shipping	AUS	1
Shipping	NZL	2
Shipping	USA	3

This allows sales and shipping data to be analysed through the same country dimension, even though the source systems use different local codes.

When done appropriately, a conformed view can empower decision-makers at the most senior levels of the organisation.

Conformed dimensions are studied in [Reference data](#).

24.3.2 Defining primary keys

Defining primary keys closes the quality gap between database rows and real-world entities by making their link explicit.

Primary keys serve as the [link between data records and their counterparts in the real world](#). Unfortunately, some business processes do not rigorously define primary keys.

This can lead to slippery definitions in the database. In other cases, the primary key is defined at the application layer but remains invisible at the data layer, leaving the business user without a clear way to interpret the data.

The data engineer should establish the primary key where it is not meaningfully defined.

The primary key articulates the data engineer’s view of how the data row should be interpreted as a business entity.

Most primary keys can be traced by examining how a business process creates, retrieves, and updates a data record. Three common patterns are sequence numbers, version numbers, and temporality.

Sequence numbers

Sequence numbers close the quality gap where the source system stores a list, but the data product needs stable business identity for each row.

Sequence numbers are useful when a header has multiple detail rows and the detail table lacks a meaningful primary key.

For example, a sales order may have multiple items. The order is stored in **Sales**, with **[Order number]** as the primary key. The items are stored in **SalesItems**, which may be a miscellaneous list without a meaningful key.

If each product can appear only once, then **[Order number]** and **[Product ID]** may suffice as a primary key. But if a product can appear more than once, this fails.

Example structure of SalesItems before sequence numbering

Order number	Product ID	Sales item ID	Quantity
O1001	P100	501	1
O1001	P100	502	2
O1001	P200	503	1

[Order number], **[Product ID]** cannot identify a row because P100 appears twice for the same order.

A simple solution is to treat the list of items as a sequence and create an artificial column **[Sales item sequence number]**, forming the key **[Order number]**, **[Sales item sequence number]**.

Example structure of SalesItems after sequence numbering

Sales item sequence		Product ID	Sales item ID	Quantity
Order number	number			
O1001	1	P100	501	1
O1001	2	P100	502	2
O1001	3	P200	503	1

Sometimes a system will implement a meaningless key such as **[Sales item ID]**, a simple integer used for UI or database constraints. Even in these cases, the data engineer may still define a sequence number and use **[Order number]**, **[Sales item sequence number]** as the business-facing primary key. **[Sales item ID]** can be retained for joins.

In general, sequence numbers are effective wherever there is a miscellaneous list of line items within a header. Care must be taken to ensure the sequence is deterministic by breaking ties—using a surrogate key like `[Sales item ID]` as a sort order is a reliable approach.

Version numbers

Version numbers close the quality gap where the source system records change, but does not make the continuity of the entity clear.

Version numbers are useful for [entities whose changes should be preserved as new versions](#).

For example, a customer order may be submitted, revised, and resubmitted before it is fulfilled. The business may still regard this as the same order, but each revision changes the content that was true at a particular point in time.

Digital systems are not always consistent in how they manage versions. A common but problematic approach is to allow the entity ID to change each time the record is revised and record the relationship using `[Previous order ID]` or `[Superseded order ID]`. This may work for rendering a web UI but can cause confusion in analysis.

Where versioning is lost or muddled, the data engineer can restore clarity using a consistent pattern such as `[Entity ID]`, `[Version number]`.

The order can be represented with a stable `[Order ID]` and a changing `[Order version number]`.

Example structure of `Sales.Order`

Order ID	Customer ID	Current order version number	Order status
O1001	C042	3	Submitted
O1002	C087	1	Fulfilled

Example structure of `Sales.OrderVersion`

Order ID	Order version number	Version datetime	Product ID	Quantity	Order value
O1001	1	2025-04-01 09:15	P100	10	250.00
O1001	2	2025-04-01 11:40	P100	12	300.00
O1001	3	2025-04-02 08:30	P200	12	420.00
O1002	1	2025-04-03 14:10	P300	5	175.00

The primary key for `Sales.OrderVersion` is `[Order ID]`, `[Order version number]`.

Temporality

Temporality closes the quality gap where the business needs historical interpretation but the source system only presents the current state.

Temporality refers to tracking changes over time. In data warehousing, this is commonly handled through Type II tracking.

Some business processes are designed to handle only the current event, without tracking history. But when the business is interested in change over time, the [entity is mutable](#).

Mutable entities should usually include a time component in their business key. The data engineer may define the business key as [Employee ID], [Start datetime], with [End datetime] marking the end of the validity period.

In implementation, it is necessary to add a surrogate key as well.

Example structure of Employee.TeamAssignment

Team assignment SK	Employee ID	Start datetime	End datetime	Team name
1	E1001	2024-01-01 00:00	2025-03-15 09:30	Finance
2	E1001	2025-03-15 09:30	9999-12-31 00:00	Analytics
3	E1002	2024-07-01 00:00	9999-12-31 00:00	Operations

The business key is [Employee ID], [Start datetime]. [End datetime] marks the end of the validity period. [Team assignment SK] provides a simple surrogate key for joins, relationships, and downstream modelling.

Recovering temporality depends on how history is stored. Sometimes it is available in audit tables. Sometimes it must be reconstructed with help from business experts.

24.3.3 Defining the primary record

Defining the primary record closes the quality gap between multiple records and one underlying entity.

This occurs when the database has primary keys, but different primary keys relate to the same underlying entity. It is common in business processes that gather decentralised observations.

Consider the example of whale sightings. A database may collect observations from citizen scientists, stored in `Whale.Observation`, with [User ID], [Observation number] identifying each user’s submissions. A surrogate key such as [Observation ID] may also exist for system retrieval.

Multiple citizens may report sightings of the same whale. As a result, multiple [Observation ID] values relate to the same real-world entity.

Example structure of Whale.Observation

Observation ID	User ID	Observation			GPS area	Observation date
		number	Species			
101	U01	1	Blue whale		A17	2025-05-01
102	U02	1	Blue whale		A17	2025-05-01
103	U03	1	Humpback whale		B04	2025-05-01

Suppose the business has a rule such as:

A whale species at one GPS proximity should only appear once within a day.

This rule can be used to group observations and identify a representative record.

The data engineer can express this by identifying the primary observation. For example, the pipeline may select the first observation of the day at a location and store it as `[Primary observation ID]` in a table called `Whale.PrimaryObservation`.

Example structure of `Whale.PrimaryObservation`

Observation ID	Primary observation ID
101	101
102	101
103	103

`[Observation ID]` is the key for the original record. `[Primary observation ID]` identifies the chosen representative.

Subsequent transformation can focus on the `[Primary observation ID]` grain. The same key can also be used in Power BI to return the true count of whale sightings.

Care must be taken to resolve race conditions. If two users submit observations at the same time, only one record should be selected as primary. This can be resolved deterministically using a surrogate key such as `[Observation ID]`.

24.3.4 Defining relationships

Defining relationships closes the quality gap where analytically important relationships were not recorded by the source system.

Digital systems are usually good at recording relationships needed by operational workflows. They are less reliable at recording relationships that are useful for analysis.

This is especially common when the system is designed for operational efficiency rather than analytical understanding.

The data engineer can sometimes close this quality gap by defining relationships that the source system did not record.

Two useful techniques are:

- nearest temporal joins;
- mapping tables.

Nearest temporal join

Nearest temporal joins close the quality gap between related events whose relationship was not explicitly recorded.

Sometimes two sets of events are related, but the relationship is not recorded as a database key.

If business knowledge suggests that one set of events is expected to precede another, the relationship can sometimes be recovered by identifying the nearest preceding event. This technique is known as a nearest temporal join.

Consider a building fire-safety process.

Building owners submit fire-safety certificates periodically, or when the building’s fire-safety arrangements change. Fire-safety officers conduct inspections separately. Some inspections may be scheduled. Others may be triggered by complaints, incidents, risk reviews, or follow-up action.

The two processes are related, but they are not one-to-one. One certificate may be followed by several inspections. Several certificates may be submitted before the next inspection. Some inspections may have no recent certificate at all.

The inspection system records the building and inspection date, but it does not record which certificate was in force at the time of inspection. This makes the relationship analytically important but structurally missing.

The fire-safety certificates are stored in `FireSafety.Certificate`.

Example structure of `FireSafety.Certificate`

Fire safety certificate		Certificate submitted datetime	Certificate type	Declared risk level
SK	Building ID			
1	B001	2025-04-01 09:00	Annual certificate	Low
2	B001	2025-05-01 09:00	Revised certificate	Medium
3	B002	2025-04-10 14:30	High-risk occupancy	High
4	B002	2025-04-18 16:00	Remediation certificate	Medium

The inspections are stored separately in `FireSafety.Inspection`.

Example structure of `FireSafety.Inspection`

Inspection SK	Building ID	Inspection datetime	Inspection trigger	Inspection outcome
101	B001	2025-04-18 10:00	Scheduled	Pass
102	B001	2025-05-20 11:30	Complaint	Fail
103	B002	2025-04-15 09:00	Risk review	Fail
104	B002	2025-04-30 13:15	Follow-up	Pass

Although both tables contain [Building ID], there is no direct relationship between a specific inspection and a specific fire-safety certificate. The source system does not say which certificate the inspection should be read against.

To support analysis, the data engineer can create a nearest temporal join.

For example, the rule might be:

An inspection relates to the most recent fire-safety certificate submitted for the same building before the inspection.

The result can be stored in `FireSafety.InspectionCertificate`.

Example structure of `FireSafety.InspectionCertificate`

Inspection SK	Fire safety certificate SK	Days from certificate to inspection
101	1	17
102	2	19
103	3	5
104	4	12

This table re-establishes an analytical relationship between the inspection and the fire-safety certificate that preceded it.

The relationship is not “true” in the same way as a source-system foreign key. It is an inferred relationship created from business logic. For that reason, it should be documented and tested.

The data engineer should monitor cases where the assumption may be unsafe, such as:

- inspections with no preceding fire-safety certificate;
- inspections where the most recent certificate is too old;
- buildings with multiple certificates close together;
- certificate types that should not be associated with inspections;
- changes in the inspection process that alter the expected timing.

For example, the business may decide that an inspection should only be linked to a fire-safety certificate if it occurred within 60 days. If the most recent certificate is older than that, the inspection should be flagged for review rather than automatically linked.

Nearest temporal joins are especially valuable at the macro level.

At unit level, the problem is often that two records should be related but no foreign key was recorded. At macro level, the problem is broader: two business processes are related, but they operate in bulk, at different grains, and without unit-level traceability.

In the fire-safety example, certificate submission and inspection are not the same business process. They are separate processes that become analytically meaningful only when the data engineer defines a relationship between them.

This is the discipline of the pattern: use time to restore a relationship only where business knowledge says the relationship is meaningful, then monitor the assumptions that make it safe.

Mapping tables

Mapping tables close the quality gap between entities that must be related analytically but are not related operationally.

Mapping tables are introduced when the original system does not record relationships needed for analysis.

A common scenario is a header table with two separate detail tables at different grains. Both detail tables relate to the header, but not to each other.

The data engineer may introduce a mapping table between them based on a business rule.

Consider a club restaurant with data captured in three tables:

- **Club.TableSitting**—records of customers seated at the restaurant, one per sitting, keyed by [Table sitting ID]
- **Club.TableCustomer**—club members identified by [Member ID], linked to [Table sitting ID]
- **Club.TableFoodOrder**—menu items ordered per [Table sitting ID], with food items identified by [Food item ID]

Example structure of Club.TableCustomer

Table sitting ID	Member ID
T001	M001
T001	M002

Example structure of Club.TableFoodOrder

Table sitting ID	Food order ID	Food item ID	Food item cost
T001	FO001	F100	12.00
T001	FO002	F200	18.00

The system does not record which member ordered which item. For analysis, the business may wish to associate members with food items. However, this relationship is not captured in the source system.

To support analysis, the data engineer may introduce a mapping table called `Club.CustomerFoodOrderMap`. The logic may be based on a business rule such as:

Every member at the table shares the cost of all items ordered.

Example structure of `Club.CustomerFoodOrderMap`

Table sitting ID	Member ID	Food order ID	Allocation weight
T001	M001	FO001	0.5
T001	M002	FO001	0.5
T001	M001	FO002	0.5
T001	M002	FO002	0.5

Because the relationship is inferred rather than recorded, it should be tested. Techniques for validating mapping logic are described in [Tests and assumptions](#).

Variants of this are studied in [Meaningful fragments](#).

24.4 Fuzzy logic: when intent can only be approximated

There are cases where precise rules do not apply.

Fuzzy logic closes data quality gaps by approximating business intent where exact rules are not available. Its main monitoring need is to detect drift in match rates, validation results, or alignment with a known reference.

A common example is natural language processing. When working with free text, it is often impossible to define exact rules that meet business intent.

Fuzzy logic is appropriate when the business intent is real but cannot be captured precisely. In many scenarios, something is better than nothing, and a perfect match with reality does not justify the cost.

This aligns with the view that data quality is not data versus reality, but data versus business intent.

Examples of fuzzy logic include:

- optical character recognition to extract information from images;
- large language models to infer sentiment from user text;
- entity resolution techniques to identify the underlying entity behind multiple records;
- pattern extraction from free text, such as phone numbers, invoice numbers, or addresses.

Fuzzy logic often pushes into the domain of data science, where statistical and mathematical disciplines bring rigour to its application. However, there are cases where a data engineer can apply a simple and practical approach.

The following is a three-step process, a kind of “poor man’s data science”:

1. Find the pattern through loose–tight iteration.
2. Check the pattern with random validation.
3. Monitor the pattern by watching for drift.

This approach can be highly effective in common scenarios that require extracting information from free text. A data engineer should be comfortable considering this approach and ready to pivot to a more sophisticated data science method when necessary.

24.4.1 Step 1—Loose–tight iteration

Loose–tight iteration closes the quality gap where a pattern is real enough to use, but too messy to define perfectly at the start.

Every pattern has matches and rejects. The matches are records that meet the criteria. The rejects are records that do not.

The data engineer tunes the pattern by adjusting the criteria:

- tightening to reduce over-matching;
- loosening to recover missed records.

The goal is not to maximise matches. The goal is to reduce mistakes on both sides.

Consider the example of extracting phone numbers from a free-text field. Phone numbers may appear in formats such as (02) 1234 5678, 0412345678, 12345678, or 02-1234-5678.

The challenge is that the field may also contain other numbers, such as invoice numbers, dates, quantities, account numbers, or address numbers.

Example structure of 'Customer note'

Customer note ID	Customer note
1	Please call me on (02) 1234 5678 after 3pm.
2	Mobile is 0412345678. Invoice 889912 is unrelated.
3	Address is 12 Hill Street. No phone provided.
4	Try 02-9876-5432 or office 98765432.
5	Order 12345678 was delayed.
6	Phone number changed to 0412 345 678 on 2025-04-01.

A rough first pattern might look for any eight-digit number.

This will produce matches, but it will also over-match.

Example match set after rough pattern

Customer note ID	Extracted value	Correct extraction?	Comment
2	0412345678	true	Mobile number
4	98765432	true	Local phone number
5	12345678	false	Order number, not phone number

Customer note ID	Extracted value	Correct extraction?	Comment
------------------	-----------------	---------------------	---------

The pattern found real phone numbers, but also picked up an order number. This is a false match. The pattern is too loose.

The data engineer may tighten the pattern by looking for phone-like context, such as nearby words like **phone**, **mobile**, **call**, or **office**, or by excluding known phrases such as **order** and **invoice**.

After tightening, the match set may improve, but some genuine phone numbers may now be missed.

Example reject set after tightened pattern

Customer note ID	Customer note	Should have matched?	Comment
1	Please call me on (02) 1234 5678 after 3pm.	true	Landline with area code
3	Address is 12 Hill Street. No phone provided.	false	No phone number
6	Phone number changed to 0412 345 678 on 2025-04-01.	true	Mobile with spaces

The rejected records reveal false rejects. The pattern is now too tight, or it does not recognise enough valid phone-number formats.

The data engineer then loosens the pattern to recognise area codes, mobile numbers with spaces, and hyphenated numbers while still avoiding obvious order numbers, invoice numbers, dates, and addresses.

A later iteration may produce a more useful extraction.

Example extraction after loose-tight iteration

Customer note ID	Extracted phone number	Extraction status
1	0212345678	Matched
2	0412345678	Matched
3		No phone detected
4	0298765432; 98765432	Matched
5		Rejected as likely order number
6	0412345678	Matched

The important point is not the exact pattern. The important point is the discipline of iteration. Finding the best pattern is an iterative process:

1. Define a rough pattern.
2. Inspect the match set for false matches.
3. Inspect the reject set for false rejects.
4. Adjust and repeat.

Each adjustment changes the match set and reject set. It is usually better to tighten or loosen one thing at a time, then inspect the result.

The key idea is that focusing only on matches creates a blind spot. The rejected records are equally significant. A useful pattern is found by examining both incorrect matches and incorrect rejects.

24.4.2 Step 2—Random validation

Random validation closes the quality gap where approximate logic looks plausible, but has not been checked against independent judgement.

Once the pattern has been tuned, it should be validated by a business expert.

This is done by randomly sampling records from both the match set and the reject set.

The sample must be unbiased. It should not focus only on records where the engineer has low confidence, nor only on records that seem easy to validate. The point is to test the pattern across its full range of behaviour.

For example, the data engineer may extract phone numbers from 50,000 customer notes. The pattern may return 18,000 notes with phone numbers and 32,000 notes without phone numbers. The validation sample should include records from both groups.

Example validation sample

Sample ID	Source set	Customer note	Pattern result	Expert judgement	Validation outcome
1	Match set	Mobile is 0412345678.	Phone found	Phone present	Correct match
2	Match set	Order 12345678. Please follow up.	Phone found	No phone present	False match
3	Reject set	Call through reception on 98765432.	No phone found	Phone present	False reject
4	Reject set	Address is 12 Hill Street.	No phone found	No phone present	Correct reject

The validation result can then be summarised.

Example validation summary

Validation outcome	Count
Correct match	86
False match	6
Correct reject	92
False reject	16

This tells the data engineer how the approximation behaves. It may be good enough for some purposes and not good enough for others.

If the business only needs a rough indication of whether contact details are likely to be present, this may be acceptable. If the extracted phone number will be used for direct customer contact, the tolerance for false matches and false rejects will be much lower.

This is why fuzzy logic still depends on business intent. The question is not whether the pattern is perfect. The question is whether the pattern is good enough for the decision or action it supports.

This validation process should be repeated periodically. Even if the pattern was correct at the time of implementation, changes in business processes or user behaviour may cause it to degrade.

Regular validation helps ensure the pattern continues to serve its intended purpose.

24.4.3 Step 3—Monitoring for drift

Monitoring for drift closes the quality gap where an approximation was once good enough, but may stop behaving as intended over time.

In practice, random validation cannot be performed continuously. Drift monitoring provides a lightweight alternative.

The idea is to identify a statistic that should remain relatively stable over time. If the statistic changes significantly, the pattern may no longer be behaving as expected.

For example, suppose a free-text field is used to extract phone numbers. Not all extracted numbers will match the customer database, but a certain percentage—say 85%—typically do. This percentage reflects the stability of the pattern.

The data engineer can monitor this statistic over time.

Example drift monitoring

Month	Notes processed	Notes with extracted phone number	Extracted numbers matching customer records
2025-01	12,400	4,820	86%
2025-02	13,100	5,040	85%
2025-03	12,850	4,910	84%
2025-04	13,300	5,110	85%
2025-05	13,050	7,900	61%

A sudden drop from the usual range may indicate that the pattern is no longer behaving as expected.

Users may have started entering phone numbers in a new format. The source system may have changed its note template. Another number, such as a case number or ticket number, may have started appearing near phone-like words. The customer database may also have changed, reducing the apparent match rate.

This is a form of assumption monitoring. The assumption is that the extracted values will continue to resemble the known population. If the assumption fails, the pattern may need to be revisited.

Drift monitoring is especially useful when the business has a relatively static reference point, such as a customer table, a list of known codes, or a set of standard formats. However, any stable statistic can be used.

Useful drift statistics may include:

- percentage of records matched;
- percentage of extracted values that match a known reference table;
- number of distinct extracted values;
- proportion of records with multiple extracted values;
- proportion of records requiring manual review;
- proportion of null or rejected records.

Key ideas.

Data quality is fitness for business intent, not perfect correspondence with reality.

Different intents create different quality requirements.

Human curation is appropriate when business judgement is needed.

Precise rules are appropriate when business intent can be formalised.

Fuzzy logic is appropriate when business intent can only be approximated.

All three approaches depend on monitoring the assumptions that make the intervention safe.

Tests and assumptions

Good engineering makes failure visible before it reaches the user.

25.1 Anticipating errors

A data product is not trustworthy merely because it is correct today. It is trustworthy when users are not caught unawares by eventual failures.

This is the third principle of data engineering:

Instead of stopping with what works now, anticipate errors that may occur.

The principle follows from two facts.

First, the world changes. What is true today may not be true tomorrow. New data may arrive outside previously conceived parameters. Business processes may evolve. Source systems may change. Another developer may alter existing code.

Second, engineers make mistakes. Complex transformations create many opportunities for small errors to become consequential.

The danger is not only that errors occur—errors are inevitable.

The deeper danger is that errors occur silently.

A silent error allows a data product to remain polished and apparently authoritative while no longer being safe to use.

Therefore:

Data engineering is not only building the transformation that works. It is building the means by which failure becomes visible.

This chapter covers two such mechanisms:

Mechanism	Question	Output
Tests	Did two independent calculations produce the same result?	Pass or fail
Monitored assumptions	Has something appeared that requires human attention?	A list of violating records

Mechanism	Question	Output

A test performs the same calculation in two different ways and checks whether the results agree.

A monitored assumption checks whether a condition presumed to hold has been violated, and returns the records requiring attention.

Tests protect the correctness of implemented logic. Monitored assumptions protect the safety of the conditions around that logic.

Both should run regularly, such as once per pipeline batch. Their purpose is not to eliminate failure entirely. Their purpose is to prevent avoidable failure from remaining hidden.

25.2 Tests: checking the same result two ways

A data test is strongest when expected and actual results are calculated independently.

The premise of testing is this: when the same problem is solved in two different ways, and both methods arrive at the same result, confidence is justified.

In data engineering, a test has two parts:

Test part	Meaning
Expected result	A calculation of what the result should be, preferably from a different source or logic path.
Actual result	The result produced by the pipeline, model, or measure being tested.

The test passes if expected and actual results match.

In a data pipeline, the expected part of a test may be calculated from raw or partly transformed data. The actual part uses the transformed output.

In a Power BI semantic model, the expected part of a test may be calculated from the underlying data source, such as SQL or a data lake. The actual part may be calculated using DAX. This approach is effective for validating complex DAX measures, especially measures that rely on embedded grain or complex DAX.

Power BI tests are easy to create. The data engineer can drag and drop to build a table visual with dimensions and measures. The Performance Analyzer in Power BI Desktop reveals the DAX query behind the visual. This can serve as the actual portion of the test. The engineer then compares the DAX evaluation with a calculation from the underlying data source.

Tests lose effectiveness when the expected query is a copy of the pipeline code that produced the actual result. This entanglement reduces independence and narrows the surface of possible errors caught by the test.

A thoughtful test maximises the difference in logic between expected and actual results.

The way to write a good test is to ask:

Where is the most complex or fragile part of the transformation, and how can this result be calculated in a different way?

The following examples are not exhaustive. They illustrate the mindset of checking weak points by calculating the same result through a different path.

25.2.1 Row counts

Row count tests check whether records have appeared, disappeared, or duplicated unexpectedly.

The simplest test is to compare row counts before and after transformation. The expected query counts rows from the raw data with appropriate filters. The actual query counts rows from the transformed output.

Example row count test

Test	Expected result	Actual result
Count sales records for April 2025	10,240	10,240

Even small variations can make row count tests more effective.

For example, instead of only counting all rows, the data engineer may group by a reference value.

Example grouped row count test

Sales region	Expected sales rows	Actual sales rows
North	2,450	2,450
South	3,100	3,100
West	1,920	1,918
New store not annotated	12	14

The total row count may still match because one group is undercounted while another is overcounted. Grouped counts reveal the problem, helping catch update errors, join errors, and mapping issues.

The expected and actual calculations should use different logical paths. For example, the expected query may group raw records using source fields, while the actual query groups transformed records using curated dimensions.

Row count tests are especially important for incremental loads.

If the extract relies on an architectural column such as `[Row update datetime]`, the test should use a different datetime column, such as `[Staff update datetime]`, `[Submission datetime]`, or another business-centric column. This separation avoids self-confirmation.

Selecting a sample of recent records is also useful. A recent-record sample can check creation, update, and deletion behaviour across many columns, not only row counts.

25.2.2 Checksums on key results

Checksum tests check whether an important summary result still balances. They are useful when a transformation rolls detailed records up into a business concept. Consider the example of manufacturing batch quality.

A business process may record individual quality-control criteria. The data engineer then rolls these up to define the overall quality outcome for each batch.

Example structure of 'Batch quality criterion'

Batch ID	Quality criterion	Quality result	Is critical criterion
B1001	Temperature control	Pass	true
B1001	Packaging integrity	Pass	false
B1001	Labelling accuracy	Fail	false
B1002	Temperature control	Fail	true
B1002	Packaging integrity	Pass	false

The transformed output may look like this.

Example structure of 'Batch quality outcome'

Batch ID	Critical criteria failed	Non-critical criteria failed	Batch quality outcome
B1001	0	1	Pass
B1002	1	0	Fail

A checksum can test the core logic without copying the transformation. For example, every batch should fall into one and only one of two groups: pass or fail.

Example checksum test

Test	Expected result	Actual result
Count distinct batches from criterion table	2	2
Count batches marked pass or fail in outcome table	2	2
Count batches with any critical failure from criterion table	1	1
Count batches marked fail in outcome table	1	1

This test is not a copy of the transformation. It checks whether the transformed result still balances against the source logic.

A more comprehensive version can group by values such as [Product type], [Manufacture site], or [Inspection month]. This increases the chance of catching errors that affect only a segment of the data.

25.2.3 Bypassing mapping tables

Mapping tables are often introduced where the source system did not record an analytical relationship.

A good test for a mapping table is to compare results with and without the mapping table.

Consider the restaurant example from the previous chapter. The source system records table sittings, table customers, and food orders. It does not record which customer ordered which food item.

The data engineer introduces `Club.CustomerFoodOrderMap` to allocate food orders across members seated at the table.

Example structure of `Club.TableCustomer`

Table sitting ID	Member ID
T001	M001
T001	M002

Example structure of `Club.TableFoodOrder`

Table sitting ID	Food order ID	Food item ID	Food item cost
T001	FO001	F100	12.00
T001	FO002	F200	18.00

Example structure of `Club.CustomerFoodOrderMap`

Table sitting ID	Member ID	Food order ID	Allocation weight
T001	M001	FO001	0.5
T001	M002	FO001	0.5
T001	M001	FO002	0.5
T001	M002	FO002	0.5

Regardless of how complex the mapping logic becomes, some totals should remain the same. The total cost of food ordered at the table should not change after allocation.

Example mapping checksum

Table sitting ID	Expected food cost from <code>Club.TableFoodOrder</code>	Actual allocated food cost from <code>Club.CustomerFoodOrderMap</code>
T001	30.00	30.00

This is a powerful way to validate mapping logic. The test does not need to repeat every detail of the mapping. It checks whether the mapping preserves an invariant.

More specific tests can then check the business rule itself.

For example, if the business rule assumes food cost is evenly distributed among seated members, the test can check that allocation weights sum to 1 per food order.

Example allocation-weight test

Food order ID	Expected allocation weight total	Actual allocation weight total
FO001	1.0	1.0
FO002	1.0	1.0

Grouping by reference columns such as `[Food item ID]`, `[Food item type]`, or `[Table sitting ID]` can make the test more sensitive.

25.2.4 Checking boundary cases

Boundary cases are common sources of error.

A typical example is a rolling-window measure. Any implementation of a rolling window should be tested at boundary points.

Suppose there is a Power BI measure `[Total sales volume]`, and a derived rolling 12-month version `[Total sales volume past 12 months]`.

The rolling measure is easy to get subtly wrong. It may include the wrong start date, exclude the current date, mishandle leap years, or behave incorrectly at the beginning of the available history.

One useful test is to compare the rolling result against known calendar boundaries.

For example, if `[Total sales volume past 12 months]` is evaluated at 31 December 2025, it should match total sales volume for the 2025 calendar year.

Example boundary test

Evaluation date	Expected result	Actual rolling 12-month result
2025-12-31	Total sales from 2025-01-01 to 2025-12-31	Total sales volume past 12 months at 2025-12-31
2026-06-30	Total sales from 2025-07-01 to 2026-06-30	Total sales volume past 12 months at 2026-06-30

This checks that the measure behaves properly at calendar-year and financial-year boundaries.

However, this test would still pass if `[Total sales volume past 12 months]` were mistakenly

implemented as a copy of `[Total sales volume]` in a model that is already filtered to one year. To guard against this, the data engineer should also test turning points, such as the first and last day of a month.

Example turning-point test

Evaluation date	Boundary being tested
2025-02-28	End of February
2025-03-01	Start of March
2025-06-30	End of financial year
2025-07-01	Start of financial year

A rolling-window measure that passes at calendar-year boundaries, financial-year boundaries, and interim month turning points provides much stronger assurance than one tested only at convenient dates.

25.2.5 Using subject matter knowledge

Subject matter knowledge can introduce independence into a test.

Normally, the data engineer should avoid building unverified assumptions into the pipeline. This prevents blind spots and reduces the risk of silent errors.

However, tests are one place where business knowledge can be safely used to strengthen robustness. If the business knowledge is wrong, the test will reveal that. If the business knowledge was once right but stops being right, the test will reveal a change in the process.

For example, a business expert may know that every table sitting must have at least one customer. The data engineer should not blindly assume this in the pipeline, because load errors may still occur. But the knowledge can be used in a test.

Example subject-matter test

Business knowledge	Expected calculation	Actual calculation
Every table sitting has at least one customer	Count table sittings using an inner join to <code>Club.TableCustomer</code>	Count all rows in <code>Club.TableSitting</code>

If the results differ, either the data has an error, the pipeline has an error, or the business knowledge was incomplete.

Business knowledge can also validate roll-up logic.

Consider a help desk system where cases escalate from Tier 1 to Tier 4. Escalations are recorded in `Helpdesk.Escalation`, keyed by `[Case ID]`, with `[Tier]` indicating escalation level. The table `Helpdesk.CaseEscalation` rolls this information up to case grain. A reference table `Helpdesk.RefCaseEscalation` includes `[Highest escalation]` to summarise each case.

Tests can use business knowledge to validate the roll-up.

Business knowledge	Expected	Actual
Every case starts at Tier 1	Count distinct [Case ID] in Helpdesk.Escalation where [Tier] = "Tier 1"	Count rows in Helpdesk.CaseEscalation
Tier 4 is the maximum escalation	Count distinct [Case ID] in Helpdesk.Escalation where [Tier] = "Tier 4"	Count rows in Helpdesk.CaseEscalation where [Highest escalation] = "Tier 4"
Cases do not skip the existence of earlier tiers	Count Tier 4 cases with no Tier 1 record	Should be zero

In general, existence and uniqueness conditions known to business experts can be used to bypass or vary the original pipeline logic.

This is one of the strongest ways to write tests because it introduces a source of independence outside the code itself.

25.3 Monitored assumptions: surfacing records that require attention

Data engineers must make assumptions.

However, assumptions may fail over time. Sometimes the failure is unanticipated. At other times, the failure is anticipated but still needs to be detected.

A monitored assumption is a query that returns records if and only if human attention is required.

If no rows are returned, the assumption remains valid. If rows are returned, each row is a violation that needs attention.

The key to monitoring assumptions is recognising that an assumption exists.

The following are common cases.

Assumption	Returns rows when...
Source data is up to date	The latest expected batch has not arrived.
Reference data is complete	New values need mapping.
Known values remain stable	A new hard-coded or unhandled value appears.
Data quality rules still hold	Invalid dates, duplicate keys, or out-of-range values appear.
Fuzzy logic has not drifted	Match rates or validation results move outside tolerance.

25.3.1 Is source data up to date?

Source data cannot always be assumed to arrive on time.

Any number of failures may delay the arrival of new data: an upstream system may be unavailable, an extract may fail, a file may not arrive, or a source team may change a schedule without warning.

A monitored assumption can check whether the latest expected data has arrived.

Example source freshness assumption

Source table	Expected latest date	Actual latest date	Requires attention
<code>Sales.Order</code>	2025-05-01	2025-05-01	false
<code>Sales.Payment</code>	2025-05-01	2025-04-28	true

If the query returns `Sales.Payment`, the issue needs attention. The data product may be incomplete even though the pipeline has technically run.

This assumption is especially important when multiple source systems are combined. A report may look normal while one process has updated and another has not.

25.3.2 Is reference data complete?

Reference data is vital to a quality warehouse.

It enables different systems to map local values to conformed values. But reference mappings require maintenance as new source values arrive.

For example, one system may start sending a new country code that is not yet mapped.

Example structure of 'Country code map'

Source system	Source country code	Country SK
Sales	AU	1
Sales	NZ	2
Sales	US	3
Shipping	AUS	1
Shipping	NZL	2
Shipping	USA	3

A monitored assumption can check whether any source country codes have arrived without a mapping.

Example assumption output

Source system	Source country code	Records affected
Shipping	SGP	14

The query should return only the values that require human attention. In this case, someone needs to decide whether *SGP* maps to an existing country record or whether the country reference data need to be extended.

25.3.3 Are there unanticipated values?

Hard-coded values are sometimes unavoidable.

This is acceptable when a source column contains a small number of stable values. For example, a status column may have contained *Open*, *Closed*, and *Cancelled* for many years.

However, if a new value appears, the transformation logic may no longer be safe.

Example status assumption

Source status	Records affected
Pending review	37

This assumption does not necessarily mean the pipeline is broken. It means the business process has produced a value not anticipated by the transformation. The data engineer or business owner must decide how it should be handled.

This is especially important when a transformation uses *case* logic, hard-coded mappings, or manually curated categories.

25.3.4 Are there data quality issues?

Source data may contain issues from unvalidated collection.

These can include duplicate keys, dates outside expected ranges, negative values where only positive values make sense, or records that violate business rules.

In some cases, the pipeline may treat the issue to preserve analytical usefulness. For example, a future date outside the allowed range may be converted to blank and flagged.

Example assumption output for invalid dates

Record ID	Source date	Issue
2	2300-06-15	Date exceeds allowed future range

Similarly, duplicate records may be removed from the main analytical table and written to a rejected-records table.

Example assumption output for duplicate submissions

Submission ID	Customer ID	Submission date	Rejection reason
S1003	C001	2025-04-01	Duplicate customer submission for date

These are monitored assumptions because the business needs to know that the issue occurred. The pipeline may be able to continue, but the underlying source issue still requires attention.

25.3.5 Is there data drift?

Statistical and fuzzy logic often rely on assumptions about the distribution or pattern of input data.

These assumptions can change over time. This is commonly known as data drift.

Data engineers may encounter this when applying fuzzy patterns to extract information from free-text fields. A pattern may work well when users write notes in one style, then fail when a source form changes or users begin entering data differently.

For example, suppose a pattern extracts phone numbers from customer notes. The data engineer may monitor the proportion of extracted numbers that match known customer phone records.

Example drift monitoring

Month	Notes processed	Notes with extracted phone number	Extracted numbers matching customer records
2025-01	12,400	4,820	86%
2025-02	13,100	5,040	85%
2025-03	12,850	4,910	84%
2025-04	13,300	5,110	85%
2025-05	13,050	7,900	61%

The May result may indicate drift. Users may have started entering phone numbers in a new format. Another number may have started appearing near phone-like words. The customer reference table may also have changed.

A monitored assumption should return rows, periods, or segments where the statistic moves outside tolerance.

The important point is that fuzzy logic should not be treated as set-and-forget. Its assumptions must be monitored.

25.4 Tests and assumptions together

Tests and monitored assumptions are closely related, but they protect different things.

Tests detect whether implemented logic has failed. Monitored assumptions detect whether the

world has changed in a way that makes the logic unsafe.

A condition can often be expressed either as a test or as a monitored assumption by rephrasing. For example, row counts before and after transformation should usually be expressed as tests. A discrepancy may indicate a serious error in the pipeline.

New country codes requiring annotation are better expressed as monitored assumptions. Their appearance does not necessarily mean the pipeline has failed. It means the world has produced something that requires human attention.

A useful rule of thumb is:

Condition	Prefer	Reason
Critical correctness condition	Test	Failure suggests the product may be wrong.
Slow-moving business condition	Monitored assumption	Failure suggests human attention is needed.
New values or mappings	Monitored assumption	The product needs curation, not necessarily shutdown.
Core reconciliation or row preservation	Test	Failure suggests the transformation may have broken.
Statistical or fuzzy pattern stability	Monitored assumption	Failure suggests logic may need review.

25.5 Conclusion

Tests and assumptions accelerate delivery rather than slow it down. They reduce the cost of change. They allow data engineers to modify complex transformations with confidence. They make it safer to refactor code, adjust business logic, optimise performance, and release improvements without relying on hope.

If something is easy to test, it should be tested because the cost is low. If something is hard to test, it definitely should be tested because the logic is complex. Either way, it should be tested.

Thoughtful tests are best, but even simple tests are valuable. Many errors in data products are not profound misunderstandings. They are careless mistakes made during rapid development. Simple tests and assumptions can catch these before they reach the user.

New engineers naturally spend more time choosing patterns and building their implementation. With experience, design becomes more mechanical and rapid. Mature engineers dealing with greater complexity spend proportionately more time anticipating failure: defining tests, monitoring assumptions, and making sure that the data product will reveal when it is no longer safe.

Key ideas.

Data engineering is not only building the transformation that works. It is building the means by which failure becomes visible.

The third principle of data engineering is to **anticipate errors**.

Tests check the same result in two independent ways.

Tests are strongest when expected and actual calculations are not entangled.

Monitored assumptions return records that require human attention.

Tests detect failures in implemented logic.

Monitored assumptions detect changes that make the logic unsafe.

Fault tolerance

Good engineering contains errors before they spread through the pipeline.

26.1 Containing failure

Good engineering contains failure before it spreads.

A brittle pipeline treats every error as catastrophic. A fault-tolerant pipeline distinguishes between errors that should stop everything, errors that should stop one table, and errors that should be isolated to a small set of records.

The goal is not to ignore errors. The goal is to keep the rest of the data product usable while surfacing failure clearly.

On the other hand, surfacing failure is not the same as producing endless alerts.

If the same warning appears every day and no one knows what to do with it, the warning becomes background noise rather than a warning.

This is the cry-wolf problem of monitoring. Repeated, unactionable alerts train people to ignore the pipeline.

For this reason, fault tolerance requires judgement. Some errors should abort a load. Some occasional errors should be isolated into reject tables while the rest of the pipeline proceeds. Some recurring errors should be given targeted handling so that the pipeline explicitly represents the known issue rather than repeatedly rediscovering it.

This is fault tolerance.

Fault tolerance is therefore an extension of the third principle of data engineering: anticipate errors.

This chapter introduces three common fault patterns:

Fault pattern	Core question	Example failure
Uniqueness	Does one real-world entity correspond to the right number of records?	Duplicate submissions, duplicated incremental loads, or two people sharing one identifier.
Existence	Are required records and values present where they should be?	A detail row without a header, a missing sales amount, or a deleted source record not removed downstream.

Fault pattern	Core question	Example failure
Stability	Are small real-world changes producing proportionate data-world changes?	One source update timestamp causes every historical row to reload.

These patterns illustrate the mindset of building a fault-tolerant pipeline.

26.2 Uniqueness

Uniqueness faults occur when the relationship between real-world entities and database records breaks down.

If the basic task of data engineering is to represent real-world objects and processes in the data world, then uniqueness is one of the most fundamental expectations.

In the simplest case, one real-world entity should correspond to one database record.

Uniqueness is violated when one real-world entity appears as multiple database records. For example, a staff member may accidentally enter the same transaction twice.

The reverse can also occur. Two distinct real-world entities may be forced to occupy the same database record. For example, if staff are identified only by first and last name, two staff members with the same name may collapse into one identifier value.

Uniqueness violations can arise from business process failures, such as duplicate entry. They can also arise from mechanical failures, such as incremental load logic incorrectly loading the same record twice.

The data engineer must express uniqueness expectations in the warehouse. They are physical expressions of business intent.

If the constraint is violated, the pipeline has several possible responses:

Response	Use when...
Abort the load	The violation suggests the table is unsafe to publish.
Reject the violating records	The invalid records can be isolated while the rest of the table remains usable.
Deduplicate automatically	The business rule for choosing the accepted record is clear and safe.

For example, a source table may contain duplicate submissions.

Example structure before deduplication

Submission ID	Customer ID	Submission date	Submission amount
S1001	C001	2025-04-01	120.00
S1002	C002	2025-04-01	95.00

Submission ID	Customer ID	Submission date	Submission amount
S1003	C001	2025-04-01	120.00
S1004	C003	2025-04-02	80.00

If the business expects one submission per customer per date, the duplicate can be separated from the main analytical output.

Example structure of accepted records

Submission ID	Customer ID	Submission date	Submission amount
S1001	C001	2025-04-01	120.00
S1002	C002	2025-04-01	95.00
S1004	C003	2025-04-02	80.00

Example structure of rejected records

Submission ID	Customer ID	Submission date	Submission amount	Rejection reason
S1003	C001	2025-04-01	120.00	Duplicate customer submission for date

The accepted records remain available for users. The duplicate record is not silently lost but surfaced for attention. This approach localises the fault.

If uniqueness violations are rare, a reject-record pattern may be sufficient. If violations are frequent, the table needs targeted handling. Repeated alerts can create a cry-wolf effect, dulling attention and undermining the purpose of monitoring.

In such cases, the data engineer should implement a specific treatment for the table. This may include deterministic deduplication, a remediation workflow, or a data quality report.

26.3 Existence

Existence faults occur when required records or values are missing, or when records remain in the data world after their real-world counterpart has disappeared.

Existence is violated when a real-world entity exists but does not appear in the database. For example, a completed sale may be missing its sales amount.

Existence is also violated when a database record exists but the real-world entity no longer does. For example, a record may fail to be deleted during an incremental load.

Some existence expectations are simple. A completed sales record must have a sales amount. This can be expressed through a not-null constraint.

Example structure of 'Sales' with missing value

Sales ID	Sales date	Customer ID	Sales amount
S1001	2025-04-01	C001	120.00
S1002	2025-04-02	C002	
S1003	2025-04-03	C003	95.00

If [Sales amount] is required for completed sales, then the missing value should be treated as a fault. Depending on the use case, the row may be rejected, flagged, or excluded from measures that require a sales amount.

More complex existence faults occur between related tables.

For example, a sales item should not exist without a corresponding sales header.

Example structure of Sales.Sales

Sales ID	Sales date	Customer ID
S1001	2025-04-01	C001
S1002	2025-04-02	C002

Example structure of Sales.SalesItems

Sales item ID	Sales ID	Product ID	Quantity
SI001	S1001	P100	1
SI002	S1001	P200	2
SI003	S999	P300	1

In this example, SI003 refers to S999, but the sales header does not exist.

This may occur because the header failed to load, the item arrived earlier than the header, or the source system produced an invalid relationship.

The data engineer can detect this by left-joining the detail table to the header table, then checking whether the header key is missing.

Conceptually, the check looks like this.

Example structure after checking for header existence

Sales item ID	Sales ID	Product ID	Quantity	Header sales ID
SI001	S1001	P100	1	S1001
SI002	S1001	P200	2	S1001
SI003	S999	P300	1	

The existence expectation can then be expressed as a not-null requirement on [Header sales ID]. If [Header sales ID] is blank, the detail row has no corresponding header.

When the not-null check fails, the pipeline can send the violating rows into a reject table and allow the rest of the detail table to proceed.

Example structure of accepted sales items

Sales item ID	Sales ID	Product ID	Quantity
SI001	S1001	P100	1
SI002	S1001	P200	2

Example structure of rejected sales items

Sales item ID	Sales ID	Product ID	Quantity	Rejection reason
SI003	S999	P300	1	Missing sales header

The rest of the table can continue to load, while the missing relationship is surfaced.

This is the same pattern as uniqueness handling: detect the violation, isolate the unsafe rows, preserve them for attention, and allow the safe records to continue.

Existence faults are especially important in batch and incremental pipelines. Tables may be extracted at different times. A header may arrive in one batch and detail records in another. In traditional warehousing this is often discussed as late-arriving data.

The data engineer should not assume that every related record will always arrive in the expected order.

For existence, the best mindset is to design the pipeline as if it were streaming.

Even in a batch pipeline, this is a useful design mindset:

- tables may load continuously, or at least several times a day;
- some tables may fail to load from time to time;
- related records may arrive out of order;
- records may catch up in a later batch.

In this mindset, temporary existence violations are expected. The pipeline should handle them safely.

However, safe handling must not become silent handling.

If a missing header catches up in the next batch, the issue may resolve. If it does not catch up, tests or monitored assumptions should surface the prolonged discrepancy.

26.4 Stability

Stability faults occur when small real-world changes produce disproportionate changes in the data world.

A stable pipeline changes in proportion to the world it represents.

If ten sales changed in the source system, the pipeline should not update ten million downstream records. If one reference value changed, half the warehouse should not change as a side effect.

Stability means that ordinary changes should produce ordinary effects.

This is important because the real world rarely rewrites all historical records at once. But the data world can suffer from massive accidental changes that do not correspond to real-world change.

For example, a source system update may refresh `[Row update datetime]` for every historical row. If the pipeline relies on `[Row update datetime]` for incremental extraction, it may attempt to reload the entire history.

Another example is a non-deterministic ranking. If a row number is assigned using an incomplete sort order, the selected “first” row may change between runs even when the underlying data has not changed.

A third example is a reporting column such as `[Today’s date]`. If this is stored on every row, every row changes every day even though the underlying business entity has not changed.

The data engineer should therefore design pipelines so that changes are commensurate with real-world change.

One way to do this is to avoid unstable elements:

Unstable element	Why it is risky
<code>[Today’s date]</code> stored on every row	Causes every row to change every day.
Non-deterministic ranking	Causes rows to change between runs without real-world change.
Incomplete tie-breaking logic	Makes selected records unstable.
Single-row control tables	Causes downstream volatility when append-only history would be safer.
Wide miscellaneous tables	Amplifies small changes because unrelated attributes share one row.

Another way is to use stability thresholds.

For example, the pipeline may abort or quarantine a table load if more than 5% of records are about to be deleted.

Example stability threshold

Table	Existing rows	Rows proposed for		Action
		deletion	Deletion percentage	
<code>Sales.Sales</code>	1,000,000	2,500	0.25%	Proceed
<code>Sales.Customer</code>	250,000	180,000	72.00%	Abort and alert

The threshold does not imply that the change is wrong. It says the proposed change is large enough to require attention before it is allowed to spread.

One important habit is to avoid wide tables with miscellaneous attributes. In a wide table, a

change in any cell can cause the whole row to update. This amplifies small real-world changes into broad database changes.

Narrow meaningful fragments are more stable. When each table has a clear informational purpose, changes to that table correspond more closely to changes in the thing it represents.

Another habit is to manage dependencies carefully. Small reference tables can have large downstream effects. If a country reference table fails to load, and downstream transformations depend on it early in the pipeline, country data may disappear across many products.

These design habits are developed further in [Load mechanics](#) and [Load dependencies](#).

26.5 Conclusion

Uniqueness, existence, and stability are three common fault patterns. But the possibilities of error are limitless. Parallel loads may deadlock. Source systems may change without notice. Files may arrive late. Users may enter unexpected values.

It is not useful to enumerate every possible fault.

The important thing is the mindset: anticipating errors.

The experienced data engineer designs for what the world might do to the pipeline, not only what the pipeline does to the data.

This is why the data engineer builds with fault tolerance: not every error is fatal, but none should pass silently.

Key ideas.

Fault tolerance is the discipline of containing failure.

Good engineering contains errors before they spread through the pipeline.

A fault-tolerant pipeline distinguishes between errors that should stop everything, errors that should stop one table, and errors that should be isolated to a small set of records.

Uniqueness faults occur when the relationship between real-world entities and database records breaks down.

Existence faults occur when required records or values are missing, or when records remain after their real-world counterpart has disappeared.

Stability faults occur when small real-world changes produce disproportionate changes in the data world.

Not every error is fatal, but none should pass silently.

Efficient & stable pipeline

Efficiency and stability

A mature pipeline keeps changes in computation aligned to changes in information.

27.1 Correspondence between information and computation

A mature pipeline maintains correspondence between changes in information and changes in computation.

When little has changed in the business world, little should need to change in the data world, and little should need to be recomputed. When that correspondence breaks down, the pipeline becomes inefficient, unstable, or both.

This chapter introduces efficiency and stability as two disciplines for preserving that correspondence.

Concept	Question
Efficiency	Is the pipeline doing more computational work than the information change requires?
Stability	Is the data world changing in proportion to the business world?

This is also the fourth principle of data engineering:

Instead of wholesale response, maintain proportionate change.

27.2 Efficiency

Developers often talk about fast and slow. But fast and slow are slippery terms.

Is one second fast? Is ten seconds slow? One second to process ten rows may be unacceptably slow. Ten seconds to process ten million rows may be impressively fast.

These judgements depend on infrastructure, data shape, implementation choices, and the value of the information being processed.

Because performance depends on many factors, it is more useful to focus first on efficiency.

Efficiency means using the minimum necessary resource to calculate the required information.

There are two kinds of efficiency:

Kind	Concern
Informational efficiency	Does the pipeline process only the information needed to reflect real change?
Algorithmic efficiency	Does the pipeline calculate that information as cheaply as the environment allows?

Informational efficiency is about processing as little information as possible. A pipeline is informationally inefficient if it processes hundreds of millions of rows when only a handful have changed.

Algorithmic efficiency is about computing those changes as cheaply as possible in a given computational environment. A pipeline is algorithmically inefficient if it takes seconds to process one row when the same result could be calculated in milliseconds.

Informational efficiency is largely universal across technologies. Algorithmic efficiency is more technology-dependent and varies from site to site.

Efficiency is important because it is central to sustainable warehouse growth. It also affects how quickly users receive results.

Some performance factors sit outside the data engineer’s immediate control: server configuration, infrastructure limits, source-system behaviour, or the business value of the information being processed. But the data engineer can still ensure that the pipeline does only the necessary work, and no more, to reflect real-world change.

27.3 Stability

Stability is closely related to efficiency.

Stability means that small changes in the business world should produce proportionate changes in the data world.

An unstable pipeline amplifies change. A small input change creates a large output change.

For example, changing a country name in a reference table might ripple through and rewrite many downstream tables. A more dramatic example was the fear of the Y2K bug, where systems storing years using only two digits risked treating the change from 1999 to 2000 as a catastrophic discontinuity rather than an ordinary date transition.

Stability is important because it shapes the user’s experience of reliability.

Reports that suddenly become empty or radically different undermine trust. There is little value in explaining that the issue came from a small reference-data change, a refreshed update timestamp, or a ripple effect in the pipeline. As far as the user is concerned, the product became unreliable.

A stable pipeline does not merely run successfully. It changes in ways that correspond to changes in the world it represents.

27.4 Efficiency and stability together

Efficiency and stability are related but distinct.

A pipeline can be algorithmically inefficient without being unstable. For example, a pipeline may scan a full source table every night to identify a few changed rows, then update only those changed rows in the warehouse. The final data-world change is proportionate, because only the changed rows are updated. However, the computation is inefficient, because the pipeline read far more source data than the information change required.

A pipeline can also be unstable without being algorithmically inefficient. For example, truncate-and-reload may be the cheapest implementation for a small table. In computational terms, this may be acceptable. But it is unstable because the table temporarily disappears or becomes incomplete during the load. If the reload fails after the truncate, the data product is left in an unsafe state.

Sometimes inefficiency and instability overlap. For example, a large table with a [Today's date] column may update every row every day. This is unstable because the table changes constantly even when the underlying business entities have not changed. It is also informationally inefficient because the pipeline performs large amounts of work for little business value.

In all cases, the problem is broken correspondence.

The amount of data change and computational work no longer reflects the amount of information change.

Meaningful fragments help preserve this correspondence. When each table has a clear informational purpose, changes can stay local. A change in one concept does not need to churn unrelated data. This makes the pipeline easier to load, easier to test, easier to reason about, and more stable under change.

The chapters in **Efficiency & stability** explore how to build pipelines that are efficient and stable over time.

Key ideas.

The fourth principle of data engineering is **proportionate change**.

Informational efficiency means processing only the information needed to reflect real-world change.

Algorithmic efficiency means calculating that information as cheaply as the environment allows.

Stability means that small changes in the business world should produce proportionate changes in the data world.

Inefficiency and instability both occur when pipeline work stops corresponding to information change.

Meaningful fragments help preserve correspondence by keeping change local, legible, and proportionate.

Load mechanics

A mature load applies only genuine changes, after checking that they are safe.

28.1 Controlling change

Efficiency and stability both depend on the warehouse remaining in control of change.

Efficiency requires the warehouse to know which rows genuinely changed, so it does not waste work reprocessing rows whose information stayed the same.

Stability requires the warehouse to know whether the proposed changes are safe, so abnormal changes do not spread through the pipeline.

Therefore, the purpose of a load is not only to keep the target table current. The purpose is to keep the warehouse in control of change.

For every change applied to a table, the warehouse should be able to explain why the change happened, whether it should happen, what changed, and when it changed.

A naive load loses this control.

When a table is replaced wholesale, the pipeline can no longer reason precisely about change. Every row may appear new. Unchanged rows are mixed with changed rows. Unsafe rows are mixed with safe rows. Deletes are difficult to distinguish from load failures. Downstream tables cannot respond with precision because the warehouse has not preserved the meaning of what happened.

This is inefficient because downstream tables may need to respond to rows that did not genuinely change. It is unstable because one abnormal load may cause a disproportionate change in the target and downstream tables.

The extra work in controlling change takes effort, but it lays the foundation for an efficient and stable pipeline. This is the topic of **load mechanics**.

The standard pattern has three steps:

Step	Question
Stage	What incoming data is available?
Check	Which rows genuinely changed, and are those changes safe?
Apply	How should the target table be updated without losing history or spreading faults?

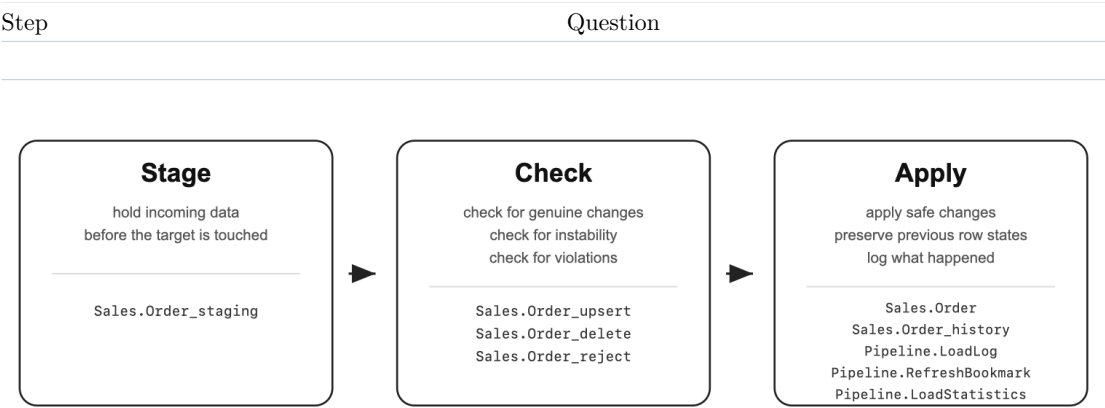


Figure 1. The load mechanics pattern: stage incoming data, check proposed changes, then apply safe changes and log what happened.

28.2 The three-step load pattern

Suppose the target table is `Sales.Order`. The goal is to update `Sales.Order` with the latest batch of data.

For this pattern to work, the target table must have a defined primary key.

In this example, `Sales.Order` is keyed by `[Order ID]`.

For any target table, the standard load pattern may create the following artefacts:

Artefact	Purpose
Sales.Order_staging	Incoming data before Sales.Order is touched.
Sales.Order_upsert	Rows that are new or genuinely changed.
Sales.Order_delete	Rows that should be removed from the current target.
Sales.Order_reject	Changed rows that violate constraints and should not enter the target.
Sales.Order_history	Previous versions of rows that were updated or deleted.

This may seem like a lot of machinery. The rest of this chapter explains why each part exists.

28.2.1 Step 1—Stage the incoming data

The first step is to load the latest batch of data into a staging table.

The staging table holds incoming data before the target table is touched. For `Sales.Order`, this table is `Sales.Order_staging`.

`Sales.Order_staging` may contain a full extract of the source data. It may also contain only

a subset of recently changed records. The latter is an incremental extract.

An incremental extract can reduce the number of records to stage, check, and apply from hundreds of millions to a few thousand. The ways to do this reliably are covered in [Tracking changes](#) and [Responding to changes](#).

The staging step applies regardless of full or incremental extract.

Suppose the current target table looks like this.

Current Sales.Order

Order ID	Customer ID	Order reference	Order status	Order amount	Row insert datetime	Row update datetime
O1001	C001	REF-1001	Submitted	120.00	2025-04-01 09:00	2025-04-01 09:00
O1002	C002	REF-1002	Submitted	95.00	2025-04-01 09:05	2025-04-01 09:05
O1003	C003	REF-1003	Cancelled	80.00	2025-04-01 09:10	2025-04-01 09:10

The incoming staging table may look like this.

Incoming Sales.Order_staging

Order ID	Customer ID	Order reference	Order status	Order amount
O1001	C001	REF-1001	Submitted	120.00
O1002	C002	REF-1002	Fulfilled	95.00
O1004	C004	REF-1004	Submitted	140.00
O1005		REF-1005	Submitted	75.00
O1006	C006	REF-1003	Submitted	160.00

At this point, the target table has not yet changed. The incoming data is available for checking.

28.2.2 Step 2—Check the incoming data

The Check step compares `Sales.Order_staging` against `Sales.Order` before loading data into the target.

The Check step asks three kinds of question:

Check	Question
Genuine change	Which rows should be inserted, updated, or deleted?
Instability	Are there abnormally high numbers of inserts, updates, or deletes?
Violations	Are there rows that should be rejected rather than loaded?

Checking for genuine changes is essential. Checking for instability is highly desirable. Checking for violations is a practical way to improve fault tolerance, because a small number of bad rows can be rejected while the rest of the table proceeds.

Instability and violation checks are part of [Fault tolerance](#).

Check for genuine changes

Not every row in `Sales.Order_staging` is a genuine change.

During a full extract, millions of rows may arrive, but only a small number may differ from the current target. During an incremental extract, the incoming rows may only be candidates for change. Some may still be identical to the current target.

The Check step identifies which rows genuinely need action by comparing rows on the primary key, then comparing non-key values for rows whose primary key already exists.

There are three possibilities:

Change type	Meaning
Insert	A primary key exists in <code>Sales.Order_staging</code> but not in <code>Sales.Order</code> .
Update	A primary key exists in both tables, but one or more non-key values differ.
Delete	A primary key exists in <code>Sales.Order</code> but should no longer exist in the final target.

In the example:

Order ID	Interpretation
O1001	Unchanged.
O1002	Existing row with changed status, so it is an update.
O1003	Missing from a full extract, so it is a delete.
O1004	New row, so it is an insert.
O1005	New row, so it is an insert, but <code>[Customer ID]</code> is missing.
O1006	New row, so it is an insert, but its <code>[Order reference]</code> clashes with existing order O1003.

Inserts and updates are isolated into an upsert table.

The first version of the upsert table contains all rows that are new or genuinely changed. It also has a column `[Is new row]` to indicate whether the row is to be inserted or not.

Example structure of Sales.Order_upsert before checking violations

Order ID	Customer ID	Order reference	Order status	Order amount	Is new row
O1002	C002	REF-1002	Fulfilled	95.00	false
O1004	C004	REF-1004	Submitted	140.00	true
O1005		REF-1005	Submitted	75.00	true
O1006	C006	REF-1003	Submitted	160.00	true

O1001 does not appear because it has not changed.

If `Sales.Order_staging` is a full extract, deletes can be identified by finding rows in `Sales.Order` whose primary key values no longer appear in `Sales.Order_staging`.

Example structure of Sales.Order_delete

Order ID
O1003

If `Sales.Order_staging` is an incremental extract, delete detection must be tailored to the incremental logic. The staging table may contain only a fraction of the desired final table, so absence from staging does not necessarily mean deletion from the target.

This distinction is covered in [Responding to changes](#).

Check for instability

Once `Sales.Order_upsert` and `Sales.Order_delete` have been calculated, the pipeline can check whether the proposed changes are abnormal.

These checks are stability thresholds.

A stability threshold does not prove that the proposed change is wrong. It says the proposed change is large enough to require attention before it is allowed to spread.

For example, suppose `Sales.Order` normally changes by a small percentage each day. If half the table is about to be updated or deleted, this may indicate a source-system issue, extraction error, or accidental change in load logic.

Example stability check

Check	Existing rows	Proposed changed rows	Percentage changed	Action
Upsert threshold	100,000	2,500	2.5%	Proceed
Delete threshold	100,000	45,000	45.0%	Abort and alert

The threshold for deletes should usually be lower than the threshold for upserts. In many business processes, deletes are rarer than inserts or updates.

A simple default threshold can catch most serious abnormalities across a pipeline. Specific tables may need tailored thresholds based on their normal pattern of change.

Logging change statistics over time helps set these thresholds. If a table normally updates between 0.5% and 2% of rows each day, then a 20% update is worth investigating.

If a table frequently breaches its threshold, the data engineer should not simply increase the threshold until the alert disappears. The extraction logic, source behaviour, and business process should be understood. The alert may be exposing instability that should be fixed.

Check for violations

After genuine changes have been identified, the pipeline should check whether any rows in `Sales.Order_upsert` are unsafe to load.

A brittle load may fail the whole table. A fault-tolerant load can move the unsafe row from `Sales.Order_upsert` into `Sales.Order_reject`, then allow the remaining safe rows to proceed.

While the pipeline can contain tailored rules for each table, it is useful to check for two common violations: existence and uniqueness constraints.

For example, if `[Customer ID]` is required for `Sales.Order`, then an incoming order with a null `[Customer ID]` violates the existence expectation for the table.

Checking for uniqueness is more complex.

Suppose `Sales.Order` has a unique column `[Order reference]`. The pipeline needs to check two kinds of uniqueness violation.

First, the incoming rows may contain duplicates within the upsert set itself. This occurs when two rows in `Sales.Order_upsert` have the same `[Order reference]`.

Second, an incoming row may clash with an existing row in `Sales.Order`. This occurs when applying the upsert would create an `[Order reference]` that already exists on another row in the target table.

The second case must be checked carefully. It is not a violation if the incoming row is updating its own existing record and keeping the same `[Order reference]`. It is also not necessarily a violation if another row is changing away from that value in the same load. The question is whether the final target table would contain two rows with the same unique value after the upsert is applied.

These violations should be sent to a reject table.

Example structure of `Sales.Order_reject`

Order ID	Customer ID	Order reference	Order status	Order amount	Rejection reason
O1005		REF-1005	Submitted	75.00	Existence violation

Order ID	Customer ID	Order reference	Order status	Order amount	Rejection reason
O1006	C006	REF-1003	Submitted	160.00	Uniqueness violation

After rejection, `Sales.Order_upsert` contains only safe rows to apply.

Example structure of `Sales.Order_upsert` after checking violations

Order ID	Customer ID	Order reference	Order status	Order amount	Is new row
O1002	C002	REF-1002	Fulfilled	95.00	false
O1004	C004	REF-1004	Submitted	140.00	true

The table is now safe to load.

28.2.3 Step 3—Apply the changes

After the Check step, the pipeline has separated incoming data into the actions it needs to take:

Artefact	Action
<code>Sales.Order_upsert</code>	Insert new rows and update changed rows.
<code>Sales.Order_delete</code>	Remove rows that should no longer be current.
<code>Sales.Order_reject</code>	Preserve unsafe rows for attention, without loading them into the target.

The Apply step updates the target table in a controlled way.

First, rows marked for deletion should be removed from the current target. But they should not simply disappear. Their previous values should be preserved in `Sales.Order_history`. This is not merely for archival purposes, but to inform downstream processes which rows have been deleted, so they can respond accordingly.

Second, rows in `Sales.Order_upsert` where `[Is new row] = false` should be used to update existing rows in the target. Again, the previous values should be preserved in `Sales.Order_history`.

Third, rows in `Sales.Order_upsert` where `[Is new row] = true` should be inserted into the target.

The Apply step should also manage architectural columns for change datetimes:

Column	Meaning
<code>[Row insert datetime]</code>	When the row, defined by its primary key, first entered the target table.
<code>[Row update datetime]</code>	When the row, defined by its primary key, was most recently updated with a genuine change.

Column	Meaning
[Row delete datetime]	When the row stopped being current. Current rows use 9999-12-31 00:00:00 to indicate that they have not been deleted.

Because the Check step removed unchanged rows, [Row update datetime] is updated only when row content genuinely changes.

As we will see in [Tracking changes](#), these artefacts are necessary for downstream processing.

After applying the change, the target may look like this.

Updated Sales.Order

Order ID	Customer ID	Order reference	Order status	Order amount	Row insert datetime	Row update datetime	Row delete datetime
O1001	C001	REF-1001	Submitted	120.00	2025-04-01 09:00	2025-04-01 09:00	9999-12-31 00:00:00
O1002	C002	REF-1002	Fulfilled	95.00	2025-04-01 09:05	2025-05-01 08:00	9999-12-31 00:00:00
O1004	C004	REF-1004	Submitted	140.00	2025-05-01 08:00	2025-05-01 08:00	9999-12-31 00:00:00

O1001 remains unchanged. O1002 has been updated. O1003 has been deleted from the current table. O1004 has been inserted. O1005 has been rejected because [Customer ID] is missing. O1006 has been rejected because its [Order reference] clashes with an existing target row.

The history table preserves previous versions of updated and deleted rows.

Example structure of Sales.Order_history

Order ID	Customer ID	Order reference	Order status	Order amount	Row insert datetime	Row update datetime	Row delete datetime
O1002	C002	REF-1002	Submitted	95.00	2025-04-01 09:05	2025-04-01 09:05	2025-05-01 08:00
O1003	C003	REF-1003	Cancelled	80.00	2025-04-01 09:10	2025-04-01 09:10	2025-05-01 08:00

28.3 After the load

The loading process finishes with cleaning up temporary artefacts and logging what happened.

28.3.1 Clean up temporary artefacts

If there are no errors, temporary tables such as `Sales.Order_staging`, `Sales.Order_upsert`, and `Sales.Order_delete` do not usually need to be retained.

`Sales.Order_history` may also be purged after a suitable period, once downstream tables have had time to respond to updates and deletes. In Delta table contexts, purging old removed rows is often known as vacuuming.

If there are errors, temporary artefacts and `Sales.Order_reject` may need to be retained for troubleshooting.

These artefacts should be secured in the same way as the target table. Temporary load tables can contain the same sensitive information as the production table. They should not become a source of unintended data leakage.

28.3.2 Log, bookmarks and change statistics

The pipeline should log enough information to explain what happened during each load.

At minimum, the log should record whether the load succeeded, failed, or was aborted, along with any associated failure message.

Example structure of Pipeline.LoadLog

Load ID	Table name	Load start datetime	Load end datetime	Load status	Failure message
10001	Sales.Order	2025-05-01 08:00	2025-05-01 08:03	Succeeded	
10002	Sales.Customer	2025-05-01 08:03	2025-05-01 08:05	Succeeded	
10003	Sales.Payment	2025-05-01 08:05	2025-05-01 08:06	Failed	Source extract timeout
10004	Sales.Product	2025-05-01 08:06	2025-05-01 08:06	Aborted	Delete threshold exceeded

For incremental processing, the pipeline should also log the table's refresh bookmark.

A refresh bookmark records when the table started reading from its source data, if the load completed successfully. Suppose `Sales.Order` draws from several source tables. Any source records that appear after the bookmark belong to a later load.

This bookmark gives the next load a safe starting point for incremental extraction. This is explained in greater detail in [Tracking changes](#).

The refresh bookmark should only be advanced when the load succeeds. If the load fails or aborts, the bookmark should not move forward, because the next load still needs to consider the same source period.

Example structure of Pipeline.RefreshBookmark

Table name	Load ID	Refresh bookmark datetime
Sales.Order	10001	2025-05-01 08:00
Sales.Customer	10002	2025-05-01 08:03

In this example, **Sales.Order** completed successfully, so its bookmark is set to 2025-05-01 08:00. **Sales.Payment** failed, so no bookmark is recorded. **Sales.Product** was aborted by a stability threshold, so its bookmark is also not advanced.

The Check step also calculates useful change statistics. Since these statistics have already been calculated as part of the load, they are worth logging.

Example structure of Pipeline.LoadStatistics

Load ID	Rows staged	Rows inserted	Rows updated	Rows deleted	Rows rejected	Target row count
10001	5	1	1	1	2	3
10002	1,250	12	34	0	0	42,810

The change statistics should normally be logged only for successful loads. For failed or aborted loads, the failure is recorded in **Pipeline.LoadLog**, but the bookmark and final change statistics are not advanced.

Over time, these logs form a history of how each table behaves.

That history is valuable for troubleshooting, setting stability thresholds, capacity planning, and understanding whether a table’s behaviour has changed. For example, if **Sales.Order** normally updates between 1% and 3% of rows each day, a load that updates 40% of rows is immediately suspicious. Without change statistics, the pipeline has no memory of what normal looks like.

28.4 Is the extra work worth it?

At first glance, the three-step load pattern looks like overhead.

For one target table, the pattern may create several temporary or supporting artefacts: staging, upsert, delete, reject, and history tables. It also creates architectural columns such as **[Row insert datetime]**, **[Row update datetime]**, and **[Row delete datetime]**. The logic is much more complex than dropping a table and replacing it.

In a small environment, this may be unnecessary. If there are few tables, little downstream dependency, and low business risk, a simpler approach may be enough.

But as the pipeline grows, the calculation changes.

The extra work is necessary because the warehouse needs to know and control every change that happens within it. Each artefact preserves a distinction that the warehouse needs in order to achieve this.

Artefact	Why it matters
<code>Sales.Order_staging</code>	Holds incoming data separately so the target is not touched until the batch has been checked.
<code>Sales.Order_upsert</code>	Identifies candidate inserts and updates before they are applied, allowing the pipeline to separate genuine changes from unchanged rows.
<code>Sales.Order_delete</code>	Identifies candidate deletes before they are applied, allowing the pipeline to check whether the volume of deletion is safe before rows are removed from the current target.
<code>Sales.Order_reject</code>	Isolates unsafe changed rows so they can be reported and remediated without contaminating the target or stopping safe rows from loading.
<code>Sales.Order_history</code>	Preserves previous versions of updated and deleted rows. This is not merely archival: downstream tables need history to know that a row changed or stopped being current.
Row change datetimes	Allow downstream incremental processing to respond precisely to inserts, updates, and deletes.
<code>Pipeline.LoadLog</code>	Records success, failure, aborts, and error messages, so the pipeline can be troubleshooted and audited.
<code>Pipeline.RefreshBookmark</code>	Records where the next incremental load should restart, but only after a successful load.
<code>Pipeline.LoadStatistics</code>	Builds a history of how each table changes over time, supporting stability thresholds, capacity planning, and diagnosis of abnormal behaviour.

This is the price of controlled change.

Without these artefacts, the warehouse may still produce a current-looking table, but it cannot explain how that table changed. It cannot reliably tell downstream processes which rows require action. It cannot distinguish genuine change from noisy candidate change. It cannot isolate unsafe rows and report on errors. It cannot remember what happened during the load.

The value of the pattern grows with scale. On one table, the extra machinery may seem excessive. Across a serious pipeline, it becomes the backbone of efficiency, stability, and fault tolerance.

Investment in automation or appropriate technology ensures that this logic can be applied by default, without requiring the data engineer to handcraft every part for every table.

Key ideas.

A mature load applies only genuine changes, after checking that they are safe.

Load mechanics are about keeping the warehouse in control of change.

Drop-and-replace loading is simple, but it hides genuine change, creates instability, and magnifies failure.

The standard load pattern is stage, check, and apply.

Staging tables hold incoming data before the target table is touched.

Upsert, delete, and reject tables separate genuine changes, unsafe changes, and candidate deletions before anything is applied.

History tables preserve previous versions of updated and deleted rows for downstream processing.

Row change datetimes allow downstream tables to respond precisely to inserts, updates, and deletes.

Load logging records bookmarks, change statistics, and success or failure so the pipeline can explain what happened.

Load orchestration

Load dependencies are one of the first bottlenecks in a growing warehouse.

29.1 Load stack

Load dependencies are one of the first bottlenecks in a growing warehouse.

The previous chapter explained how to load one table safely. A real pipeline must load many tables safely, in the right order, without making independent tables wait unnecessarily.

At small scale, load order can be managed manually. A developer can hard-code that customers load before orders, orders load before order items, and reference tables load before the facts that use them.

This does not last.

As the warehouse grows, dependencies multiply. A pipeline may contain hundreds or thousands of tables, with tens of thousands of dependency relationships between them. At that scale, load order cannot be safely maintained as a hand-coded sequence.

The **load stack** approach begins from a simple observation: if the warehouse knows which tables depend on which other tables, load order can be computed at run time.

29.2 Orchestrating a load

The load stack approach rests on three objects.

Object	Role
<code>Pipeline.LoadDependency</code>	Records dependency structure: which tables depend on which upstream tables.
<code>Pipeline.LoadStack</code>	Records execution state for the current workflow: which tables have started and ended.
<code>Pipeline.LoadCandidate</code>	A view which combines dependency structure and execution state to show which tables are ready to load now.

The dependency table by itself knows the structure of the pipeline, but not what has already

loaded.

The load stack by itself knows what has started and ended, but not what depends on what.

The load candidate view combines both. It asks: given the dependency structure, and given the current execution state, which tables are ready to load now?

Once readiness is visible, workers can claim ready tables independently at run-time, without pre-planned orchestration.

29.2.1 Dependency metadata

Dependency metadata records which tables must be loaded before another table can load.

Suppose a sales pipeline contains six tables:

- Sales.Customer
- Sales.Product
- Reference.Calendar
- Sales.Order
- Sales.OrderItem
- Sales.OrderSummary

The dependency graph might look like this.

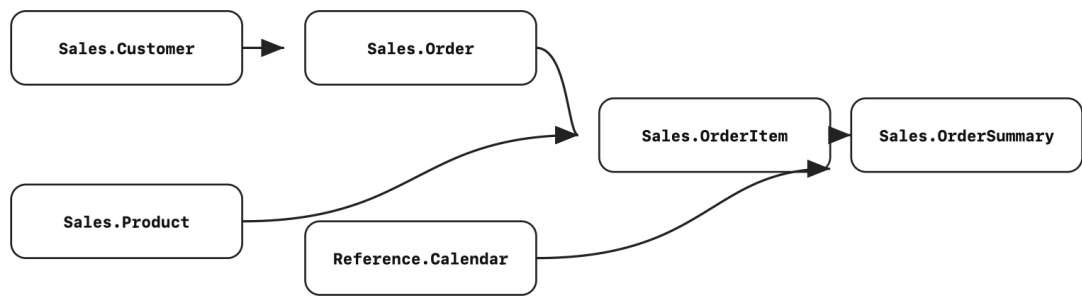


Figure 1. Example dependency graph for a sales pipeline. A table can load only after all upstream tables pointing into it have finished.

The same dependency structure can be recorded in a metadata table.

Example structure of Pipeline.LoadDependency

Table name	Depends on table name
Sales.Order	Sales.Customer
Sales.OrderItem	Sales.Order
Sales.OrderItem	Sales.Product
Sales.OrderSummary	Sales.Order
Sales.OrderSummary	Sales.OrderItem
Sales.OrderSummary	Reference.Calendar

Each row states one direct dependency.

`Sales.Order` depends on `Sales.Customer`. `Sales.OrderItem` depends on both `Sales.Order` and `Sales.Product`. `Sales.OrderSummary` depends on `Sales.Order`, `Sales.OrderItem`, and `Reference.Calendar`.

Some tables have no dependencies. In this example, `Sales.Customer`, `Sales.Product`, and `Reference.Calendar` can load at the beginning of the workflow because nothing upstream needs to finish first.

29.2.2 The load stack

The load stack is the list of tables to be loaded in a workflow.

At the beginning of the workflow, the stack is refreshed. Every table that belongs to the workflow is added to `Pipeline.LoadStack`, with `[Is started] = false` and `[Is ended] = false`.

Initial `Pipeline.LoadStack`

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Customer	false	false	
9001	Sales.Product	false	false	
9001	Reference.Calendar	false	false	
9001	Sales.Order	false	false	
9001	Sales.OrderItem	false	false	
9001	Sales.OrderSummary	false	false	

The load stack records execution state. It does not itself decide which table is ready.

A table is ready only when all the tables it depends on have ended. This condition is called execution readiness.

29.2.3 Load candidates

A load candidate is a table in the load stack that is ready to load.

At the beginning of the workflow, only tables with no unfinished dependencies are ready.

Conceptually, a table is a candidate when:

- it is in the current load stack;
- it has not started;
- none of its dependencies are unfinished.

This logic also works for tables with no dependencies. Their dependency list is empty, so they have no unfinished dependencies.

A load candidate view can expose this list at any time. The view can be simplified if `Pipeline.LoadDependencies` has already been expanded to contain all upstream dependencies, not only direct dependencies.

Initial Pipeline.LoadCandidate

Table name	Reason
Sales.Customer	No dependencies.
Sales.Product	No dependencies.
Reference.Calendar	No dependencies.

29.3 Using the load stack

The load stack can be used to implement dependency-aware parallel loading.

The basic algorithm is simple.

29.3.1 Step 1—Refresh the load stack

At the beginning of the workflow, populate `Pipeline.LoadStack` with the tables that need to load.

Set `[Is started] = false` and `[Is ended] = false` for all rows. This creates a visible queue of work for the workflow.

The stack may contain the full pipeline, or only a selected part of it. If a subset of tables is being loaded, the stack should include the necessary upstream tables required for that subset to load correctly.

29.3.2 Step 2—Start workers

Start any number of workers.

Workers may be stored procedures, functions, notebooks, jobs, or other executable units. Each worker repeatedly checks the load stack until there is no more work to claim.

A worker performs the following loop:

1. Check whether any unstarted tables remain in `Pipeline.LoadStack`.
2. Query `Pipeline.LoadCandidate`.
3. If no candidate is available, pause briefly and poll again.
4. If a candidate is available, claim one table by setting `[Is started] = true`.
5. Load the table.
6. Set `[Is ended] = true` when the load finishes, fails, or aborts.
7. Repeat until no unstarted tables remain.

The worker exits when there are no rows in `Pipeline.LoadStack` where `[Is started] = false`.

29.3.3 Step 3—Claim a candidate

Once the workers have started, they can pick up load candidates for load.

If three workers are available at the start of the example workflow, each can claim one initial candidate.

After workers claim the initial candidates

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Customer	true	false	W01
9001	Sales.Product	true	false	W02
9001	Reference.Calendar	true	false	W03
9001	Sales.Order	false	false	
9001	Sales.OrderItem	false	false	
9001	Sales.OrderSummary	false	false	

A table that has started but not ended is not available to other workers. Downstream tables also remain blocked until their dependencies have ended.

Claiming from the load stack must be done in a way that prevents multiple workers from claiming the same table. This can be achieved by wrapping the read from the load candidate view and the update to the load stack in a transaction. This avoids race conditions.

A simplified claim pattern is:

```
begin transaction;

select top 1
    [Table name]
from Pipeline.LoadCandidate
where [Workflow ID] = 9001
order by [Table name];

update Pipeline.LoadStack
set
    [Is started] = 1,
    [Worker ID] = 'W01',
    [Load start datetime] = current_timestamp
where
    [Workflow ID] = 9001
    and [Table name] = @TableName
    and [Is started] = 0;

commit transaction;
```

The exact syntax will vary by SQL platform. The important point is that the worker should not select and claim a candidate in two unsafe steps. The claim should happen atomically.

29.3.4 Step 4—Load, end, and repeat

When a table finishes loading, the worker marks it as ended.

```
update Pipeline.LoadStack
set
  [Is ended] = 1,
  [Load end datetime] = current_timestamp
where
  [Workflow ID] = 9001
and [Table name] = 'Sales.Customer'
and [Worker ID] = 'W01';
```

Suppose `Sales.Customer` finishes first.

After `Sales.Customer` ends

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Customer	true	true	W01
9001	Sales.Product	true	false	W02
9001	Reference.Calendar	true	false	W03
9001	Sales.Order	false	false	
9001	Sales.OrderItem	false	false	
9001	Sales.OrderSummary	false	false	

`Sales.Order` now becomes a load candidate because its only dependency, `Sales.Customer`, has ended.

Current `Pipeline.LoadCandidate`

Table name	Reason
Sales.Order	Sales.Customer has ended.

A worker can now claim `Sales.Order`.

After `Sales.Order` is claimed

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Customer	true	true	W01
9001	Sales.Product	true	false	W02
9001	Reference.Calendar	true	false	W03
9001	Sales.Order	true	false	W01
9001	Sales.OrderItem	false	false	
9001	Sales.OrderSummary	false	false	

Suppose `Sales.Product`, `Reference.Calendar`, and `Sales.Order` have all finished.

After Sales.Product, Reference.Calendar, and Sales.Order end

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Customer	true	true	W01
9001	Sales.Product	true	true	W02
9001	Reference.Calendar	true	true	W03
9001	Sales.Order	true	true	W01
9001	Sales.OrderItem	false	false	
9001	Sales.OrderSummary	false	false	

Sales.OrderItem is now ready because both of its dependencies have ended.

Current Pipeline.LoadCandidate

Table name	Reason
Sales.OrderItem	Sales.Order and Sales.Product have ended.

Sales.OrderSummary is not ready yet. Even though Sales.Order and Reference.Calendar have ended, Sales.OrderItem has not.

After a worker claims and finishes Sales.OrderItem, the stack looks like this.

After Sales.OrderItem ends

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Customer	true	true	W01
9001	Sales.Product	true	true	W02
9001	Reference.Calendar	true	true	W03
9001	Sales.Order	true	true	W01
9001	Sales.OrderItem	true	true	W02
9001	Sales.OrderSummary	false	false	

Sales.OrderSummary is now ready.

Current Pipeline.LoadCandidate

Table name	Reason
Sales.OrderSummary	Sales.Order, Sales.OrderItem, and Reference.Calendar have ended.

After Sales.OrderSummary ends, the workflow is complete.

Completed Pipeline.LoadStack

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Customer	true	true	W01

Workflow ID	Table name	Is started	Is ended	Worker ID
9001	Sales.Product	true	true	W02
9001	Reference.Calendar	true	true	W03
9001	Sales.Order	true	true	W01
9001	Sales.OrderItem	true	true	W02
9001	Sales.OrderSummary	true	true	W03

The workflow ends when there are no rows in `Pipeline.LoadStack` where `[Is started] = false`.

29.3.5 Handling failure

If a table fails or aborts, the worker still marks it as ended.

This is deliberate. In this pattern, `[Is ended]` does not mean the load succeeded. It means the worker has finished attempting the load, so the table no longer blocks the workflow.

```
update Pipeline.LoadStack
set
    [Is ended] = 1,
    [Load end datetime] = current_timestamp,
    [Load status] = 'Failed',
    [Failure message] = 'Source extract timeout'
where
    [Workflow ID] = 9001
and [Table name] = 'Sales.Order'
and [Worker ID] = 'W01';
```

This allows the pipeline to continue. Downstream tables can still run, usually against the last successfully loaded version of the upstream table.

This is consistent with the principle of fault tolerance. A failed table should be visible, logged, and available for repair, but it should not automatically stop unrelated or downstream work unless the pipeline has a specific reason to enforce that behaviour.

The important point is that the decision is visible. The load stack records what started, what ended, and what remains available to run. Errors should be separately logged.

29.4 Consequences of the load stack

29.4.1 Correct order is computed, not hard-coded

The load candidate view ensures that a table is offered for loading only when its upstream dependencies have ended.

This means correct order does not depend on a developer manually maintaining a sequence of

execution steps. The order emerges from dependency metadata and the current state of the load stack.

When a new table is added, the data engineer adds its dependency metadata. The load candidate view uses that metadata to determine when it is ready. The orchestration logic does not need to be rewritten for every new dependency path.

29.4.2 Parallelism follows from readiness

Parallelism is a consequence of the readiness rule.

If three independent tables are ready, three workers can claim them. If ten independent tables are ready, ten workers can claim them. If only one table is ready, only one table is available.

There is no need for a central process to decide in advance which tables should run side by side. Workers simply claim tables that are ready.

The number of workers can be adjusted according to available resources. The load stack does not need to know how many workers exist. It only needs to expose which tables are ready.

Moreover, the algorithm is load-balanced by default with a greedy approach to assigning work. As soon as a load candidate is ready, a worker picks up the job. There are no situations where a table is ready to load and a worker is available, but the load does not happen.

29.4.3 Execution state is visible and manipulable

Because the load stack is a table, the state of the workflow can be inspected while the load is running.

The data engineer can see:

- which tables have not started;
- which tables are currently loading;
- which tables have ended;
- which worker claimed each table;
- which tables are blocked;
- which tables failed or aborted.

But visibility is only part of the value.

Because execution state is stored as data, it can also be manipulated. A table can be placed back on the stack by setting `[Is started] = false` and `[Is ended] = false`. A subset of tables can be added to the stack for a targeted repair load.

This makes the workflow controllable while it is running.

The load is no longer a hidden procedural state inside a scheduler. It is visible and editable execution state.

29.4.4 Higher-frequency loads

Most warehouses run in daily batches, but some parts of a warehouse may need to load more frequently.

A load stack allows a selected path through the pipeline to be loaded more often than the whole warehouse. This works if the tables in that path are efficient enough to meet the required frequency, but the orchestration pattern does not need to change.

A path that is incrementally processed can be loaded hourly, every few minutes, or continuously, while the rest of the warehouse remains on its normal batch schedule.

This gives the data engineering team a gradual path toward higher-frequency loading. Instead of adopting a dedicated streaming platform for the whole warehouse, the team can increase load frequency only where the business need and pipeline efficiency justify it.

29.4.5 Cross-technology loading requires shared state, not shared tooling

The load stack can also support orchestration across technologies.

A complex pipeline may use SQL stored procedures, Spark notebooks, Python functions, or other processing technologies. A central orchestration product may provide connectors for each of these technologies, but the load stack takes a different approach.

The only requirement is that each worker can read and update the shared load stack.

For example, `Pipeline.LoadStack` and `Pipeline.LoadCandidate` may be implemented in a SQL database. A SQL procedure, Spark notebook, and Python process can all participate as workers if they can connect to the database, claim candidates, and update the stack.

This makes the load stack technology-agnostic. The coordination happens through shared execution state.

Key ideas.

Load dependencies are one of the first bottlenecks in a growing warehouse.

A load stack exposes the pipeline's execution queue as data. Allowing dependency-aware parallel loading without a central scheduler.

Because execution state is data, the load stack can be inspected, retried, edited, and reused for partial or continuous loading.

The same pattern supports partial loads, higher-frequency loads, and cross-technology orchestration.

Load dependencies

Every dependency is a trade-off between reuse, efficiency, and stability.

30.1 Dependency as coupling

A dependency occurs when one object relies on another object for information or logic.

Dependencies are useful because they allow the warehouse to reuse information. A table can calculate a piece of meaning once, persist it, and allow downstream tables to build on it. This improves efficiency because the same information does not need to be recalculated repeatedly. It also improves quality because the same logic is not copied into many places where it can drift out of sync.

But a dependency is also coupling.

When one table depends on another, change can propagate. A failure upstream can affect downstream tables. A small logic change upstream may require many downstream tables to reload. A small reference-data change can trigger large downstream updates.

What improves reuse can also reduce stability.

Therefore:

Every dependency is a trade-off between reuse, efficiency, and stability.

The data engineer therefore has to decide when a dependency is worth creating.

30.2 Load dependencies and view dependencies

Dependencies occur in two important ways.

A **load dependency** occurs when a materialised table selects from another table during its load.

A **view dependency** occurs when a view selects from another table or view.

Both propagate information through the warehouse, but they do so differently.

Dependency type	What it propagates
Load dependency	Persisted information
View dependency	Logic

Load dependencies propagate persisted information. The downstream table stores the result of the dependency at load time.

View dependencies propagate logic. The downstream object does not store the result. Instead, the join, filter, or calculation is evaluated when the view is queried.

This chapter focuses on load dependencies. Whenever the word **dependency** is used in this chapter, it refers to a load dependency unless otherwise stated.

Views are discussed later as an alternative to load dependencies.

30.3 The dependency trade-off

At every turn, the data engineer chooses between recalculating information and reusing existing information.

Recalculation avoids coupling, but may repeat logic or waste compute. Reuse improves efficiency and consistency, but creates a path for upstream change to propagate downstream.

The decision depends on whether the value of reuse exceeds the cost of coupling. This can be determined by the rule:

A dependency should be valuable, targeted, and stable.

Criterion	Question
Valuable	Is the information worth reusing rather than recalculating?
Targeted	Is the dependency narrow enough to match the actual purpose?
Stable	Will small source changes avoid causing disproportionate downstream change?

30.4 Valuable dependencies

A dependency should carry information worth reusing.

Using dependencies to propagate information is convenient in the short term. This can lead data engineers to create dependencies casually. The problem may not appear immediately. It surfaces later, when the pipeline becomes harder to reload, debug, or change.

The first test is therefore value.

A dependency is valuable when the source table contains information that is expensive, difficult, important, or risky to recalculate.

A dependency is weak when it exists only for convenience.

For example, a transaction table may select a country name from a reference table simply for display or debugging. This may be convenient, but it often creates unnecessary coupling. The transaction table may only need the country code. The country name can be joined later in the presentation layer, or exposed through a view.

Another example is the use of standard default values. Suppose the surrogate key for unknown values is always 1. A transaction table can assign 1 directly for unknown references, rather than joining to a reference table to retrieve the unknown surrogate key by name. The hardcoded value is safe because it is a stable convention, not a fragile business rule.

The principle is simple:

Do not create a dependency for information that is cheap, obvious, and stable enough to calculate locally.

Dependencies should be reserved for information that is worth propagating.

30.5 Targeted dependencies

A dependency should be targeted.

A targeted dependency selects from a source table whose informational purpose matches the dependency being created.

An untargeted dependency selects from a broad table that contains many unrelated columns. This creates more coupling than necessary.

The smallest unit of load dependency is a table. If a downstream table selects one column from a source table, it depends on the source table as a whole. This means wide miscellaneous tables create broad dependency surfaces.

For example, suppose there is a table called **Cake.Everything**.

It contains:

- cake sales;
- cake milestones;
- cake quality scores;
- cake production notes;
- cake marketing categories;
- cake seasonal flags.

If many tables select from **Cake.Everything**, then many tables depend on it. A change to one concept in **Cake.Everything** may force downstream work that has nothing to do with that concept. The dependency graph becomes tangled because it is unclear what information is being passed downstream.

Meaningful fragments reduce this problem.

Instead of `Cake.Everything`, the pipeline could create:

- `Cake.Sales`
- `Cake.Milestone`
- `Cake.Quality`
- `Cake.Season`

A downstream table that needs sales information can depend on `Cake.Sales`. A downstream table that needs quality information can depend on `Cake.Quality`.

The dependency becomes more targeted because the source table has a narrower informational purpose.

This is one reason it is often better to create a new fragment than to add a new column to an existing table, especially when the new column introduces a different concept.

For example, suppose `Cake.RefCalendar` describes dates. A data engineer may need to identify sales seasons. It may be tempting to add `[Is sales season]` to `Cake.RefCalendar`.

That may be acceptable for display. But if sales season becomes part of analytical computation, adding it to `Cake.RefCalendar` increases the dependency weight of the calendar table. Downstream tables interested only in sales seasons now depend on the whole calendar reference.

A better design may be to create `Cake.RefSaleSeason`, keyed by `[First date of week]`, because sales seasons operate at the week grain. Downstream tables that need sales seasons can depend on `Cake.RefSaleSeason`. The dependency is clearer, narrower, and more targeted.

The principle is:

A dependency should point to the smallest meaningful fragment that carries the required information.

30.6 Stable dependencies

A dependency should be stable.

Stability means that small changes in the source should not create disproportionate downstream change.

A common example is a transaction table that stores a country name from a reference country table. If the country name changes, that single reference-data update may trigger millions of downstream transaction updates.

The underlying business event is small: one name changed.

The data-world effect is large: many rows update.

This is dependency instability.

Where possible, unstable dependencies should be avoided. Where they cannot be avoided, their

risk should be understood and mitigated.

There are two common forms of instability:

Instability type	Description
Row-wise instability	One source row affects many downstream rows.
Column-wise instability	A source column changes frequently or for reasons unrelated to business meaning.

30.6.1 Row-wise instability

Row-wise instability occurs when a small number of source rows drive many downstream rows.

Reference tables are the most common example. A single row in a reference table may be used by millions of transaction rows. If that reference row changes, the downstream impact can be large.

For example, if a transaction table denormalises `[Country name]` from `Reference.Country`, then a name change in `Reference.Country` can update every transaction row for that country.

This may be unnecessary.

The transaction table may only need `[Country code]`. The name can remain in the reference table and be joined later for presentation.

The same principle applies more generally. Information that describes a reference entity should usually remain on the reference table unless there is a strong reason to materialise it downstream.

Row-wise instability can also occur with metadata-driven loads. If a pipeline reads a metadata table to decide what to load, an error in that metadata can cascade into widespread change. The metadata table may be small, but it controls a large execution surface.

Small tables are often sources of instability because one row can affect many downstream rows.

This does not mean small tables should never be dependencies. It means the data engineer should be careful when a small table is used to drive large downstream effects.

30.6.2 Column-wise instability

Column-wise instability occurs when the source column itself is volatile.

For example, selecting `[Update datetime]` into a downstream table can introduce instability. A source system may refresh update timestamps across many rows as part of routine maintenance. From the application's point of view, little has changed. From the pipeline's point of view, every row may appear to have changed.

The same problem can occur with columns such as `[Is archived]`, `[Last viewed datetime]`, or other operational fields that can be updated in bulk.

These columns may matter for the source system, but they may not represent meaningful business change for the warehouse.

A dependency on such a column can cause false downstream changes. It can make stable business information appear unstable.

The mitigation is to separate change detection from business meaning. If a volatile column is only needed for polling or extraction, it should not necessarily be propagated downstream as information. Techniques for managing this are discussed in [Tracking changes](#).

The principle is:

Do not propagate volatile columns unless their volatility is meaningful to the downstream table.

30.7 The case of surrogate keys

Surrogate keys are a special case of dependency.

A surrogate key is a stand-in for the primary key of a table. It is usually a single integer used in place of a natural or composite key.

Surrogate keys are useful in a warehouse for several reasons.

First, Power BI relationships use single-column relationships. If the natural key consists of multiple columns, a surrogate key may be required to implement the relationship cleanly.

Second, composite keys can be clumsy to carry. This is especially true for storytelling dimensions where the primary key may consist of many binary columns. Carrying ten columns across transaction tables is awkward, and joining on all ten repeatedly is error-prone.

Third, some keys are slow or difficult to join on. Type II dimensions often require inequality joins across validity periods, such as `[Start datetime]` and `[End datetime]`. Long string keys can also be slow or cumbersome.

For these reasons, it is often desirable to retrieve a surrogate key. This involves looking up the table using its primary key and selecting the surrogate key column. In this way, surrogate key retrieval can be a valuable dependency.

However, surrogate keys can also become magnets for dependency.

If downstream tables store a surrogate key, they depend on the table that generated it. If that table is rebuilt and the surrogate keys are regenerated, downstream key values may reshuffle. This can force large reloads and create instability.

There are two main ways to manage this.

The first is to defer the surrogate key lookup until later in the pipeline. If the original business key is simple and join performance is acceptable, downstream tables can carry the business key for longer. The surrogate key can be added only when needed for the semantic model or final presentation layer.

The second is to compute the surrogate key rather than look it up.

For example, suppose a storytelling dimension has five binary columns as its primary key:

- [Is A]
- [Is B]
- [Is C]
- [Is D]
- [Is E]

If each column is either 0 or 1, the surrogate key can be calculated with a stable formula:

$$[SK] = 1 + [Is A] + 2 * [Is B] + 4 * [Is C] + 8 * [Is D] + 16 * [Is E]$$

This gives each combination a stable integer without requiring a lookup. The downstream table gets the benefit of a surrogate key without forming a dependency on a generated key table.

This approach only works when the surrogate key can be computed safely and permanently. Many surrogate keys cannot be managed this way. But when the logic is simple and stable, calculation may be better than lookup.

The principle is:

A surrogate key dependency is justified when the value of the simplified key exceeds the coupling risk created by the lookup.

30.8 Views as an alternative

Views can sometimes avoid load dependencies.

A load dependency propagates persisted information.

A view dependency propagates logic.

For example, instead of denormalising [Country name] into a transaction table, a view can join the transaction table to **Reference.Country** at query time. If the country name changes, no downstream transaction table needs to reload. The updated name appears when the view is queried.

This can be useful when:

- query performance is acceptable;
- instant propagation of source changes is acceptable;
- the result does not need to be persisted;
- long-term historical retention is not required;
- downstream processes do not need row-level change tracking.

Views are especially useful in presentation scenarios. If a user is querying a small section of data through a report, it may be unnecessary to materialise every display attribute into a downstream table.

But views are not a complete substitute for loaded tables.

Views can propagate errors instantly. If the source table contains a bad value, the view exposes it immediately. A materialised table can provide a buffer because it changes only when loaded.

Views also do not track row-level change. As explained in [Load mechanics](#), the Check step of a load determines whether a row has genuinely changed. This allows downstream tables and processes to respond precisely to inserts, updates, and deletes. A view does not provide this kind of change event.

Views also do not preserve history when source systems delete old records. If a view points to a source that no longer contains a record, the view cannot recover it. A materialised warehouse table can preserve the information.

The choice between a load dependency and a view dependency is therefore another trade-off.

Use a load dependency when	Use a view when
The result should be persisted.	The result can be calculated at query time.
Downstream processes need change tracking.	Downstream processes only need current logic.
Historical retention matters.	Historical retention is not required.
Query performance requires materialisation.	Query performance is acceptable without materialisation.
A controlled loading buffer is valuable.	Instant propagation is acceptable.

30.9 Healthy dependency depth

The previous chapter showed how a load stack can make dependency orchestration manageable. That capability is powerful, but it also creates a second-order risk.

Once dependencies are easy to orchestrate, they can grow too freely.

A load stack prevents the pipeline from becoming tangled by execution order. It does not decide whether each dependency is valuable, targeted, and stable. That judgement still belongs to the data engineer.

A good pipeline should contain a healthy level of dependencies.

This may seem counterintuitive after a chapter about dependency risk, but a pipeline with almost no dependencies is usually not mature. It may reflect a garbage-in, garbage-out approach where raw tables are merely copied forward. It may also mean that complex transformation logic has been buried in scripts rather than expressed as reusable data products.

Dependencies are how the warehouse accumulates meaning.

A source system with 10 to 20 raw tables may produce 5 or 6 layers of dependency in the transformation pipeline. A larger system with 30 to 50 tables, especially when integrated with other systems, may produce 10 or more layers.

The aim is not to minimise dependencies, but to create the right ones.

A healthy dependency is valuable, targeted, and stable. It propagates meaningful information without creating unnecessary ripple effects.

Key ideas.

Every dependency is a trade-off between reuse, efficiency, and stability.

A load dependency occurs when a materialised table selects from another table during its load.

Load dependencies propagate persisted information; view dependencies propagate logic.

Dependencies improve efficiency and quality by allowing information and logic to be reused.

Dependencies reduce stability when small upstream changes cause disproportionate downstream reloads or updates.

A dependency should be valuable, targeted, and stable.

Wide miscellaneous tables create broad dependency surfaces and should often be split into meaningful fragments.

Surrogate keys are useful dependencies, but can become unstable if they are regenerated.

Views can avoid materialised reloads by propagating logic rather than persisted information, but they do not provide the same buffering, persistence, or row-level change tracking as loaded tables.

Incremental load: tracking changes

Incremental work begins with reliable knowledge of what changed.

31.1 Change observability

The aim of information efficiency is to process only what has changed.

If no records have changed since a table was last processed, the ideal load would spend no time processing that table. In practice, even when there are no input changes, a load still takes time to check its inputs and confirm that nothing needs to happen.

Efficiency improves when downstream tables can reliably identify the records that may have changed and ignore the rest.

Doing this well requires systematic change tracking.

Tracking change is the discipline of establishing a reliable relationship between source change and the pipeline's own processing state, so we can answer the question:

What source records may have changed since this target table last loaded successfully?

31.2 The problem of time

Incremental loading depends on a deceptively simple question:

What has changed since this table was last processed?

The difficulty is that this question does not refer to one time. It refers to a relationship between two states.

There are multiple times, and they are not always comparable. Confusing them can lead to missed records, unnecessary scans, or incorrect incremental loads.

Time	Meaning
Source row change time	The time the row in the source table was changed.
Source arrival time	When the row became visible to the warehouse.
Target bookmark time	When the target last started a successful load.

Time	Meaning
Target row change time	The time a row in the target table was changed.

The key distinction is whether a time is **in-sync** or **out-of-sync** with the pipeline.

A time is in-sync when it is created by the pipeline or can be safely compared for processing.

A time is out-of-sync when it belongs to another system and may lag behind, arrive late, or reflect a different notion of change.

The question:

Which source rows changed after the target last loaded successfully?

is only safe when the source change time is in-sync with the target's processing time.

Doing this systematically and accurately requires additional artefacts: refresh bookmarks and polling tables.

31.3 The simple update-datetime approach

To understand the general approach, it is useful to start with the simplest one.

The most basic approach is to include an update datetime column from the source system and filter on it.

Suppose the pipeline loads from `Raw.Event` into `Curated.Event`.

Example source table: `Raw.Event`

Event ID	Event type	Update datetime
E1001	Login	2021-05-01 07:45
E1002	Payment	2021-05-01 08:03
E1003	Refund	2021-05-01 08:07

Example target table: `Curated.Event`

Event ID	Event type	Update datetime
E1001	Login	2021-05-01 07:45

We assume that:

- the source table does not delete rows;
- the target table mirrors the source table row for row, with the same columns;
- `[Update datetime]` is carried from the source into the target.

In the simple approach, the target table remembers how far it has processed by storing the source update datetime. The next extract compares the source table against the maximum `[Update datetime]` already loaded into the target.

In this example, the maximum `[Update datetime]` in `Curated.Event` is 2021-05-01 07:45.

A simple incremental extract might therefore filter:

```
-- Step 1: Find the latest source update datetime loaded in target.
declare @latest_update_datetime datetime2(7);

set @latest_update_datetime =
(
    select max([Update datetime])
    from Curated.Event
);

-- Step 2: Pull only newer rows from the source.
select
    [Event ID],
    [Event type],
    [Update datetime]
from Raw.Event
where [Update datetime] > @latest_update_datetime;
```

This retrieves E1002 and E1003.

The approach is straightforward, but it suffers from several problems.

First, it does not scale to complex transformations. If `Curated.Event` draws from several source tables, it is not practical to carry every source update datetime into the target. A compound update datetime can be created, but this becomes error-prone as the query grows.

Second, it is unstable. If a source system update shifts all values in `[Update datetime]`, the pipeline may reprocess every row. This can cause a blowout in processing time and trigger unnecessary downstream work.

Third, it does not scale well to continuous or high-frequency loading. For example, if `Curated.Event` filters `Raw.Event` for rare events, then the maximum update datetime in `Curated.Event` does not represent the latest processing time of `Raw.Event`. The simple approach may force the pipeline to rescan too far back.

The problem is not that the simple approach is wrong. It is that it makes the target remember its processing state through a source business column. That is only safe when the target remains close to the source.

A more scalable pattern separates the time-tracking artefacts from the data content itself.

31.4 Refresh bookmarks

As explained in [Load mechanics](#), each successful table load should record a refresh bookmark.

A **refresh bookmark** is target-side state. It records how far the target table has successfully refreshed, without relying on the target table’s business columns to remember that state.

In the simplest update-datetime approach, the target table carries the source `[Update datetime]`, and the next load compares the source against the maximum value already loaded into the target. Thus, a business column is used as the marker for the processing cut-off.

A refresh bookmark separates this processing state from the target’s business data.

The bookmark records the starting datetime of a successful load. This becomes the target table’s processing boundary. If the load fails or aborts, the bookmark should not advance. The next load should resume from the last successful boundary.

The bookmark has no necessary relationship with the business datetime in the source column. They may be minutes apart, days apart, or years apart. One belongs to the pipeline’s processing time. The other belongs to the source system’s business or application time.

Example structure of Pipeline.RefreshBookmark

Table name	Load ID	Bookmark datetime
Curated.Event	10001	2026-05-01 08:01

In this example, `Curated.Event` last completed a successful load that began at 2026-05-01 08:01—five years after the events themselves.

Fetching the refresh bookmark is simple.

```
declare @refresh_bookmark_datetime datetime2(7);

-- Step 1: Look up the refresh bookmark datetime for Curated.Event.
-- This is when the previous successful load started.
select @refresh_bookmark_datetime =
(
    select [Bookmark datetime]
    from Pipeline.RefreshBookmark
    where [Table name] = 'Curated.Event'
);
```

The refresh bookmark tracks the target table’s last successful processing state. It does not, by itself, identify source changes.

To identify source changes, the pipeline needs a source-side change signal.

31.5 The source time problem

The refresh bookmark is in pipeline time. It records when the target table last started a successful load.

A source `[Update datetime]` may be in source time. It records when the source system says a row changed.

These two datetimes are not automatically comparable.

In our example, `Curated.Event` last refreshed at `2026-05-01 08:01`, but `Raw.Event[Update datetime]` contains business events from 2021.

The following filter to fetch the next batch of records would make no sense:

```
where Raw.Event.[Update datetime] > '2026-05-01 08:01'
```

It compares a source-system datetime with a pipeline refresh datetime. The result would incorrectly return no rows, even though new records may have arrived in the warehouse since the target last loaded.

A target refresh bookmark tells the pipeline where the target got to. It does not necessarily tell the pipeline what source update datetime had been safely observed at that point.

To use source update datetimes safely, the pipeline must know how source time relates to pipeline time.

There are two cases:

Case	Meaning	Consequence
In-sync source	The source change datetime is created by the pipeline or safely comparable with pipeline time.	The refresh bookmark can be compared directly to the source row change datetime.
Out-of-sync source	The source change datetime belongs to another system and may lag behind arrival in the warehouse.	A polling table is needed to map pipeline time to source time.

External raw tables are often out-of-sync. Even if they contain an `[Update datetime]`, that value may record when the row changed in the source system, not when the row became visible to the warehouse.

When datetimes are out-of-sync, the pipeline needs a polling table.

31.6 Polling tables

A polling table is source-side state.

It records how far the source's update timeline was safely observable at particular points in pipeline time. Its purpose is to let the pipeline translate the target's refresh bookmark into the source's own update timeline.

A polling table should be much faster to query than the source table itself. Ideally, the polling query should be close to zero time so that it can support frequent or continuous loads.

Suppose `Raw.Event[Update datetime]` is out-of-sync with the refresh bookmark for `Curated.Event`. There may be an unknown lag between the source update timestamp and the row's arrival in the warehouse. The only safe assumption is that `Raw.Event[Update datetime]` increases monoton-

ically within the source system.

A polling table provides a way to map pipeline time to source time.

For example, the pipeline may append a row to `Raw.Bookmark` each time it checks `Raw.Event`.

```
insert into Raw.Bookmark
(
    [Source table name],
    [Refresh datetime],
    [Bookmark datetime]
)
select
    'Raw.Event',
    sysutcdatetime(),
    max([Update datetime])
from Raw.Event;
```

Example structure of `Raw.Bookmark`

Source table name	Refresh datetime	Bookmark datetime
Raw.Event	2026-05-01 07:55	2021-05-01 07:48
Raw.Event	2026-05-01 08:00	2021-05-01 07:56
Raw.Event	2026-05-01 08:05	2021-05-01 08:02
Raw.Event	2026-05-01 08:10	2021-05-01 08:08

The columns have different meanings:

Column	Meaning
[Source table name]	The source table being polled.
[Refresh datetime]	The pipeline datetime when the polling row was created. This is in-sync with the pipeline.
[Bookmark datetime]	The maximum <code>Raw.Event</code> [Update datetime] observed at that polling moment. This is in the source system’s time-world.

The refresh bookmark datetime of `Curated.Event` is now in-sync with `[Refresh datetime]` in `Raw.Bookmark` because they are both managed by the pipeline. `Raw.Bookmark`[Refresh datetime] is linked to the source system’s update timeline through `Raw.Bookmark`[Bookmark datetime].

With the polling table as the bridge, we can now ask:

When the target last refreshed, how far through the source’s update timeline had the source safely arrived?

Suppose `Curated.Event` last successfully started at 2026-05-01 08:01.

The polling table shows that, at pipeline time 2026-05-01 08:01, the latest observed source bookmark was 2021-05-01 07:56.

The next extract should therefore consider:

```
Raw.Event[Update datetime] > '2021-05-01 07:56'
```

The query pattern is:

```
declare @refresh_bookmark_datetime datetime2(7);
declare @latest_process_datetime datetime2(7);

-- Step 1: Look up the refresh bookmark datetime for Curated.Event.
-- This is when the previous successful load started.
select @refresh_bookmark_datetime =
(
    select [Bookmark datetime]
    from Pipeline.RefreshBookmark
    where [Table name] = 'Curated.Event'
);

-- Step 2: Obtain the latest source update datetime that was
-- safely observable at or before that refresh bookmark.
select top 1
    @latest_process_datetime = [Bookmark datetime]
from Raw.Bookmark
where [Source table name] = 'Raw.Event'
    and [Refresh datetime] <= @refresh_bookmark_datetime
order by [Refresh datetime] desc;

-- Step 3: Fetch rows whose source update is newer than what
-- the previous target load had safely captured.
select
    e.*
from Raw.Event as e
where e.[Update datetime] > @latest_process_datetime;
```

In the example, this returns E1002 and E1003:

Event ID	Event type	Update datetime
E1002	Payment	2021-05-01 08:03
E1003	Refund	2021-05-01 08:07

The relationship between the source update time, the translation to pipeline time using the polling table, and the refresh bookmark on the target table to find the source records to update can be visualised as two parallel timelines.

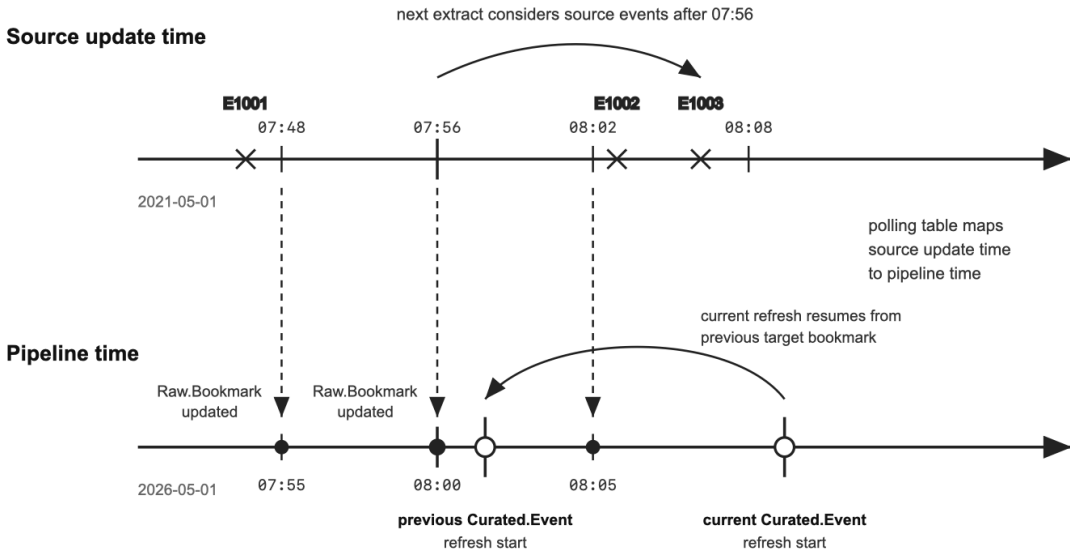


Figure 1. *Raw.Bookmark* polling rows map source update time to pipeline time. The previous *Curated.Event* refresh used the latest polling row available when it started, mapping its target bookmark to source boundary 07:56. The current refresh resumes from that target bookmark, so it considers source events after 07:56.

In theory, refreshing the polling table continuously would allow the pipeline to translate pipeline time into source time at any point. In practice, the refresh frequency should align with the pipeline cadence and business need.

A single polling table can store bookmarks for multiple source tables. It does not need to be one polling table per source table.

Advanced note: moving source boundaries.

If the source is continuously loading, source records may arrive while the target load is running.

There are two safe ways to handle this.

The first is to freeze the source boundary. The load processes only the source records that were safely observable at the start of the target load. Records that arrive later are picked up in the next batch.

The second is to allow deliberate overlap. The next load resumes from an earlier safe bookmark, or from the previous source boundary, so that late-arriving records are read again. This may re-extract records that were already processed, but the load mechanics should treat them as upserts and apply only genuine changes.

Overlap is often safer than trying to make the boundary too precise. It is acceptable for an incremental extract to read a little too much, provided the downstream load is idempotent and unchanged rows are ignored.

31.7 When polling tables can be skipped

Polling tables are needed when the source change datetime is out-of-sync with pipeline time.

They are not usually needed when the source table was created by the pipeline itself.

If the source table was created by the pipeline, its row change datetimes are reliable pipeline-managed datetimes. These include:

- [Row insert datetime]
- [Row update datetime]
- [Row delete datetime]

Because these datetimes are created by the pipeline, they are in-sync with the refresh bookmark.

Example pipeline-managed source table: Filtered.Event

Event ID	Event type	Row insert datetime	Row update datetime	Row delete datetime
E1001	Login	2026-05-01 07:40	2026-05-01 07:40	9999-12-31 00:00:00
E1002	Payment	2026-05-01 08:03	2026-05-01 08:03	9999-12-31 00:00:00
E1003	Refund	2026-05-01 08:07	2026-05-01 08:07	9999-12-31 00:00:00

If the refresh bookmark for the target is 2026-05-01 08:00, then E1002 and E1003 are candidates for processing because their insert or update datetimes are after the bookmark.

In this case, the target’s refresh bookmark can be compared directly against the source table’s row change datetimes.

```
select
    e.*
from Filtered.Event as e
where
    e.[Row insert datetime] > @refresh_bookmark_datetime
    or e.[Row update datetime] > @refresh_bookmark_datetime
    or e.[Row delete datetime] > @refresh_bookmark_datetime;
```

This is one of the reasons why the [Filter step](#) is important. Once external source data has passed through a controlled pipeline load, it receives architectural row change datetimes that are in-sync with the pipeline. Downstream tables can then process incrementally without needing to reinterpret the source system’s update timeline.

31.8 The role of the filter step

Three themes occur when tracking inserts, updates, and deletes.

First, datetimes that are synchronised with the pipeline’s processing time are easy to use.

Second, architectural artefacts are more reliable than business artefacts for change tracking.

Third, deletes are very hard to track unless they have been architecturally managed.

When these conditions are absent, tracking change becomes complex or even impossible.

This is one reason pipelines often begin with a [Filter step](#) that keeps transformation minimal. The Filter step is the first controlled interaction with the source data. It extracts the necessary rows and columns, then annotates them with pipeline-managed architectural columns. It also tracks deletes.

The Filter step may look like low-value work because it performs little transformation. But it provides a critical foundation. Once source data has passed through a properly designed load process, downstream tables can rely on `[Row insert datetime]`, `[Row update datetime]`, and `[Row delete datetime]` to track change.

The optimal case is:

1. incoming data is incrementally extracted using source change detection or polling tables;
2. the Filter step applies [load mechanics](#) to detect genuine change;
3. the resulting pipeline table has reliable row change datetimes, and deleted rows are stored in a history table;
4. downstream tables can process incrementally using those pipeline-managed datetimes.

In this way, the Filter step converts fragile or out-of-sync source change signals into reliable pipeline change artefacts.

Key ideas.

Incremental work begins with reliable knowledge of what changed.

Refresh bookmarks track the target table’s last successful processing state.

Source change detection columns can be in-sync or out-of-sync with the pipeline’s processing time.

Polling tables map out-of-sync source timestamps to in-sync pipeline timestamps.

Inserts and updates are easier to track than deletes because deleted rows disappear.

Architectural change artefacts are more reliable than business columns for detecting change.

The Filter step converts source change signals into reliable pipeline change artefacts for downstream processing.

Incremental load: responding to change

Source changes must be translated through query logic before the target can respond correctly.

32.1 Change translation

[Tracking changes](#) tells the pipeline what changed upstream. It does not tell the pipeline what should change downstream.

That second step depends on query logic.

The central problem is this:

Source inserts, updates, and deletes do not necessarily become target inserts, updates, and deletes.

Responding to change is the discipline of translating upstream source changes into the smallest correct set of target actions.

A systematic approach focuses on the primary keys. Responding to change starts by calculating the *target keys* whose result may have changed, then re-running the normal query only for those keys.

A **driver set** is the set of primary keys, or primary key components, that may need action because upstream source rows changed.

There are two main kinds of driver set.

Driver set	Meaning
Upsert driver	Keys whose rows should be recalculated and then inserted or updated in the target.
Delete driver	Keys whose rows should be removed from the target.

32.2 Source changes are not target actions

The relationship between source change and target action is not straightforward.

Consider the following examples.

Upstream change	Possible downstream action	Example reason
Insert	Delete	A new row causes a record to no longer qualify under an anti-join.
Delete	Update	Removing a row changes an aggregate.
Update	Insert	A changed value now passes a filter threshold.

For example, suppose `Bank.CustomersToFollowUp` contains customers who have not made recent deposits and need service-team follow-up. This table uses `Bank.Transaction` as an input. If a new deposit is inserted into `Bank.Transaction`, that customer may no longer require follow-up. A source insert has triggered a target delete.

Suppose `Bank.AccountSummary` contains account-level aggregates, such as total number of holders and balance per account. If a joint holder is deleted from `Bank.AccountHolder`, the holder count changes. A source delete has triggered a target update.

Suppose `Bank.GoldCustomer` contains customers whose account balance exceeds a threshold. If a row in `Bank.AccountBalance` is updated so that the customer now exceeds the threshold, the customer may newly qualify. A source update has triggered a target insert.

These examples are simple, but the general lesson is that:

The target response is determined by the query.

32.3 Analysing the query

Suppose two source tables, X and Y, are used to produce a target table T.

The setup is:

- X has columns `[Header ID]`, `[Value]`, and `[Status]`, where `[Header ID]` is the primary key.
- Y has columns `[Header ID]`, `[Line number]`, `[Value]`, and `[Status]`, where `[Header ID]` and `[Line number]` are the primary key.
- `Y[Header ID]` is a foreign key to `X[Header ID]`.

There may be inserts, updates, and deletes on both X and Y.

Inserts and updates are considered together as upserts. The possible source changes are:

- upserts from X;

- deletes from X;
- upserts from Y;
- deletes from Y.

The goal is to determine which rows in T need to be recalculated or removed.

The way to calculate these depends on the shape of the query.

32.3.1 Common query shapes

The following table summarises several common query shapes and how changes may propagate.

Scenario	Target primary key	Approach	Keys to upsert in T	Keys to delete from T
Straight select of X	[Header ID]	Source rows map directly to target rows.	Headers from X that were inserted or updated.	Headers from X that were deleted.
Filter of X on [Status]	[Header ID]	Filter condition means updates can add or remove rows.	Headers from X that were inserted or updated and now pass the filter.	Headers from X that were deleted, or updated and no longer pass the filter.
X inner join Y	[Header ID], [Line number]	Join means changes on either side can add or remove joined pairs.	Pairs for Y rows inserted or updated, plus pairs for changed X headers with matching Y rows.	Pairs for Y rows deleted, plus all pairs for headers deleted from X.
X inner join Y, grouped by [Header ID]	[Header ID]	Deleting a Y row can change the aggregate or remove the header.	Headers from X or Y whose values may have changed, including deletes from Y that change the aggregate.	Headers deleted from X, or headers that now have zero Y rows.
X left join Y with synthetic line 0	[Header ID], [Line number]	Synthetic line 0 appears only when no Y rows exist.	Pairs for Y rows inserted or updated; (Header ID, 0) for headers with no Y rows; pairs for new X headers.	Pairs for Y rows deleted; remove (Header ID, 0) when Y rows appear; remove all pairs for headers deleted from X.
Y left join X	[Header ID], [Line number]	Target follows Y lines; X changes matter only if Y lines exist.	Pairs from Y inserted or updated, plus pairs under headers in X that changed.	Pairs from Y that were deleted.

Scenario	Target primary key	Approach	Keys to upsert in T	Keys to delete from T
X left join Y, grouped by [Header ID]	[Header ID]	X controls row presence; Y changes aggregates.	Headers in X inserted, plus headers whose values changed due to changes in X or Y, including deletes from Y.	Headers deleted from X.
Union of [Header ID]	[Header ID]	Header exists if present in either X or Y.	Headers inserted or updated in X or Y; headers projected from one side but still existing on the other.	Headers deleted from both X and Y.
X left anti-join Y	[Header ID]	Row appears only when X exists and Y does not.	Headers inserted or updated in X with no Y rows; headers where Y rows were deleted and now qualify.	Headers deleted from X; headers that gained a Y row.

The table is not meant to memorise every case. Its purpose is to show that target response depends on query shape.

With more source tables, the analysis can become complicated quickly. A query with ten inputs may have many possible source changes, and each source change may affect the target differently.

32.3.2 Robustness

For robustness, upsert drivers may be broader than strictly necessary. Delete drivers should be exact.

This asymmetry exists because load mechanics apply upserts only for rows that have genuinely changed. If the upsert driver is too broad, the pipeline may recalculate extra rows, but the staging table is still compared with the target and only genuine inserts or updates are applied.

A broad upsert driver is therefore usually safe, provided it remains performant. For example, it may be acceptable to construct the driver set using part of a primary key rather than the full primary key, causing the load to recalculate a slightly wider set of rows.

Deletes are different.

A delete driver does not ask the pipeline to recalculate a row. It tells the pipeline that a row should be removed from the target. If the delete driver contains extra keys, correct rows may disappear.

A common mistake is to over-delete with the intention of reinserting extra records. This can be tempting because a delete-and-reinsert pattern can be easier to calculate than a finely targeted

delete-and-upsert pattern. However, over-deleting is dangerous. If the source system has a bulk update on a column that has no impact on the final output, the entire table may be deleted and reinserted. This can create a server bottleneck and cause downstream tables to treat unchanged rows as changed.

The rule is:

Upsert drivers may be conservative and include some extra keys. Delete drivers should contain only rows that no longer satisfy the target query’s presence rule.

32.4 Worked examples

The following examples show how source changes are translated through query logic.

32.4.1 Worked example 1—Filter

Suppose the target table contains only active headers.

```
select
  [Header ID],
  [Value],
  [Status]
from X
where [Status] = 'Active';
```

Before source change: X

Header ID	Value	Status
H100	10	Active
H101	20	Inactive
H102	30	Active

Before target load: T

Header ID	Value	Status
H100	10	Active
H102	30	Active

Now suppose two source rows are updated.

Source changes in X

Header ID	Old status	New status	Source change
H101	Inactive	Active	Update

Header ID	Old status	New status	Source change
H102	Active	Inactive	Update

The source action is the same in both cases: an update.

The target action is different.

H101 now passes the filter, so it belongs in the upsert driver. It should be inserted into T.

H102 no longer passes the filter, so it belongs in the delete driver. It should be removed from T.

Upsert driver

Header ID	Reason
H101	Updated row now passes the filter.

Delete driver

Header ID	Reason
H102	Updated row no longer passes the filter.

After incremental load: T

Header ID	Value	Status
H100	10	Active
H101	20	Active

This example shows that a source update can become either a target upsert or a target delete. The filter controls row presence.

32.4.2 Worked example 2—Aggregation

Suppose the target table contains the total line value for each header.

```
select
    [Header ID],
    sum([Value]) as [Total value]
from Y
group by [Header ID];
```

Before source change: Y

Header ID	Line number	Value
H100	1	10
H100	2	15

Header ID	Line number	Value
H101	1	20

Before target load: T

Header ID	Total value
H100	25
H101	20

Now suppose one source row is deleted.

Source changes in Y

Header ID	Line number	Value	Source change
H100	2	15	Delete

The source action is a delete.

But the target row for H100 should not be deleted. The header still has another line. Instead, the aggregate must be recalculated.

H100 belongs in the upsert driver because its total value may have changed.

Upsert driver

Header ID	Reason
H100	Deleted line changes the aggregate.

Delete driver

No rows.

After incremental load: T

Header ID	Total value
H100	10
H101	20

This example shows that a source delete can become a target update. Aggregations turn row-level changes into value changes at a higher grain.

32.4.3 Worked example 3—Anti-join

Suppose the target table contains headers in X that do not have any matching rows in Y.

```

select
    x.[Header ID],
    x.[Value]
from X as x
where not exists
(
    select 1
    from Y as y
    where y.[Header ID] = x.[Header ID]
);

```

Before source change: X

Header ID	Value
H100	10
H101	20
H102	30

Before source change: Y

Header ID	Line number	Value
H100	1	5
H102	1	8

Before target load: T

Header ID	Value
H101	20

H101 appears in T because it exists in X but has no matching row in Y.

Now suppose a new row is inserted into Y.

Source changes in Y

Header ID	Line number	Value	Source change
H101	1	12	Insert

The source action is an insert.

But because the target is an anti-join, this insert causes H101 to stop qualifying. The target action is a delete.

Upsert driver

No rows.

Delete driver

Header ID	Reason
H101	Inserted Y row means the header no longer satisfies the anti-join.

After incremental load: T

No rows.

This example shows that a source insert can become a target delete. Anti-joins reverse the usual intuition because the target row exists only while a matching source row is absent.

32.5 Applying the change

Once the query has been analysed, the transformation can be converted from a full load to an incremental load.

Recall from [Load mechanics](#) that the differences between incremental extract and full extract are:

- The staging table for the load has only a minimal set of records that is much smaller than the full set, but still covers all the records that would need to be upserted in the current batch. This is what makes the load fast.
- Deletes cannot be done automatically by comparing the full set of primary keys between the staging and the target. Instead, they need to be customised for the query by analysing the impact of changes in source.

The detailed implementation of an incremental extract follows a consistent pattern.

32.5.1 Step 1—Write the full query

Begin with the full query that expresses the business logic.

For example, suppose the full target query is an inner join between X and Y. They are used to load T.

```
select
    X.*,
    Y.*
from      X
inner join Y on Y.[PK] = X.[PK];
```

This query is the definition of the target table. It should be understandable and testable as a full load before it is made incremental.

32.5.2 Step 2—Fetch the refresh bookmark

At the start of the load, retrieve the refresh bookmark that records the target table's last successful processing boundary.

This bookmark defines the boundary for upstream change detection.

```
declare @refresh_bookmark_datetime datetime2(7);

select @refresh_bookmark_datetime =
(
    select [Bookmark datetime]
    from Pipeline.RefreshBookmark
    where [Table name] = 'T'
);
```

32.5.3 Step 3—Create the upsert driver

Using the query analysis, create a temporary table of target keys to upsert.

In this example, changes from either X or Y may affect rows in T, so the upsert driver takes keys from both source tables.

```
drop table if exists #keys_to_upsert;

-- Upserts from X.
select
    X.[PK]
into #keys_to_upsert
from X
where X.[Row update datetime] > @refresh_bookmark_datetime

union

-- Upserts from Y.
select
    Y.[PK]
from Y
where Y.[Row update datetime] > @refresh_bookmark_datetime;
```

In these examples, the driver table has the full target key. In practice, it may contain only part of a multi-column primary key, depending on the grain of the target table.

If performance requires, add an index to the driver table before joining it to the full query.

```
create clustered index cix_keys_to_upsert
on #keys_to_upsert ([PK]);
```

The upsert driver is then joined to the full query.

```

drop table if exists #T_staging;

select
    X.*,
    Y.*
into #T_staging
from      X
inner join Y          on Y.[PK] = X.[PK]
inner join #keys_to_upsert as U on U.[PK] = X.[PK]; -- downfilter original query
→ for smaller staging

```

This produces a minimal staging table. It contains only rows whose target values may have changed, but the row values still come from the normal business query.

In other query shapes, deletes may also belong in the upsert driver. For example, in an aggregation, deleting a contributing row changes the aggregate value and therefore requires recalculation.

32.5.4 Step 4—Create the delete driver

Using the query analysis, create a temporary table of target keys to delete.

For an inner join between X and Y, a target row should disappear if the corresponding key no longer exists on either side of the join.

If deleted rows are preserved in history tables, the delete driver can be calculated from those histories.

```

drop table if exists #keys_to_delete;

-- Deletes from X.
select
    X_History.[PK]
into #keys_to_delete
from      X_History
left join X          on X.[PK] = X_History.[PK]
where X_History.[Row delete datetime] > @refresh_bookmark_datetime -- recently
→ deleted rows
    and X.[PK] is null; -- truly deleted

union

-- Deletes from Y.
select
    Y_History.[PK]
from      Y_History
left join Y          on Y.[PK] = Y_History.[PK]
where Y_History.[Row delete datetime] > @refresh_bookmark_datetime -- recently
→ deleted rows
    and Y.[PK] is null; -- truly deleted

```

The delete driver should contain only target keys that no longer satisfy the target query's

presence rule.

Once the delete driver has been calculated, deleting from the target is straightforward.

```
delete T
from      T
inner join #keys_to_delete as D on D.[PK] = T.[PK];
```

32.5.5 Step 5—Apply load mechanics

After the minimal staging table and delete driver have been created, the normal load mechanics can apply.

The pipeline can:

- compare staging rows with the target to identify genuine upserts;
- check stability thresholds;
- reject unsafe rows;
- apply updates and inserts;
- apply deletes;
- preserve history;
- record load statistics and bookmarks.

This means full and incremental loads share the same business query and load mechanics. The difference is that incremental loads add driver sets to reduce the work.

32.5.6 Full incremental extract pattern

Putting the steps together, the full pattern is:

```
-- 1. Fetch the refresh bookmark, which is the point to resume.
declare @refresh_bookmark_datetime datetime2(7);

select @refresh_bookmark_datetime =
(
    select [Bookmark datetime]
    from Pipeline.RefreshBookmark
    where [Table name] = 'T'
);

-- 2. Determine the upsert driver table.
drop table if exists #keys_to_upsert;

-- Upserts from X.
select
    X.[PK]
into #keys_to_upsert
from X
```

```

where X.[Row update datetime] > @refresh_bookmark_datetime

union

-- Upserts from Y.
select
    Y.[PK]
from Y
where Y.[Row update datetime] > @refresh_bookmark_datetime;

-- 3. Create the minimal staging table by downfiltering the full query.
drop table if exists #T_staging;

select
    X.*,
    Y.*
into #T_staging
from X
inner join Y on Y.[PK] = X.[PK]
inner join #keys_to_upsert as U on U.[PK] = X.[PK]; -- downfilter original query for
↳ smaller staging

-- 4. Determine the delete driver table.
drop table if exists #keys_to_delete;

-- Deletes from X.
select
    X_History.[PK]
into #keys_to_delete
from X_History
left join X on X.[PK] = X_History.[PK]
where X_History.[Row delete datetime] > @refresh_bookmark_datetime -- recently
↳ deleted rows
    and X.[PK] is null; -- truly deleted

union

-- Deletes from Y.
select
    Y_History.[PK]
from Y_History
left join Y on Y.[PK] = Y_History.[PK]
where Y_History.[Row delete datetime] > @refresh_bookmark_datetime -- recently
↳ deleted rows
    and Y.[PK] is null; -- truly deleted

-- 5. Apply the deletes.
delete T
from T
inner join #keys_to_delete as D on D.[PK] = T.[PK];

-- 6. Continue upsert using standard load mechanics...

```

This script illustrates the complete pattern:

- 1. fetch the target refresh bookmark;
- 2. calculate the upsert driver;
- 3. downfilter the full query into a minimal staging table;
- 4. calculate the delete driver;
- 5. apply the target deletes.

Visually the procedure looks like:

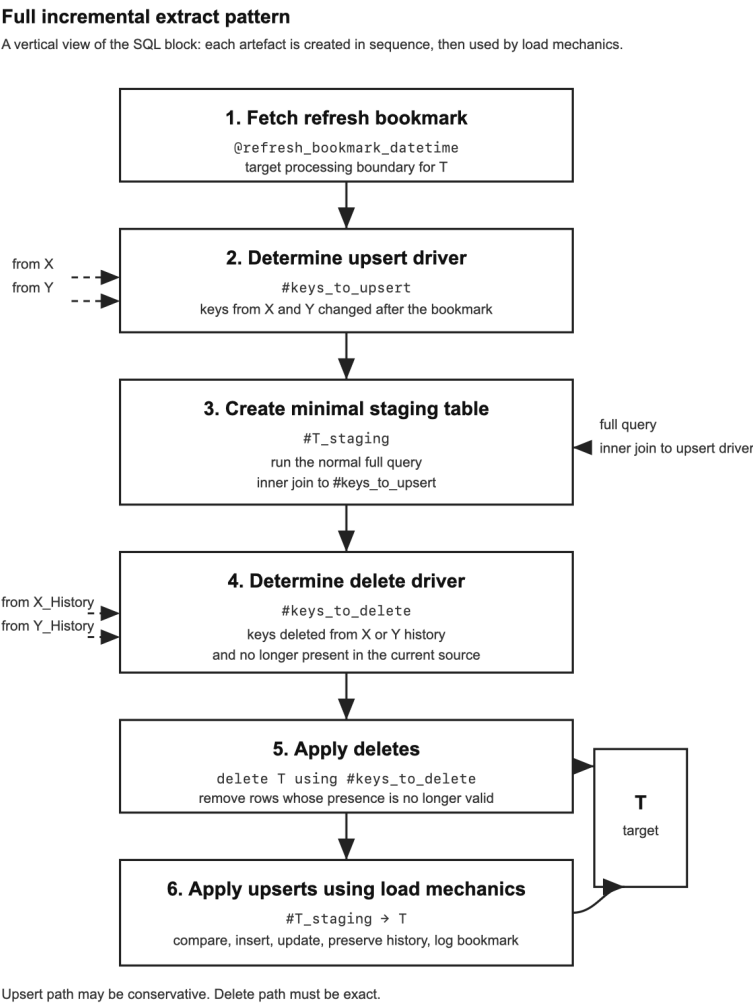


Figure 1. The incremental extract pattern follows the SQL script from top to bottom. The refresh bookmark defines the change boundary. The upsert driver creates a minimal staging table by downfiltering the normal query. The delete driver identifies rows to remove from T. Load mechanics then applies exact deletes and genuine upserts.

32.6 Why this pattern works

This workflow has several advantages.

32.6.1 Uniformity

The same query logic underpins both full loads and incremental loads.

The difference lies in the driver set. A full load runs the query for all target keys. An incremental load runs the same query for only the keys that may have changed.

This reduces the risk that the incremental load becomes a different definition from the full load.

32.6.2 Idempotency

The process works even if it runs more than once over the same interval.

If a load fails and is retried, the refresh bookmark ensures that the pipeline considers the same source change window again.

If the load runs more frequently than usual, the outcome remains consistent. There may simply be fewer source changes to respond to.

32.6.3 Graceful fallback

If upstream changes touch many rows, the driver set expands, but the final result set may have minimal changes because the touched column is not selected for the target table.

Using this approach, the incremental load will behave like a full load. This is acceptable. The staging table is still compared to the target, so only genuinely changed rows should be applied.

32.7 Best-practice workflow

Given the complexity of responding to change, incremental logic should be developed as a controlled workflow rather than assembled all at once.

The goal is to prove that the incremental load produces the same result as the full query, while doing less work.

The following is a 9 steps pattern.

32.7.1 Step 1—Create a full-load comparison test

Start by creating a test that compares the incrementally maintained target table with the expected result of the full query.

This may include:

- row counts;
- primary key comparison;
- sample selection.

The test establishes the standard of correctness. A correct incremental load should produce the same final target as a full load.

A datetime filter can be used for performance during development, but it should be independent of the extract datetime being tested.

32.7.2 Step 2—Create a realistic change window

Load the table in full, then wait for source data to change or create controlled test changes.

This gives the developer a realistic incremental window. The aim is to test the logic against actual source changes, not merely against an abstract query.

32.7.3 Step 3—Build the upsert driver

Build the upsert driver for one source table at a time.

As each source table is added, re-run the driver query and check both correctness and performance. Add indexes if performance degrades.

The upsert driver should include every key whose target values may need recalculation. It may be slightly broader than necessary, provided it remains performant.

32.7.4 Step 4—Test the minimal staging table

Join the upsert driver to the full query.

This creates the minimal staging table. It should contain only the rows that may need to be inserted or updated, while preserving the same business logic as the full query.

Check that the staging table is both correct and performant.

32.7.5 Step 5—Build the delete driver

Build the delete driver separately from the upsert driver.

The delete driver should contain only rows that no longer satisfy the target query's presence rule.

Special attention should be paid to performance because finding deletes often requires complex scans.

32.7.6 Step 6—Simulate deletes before applying them

During development, simulate deletes with a `select` rather than immediately applying them.

The result set should be examined and compared with the correct result.

Simulating the delete reduces the risk of deleting data while the delete logic is being tested, causing rework to load the data again.

32.7.7 Step 7—Apply changes through load mechanics

Apply the upserts and deletes through normal load mechanics.

The load should:

- compare staging rows with the target;
- apply only genuine inserts and updates;
- apply exact deletes;
- preserve history;
- record load statistics and bookmarks.

Then compare the incrementally maintained target with a fully loaded copy of the same target. The two should match.

32.7.8 Step 8—Run a zero-load benchmark

After a successful load, rerun the upsert and delete drivers.

They should return no rows, or near zero rows, and should run quickly.

This is the fastest the incremental load can ever be. It is also a useful baseline for detecting unnecessary work.

32.7.9 Step 9—Run incrementally over time

Continue loading incrementally over multiple days.

The full-load comparison test should continue to pass. If it fails, the incremental response logic is likely missing a source change condition, over-deleting, or failing to handle a query shape correctly.

32.8 Using AI as a reviewer

Analysing source-change propagation through a complex query can be error-prone.

This is a useful place to use generative AI as a reviewer. A model can propose how source inserts, updates, and deletes may propagate through joins, filters, aggregations, anti-joins, and

set operations.

A useful prompt is to provide the query shape, source keys, target key, and the possible source changes, then ask for:

- target keys to upsert;
- target keys to delete;
- reasons for each driver.

The output should then be tested against the full-load result.

32.9 Conclusion

Tracking changes identifies which source rows changed. Responding to change determines what those source changes mean for the target.

The difficulty of incremental loading depends not only on data volume, but on query shape. Filters, joins, aggregations, anti-joins, window functions, and set operations all affect whether source changes become target upserts or deletes.

The standard pattern is to fetch the refresh bookmark, compute the upsert driver, create a minimal staging table from the normal query, compute the delete driver, and then apply normal load mechanics.

The complexity comes from analysing the query. Generative AI is helpful for this part.

Together, the two steps—tracking changes and responding to change—apply the principle of proportionate change.

Key ideas.

Tracking change identifies which source rows have changed. Responding to change determines what those source changes mean after they pass through the target query.

Source actions do not map directly to target actions. A source insert can cause a target delete, a source delete can cause a target update, and a source update can cause a target insert.

The query is the translation layer between source change and target action.

A driver set contains the keys that may need action because upstream source rows changed.

Upsert drivers identify rows to recalculate. They may be conservative, provided they remain performant.

Delete drivers identify rows that no longer satisfy the target query's presence rule. They must be exact.

The goal of an incremental load is to apply proportionate computational change.

Optimising Power BI load

Power BI load is the final expression of pipeline design.

33.1 Power BI as the final load boundary

Loading a Power BI model is often one of the longest steps in the delivery of a data product. This is particularly true for models with large fact tables.

New technologies such as DirectLake promise to remove traditional import refresh in some scenarios. Even so, Import mode remains one of the most reliable ways to deliver a fast and responsive user experience.

When refreshing Power BI, the core idea is:

Power BI should assemble information, not manufacture it.

This is why Power BI performance depends on decisions made far earlier in the pipeline. If upstream data has been shaped into meaningful fragments, if change has been tracked carefully, and if downstream tables respond proportionately to change, then Power BI can load efficiently. If those things have not been done, Power BI refresh becomes the place where all upstream disorder is paid for.

Thus, Power BI load is not merely a Power BI problem, but is the final expression of pipeline design.

There are three main ways to optimise Power BI loads:

- avoiding load with DirectQuery;
- using efficient underlying source tables;
- partitioning the model, with further enhancements through rolling windows and incremental refresh.

This chapter assumes the model sources data from SQL, where purpose-built views or tables act as source tables that map one-to-one with the Power BI model tables.

33.2 Avoiding load with DirectQuery

DirectQuery for fact tables avoids the load entirely by sending DAX queries to the source at report time. To provide a responsive experience, the underlying source table often needs to be materialised as a columnstore table dedicated to this purpose. While materialising a large fact table is costly, it can also be incrementally loaded so that refresh remains minimal.

By using DirectQuery over an incrementally loaded columnstore table, the pipeline can achieve high efficiency in terms of information movement. The Power BI model does not need to import the fact table because the source table is queried directly.

However, DirectQuery has serious drawbacks for user experience:

- DAX queries become noticeably slower.
- Some complex DAX expressions are not supported.
- Certain DAX functions behave differently in Import mode and DirectQuery mode.
- Power BI limits how many rows can be retrieved in DirectQuery for certain operations, which restricts what reports can be built.
- Dimension values can appear blank in filter lists even when the table does not actually contain blanks.

DirectQuery works best when:

- the table is far longer than it is wide, so queries touch fewer attributes;
- queries against the table are simple, such as sums, counts, minimums, and maximums that work well against columnstore tables;
- users are not expected to browse unit records and do not hit Power BI's retrieval limits.

Where dimensions are shared by both Import and DirectQuery facts, dual mode must be implemented. Dual mode keeps a copy of the dimension for Import while still allowing the engine to query it in DirectQuery when needed. This ensures relationships remain fast for Import facts and correct for DirectQuery facts.

DirectQuery can be powerful, but it is not a universal escape from model refresh. It shifts work from refresh time to query time. If the source is not designed for this workload, the user experiences the cost directly.

33.3 Efficient underlying source tables

One of the biggest factors influencing Power BI load times is the efficiency of the underlying source tables, which are often implemented as views. A source view should be a straightforward join of ready-to-use tables that have been indexed appropriately to support high-performance execution.

The views may have simple logic such as:

- looking up surrogate keys to replace composite primary keys;
- backfilling null values with defaults;

- adding on-demand columns, such as `[Days expired since creation]` using functions like `getdate()`.

However, heavy transformations such as windowing, nested logic, and complex aggregation should be avoided. These belong upstream in curated layers where they can be tested, reused, and incrementally refreshed.

String aggregation for display should also be avoided where possible, as this can often be handled with appropriate DAX measures, as explained in [Designing measures](#).

The ideal case is to create fragments that are both meaningful and effective for loading into the Power BI model. If a source table still requires elaborate logic to stitch pieces together, the issue is not merely SQL tuning. The deeper issue is that upstream fragments have not been adequately prepared.

Materialisation of the view is sometimes necessary to ensure the source table loads rapidly. This is especially true when data needs to be rearranged to support fast retrieval of load partitions against the partition key.

In short:

Power BI source views or tables should be tuned specifically for fast retrieval, not burdened with complex logic.

33.4 Partitioning the model

Defining a table into partitions is the most effective way to reduce Power BI load times when fact tables grow large.

Partitioning a table offers increasing levels of enhancement that can be adopted as the situation requires:

1. load partitions in parallel;
2. apply a rolling window to drop older partitions;
3. incrementally refresh only the partitions that changed.

33.4.1 Analogy for partitions

Partitions are akin to having a large box of books from multiple publishing years. Batches of books arrive at regular intervals, and the box needs to be updated with the incoming batch. There can be new books, and old books can be updated. Updating the whole box can take a long time if the number of books is huge.

Splitting the books into boxes by publishing year allows a divide-and-conquer approach. Each year's box can be updated separately. This is the first benefit of partitioning: partitions can be loaded in parallel.

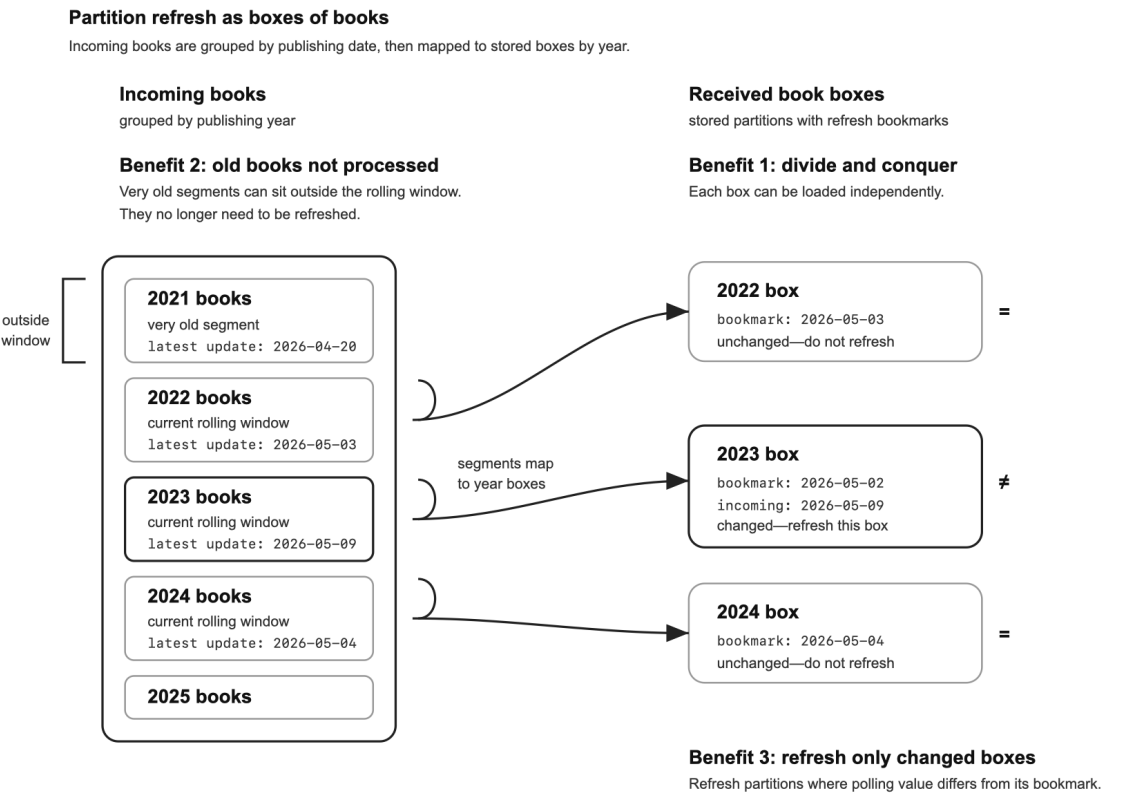
If boxes of books that are too old become unnecessary, they do not need to be kept. The oldest boxes can be dropped. This is the second benefit of partitioning: a rolling window can keep the

model from growing indefinitely.

Finally, if the incoming books can be sorted by year, and only some years have new or updated books, then only those boxes need updating. It may take additional effort to track which years changed, but this effort can dramatically reduce total refresh time. This is the third benefit of partitioning: incremental refresh can refresh only the partitions that changed.

Each step is increasingly sophisticated, and corresponds to a deeper level of gain from partitioning Power BI tables.

This is visualised below:



To make partitions effective, the source must return a single partition quickly. If the underlying view is complex, the solution is not to push SQL harder but to move complexity upstream and materialise the result with an index on the partition key. This ensures that Power BI can request “give me this month” and receive the data quickly.

Continuing the analogy, the incoming batches of books must be readily accessible by publishing year. If not, it may take longer to fetch each year’s books than the time gained from divide-and-conquer. The optimal solution is to organise the incoming batches by publishing year. In SQL terms, this means storing the source data so it can be efficiently retrieved by the partition key used to define the Power BI partition.

There is a trade-off between load time and query time. Splitting one big box of books into multiple boxes by publishing year increases the total number of objects that need to be managed. Too many small partitions can create overhead. Too few large partitions can reduce the benefit of parallelism. The choice of interval size—year, month, day, or hour—depends on the volume of data, refresh frequency, and query behaviour.

When partitioning, new developers often confuse which date to use. The partition key must be a business date rather than a change-tracking column such as `[Row update datetime]`. Using the same column for partitioning and change detection pushes rows toward the end of the range and breaks the logic. Following the analogy, the date should be the publishing year rather than the year the book arrived in a batch. The latter will always be in the last batch and does not correctly update older books.

33.4.3 Rolling windows

When partitions are in place, rolling windows provide the next level of efficiency. Power BI allows a data engineer to define the number of partitions to keep, and older partitions are dropped from the model. This keeps the model size under control and prevents refresh times from growing indefinitely.

Boundary points in the window are error-prone. Two details deserve attention.

First, if the SQL source table itself rolls history, its window must be kept in sync with the Power BI table’s rolling window to avoid mismatches.

Second, the boundaries of fact tables must match related dimensions and other facts. This is particularly true for ID dimensions, which may contain keys only because related fact rows still exist. If dimensions are not kept in sync, they may contain primary key values that no longer exist in the fact tables. If a dimension drops keys too aggressively, relationships may break for facts that still remain.

33.4.4 Incremental partition refresh

Incremental refresh adds change detection on top of partitions so only partitions with changes are refreshed.

The native Power BI interface allows the data engineer to choose a datetime column in the table for change detection. During refresh, Power BI evaluates the maximum value of that column for

each partition. This is the polling query. Power BI manages the partition refresh bookmarks internally. Each partition stores the polling value that was observed when that partition was last refreshed successfully.

If the next polling query returns a different value from the value stored internally against the partition, Power BI refreshes that partition. After the partition refresh succeeds, Power BI stores the new polling value as the partition's bookmark.

Following the analogy, each box of books by publishing year has a label showing the last update datetime observed for that box. This is the bookmark. When batches of books arrive, each year can be checked to see whether any book has an update datetime greater than the label on the box. This is the polling query. This is effective only if incoming batches are organised so that it is easy to find the maximum update datetime for each publishing year.

33.4.5 Custom polling tables

The native approach offered by Power BI can be limited if the source table is a view that joins multiple tables, and thus there is no single column representing change detection.

It is also not sufficient if partitions have deletes because a normal update datetime cannot reflect delete datetimes. When the native approach is not sufficient, the user can specify a custom polling query with arbitrary M code through XMLA-based configuration.

Polling tables can facilitate rapid evaluation of a polling query for complex cases. A polling table is simply a list of partition values and the datetime the partition was last updated. During refresh, the polling query looks up this update datetime for a partition.

Following the analogy, the incoming batches of books may have a companion spreadsheet that tracks publishing years and the latest update datetime of books for that year. During updates, this spreadsheet is consulted to decide whether that year's box needs updating.

Power BI does not support custom polling queries in its normal interface. XMLA-based configuration is required.

If Power BI can rapidly evaluate whether a partition needs refreshing and retrieve the data for that partition quickly from the source table, this can lead to substantial improvements in load times. Supporting this requires the data engineer to prepare the data through indexing, sorting, materialising source tables, or constructing dedicated polling tables.

This preparation itself takes time, and any base tables or polling tables should themselves be incrementally refreshed. Consequently, optimising Power BI loads means trading Power BI load time for source-side preparation time. If not done well, these artefacts can take longer to maintain than the time saved through incremental partition refresh.

33.4.6 Example refresh bookmarks and polling values

Suppose `PBI.Sale` is partitioned monthly using `[Sale date]`.

The source table contains sales across several months. Each row also has a `[Row update datetime]`, which records when that row was last inserted or updated in the pipeline.

Source table: PBI.Sale

Sale ID	Sale date	Customer ID	Amount	Row update datetime
S1001	2025-01-05	C001	120.00	2026-05-01 09:15
S1002	2025-01-19	C002	85.00	2026-05-03 14:20
S1003	2025-01-28	C003	64.00	2026-05-03 16:45
S2001	2025-02-03	C004	230.00	2026-05-02 10:30
S2002	2025-02-14	C005	75.00	2026-05-09 08:40
S2003	2025-02-24	C006	310.00	2026-05-09 11:10
S3001	2025-03-08	C007	50.00	2026-05-04 12:00
S3002	2025-03-18	C008	140.00	2026-05-04 15:30
S3003	2025-03-29	C009	95.00	2026-05-04 18:05

The Power BI partitions are done per month on [Sale date].

Partition start of month	Partition filter	Rows
2025-01	[Sale date] >= 2025-01-01 and [Sale date] < 2025-02-01	S1001, S1002, S1003
2025-02	[Sale date] >= 2025-02-01 and [Sale date] < 2025-03-01	S2001, S2002, S2003
2025-03	[Sale date] >= 2025-03-01 and [Sale date] < 2025-04-01	S3001, S3002, S3003

PBI.SalePolling is a companion table to PBI.Sale. It contains one row per partition and stores the maximum [Row update datetime] found inside that partition.

Companion polling table: PBI.SalePolling

Partition	Latest update datetime
2025-01	2026-05-03 16:45
2025-02	2026-05-09 11:10
2025-03	2026-05-04 18:05

The values in PBI.SalePolling correspond to the maximum row update datetime in each partition.

Partition	Row update datetimes in PBI.Sale	Latest update datetime in PBI.SalePolling
2025-01	2026-05-01 09:15; 2026-05-03 14:20; 2026-05-03 16:45	2026-05-03 16:45
2025-02	2026-05-02 10:30; 2026-05-09 08:40; 2026-05-09 11:10	2026-05-09 11:10

Partition	Row update datetimes in PBI.Sale	Latest update datetime in PBI.SalePolling
2025-03	2026-05-04 12:00; 2026-05-04 15:30; 2026-05-04 18:05	2026-05-04 18:05

Each Power BI partition also has a stored refresh bookmark. This bookmark records the latest polling value observed when that partition was last refreshed successfully. Power BI manages these partition bookmarks internally.

Power BI partition bookmarks before refresh

Partition	Stored refresh bookmark
2025-01	2026-05-03 16:45
2025-02	2026-05-02 10:30
2025-03	2026-05-04 18:05

During refresh, Power BI compares the stored refresh bookmark for each partition against the current polling value from PBI.SalePolling.

Partition	Stored refresh bookmark	Latest update datetime	Changed?	Action
2025-01	2026-05-03 16:45	2026-05-03 16:45	No	Do not refresh
2025-02	2026-05-02 10:30	2026-05-09 11:10	Yes	Refresh partition
2025-03	2026-05-04 18:05	2026-05-04 18:05	No	Do not refresh

Only the 2025-02 partition needs to refresh. When it is completed, the partition bookmark is advanced to match the polling value.

Power BI partition bookmarks after refresh

Partition	Stored refresh bookmark
2025-01	2026-05-03 16:45
2025-02	2026-05-09 11:10
2025-03	2026-05-04 18:05

33.5 Conclusion

DirectQuery, efficient source tables, and partitioned loads are the main ways to improve Power BI load performance.

Method	Basic idea	Best suited to	Consideration
DirectQuery	Avoid importing the table.	Very large, simple, narrow fact tables.	Slower report interaction and DAX limitations.
Efficient source views or tables	Make import cheap by preparing the source.	Most Import models.	Complexity moves upstream.
Partitioning	Refresh the table in slices.	Large fact tables with a stable business date.	Poor partition design can add overhead or miss changes.

Partitioning also offers several levels of improvement.

Enhancement	What it does	Benefit	Extra requirement
Parallel partition refresh	Splits one large table into smaller partitions that can be loaded independently.	Reduces refresh duration by allowing multiple partitions to load at once.	The source must retrieve each partition quickly using the partition key.
Rolling window	Keeps only recent partitions and drops older ones.	Prevents model size and refresh time from growing indefinitely.	Fact tables, dimensions, and source-retention windows must remain aligned.
Incremental partition refresh	Refreshes only partitions whose polling value has changed.	Reduces refresh work when only some partitions have changed.	Requires reliable change detection, partition bookmarks, and fast polling queries or polling tables.

A single Power BI model may use different strategies for different tables. A small dimension may load fully. A large fact table may use Import mode with incremental partition refresh. A very large, narrow fact table may use DirectQuery over a dedicated columnstore source. The right strategy depends on the shape of the data, the refresh requirement, and the expected user experience.

Whatever the technique, the foundation for a fast-loading Power BI model is set far in advance in the pipeline. It depends on meaningful fragments to avoid complex transforms, planned indexes to support partition retrieval, and careful row-change tracking to enable polling tables.

In the ideal case, information efficiency is maximised end to end:

1. raw tables enter the pipeline incrementally, using polling tables to track changes and annotating rows with change-tracking artefacts;
2. downstream tables transform incrementally with in-sync datetimes and are designed for ease of assembly;
3. source tables for Power BI, whether for DirectQuery or partitioned loads, are also incrementally materialised;
4. Power BI tables themselves are incrementally refreshed using partitions, supported by incrementally refreshed polling tables for fast change detection.

At every stage, the pipeline maintains proportionate change—the amount of computation changes in proportion to the amount of information that has genuinely changed.

The implication is that the data engineer must plan far ahead. Power BI’s efficiency requirements should shape the pipeline even as the first table is built.

Key ideas.

Power BI refresh is the point at which the quality of the pipeline becomes visible.

Power BI should assemble information, not manufacture it.

Source tables for Power BI should be simple, prepared fragments rather than complex transformation views.

DirectQuery avoids import refresh, but shifts cost to query time and imposes user-experience limits.

Partitioning improves refresh by dividing large tables into independently loadable slices.

The partition key should normally be a business date, not a change-tracking date-time.

Rolling windows keep model size and refresh time under control, but require careful boundary alignment.

Incremental partition refresh depends on polling queries and partition bookmarks.

Power BI manages partition bookmarks internally, but the pipeline may need to supply fast polling tables.

Optimising Power BI load requires planning backward from the semantic model into the pipeline. ““

Judgement under ambiguity

The six principles of data engineering

Good data engineering is guided by principles, not just techniques.

The earlier sections outlined patterns and practices for data engineering scenarios.

They focused on the treatment of information rather than technology. The technology-specific sections are on Power BI, and even then, we stayed away from syntax to focus on fundamentals.

This focus on information rather than technology is deliberate. It points data engineers to underlying principles rather than situational techniques. This last section, Judgement under ambiguity, is where patterns culminate and are summarised in six principles.

34.1 The problem with shallow curation

The most common, but incorrect, approach for new data engineers is shallow curation.

The typical steps are:

1. Take the source system and map to the target environment, adjusting for names and data types.
2. Receive a list of business requirements for reporting from stakeholders.
3. Using the source data, apply transformation rules to meet stated requirements, load them into a few big tables for Power BI. A more experienced data engineer may organise these as facts and dimensions.

This mindset has many drawbacks. Firstly, it leads to garbage-in, garbage-out. Source applications are not designed for analytical intent and never has the information content to meet business needs. Requirements stated upfront are rarely adequate to be a reliable guide to address this gap. Moreover, this approach focuses on the questions at hand rather than all the questions about the business. Finally, going straight from source to reporting tables means there are no building blocks for use in different scenarios. Using the information for a different scenario, such as ML feature engineering, requires rebuilding logic that embedded in dimensional models.

In other words, shallow curation is too easily satisfied. It does not sufficiently push the data as found to meet the demands of business intent. Neither does it break down complexity in building towards advanced levels of achievement. Consequently, the difference is not between

merely “good versus poor” quality, but between what is “possible versus impossible” for a team to achieve with this mindset.

The problem with shallow curation is that there are some outputs. After all, the data engineer has faithfully reproduced data from the source system for users to gain access. Such an engineer may even feel satisfied for doing an excellent job when reality is far from the potential value. If business users are dissatisfied when business objectives are not fully met, it is possible to blame poor data quality at source or on the unclear business rules. The danger of shallow curation is that, like all mindsets that starts by reducing standards, it settles for less, and by settling for less, creates a blind spot for the engineer who cannot see the failure to reach excellence.

34.2 Six principles

A surer basis for data engineering can be summarised in six principles:

- Instead of garbage-in-garbage-out with raw data, add value through expressive entities.
- Instead of building giant tables, create meaningful fragments.
- Instead of stopping with what works now, anticipate errors that may occur.
- Instead of wholesale response, maintain proportionate change.
- Instead of reacting to requirements, build momentum through leading discovery.
- Instead of stopping at the symptoms, diagnose the root cause.

That is, instead of finding quick fixes to what is wrong, the expert data engineer focuses on what needs to be right for business intent—now and into the future. These can be remembered through six phrases: expressive entities, meaningful fragments, anticipate errors, proportionate change, build momentum, root cause.

It is not a matter of “fast workaround” versus “slow and proper.” It is the difference between what gets tangled up in a mess versus what will not. In the long term, the six principles create fast and flexible paths to success.

The earlier sections covered four of the principles—expressive entities, meaningful fragments, anticipating errors, and proportionate change. The remaining chapters introduce the final principles—build momentum and root cause.

The section ends with the chapter Hallmarks of quality. This essay was written years before this text and reproduced with minimal edits. The essay guided the years of work now summarised in this book.

Working with stakeholders

Successful data projects depend on guiding attention, not merely collecting requirements.

The most common cause of project failure is the inability of the delivery team to build momentum with stakeholders. When momentum stalls, two patterns can often be found.

The first is when the delivery team asks stakeholders for a list of “reporting requirements,” but the stakeholders struggle to provide one. From their perspective, the richness of their business cannot be reduced to a simple checklist. When the team insists on detailed requirements, the process reaches a stalemate.

The second pattern appears more promising at first. Stakeholders provide a clear list of requirements upfront—sales aggregated by date or region, non-compliance by product type, even precise business definitions. The delivery team implements these, and all seems well. Yet during testing, the product does not meet its purpose. It lacks core features, breaks on edge cases, or proves too complex to use.

Both scenarios share the same flaw of reacting to requirements. Despite its obvious and repeated failure, this pattern persists. The reason for this persistence is because experts are often familiar with the intricacies of their own discipline but underestimate the challenges of others, expecting simplistic answers for everything else to be given in clear-cut form so they can proceed with their part.

The world is not so straightforward. The most difficult part of a data project is not technical complexity but in thinking deeply about the business and its relationship with the data world. Asking stakeholders to enumerate requirements is convenient for delivery teams because it shifts the responsibility for this deep thinking to stakeholders, leaving developers with the easier task of implementing clear technical rules.

Instead, in a successful project, the delivery team guides stakeholders to express their intent, then works together to explore the data in light of that business intent. Through this process, the stakeholders uncover what they need to “see” from the data for them to achieve business outcomes. These discovered needs—not an initial list—are the real requirements. This shift turns a one-way process into a two-way dialogue where the delivery team plays a leading role by guiding collective attention.

The fifth principle of data engineering is therefore: instead of reacting to requirements, build momentum through leading discovery.

35.1 Guided attention

During development, the delivery team and stakeholders explore the data together to meet business intent. The delivery team contributes expertise by guiding the stakeholders in how they should “see” the data in light of business intent—hence guided attention. This dialogue is an iteration between business and data, between questions and answers, between thinking and testing, and between business and technical expertise. The seven principles that follow provide a practical framework for making this a success:

1. Focus on trust
2. Lead by listening
3. Own the business intent
4. Anchor a vision
5. Gather around the solution
6. Design for workflows
7. Spot the 20%

35.1.1 Focus on trust

Many assume that the primary mode of requirements gathering is that of extracting information. In this view, the question becomes: “How effectively and accurately can I get the information from stakeholders to build a product?” This is the wrong focus. The first responsibility of a delivery team is to gain the stakeholder’s trust.

The reasons for focus on growing trust are:

1. Trust is the context for information. Clearly articulating and refining requirements is arduous. Stakeholders will not be able to give the right information before they have first established a strong trust in the people asking for the information.
2. Relationship with others is the biggest factor for meaningful and enjoyable work.

Strong relationships with stakeholders are more likely to lead to enjoyable collaboration during the development phase, and subsequently the success of the team.

3. The focus on extracting information sees the stakeholders instrumentally as a vehicle for information. The focus on growing trust sees stakeholders as human beings as an end-to-themselves with whom we work.
4. Delivery teams have recurring partnerships with stakeholders that extend beyond a particular project. The focus on building trust takes the long-term view.

Without trust, no data project can succeed. After each engagement, the question should not be “Do we know more about the requirements?” but “Do our stakeholders trust us more?”

The best ways to grow trust are transparency and active listening.

- Transparency means clear communication of the construction plan and regular demonstrations of progress—weekly or twice-weekly. This is akin to a home buyer seeing the building take shape.
- Active listening means thoughtful paraphrasing and summarising stakeholder input. When stakeholders hear their own language reflected, they feel understood and affirmed. This also happens when they are invited to check metadata, which gives them a tangible way to contribute.

A common mistake is for delivery teams to assume that the purpose of speaking is to transfer information—if they already know what stakeholders are about to say, then they can skip to the next part. This is a mistake. Beneath the surface, the stakeholder is:

1. Developing a feeling of being heard.
2. Clarifying their own thinking through articulation.
3. Growing in data literacy through dialogue with technical experts.

All these build trust. By remembering that the first responsibility of the team is to gain trust, we resist the temptation to cut people off or jump in to correct them. Patience is paramount.

35.1.2 Lead by listening

In some types of partner dancing, there are two designated roles: Lead and Follow. The Lead initiates all the movements and the Follow completes the sequence with an elegant response. An analogous dynamic is needed with stakeholders. The delivery team should formulate its own statements as responding gracefully to a previous statement from stakeholders. These responses typically take one of the following forms:

- Clarifying question: “You said X, can you please clarify whether X means...?” or “You said your team needs X—can you help us understand how this fits into your goal?”
- Playback: “There’s a lot in what you just said—can I paraphrase to check I’m on the right track?” or “I’ve summarised your explanation into workflows—can I share them for confirmation?”
- Proposal: “Based on what you said about your workflows, here are wireframes of our solution hypothesis—does this align?”
- Amendment: “You gave feedback on our hypothesis—here’s a refinement. What do you think?”

By responding to stakeholder initiatives, the delivery team is, in reality, leading. This is achieved by listening closely to what has been said and naturally guiding the stakeholder’s attention through a technical problem. This approach ensures:

- Stakeholders are heard, feel heard, and are seen to be heard.
- The conversation remains structured, logical, and focused on business objectives.

There will be times when the team needs to correct an error, redirect focus, or counter unconscious bias. This principle still applies. A common scenario is when stakeholders describe solutions before the problem is fully explored—for example, asking for a detailed data dump dashboard when analysis suggests a summarised view is better.

Saying “Let’s focus on requirements instead of jumping to the solution” sounds natural but often feels dismissive. For stakeholders, the solution is the requirement.

Regardless of content, stakeholders are trying to communicate something important.

This should always be affirmed. Even if the content is incorrect, the underlying intent is valuable. Respond by continuing with their initiative rather than breaking off. For example:

- Explore the intent: “You said your goal is to detect potential non-compliance— how does the data dump help?”
- Explore the utility: “You want a detailed data dump—can you give an example of a workflow where this is used?”
- Re-orient: “Thanks for raising the data dump idea. You previously said your goal is detecting non-compliance. Can we explore scenarios where that happens, then revisit the data dump in that context?”

These approaches respect intent and apply the first principle: focus on trust. It also applies the fifth principle: gather around the solution.

This principle should never be used as a shield for blame (“I just did what you told me to do”).

35.1.3 Own the business intent

The task of data engineering is to align data to business intent. It is an experimental process that demands the data engineer see the stakeholder’s perspective “first-hand.” Without this, the team will never truly see what the stakeholder sees, and the potential of the solution will remain unrealised. This act of viewing the data through the stakeholder’s lens is owning the business intent.

The data engineer who owns the business intent must be committed to in-depth business analysis.

This document is not a guide to business analysis, except to note that business processes often conform to recurring patterns. Mastering these patterns helps the team achieve high-quality analysis quickly, even for complex scenarios. The team can do this by consistently asking stakeholders five key questions:

- Intent: What is the business intent?
- Measure: How does the business measure the achievement of this intent, either directly or indirectly?
- Sensor: What instruments does the business have for knowing whether things are going well or not?
- Controls: What levers does the business have to influence this measure?

- Drivers: What external events outside of does the business’s control may influence this measure?

Business analysis needs to permeate the project from start to finish. For example, concepts such as “good vs bad” entities or milestones for measuring processes are rarely defined in source systems. It is unrealistic to expect stakeholders to define these upfront before seeing the data. Instead, these definitions evolve through exploration and require input by technical expertise and experience with similar problems. This, once again, demands that the team see the business problem first-hand.

Owning the business intent is an extension of the first principle of trust. Nothing gains trust with stakeholders as quickly as a team who can fluently speak the details of a business language. From this perspective, business analysis is a delivery team’s way of active listening.

Whether a team owns the business intent will define its passion, drive, business knowledge, and creativity. These qualities will determine whether its output is a mediocre reflection of the current state or a solution that pushes the business forward to excellence.

35.1.4 Anchor the vision

The natural extension of owning the business intent is a vision for the business. A vision is a view of the value the team aspires to achieve. It is both ambitious and concrete.

Whilst having a vision is important for any project, it is particularly critical in exploratory data projects for the following reasons:

- Improves the probability of arrival. Complex data projects are often ambiguous and uncertain. Teams that articulate and revisit a vision are more likely to reach the intended outcome.
- Sustains engagement. Long projects involve hard work, such as resolving edge cases. A clear vision inspires a sense of meaning to maintain engagement.
- Keeps debates in perspective. Under constraints, debates are inevitable and can become flashpoints of tension. A vision keeps these debates in perspective.

Teams often relax around specific problems when they see those problems as only one part of the whole.

- Structures discussion. Data projects are open-ended and can easily drift.

Anchoring the project in a vision, and deriving each step from that vision, keeps the team on track.

Using the vision to structure discussions begins with agreement on the overall intent of the business. The project objective is situated within the business objective, which is itself a part of the broader organisation’s objectives. The discussion on each delivery feature should be anchored to this hierarchy, with constant reference back to the higher-level intent. When the conversation becomes lost, the team moves up one level and reorients.

Example dialogue:

Team & stakeholder: Introductions Stakeholder: Jumps into details about needing a dashboard
Team: “Can you start by telling us the purpose of your business, your role, and what success means to you?”

Stakeholder: Provides some context, still detailed Team: “So your organisation’s goal is to improve customer experience, and your team supports this by monitoring service quality?”

Stakeholder: “Yes, but also...”

Team: “Okay, so you track service quality, but you also need to identify recurring issues to prevent them. Is that right?”

Stakeholder: “Yes...”

Team: “At the start you mentioned a dashboard. What is its purpose in this context? Is it for your team’s internal use, or to share insights with other areas?”

This pattern—bringing stakeholders up to the highest level and then unpacking details step by step—requires fast thinking and familiarity with the organisation’s goals. It is harder than it appears, but essential for clarity.

When priorities conflict, reference to the higher-level intent provides common ground.

Agreement is easier at higher levels and can be used to resolve issues. This approach works only when the conversation has been structured from the vision downward.

Unlike a building project, which relies on well-defined specifications of the target building and strict timelines to track progress, data projects are exploratory and open-ended. In this setting, a clear vision, rather than detailed specifications, plays the role of guiding the team. It serves as a north star and compass that sustains quality and momentum across a long project. For this reason, team leaders should insist on vision, quality, and momentum rather than rigid deadlines.

35.1.5 Gather around the solution

Data projects often waste time through too much talking and not enough doing.

Instead, the best way to collaborate is to gather around a solution—start with a pen-and-paper solution hypothesis and refine it through discussion and experimentation.

The project should develop hypotheses as early as possible, even by the second engagement. Early development matters because:

- A convincing solution is the only proof the team has understood the requirements.
- Hypotheses provide a concrete point for clarifying stakeholder thinking.
- Refinement creates excitement as all parties see an early vision come to life.

A solution hypothesis is the most important part of requirements gathering because it is the only real proof the team has done the work. In an analogy of commissioning a building, it is futile if the builder only has a list of requirements (four rooms, lots of sunlight). The builder wins the contract by providing an architectural sketch—nothing less will do. No buyer would pay a mortgage deposit for a building without a sketch.

Why expect stakeholders to work with a team without showing a solution hypothesis?

When projects meander, it is often because direction is unclear. The best advice for such situations: “When the project is stuck, draw a picture of the solution.”

The best way to refine the solution hypothesis is through an open workbench format. In an open workbench, the delivery team and stakeholders meet twice a week to explore the data model, experiment with new features, test definitions, and provide feedback for the next iteration. Stakeholders may come from different business areas, making the data model a centre for converging perspectives.

This format requires developers to be confident hosting discussions and responding to impromptu questions from stakeholders. The entire team should support them in doing so. When done well, the open workbench has a transformative effect: it builds trust through transparency and enables genuine dialogue.

Teams sometimes say, “Let’s not jump into solution mode.” This statement is not acceptable with stakeholders. For them, the solution is the requirement. When asked to describe requirements, most stakeholders are painting a mental picture of the solution. Even if it is a bad solution, it still describes what they imagine they are working toward. The team should playback the intent or seek clarification.

The error is to draw a hard line between requirements and solution. This is a distinction developers make, not stakeholders. The real distinction is between the why (intent) and the what (solution). Stakeholders will often mix both. It is the team’s job to discern one from the other.

This cannot be stressed enough: to stakeholders, the solution is the requirement—not something separate from it. In a data project, it is never too early to gather around the solution.

Gathering around the solution is the ultimate way the delivery team guides stakeholders toward clarity. It helps them see how reshaped data interacts with business intent, turning abstract requirements into something tangible. Through iterative refinement—testing, visualising, and adjusting—the solution evolves until it displays fidelity with business reality. In doing so, attention is guided deliberately, ensuring that every discussion converges on what matters most: a solution that stakeholders can recognise, trust, and act upon.

35.1.6 Design for workflows

It is common for data projects to deliver reports that are rarely used. Stakeholders may specify many requirements— “I want to see count of X per country”—and show excitement during development and testing. Yet after deployment, usage often drops quickly.

This happens when solutions are not anchored in real workflows. Anchoring a solution with workflows ensure that it will have usage when deployed.

Every workflow has at least two elements: an intent and a trigger. As an example of a business working at the helpdesk, see Figure 1.

Figure 1. Example of a workflow.

A trigger can be a specific event (e.g., receiving an email from a customer) or a schedule (e.g.,

publishing a quarterly report).

Contextualising requirements in workflows offers key advantages:

1. Ensures the product delivers value because it meets the intent of an established workflow.
2. Ensures the product will be used because real-world triggers exist for its use.
3. Integrates the product seamlessly into the user's daily role.
4. Helps stakeholders recognise when a requirement is not important and can be dropped.
5. Provides a natural way to rationalise requirements—for example, one report page per workflow.

Designing for workflows is a simple tool to ensure that project teams are grounded in real world problems.

35.1.7 Spot the 20%

The 80/20 rule says 80% of results come from 20% of effort. Its flip side is that the remaining 20% consumes 80% of the effort. This creates an illusion that most work is done when the hardest part remains—the hidden, complex 20%.

The 20/80 trap is dangerous because:

1. Foundational elements are missed until production.
2. Expectations are mismanaged when stakeholders think the work is complete.
3. Technical debt piles up when essentials are discovered late.

Teams avoid the trap by:

1. Grounding requirements in thorough business analysis to uncover hidden assumptions.
2. Focusing equally on what works and what doesn't—the parts that work are often forgotten.
3. Asking not only "What should this do?" but "What could go wrong?"
4. Balancing common cases and edge cases without bias.
5. Developing in sound engineering order, not simply in the order stakeholders want to see.

As an analogy, home buyers focus on visible features—"How many rooms?" or "Is there good ventilation?"—based on obvious needs and past pain. A good builder starts with the foundation. Likewise, a good data team begins with the start of the business process and systematically work through the business processes in order, rather than jumping to flashy features.

Because of this, the team will often run counter to stakeholder instincts. This is healthy.

In effect, the team acts as:

- A counterweight to natural biases that overlook fundamentals.

- A safeguard against missed edge cases that matter.

Foreseeing hidden elements is hard. It takes strong business knowledge, technical experience, and high engagement skills to counter bias. Only the best teams do it well.

35.2 Conclusion

The hardest part of any data project is thinking deeply about the business and how it relates to data. This can only be achieved when stakeholders and the delivery team explore the data together—thinking together through a dialogue focused on the solution. In this dialogue, the relationship is not symmetrical. Stakeholders, often new to data, do not know what to look for. This is why the experienced delivery team plays an important role in guiding the dialogue’s attention to relevant observations.

The seven principles of engagement capture hard-earned wisdom for facilitating these dialogues. They run counter to ingrained habits that focus on visible details while ignoring complexity. They are treasures for any delivery team.

The principles change the team’s view of engagement from receiving requirements and getting product sign-off to that of building trust with stakeholders. Trust is the guiding principle. Fluently speaking the business language establishes trust early in the project, gathering the solution and continuing to demonstrate progress on the business vision carries trust to the end.

These principles move the team beyond reacting to requirements. Applied well, they enable the team to guide stakeholders through complexity and build momentum toward a solution that truly meets business intent.

Construction planning

A strong construction plan creates momentum by giving delivery an orderly shape.

The fifth principle of data engineering is to build momentum through leading discovery rather than reacting to requirements. This can be challenging in projects that are lengthy and overwhelmingly complex. In such cases, the engineer often feels at a loss for where to begin. Because the data engineer sits on the critical path of data projects, this uncertainty directly impacts project momentum. Conversely, a data engineer who delivers outputs in an orderly and predictable way propels the project forward—participants align on the goal and instinctively sense that the team is taking concrete steps toward achieving it.

In complex projects, a data engineer stays organised and guides the delivery team by formulating a plan. This plan is expected from an experienced engineer because:

1. No project waits in silence. The engineer cannot disappear for a month to apply ideal patterns and return later. Unless the engineer provides the team and sponsors with confidence about the weeks or months ahead, there will be no engineering work at all.
2. Logical sequencing matters in a build. Doing tasks in a sensible order helps the engineer move forward with confidence, avoids code entanglement, and reduces mistakes.
3. Only the engineer knows the effort required. The data engineer alone understands how long quality work takes and has the sole responsibility to advocate for that time.

If the engineer does not actively formulate a plan, there will be no plan—leading the project to meander. Or someone else will create the plan for the engineer—leading the project to rush. In either case, the project suffers.

Projects can succeed or fail depending on the data engineer's plan. It is one of the data engineer's leadership roles when guiding stakeholders. Formulating a plan for the team is one of the most advanced skills to master and a mark of a confident engineer.

36.1 An effective plan

A construction plan lays out, in sequence, the components the data engineer will build and provides a forecast for how long each part will take. This plan gives users a clear expectation of what features will arrive and when. In addition, a construction plan includes non-feature components such as unit tests, technical debt clean-up, and performance tuning. The plan defines the project's momentum.

A plan is effective if it provides the project with confidence and momentum. For this purpose, the focus is on sequence rather than precise timings. While it is important not to blow out project timelines, the sequence is more critical than exact estimates because delivering one feature after another is strongly associated with user satisfaction and project momentum.

An effective plan is akin to a travel itinerary for exploring a new country. An itinerary is not effective merely by meeting a predefined schedule; it is effective when it moves forward logically, providing a quality experience without wasting time.

A data engineer can craft an effective plan by aiming for three characteristics:

- Ambitious yet grounded
- Orderly yet flexible
- Rotating new features with backend consolidation

Being ambitious means having a view to tackle the full business processes and designing a model that addresses all questions, rather than limiting the scope to specific requirements raised by current stakeholders. Being grounded means supporting this vision with well-informed business analysis so that the plan is neither vague nor impossible, and ensuring that necessary information is available for reliable time estimates.

An orderly yet flexible plan is akin to a travel itinerary that is physically sensible (not looping back across half the country) but still organised into reasonable segments that can be shifted. For a data engineer, being orderly means isolating releases into promotes code clarity, avoid code entanglement, and minimises rework. Flexibility means structuring the plan so that key business priorities can shift forward or backward as a block without massively disrupting the schedule. This requires grouping deliverables into logical blocks centred on core business processes, rather than maintaining a long miscellaneous list of business questions.

New features excite stakeholders and propel project momentum. However, the data engineer must balance these visible features with invisible but equally important consolidation work. An effective plan achieves both by alternating between them. This rotation not only supports quality work by building consolidation into the schedule but also gives stakeholders time to absorb one feature before moving on to the next. This immersion is necessary because data insights are exploratory in nature. Testing a new feature requires users to interact with the output, grasp its implications, and check edge cases thoroughly.

36.2 Formulating an effective plan

A data engineer can craft an effective plan in four stages. Each stage involves increasing commitment to specific outcomes. A simple project may require only the first stage, while a complex project may demand a detailed plan at the fourth stage. The four stages are:

1. Discovery
2. Vision
3. Scope

4. Build

36.2.1 Stage 1 – Discovery

Formulating a plan begins with discovering facts about the business. The purpose of discovery is to answer key questions. At the highest level, these include:

- What are the major business processes in the end-to-end business lifecycle?
- What information is captured by each process, and where is it stored?
- What is the stakeholders' interest in each process?
- What reporting artefacts already exist, and how are they currently used in workflows?

The team answers these questions by reviewing artefacts relevant to the business processes and interviewing stakeholders.

To sharpen the focus on business objectives, the team should also answer:

- Intent: What is the business intent?
- Measure: How does the business measure the achievement of this intent, either directly or indirectly?
- Sensor: What instruments does the business have for knowing whether things are going well or not?
- Controls: What levers does the business have to influence this measure?
- Drivers: What external events outside of does the business's control may influence this measure?

The outputs of discovery should be expressed as linear process diagrams and cumulative information diagrams, as explained in the chapter Anticipating Questions.

Recall that a linear process diagram is a simple diagram that lays out the major business processes in chronological order, without loops or branches. Its purpose is to give a clear, high-level view of the end-to-end flow, making it easy to see where each process fits in the overall lifecycle. A cumulative information diagram is a table that lists business information as rows and business processes as columns, showing which process captures or inherits each piece of information. This diagram helps identify what data is available at each stage and ensures completeness for answering business questions.

36.2.2 Stage 2 – Vision

The vision for all data engineering projects is the same: advance business intent by obtaining insights across all underlying business processes. This vision is adapted to each project using findings from the discovery phase, especially the linear process diagrams.

For example, if the core business processes for an organisation are manufacturing, quality control, sales, shipping, and customer feedback, the vision statement could be:

“Understand the factors that drive sales profit and customer satisfaction through an integrated data source of complete business processes—from manufacturing, quality control, sales, shipping, to customer feedback. Obtaining timely and accurate insights into measures such as manufacturing turnaround, early detection of quality issues, changes in sales trend, efficiencies in shipping and latest customer sentiment.”

This vision statement is ambitious yet grounded. It resonates with business stakeholders, inspires confidence, and establishes trust by demonstrating that the project team understands the business. Being neither too vague nor too specific, the vision frames the project at the right level for prioritisation and dialogue.

Such vision statements are easy to craft and extend naturally from the discovery questions. Yet many teams produce poor alternatives that lead projects to failure. For example, it is common to see vision statements such as:

“Reduce pain points of manual processes and create near real-time dashboard of the operation.”

These statements are not a deep analysis of business intent but a reaction to user complaints. As such, they lack the substance to serve as a foundation for the project.

As part of vision setting, it is important to sketch pen-and-paper wireframes of sample reports that user can build off the data. The wireframe is a play-back what the team has heard, and the beginning of a solution hypothesis. Its purpose is to build trust that the team has understood the needs and inspire the project to strive for the finish line. It is akin to an artistic model of a public building prior to build—a good model generates excitement and builds public confidence. To achieve this purpose, the wireframe itself needs to be ambitious yet grounded.

When the vision is framed as above, defining the project scope becomes straightforward.

36.2.3 Stage 3 – Scope

The vision is an ambition to cover the business processes end to end. It is a visionary statement of the linear process diagram.

The full vision may be too large to tackle in one project. The scope defines a subset of business processes to focus on. For example, rather than covering manufacturing through to customer feedback, the project scope may concentrate on the first two—manufacturing and quality control—on priorities such as sales and shipping. Dividing the vision this way allows the creation of project phases: the first phase focuses on two business processes, the second on the next two, and so on.

The selection of business processes should consider three factors:

1. Availability of source data. If the information captured by a business process resides in systems that are difficult to access, it may need to be deferred to later phases.
2. Order of the business lifecycle. Where possible, work in the natural sequence—manufacturing, then quality control, then sales, and so forth.

Information accumulates along the lifecycle, and working in the same order creates building blocks for the next stage.

3. Business priorities. These often pull toward the tail end of the lifecycle, where most activity occurs. The data engineer must judge based on dependencies. For example, starting with sales without manufacturing may be acceptable, but starting with shipping without sales orders is not—no matter how much users want insights on shipping delays.

The cumulative information diagram, which tracks the flow of information and its storage, is useful for informing scope decisions.

The scope must include a time estimate for project completion. This requires input from an experienced engineer who can make a judgment based on discovery findings.

Like the vision statement, the scope is a straightforward application of the linear process diagram.

A common error is to scope by system. For example, rather than working through the business process one by one, the team works through integrating systems one by one.

This is the wrong perspective. One reason, amongst many, is that it easily leads to a situation where the project creates an unusable product because the system does not have the minimal set of information to describe one business process.

36.2.4 Stage 4 – Build

The final stage is the construction plan. This is the practical roadmap for delivery that translates scope into actionable steps and sets out how the project will progress.

A construction plan is akin to a six-month travel itinerary in a large country. The country is the organisation, and the towns are the major business processes. Features are the specific sites within a town. Building blocks are times of being en route, such as taking a long bus, while technical consolidation is the time of rest in the journey.

This travel itinerary is exploratory in nature, so it needs flexibility, but it also needs order so that the physical route makes sense and there is a continuous flow of experiences alternating with rest. This cannot be achieved if the itinerary is too vague: “Let’s see the country.” Nor can it be too detailed: “Here’s a long, ad hoc list of things we want to do all across the country in the next six months.” Instead, an orderly yet flexible itinerary sketches out the major towns and important sites, while the details of visiting a town only need to be fleshed out when getting close.

If the itinerary is well organised, it can be changed mid-way, such as swapping the order of towns or sites, with minimal disruption to the overall journey.

An engineer guiding stakeholders through a data project is akin to a tour guide crafting a travel itinerary for a traveller experiencing a country. The analogous concepts apply.

The two primary considerations are: how the delivery is divided into releases, and the sequencing of those releases.

At the highest level, build should proceed in the sequence of business processes. For example, if the scope includes manufacturing, quality control, sales, and shipping, then the build releases should follow this order, process by process. This contrasts with tackling a mix of manufacturing and sales, or sales and shipping, in one release. If the data engineer finds this cannot be done,

then the abstraction of business processes may need to be revisited. For example, if the data for sales and shipping are so entangled that they cannot be separated for build, then it is arguable that sales and shipping in the organisation are better seen as one process.

If possible, proceed in the same order as the business process. For example, avoid jumping to shipping and then doubling back to sales. However, if reverse order is necessary due to business priorities, then organising the build along the boundary of business processes can still avoid creating a mess.

The releases should be further broken down within a business process. For example, a complex project may expect to build 50 tables in total; this may be divided into releases of 3 to 8 tables at a time. The more challenging the project, the smaller the release size.

Each release should be centred on a group of business information to expose. For example, if the scope is manufacturing, one release may focus on basic attributes such as product types, production status, and production date; another release on manufacturing details such as material inputs and outputs; and another release on aggregated information such as time to manufacture. In grouping releases, priorities should be given to being as logical as possible from a business point of view.

Once defined, the construction plan must build in order of the pipeline by following table dependencies. This means a gradual increase in computation complexity. For example, the first release focuses on basic attributes with little transformation, the second release adds more complex computation, and a third release aggregates information from prior tables. This is the natural progression for a data engineer following the filter -> compute -> reduce steps explained in Creating information. This gradual layering of computation keeps complexity manageable and ensures each part is tested systematically along the way.

In a complex project with many attributes, it is bad practice to work on all the underlying tables and then all the Power BI tables and measures as one release.

Instead, Power BI releases should be interleaved as information is curated in the pipeline. This means that features are surfaced to users continuously, so they get exposure to business attributes as soon as they are added to the pipeline.

Releases should be tightly coupled with unit tests, fault tolerance, and metadata.

Whenever a feature is released, unit tests and metadata should come with the release, rather than as a bulk afterthought.

Releases should be kept short in time—approximately two weeks, or three weeks at most. They need to be small enough for thorough code review.

In a construction plan, not all releases will be user features. There is also important backend work such as building blocks or performance tuning. It is important to earmark the feature releases because they are the key driver of project momentum. Users should know when features are coming and receive them continuously in small chunks.

User features are typically exposed to users in the self-service Power BI model.

User features should rotate with backend work. Not only does this keep up project momentum, it allows data engineers to spend time on quality consolidation and gives users space to fully immerse in testing features between releases.

By anchoring the construction plan on the user features, and keeping in mind how releases build up into a user feature, it becomes easier to reshuffle development as priorities change without ending up in a tangled mess.

Finally, it is not necessary to forecast the plan in full detail for the entire project. This rigidity is not fit for the exploratory nature of data analytics. At any given moment, only the next four or so releases need to be planned in detail, while features further out do not need detailed planning. For example, when working on manufacturing, the data engineer should know the plan for the next few releases to complete the work on manufacturing, while the further processes such as sales and shipping only need to be noted.

A construction plan designed in this manner is, like a good travel itinerary, both orderly and flexible.

The following is an example of a construction plan for manufacture and quality control.

Release	Information focus	Description	Artefacts
1	Manufacture core tables	Basic tables from source system to describe the manufacture process, with metadata and minor clean-up	Cake.Manufacture; Cake.RefManufactureSite; Cake.RefManufactureStatus; Cake.ManufactureStep; Cake.RefProductType
2	Manufacture SLA tables	Define milestone points in the manufacture process, and compute time between milestones and total time-to-manufacture by aggregating over Cake.ManufactureStep, compare the results against SLA for different product types	Cake.ManufactureBatchMilestone; Cake.RefManufactureProductSla
3	Quality control core tables	Basic tables from source system to describe the quality control, with metadata and minor clean-up	Cake.SampleTesting; Cake.RefQualityControlCriteria; Cake.RefQualityControlResult
4	Quality control summary	Aggregate results from different quality control criteria testing, to compute an overall result for a batch, with the concepts of failure severity	Cake.ManufactureBatchOutcome; Cake.RefBatchOutcome

Release	Information focus	Description	Artefacts
5	Power BI model for exposure	Create the views that define the fact and dimension tables for a Power BI report, and expose them in a Power BI model with corresponding measures, including metadata	Driver views for facts & dimensions: PBI_VIEW.Manufacture; PBI_VIEW.QualityControl; PBI_VIEW.ReportingCalendar; PBI_VIEW.BatchOutcome; PBI_VIEW.ProductType etc. Corresponding Power BI tables: Manufacture, Quality control, Reporting calendar, Product type etc. Measures.

36.3 Stakeholder requirements

A typical IT project is driven by stakeholder requirements. But stakeholder requirements cannot determine the whole plan, any more than a town planner can plan a town-build simply by collecting a list of requirements from citizens. On the contrary, the town planner is expected to contribute expertise in dialogue with community.

In practice, stakeholder requirements swing between two extremes—too vague to guide development, or too specific so that the forest is lost for the trees. Consequently, the delivery team should not react to the requirements and build the first thing that is asked. Such an approach is destined to lead to a tangled build if the requirements are too specific, or meandering if they are too vague.

Instead, the data engineer takes stakeholder input as part of a broader consideration for a data model that anticipates questions across end-to-end business processes.

This can be done by reframing stakeholder requirements as objectives about core business processes. If the requirements are too specific, such as a long list of details like “I want to see count of sales by product type,” they can be brought up a level by categorising them as objectives on sales. If the requirements are too vague, such as “I just want to know about my business,” they can be made more concrete by articulating objectives on key processes such as sales, manufacturing, or shipping.

In short, stakeholder requirements should be reframed to the level explained in the chapter Anticipating questions. The delivery team has responsibility in guiding the stakeholders comfortably on this journey as described in Working with stakeholders.

36.4 Conclusion

In traditional IT projects, data engineers are often seen as “doers” who implement specified rules. This is far from their full potential. Being closest to the data, they actively shape how

the organisation understands its business by reorganising information at a fundamental level. Data engineers are true influencers.

This chapter continues the theme of a data engineer's influence. In a high-intensity project, the data engineer—working with other team members—formulates a plan that not only addresses business requirements but reshapes them, using this plan to build project momentum. The data engineer is doing the build, and only the data engineer can craft the plan.

The way to create an effective plan is to use the tools explained in Anticipating Questions: linear process diagrams and cumulative information diagrams. These tools break down organisational complexity into a level that is useful for all parties and especially conducive to the data engineer shaping data under these frames. In this sense, the linear process diagram becomes a common ground between business stakeholders and the delivery team for planning and prioritisation.

An effective plan lays the groundwork for success. In a complex project that requires a defined scope, a rough estimate is that:

- 50% of success is determined by the scope
- 35% by the construction planning
- 15% by the actual development work

This is no surprise. As developers mature, development becomes more mechanical and predictable. Familiarity with patterns makes implementation routine. The more experience builds, the more developers can foresee outcomes and issues in advance.

Consequently, success hinges increasingly on planning. This is why the mark of an experienced data engineer is an effective plan.

Sound judgement

Sound judgement means acting at the level where problems stop recurring.

Sound judgement is the defining capability of a mature data engineer.

It is tempting to think of data engineering as a sequence of technical tasks: building pipelines, fixing errors, optimising performance, and integrating systems. In practice, these tasks are secondary. What distinguishes effective engineers is not how quickly they act, but where they act.

Sound judgement is the ability to recognise at what level a problem should be addressed — whether a situation calls for surface correction, structural repair, or a reconsideration of what “good” even means. It is exercised continuously, not only when something is visibly broken.

For this reason, troubleshooting is not a special activity reserved for failures. It is the most explicit expression of a posture that pervades the entire discipline. The same judgement used to diagnose a broken pipeline is used when deciding how to model an entity, where to aggregate information, or whether a system should be conformed at all.

This chapter presents a simple diagnostic framework that makes sound judgement explicit. This framework defines the sixth and final principle of data engineering—instead of treating symptoms, diagnose root cause.

37.1 A diagnostic framework for sound judgement

Sound judgement can be practised deliberately by distinguishing four stages that are often confused:

Stage	Diagnostic question
Symptom	What is observably wrong?
Trigger	What event caused the symptom to surface?
Root cause	What structural violation allows this to recur?
Final effect	What outcome must hold for the system to be genuinely fixed?

This framework does not prescribe solutions. Its purpose is to discipline attention. It prevents engineers from reacting to what is loud instead of addressing what is consequential.

37.1.1 A simple example

Consider someone learning piano.

- Symptom: the playing sounds offbeat.
- Trigger: the finger is lifted too late before the next note.
- Root cause: the learner relies on playing by ear rather than learning the mechanics of timing and articulation.
- Final effect: the piece is played according to the intended rhythm, not merely

“close enough” to how it should sound.

Two observations matter.

First, the trigger is not the root cause. The trigger explains how the symptom occurs, not why it persists.

Second, the final effect is not the absence of mistakes. It is the presence of the intended outcome.

Sound judgement lies in acting at the level that prevents recurrence, not merely silences error.

37.1.2 Step 1: Symptoms — seeing clearly before acting

The first act of sound judgement is to systematically canvas the symptoms, rather than reacting to the first issue reported.

Stakeholders describe what they see. They do not describe the full boundary of failure.

Beginning work from a single reported symptom allows the reporting channel to define the problem — and that definition is almost always incomplete.

At this stage, interpretation is a liability. The task is mechanical observation.

In medicine, uncertainty is met with vital checks. In data systems, the equivalent is to examine surface health before forming explanations.

Typical symptom canvassing includes:

- If one report is broken, are others broken?
- If one metric is wrong, are related metrics wrong?
- If a domain disappears, did the underlying rows disappear or only their classification?
- Did row counts change? Did distributions shift? Did null rates spike?
- Did upstream pipelines succeed? Did downstream consumers fail?

Jumping ahead at this stage is a failure of judgement. Engineers tend to do so for two reasons:

- pattern-matching based on past experience

- taking the reported symptom as a complete description

Both lead to work that is precise but misdirected.

Sound judgement at this stage consists in restraint. Seeing clearly comes before acting decisively.

37.1.3 Step 2: Triggers — locating the point of exposure

The second act of sound judgement is to identify the trigger — the event that caused the symptom to surface.

Triggers are time-bound and concrete. They narrow the search space and provide orientation.

Common triggers in data engineering include:

- schema changes in source systems
- new or invalid records entering a load
- code changes or refactors
- infrastructure or platform migrations
- permission or security changes
- changes in load ordering or scheduling

Triggers are useful because they are often discoverable, and because they sometimes allow temporary containment: rollbacks, bypasses, or delayed execution.

However, sound judgement requires a strict distinction:

A trigger explains when a symptom appeared.

A root cause explains why it can reappear.

Treating the trigger as the cause leads to fragile systems that fail again under different conditions.

37.1.4 Step 3: Root causes — acting at the level that matters

The third act of sound judgement is to diagnose the root cause.

Root causes explain recurrence. They are structural violations that allow different triggers to produce the same class of symptom.

Root cause diagnosis may be:

- Theoretical: reading the system and identifying what must be true for the failure to occur
- Experimental: isolating variables through controlled tests

Both require skill. Theoretical diagnosis depends on structural understanding.

Experimental diagnosis depends on technical competence and disciplined inquiry.

A useful rule is:

Root causes violate core principles that define the ideal system.

Triggers merely activate those violations.

In data engineering, recurring root causes include:

- incorrect or ambiguous grain
- missing, unstable, or misused keys
- semantic mismatches hidden behind identical column names
- wide tables that collapse unrelated concepts
- implicit dependencies on ordering, timing, or existence
- weak business processes that produce unreliable identity
- compounded logic that mixes extraction, transformation, and aggregation

Two engineers may observe the same symptom and trigger. One applies a patch. The other reshapes the system. The difference is not effort, but judgement.

Sound judgement is the willingness to act where the system is wrong, not merely where it is noisy.

37.1.5 Step 4: Final effect — knowing when you are done

Between diagnosing root cause and completing work lies the act of applying a fix. Fixes vary: containment, refactoring, redesign, or process change.

Regardless of the fix, the final act of sound judgement is to check the final effect.

This includes:

- visual validation: not “does the code run”, but “are the outputs correct”
- environment validation: behaviour across development, pre-production, and production
- end-to-end validation: whether the information now supports the intended decision

A critical rule applies:

The final effect is not the absence of the symptom.

Error suppression creates silence, not correctness. Silence is often mistaken for success, until the failure reappears elsewhere.

Knowing the final effect is difficult because it requires seeing what “good” should be.

Engineers often fail to recognise slow systems because they do not know what fast systems look like. They fail to recognise weak models because they do not know what expressive models look like.

In data engineering, the final effect is always the same:

The business receives the information it needs to make the decision.

Pipeline success, test success, and error absence are necessary — but they are not the end.

37.2 Sound judgement throughout data engineering

Sound judgement is not exercised only when something breaks. It is present throughout the discipline of a data engineer:

- Expressiveness: o Symptom-level response: rename columns or add documentation when reports confuse users. o Root-cause response: reshape the model so business meaning is explicit in its structure.
- Fragment modelling: o Symptom-level response: patch special cases into existing tables as complexity grows. o Root-cause response: extract independent fragments and isolate responsibilities.
- Reference data and conformance: o Symptom-level response: reconcile mismatched numbers in reports. o Root-cause response: introduce shared reference data and apply it consistently.
- Aggregation and grain: o Symptom-level response: simplify visuals or add filters when reports are slow or inconsistent. o Root-cause response: aggregate correctly in the pipeline at a grain aligned with decisions.

In each case, nothing is “broken” in an obvious sense. Yet judgement is still required.

Mature engineers recognise these situations early and act at the level that prevents future failure.

37.3 Conclusion

Sound judgement is the ability to act at the right level of abstraction, at the right time, for the right reason.

The diagnostic framework introduced here makes that judgement explicit:

1. Canvas the symptoms
2. Identify the trigger
3. Diagnose the root cause
4. Check the final effect

Common failures arise from:

- acting on symptoms without understanding scope

- mistaking triggers for causes
- equating silence with success

The goal is not to avoid problems. It is to address them at the level where they stop recurring. Thus, instead of treating the symptom, diagnose the root cause.

Sound judgement is not a talent. It is a discipline — and it is the thread that runs through all effective data engineering work.

Getting started as a data engineer

A strong start in data engineering depends more on habits and judgment than on flashy technique.

The full set of concepts and patterns can feel overwhelming for a new data engineer. It is not necessary to master everything upfront. In fact, a good data engineer is defined by much more than technical proficiency.

Starting out as a data engineer is less about mastering every technique and more about developing the habits and perspective that sustain growth. The priority is to internalise the engineering principles until they become second nature.

38.1 Developing good habits

New engineers often focus on reaching great technical feats. This is the wrong approach. Instead, all new engineers should aim for two goals—exposure and technical discipline.

The greatest and most common danger for engineers is becoming narrow—focusing purely on engineering techniques rather than the business or wider architectural perspectives. Early exposure to diverse aspects of a data project and team is a healthy antidote.

The second most common danger for engineers is focusing on flashy techniques. On the contrary, the best engineer is built on a foundation of consistent discipline in technical hygiene. That discipline is formed early through small problems.

Exposure and technical discipline can be developed through the following habits:

38.1.1 Aim to be helpful, especially on small tasks

A wide range of small tasks gives exposure to the scenarios an engineer deals with. The best tasks involve addressing unit-test failures or responding to low-urgency production errors. These build the mindset of anticipating errors through exposure to fixing them.

38.1.2 Talk to the team

Constant and open conversation with team members contributes to exposure. Engineers on the frontline are often the first to see an issue. Consequently, the ability to raise risks and articulate uncertainties develops through working with a team. Learning from experienced team members is a core part of gaining exposure, though not all team members are good models. It is important

to be aware of their track record. In any case, no engineer should work silently in isolation. A new engineer is able to develop good habits of communication before the habit of solo-working takes hold.

38.1.3 Talk to the business

A new engineer benefits from learning the business problem “first-hand,” as if part of the business team. This is owning the business intent. Instead of focusing solely on code, early habits should include writing quality metadata and descriptions. Confidence in live demonstrations and hands-on experimentation with the business grows over time and requires comfort in acknowledging ignorance or mistakes.

38.1.4 Understand the architecture

Developers often focus only on the development environment rather than the complete architecture. Yet multiple environments, CI/CD practices, and orchestration of loads all influence the engineer’s work. Thinking a few steps ahead—not just about development but also about deployment and monitoring— becomes an important habit.

38.1.5 Technical hygiene

Technical hygiene begins with code management. This starts with elementary habits such as using Git version control as second nature. The most common error for new engineers is creating massive branches that release many features at once. Smaller, logically isolated releases build the capability to break down complex problems into simplified steps.

38.1.6 Check, don’t assume

New engineers often make assumptions—about data, people, business, communication, or architecture. A habit of checking every step carefully is a prerequisite for tackling complex problems.

38.1.7 Estimate and refine

Engineers often shy away from estimating effort and time-to-complete, developing an unhealthy habit of making excuses for why estimation is too difficult. Yet the ability to estimate accurately is one of the best indicators of technical competency. This will not come on day one. Developers commonly underestimate before work starts and overestimate once underway. A useful habit is making a rough guess of time-to-complete before starting any task and checking the estimate afterward.

38.1.8 Focus on the principles

Data engineers must develop mastery of techniques necessary for quality engineering. Most new engineers learn better by doing rather than reading a book. However, understanding techniques through a theoretical framework from first principles remains important. Instead of being satisfied with ticking off a delivery feature, a new engineer benefits from reflecting on how the task links to the patterns in this book, especially the principles of data engineering.

38.2 Conclusion

A new engineer benefits from balancing the mastery of techniques with confidence in the span of responsibility. A data engineer is more than a developer applying techniques to build a data model. The role involves engaging with the business, understanding its perspective, and alerting the team to issues that may not be visible.

Being helpful remains the best attitude for a new engineer.

Mastery of techniques is essential, though it develops over time. Progress comes from reflecting on these techniques consciously from first principles. This theoretical understanding forms the foundation for tackling more complex tasks.

Progress for a new engineer can be measured by the ability to estimate effort. The ultimate sign of success is the capacity to formulate an effective construction plan.

This does not require to-the-day accuracy in scheduling, but rather an orderly and flexible plan that provides clarity and builds momentum for a project team. Reasonable precision in estimates is sufficient for this purpose.

The ability to formulate an effective construction plan is built on consistent technical discipline, and that discipline begins with the earliest practices of a new engineer.

These habits are not optional—they serve as the foundation for everything else in this book.

Closing essay: Hallmarks of quality

Quality is recognised through patterns of work that endure beyond the immediate feature.

As developers, and broadly as a team with the goal of delivering products, we are under the constant temptation to build products that answer to the demands of the day.

These demands are often driven by emotions we feel towards other people. They may be happiness of appreciations from a client or the fear of reprimand if we do not deliver on time.

However, the instinctive reactions of personalities, and especially of personalities who barely understand our craft, are not reliable guides to quality. While these reactions may be justified—for example, if a project has been dragging on too long, then the people around us are right to be angry, they cannot be relied upon as the quality of work.

Our work is of quality if it enables the organisation to make sound, positive decisions.

Nevertheless, this is too abstract a goal for day-to-day work. Even other measures such as product usages or sentiment analysis of feedback are too distant to help developers make decisions in their daily work. We must seek other guides.

Our purpose is to lay down hallmarks of quality that help a team to recognise and celebrate quality. They are nothing new but are time-honoured practices.

As a developer, we must deliver the features, but also complete the following:

1. Expressive entities
2. Well-written explanation
3. Thoughtful unit-tests
4. Monitor assumptions
5. Anticipate errors
6. Adhere to patterns
7. Optimised code

These are the hallmarks of quality. When done well, we have not only delivered a feature but delivered quality work. The work will be high value, long lasting, will win trust and importantly, give us pride in the work we do. Not everyone will carry out all aspects in every instance, but as a team, these are non-negotiables to quality.

39.1 Expressive entities

When looking at data, it is easy to forget we are dealing with a real world. Yet the real world is what decision makers deal with. All models, whether they are simulation models or data models or machine learning models, are models of the world.

Correspondence to the world by the agent trying to influence it is therefore the final arbitrator of whether a data model is successful.

Expressive entities refer to the idea that the tables and relationships we create in the data warehouse should not merely reflect the data as we found it but corresponds strongly to real world business processes. It is expressive because the consumer or reviewer of the model can clearly recognise the world the model is trying to approximate. If a competent layperson cannot easily relate the model to the real world, then the model is not expressive.

The process of creating expressive entities happen at a local level when we use table and column names that are business meaningful. At the level of a single model, we often say that we are “crafting” entities or tables. At the warehouse level across several models, we often say that we are conforming entities.

A data model is not a quality data model unless it is a useful representation of the world. A data model is, therefore, a worldview—more than a worldview, but not less. A developer without a strong worldview cannot create a sound data model.

In a rapid development environment with the pressure to deliver, crafting and conforming entities are difficult tasks to achieve. They are hard because it is hard to think clearly about the world in a systematic way and then articulate it through data.

There is a temptation to keep moving on to the next feature. Moreover, the benefits of crafting entities are long term and not easily understood by stakeholders. People understand testing and monitoring assumptions, but what does a “conceptual model” mean? However, the implication of inexpressive entities is that no one had spent time thinking about the world and the processes with which we try to influence it.

A quality data model is only achievable by a developer who has thought seriously about the world, how data relates to it, and how to organise the world in a way that makes sense for decisions. The ability to create data entities that are accurate and expressive of the world is an infallible mark of quality.

39.2 Well-written explanation

Documentation is particularly important for data analysis (as opposed to say, a game programmer) because data is a projection of real-world processes and a data code is acting as an “interpreter” between data and the world. This interpretation has a plain English equivalent. Documentation is not simply “code comments” from one developer to another but is an articulation to oneself and to others how the output is reflecting reality. The writer C.S Lewis said, “If you cannot say something in common language, you either do not believe in it or you do not understand it.” If we cannot explain the logic behind our code in plain English, why should anyone believe we know what we are doing?

The primary purpose of a well-written explanation of our code is to articulate the logic back to ourselves. What are the entities we created? What do they really mean? What assumptions have we made? Writing a clear explanation is to step back from the code and do the work again from a different angle. Often, in articulating an explanation, one finds new ways of approaching the problem or discover assumptions that we missed.

The secondary purpose of documentation is to win trust. To non-developers, our work is a black box. A well-written explanation is part of the transparency that are integral to trust. Business areas also have a clearer understanding of the quality work and effort put in development. These builds trust.

A developer does not need to write documentation alone. It is the one task where other people can be intimately involved. Sharing and collaborating on documentation with subject matter experts are avenues to clarify concepts and listen to feedback.

It is interesting to see a shift of habit as developers mature. A new developer often write code with no documentation. Over time, the developer gains a habit to write documentation after completing code, then a habit to write documentation together with code. When mature, it is common for a developer to start with code documentation for complex scenarios (usually for the purpose for circulation). There is nothing surprising about this. The transition merely reflects the fact that the developer has mastered patterns in the work environment and is shifting focus from the mechanics of code-writing to the reality behind what the code is about.

39.3 Thoughtful unit-tests

At the minimum, writing proper unit-tests means that developer has solved the problem more than once and from different angles. This is enough to justify the writing of the test. Often in the writing of a test, the developer discovers a logical error or edge case that escaped attention during development.

In the long term, tests exist because the world is not static. What is true at the time of writing the code may not be true the day after. The data itself can change, or the code itself may change. Sometimes these changes can lead to significant errors that lead to incorrect decisions or harm trust.

In isolated instances, one code element may have low probability of going wrong due to external changes. When running several hundred thousand lines of codes against several thousand entities, and a dozen developers are changing several hundred lines per week, the probabilities of error add up.

Tests are about quality and not quantity nor coverage. Poorly written tests can lead to false confidence.

Resist the temptation to delay a test. If instinct says a code should be tested, do not delay to the next week. It is astonishing how often the test of which we think “I probably should write this test, but I will do this a bit later” is the one that would have caught an error on the week’s product release.

39.4 Assumptions are monitored

All reasoning involves assumptions. Dealing with incomplete data especially requires assumptions. Assumptions is the X in the “If X, then Y.” Assumptions is the short-cut to Y. They allow one to defer insignificant problems to a future date.

The world is not static. Assumptions valid at the time of writing the code may be invalid the next day. When assumptions fail, the consequence is always negative. This is because if the failure of the assumption has no impact, then the assumption is not necessary in the first place.

The failure of assumptions is often incremental rather than sudden. The world and its data usually drift from where we thought it would be rather than making a drastic break.

This is particularly relevant with the rise of AI—models that work today will behave differently as the world changes.

Whether they are sudden or gradual, we must monitor assumptions to mitigate the impact for the inevitable day when they fail. If we interpret fault-tolerance as the ability to go ahead despite errors, then assumptions monitoring is a form of macro fault-tolerance.

Developers often assume uniqueness and existence of data. Without such assumptions, there can be no analysis. But what if the product table is not unique by [Product ID] tomorrow because the application had to add a [Product sub-type]? This is quite possible if the source table came from a non-relational system. If this happens, our logic will fall apart. On the other hand, we become paralysed by risks if we constantly assume source tables can change drastically. Therefore, automated treatment of common types of assumptions should be built into any mature processes.

The key step to monitoring assumptions is to know when we are making one. More often, the issue is not that we are not monitoring the assumption, but that we are not conscious of making them. We assume the source tables will load in the daily ELT process. What if one of them does not load, or if they load but not in the order we expect? Another common type of assumption is to use hard-coded values in code.

Once assumptions have been identified, then it is a matter of being disciplined and monitoring them through an automated framework.

39.5 Anticipate errors

Tests and assumptions fall under the general category of error handling. The question which concerns a seasoned developer is not whether the code runs today, but how it may fail tomorrow. Many can write code against the actual, but writing code against the possible is a whole new level. One can often identify a superior developer by an instinctive, second-nature, obsession about how the code may fail.

Even the most seemingly inconsequential actions can trigger an avalanche of unintended consequences. By learning from these errors and improving resilience of the process, these errors decrease. However, any large, structural change, are still likely to cause unintended effects. The first task of a developer then, is to expect errors to happen and to anticipate specific ones. A

developer must assume that things will always go wrong.

The primary purpose of error handling is to limit damage and prevent catastrophe of ripple effects. If we make it a rule to expect errors, the easiest response is to limit the damage through fault-tolerant code. This is the task of preventing or silencing errors.

The second purpose of error handling is to reveal silent errors. Not all errors cause panic. In fact, the most dangerous errors are silent ones. They can be silent by intention or by accident. In either case, a human should be alerted to apply a fix, or the code should be designed to recover automatically before damage propagates. This is the task of detecting or recovering gracefully from errors.

The third purpose of error handling is for the team to grow continuously. Each error reveals a blind spot. When errors are handled properly, the team will be able to learn from the error and improve on processes. This may be implementing new automated checks or rethinking the algorithm from scratch. With each error the system becomes more mature and more resilient. This is the task of learning from errors. Learning from errors requires an open acknowledgement of failure and a willingness to share the lesson with others. This is why humility is important for developers.

Error handling is hard because the developer needs to look beyond the now and anticipate any changes in the world. It is also challenging because error handling requires an end-to-end understanding of the data architecture. The ability to anticipate, prevent, silence, detect, recover gracefully, and learn from error is a hallmark of genius. This should apply both to technical situation (what happens to the join if the column is null?) and to business rules (does the logic given by the business make sense in all circumstances?).

A delivery team that can continuously learn from errors is unstoppable. While error handling is challenging, a new developer can start by adopting the habit of expecting errors.

39.6 Adherence to patterns

There are slavish adherences to patterns that are counterproductive. However, patterns, whether they be coding styles, naming conventions, architecture, or the use of Git, are an indispensable part of a strong team. There are bad teams that follow patterns, but there are no good teams that do not have them.

The primary purpose of patterns is to pass on hard-earned wisdom acquired by the team. In a complex environment, there are solutions that are difficult to arrive at, or are counter-intuitive or are invisible to the untrained eye. This wisdom has been acquired by predecessors at great cost. It is not proper for developers to remain ignorant of their value due to laziness in observation or a naive confidence in the developer's own judgement.

The second purpose of patterns is to create a consistent experience for end users. This consistent experience ranges from design language such as table names to more abstract experience such as joining a set of unfamiliar tables. This consistency of experience over a body of work is a felt quality that cannot be overstated. This hallmark of quality is achievable only by a team where every developer plays their part adhering to the whole.

The third purpose of patterns is to support thinking by filtering out mental noise. Yes, Camel-

Case is no better than `snake_case` for table names. We could have used `[Datetime created]` as well as `[Creation datetime]` or indeed `[Created datetime]`. In isolation, whether text files have hard tabs, or 4-space, or 3-space soft tabs matter little. On the other hand, when the same standard decision is applied across the codebase, the pattern creates an important stability. In fact, when patterns are consistently applied, anomalies stand out—even anomalies which have nothing to do with the pattern. During review of code that is not written in accordance with a usual pattern, the reviewer is forced to carefully parse and interpret every line and every word.

These mental noises reduce the probability of detecting an error. It is astonishing how, after the developer fixes some code indentation, previously unseen logical errors stand out.

Judicious use of patterns in a team will advance quality by promoting solutions that survived the test of time. Patterns also accelerate delivery by removing developer uncertainty and simplifying decision making. On the other hand, an unthinking adherence to patterns through a “copy-and-paste” mindset can lead to overengineering or stifle innovation. To strike a balance between individual innovation and accumulated wisdom, team members should see themselves as participants in a conversation of innovation in which each team member can contribute with an attitude of “inquire, not debate.”

39.7 Optimised code

Performant code carries out a task with as little server resource as possible. Elegant code promotes readability and simplicity. Taken together, we say the code has been optimised.

The primary purpose of optimising code is to support long term maintainability. In a rapid development environment, a multitude of non-performant code can easily overwhelm the server. If code is not elegant, we increase the burden of changing the code. In isolated instances the problem may be insignificant. Over several hundred thousand lines of code, the problem compounds.

A direct way to write performant code in a data warehouse is to use incremental extract logic. This is by no means easy. It can be difficult to do accurately when integrating several datasets. Incremental extract can influence overall design. For example, one may be selective about what tables to use in a single script to avoid a dependency that would ruin the potential to incrementally extract.

The secondary purpose of optimising code is to think about the problem again. It is harder, and takes more thought, to write simple code than a complicated one. Often in making code elegant, the developer sees the concept in a different light. The process of a re-think can lead to conceptual breakthroughs.

Not all good developers can write optimised code, but bad developers can never write elegant, optimised code. Optimised code is one of the harder hallmarks to achieve. We can all improve with practice.

39.8 Three aspects of quality

The hallmarks describe three aspects of data-based code: expressiveness, error handling and elegant code.

39.8.1 Expressiveness

Creating expressive entities, well-written documentation.

Expressiveness is grounded in the fact that data is not a real-world entity in and of itself but is a projection of real-world processes onto computer databases. Such projections are often fuzzy, imperfect, and contingent. When we create expressive entities, our goal is to reconstruct a clear and useful picture of the world from whence the data came.

Well-written documentation serves the same purpose by articulating in plain language how a data model relates to the real world. Analysts who continually take data at face value (each row in the Task table is a task right?), or mechanically applying statistical algorithms from a textbook is divorced from the reality behind the data.

39.8.2 Error handling

Anticipating errors, thoughtful unit-tests, monitoring assumptions.

Error handling is to code what seat belts and safety breaks are to cars or monitoring and kill-switches are to a power plant. They are not "nice to haves" but are indispensable components to a complex computational environment. Unit tests and monitoring assumptions all help to mitigate against the inevitable day when an error occurs. A developer creating code without considering what may go wrong is akin to a builder who adds a floor to a building without strengthening the foundations—it will work, for now.

39.8.3 Elegant code

Adherence to patterns, optimised code.

Elegant code is key to sustainable growth. In a complex and fast-moving environment, we cannot afford spaghetti code nor free-for-all approaches to common problems.

Performance-tuned code is obvious for the simple reason that the computational resources are finite.

39.9 Final words

These hallmarks of quality are not revolutionary. They have been around since code development began. The issue is not that developers do not know about them but that they do not do them because of lack of discipline or pressures to deliver.

There is the idea that doing work properly cost time and delay delivery. The retort is that doing work improperly cost more time. Writing good tests help capture errors earlier and reduces the time it takes to fix things up. Crafted entities vastly reduce the time to deliver complex features because the model becomes more powerful. Writing a clear explanation helps all get on-board, establishes trust and builds momentum. On the other hand, technical debt quickly comes back with a vengeance. At the level of a single development cycle, doing the work properly may increase development time by 20 to 40%. Over the course of several months, the dividends of quality work pay off and enable faster delivery than hasty development could ever achieve.

The developer is constantly under the pressures of deadlines. However, if one does not have time to do the work well, the solution is not to do the work poorly. The best way to achieve the hallmarks is simply consider these as part of, and not something in addition to, the work itself. Once we make the mental leap, it becomes more natural to build these into the daily work and to the pace of projects.